A PRACTICAL GUIDE FOR WINDOWS 10 AND 11

Mastering Batch Scripting

From CMD Basics to Advanced Automation

5	24	10
parts	chapters	projects

First Edition | Version 1.0

Contents

	Preface	4
	Conventions Used in This Book	5
	Using the Practice Folder	7
	List of Figures	8
1	Foundations	9
	Chapter 1: What CMD and Batch Scripting Are	10
	Chapter 2: Setting Up Your Environment	16
	Chapter 3: Essential CMD Commands	23
	Chapter 4: Your First Batch Script	34
	Part 1 Quiz	42
2	Core Language	43
	Chapter 5: Variables	44
	Chapter 6: Arguments and Parameters	53
	Chapter 7: String Handling and Delayed Expansion	60
	Chapter 8: Arithmetic and Logic	70
	Chapter 9: Conditionals	80
	Chapter 10: Loops	90
	Chapter 11: Labels, goto, call, and Subroutines	100
	Chapter 12: Input, Output, Redirection and Escaping	109
	Part 2 Quiz	120
3	Practical Programming	121
	Chapter 13: File and Folder Operations	122
	Chapter 14: Error Handling and Robustness	131
	Chapter 15: Working with Dates, Times and Timestamps	140
	Chapter 16: Menus and User Interaction	147
	Chapter 17: Process and System Management	155
	Chapter 18: Networking from the Command Line	164
	Chapter 19: Interoperability	171
	Part 3 Quiz	179
4	Advanced Topics	180
	Chapter 20: Advanced Techniques	181

	Chapter 21: Performance and Optimization	189
	Chapter 22: Debugging and Testing	195
	Chapter 23: Security and Best Practices	204
	Chapter 24: Style Guide and Project Structure	214
	Part 4 Quiz	221
5	Projects	222
	Chapter 25: Project 1, Number Guessing Game	223
	Chapter 26: Project 2, System Information Report Generator	228
	Chapter 27: Project 3, File Organizer by Extension	233
	Chapter 28: Project 4, Bulk File Renamer	238
	Chapter 29: Project 5, Log File Analyzer	244
	Chapter 30: Project 6, Automated Backup Tool with Logging	250
	Chapter 31: Project 7, Disk Usage Monitor	256
	Chapter 32: Project 8, Scheduled Cleanup Utility	261
	Chapter 33: Project 9, Network Diagnostics Reporter	266
	Chapter 34: Project 10, Menu-Driven Administration Toolkit	271
	Part 5 Quiz	277
A-I	Reference Material	278
	Appendix A: Complete Command Reference	279
	Appendix B: Special Characters and Escape Rules	284
	Appendix C: Error Level and Exit Code Reference	286
	Appendix D: Parameter Modifier Reference	289
	Appendix E: Common Pitfalls and Troubleshooting Checklist	291
	Appendix F: Cheat Sheet	294
	Appendix G: Glossary	296
	Appendix H: Answers to All Exercises	298
	Appendix I: Recommended Next Steps and Further Resources	323
-	Index	325
	Index of Commands and Keywords	326

Preface

Batch scripting is one of the oldest automation technologies still shipping in a current operating system. Every installation of Windows 10 and Windows 11 includes `cmd.exe`, and every administrator eventually meets a `.bat` or `.cmd` file: a logon script, a build step, an installer wrapper, a scheduled maintenance job. These files are often dismissed as primitive, yet they remain the most portable way to automate a Windows machine without installing anything.

The language is also unforgiving. Its syntax grew by accretion over four decades, its parser has rules that are rarely documented in one place, and many errors fail silently. This book treats Batch as what it is: a real, if idiosyncratic, programming language with a precise execution model. Once that model is understood, the odd behaviors stop being odd.

What you will be able to do

By the end of this book you will be able to read any batch file and predict what it does, write maintainable scripts with proper structure, validation, logging and error handling, automate file, process, service and network administration, and recognize the patterns that make a script unsafe or slow. Part 5 contains ten complete projects that combine everything.

How this book is organized

Part	Chapters	Content
1. Foundations	1 to 4	What CMD is, setup, essential commands, your first script
2. Core Language	5 to 12	Variables, parameters, strings, arithmetic, conditionals, loops, subroutines, redirection
3. Practical Programming	13 to 19	Files, errors, dates, menus, processes, networking, interoperability
4. Advanced Topics	20 to 24	Data structures, performance, debugging, security, style
5. Projects	25 to 34	Ten complete projects of increasing difficulty
Appendices	A to I	References, cheat sheet, glossary, exercise answers, resources

Chapters build on each other. A concept is never used before it is introduced, so reading in order is strongly recommended. Each chapter opens with learning objectives and closes with a summary, a list of common mistakes and graded exercises. Answers to every exercise are in Appendix H. Each Part ends with a short quiz.

Conventions Used in This Book

Typography

Commands, file names, variables and switches appear in `monospaced type`. Complete code is shown in **listings**, each labelled with a file name in a dark header bar:

example.bat

```
@echo off
echo This is a script saved as example.bat
```

Text typed at the interactive prompt is shown in a grey-headed block. The prompt itself (`C:\>`) is not typed:

Command Prompt

```
C:\> echo This is typed at the prompt
```

The text a command prints is always shown separately, in a dashed block labelled **Output**:

OUTPUT

```
This is typed at the prompt
```

Outputs that contain machine-specific values (dates, drive serial numbers, addresses, user names) are samples. Your values will differ.

Call-out boxes

Best Practice: A recommended habit that prevents problems or improves clarity.

Common Mistake: A frequent error, with an explanation of what goes wrong and why.

Warning: A command that deletes or changes data, or a behavior with security impact. Every such example in this book includes a safety check.

Version note: A behavior that depends on the Windows version (Windows 10 or Windows 11), on the console host, or on whether the code runs at the prompt or inside a batch file.

Percent signs: prompt versus batch file

The single most common source of confusion in this book's examples is the percent sign in `for` loops. At the interactive prompt a loop variable is written with **one** percent sign (`%i`). Inside a batch file it must be written with **two** (`%%i`). Every example states which context it is for. The same rule applies to a literal percent sign in a batch file (`%%`).

Safety

Examples that delete, move or overwrite data operate only on a dedicated practice folder, show what they would do before doing it, and require confirmation. Always test new scripts on copies of data. The scripts in this book are defensive and administrative in nature.

Environment

All code targets `cmd.exe` on Windows 10 (version 21H2 and later) and Windows 11. Where a command is deprecated or has been removed, the text says so and gives the supported replacement. Windows is not required to read the book, but the examples cannot be run on other systems.

Using the Practice Folder

Several exercises and projects need sample files. Create a practice area once, and reuse it. In a Command Prompt window, type each line below (you will meet every command in Chapter 3):

Command Prompt

```
C:\> md "%USERPROFILE%\BatchBook"
C:\> cd /d "%USERPROFILE%\BatchBook"
C:\Users\Alex\BatchBook> md work scripts logs
```

From now on, **the practice folder** means `%USERPROFILE%\BatchBook`. Save your scripts in `scripts`, and do your experiments in `work`. Nothing in this book asks you to modify files outside the practice folder unless it says so explicitly and shows a safety check.

List of Figures

1.1 Console host, shell and programs 10

1.2 How cmd.exe processes one batch line 13

7.1 Immediate versus delayed expansion inside a block 63

9.1 Decision flow of if / else 80

10.1 Control flow of for /l 90

11.1 call and return for a subroutine 101

12.1 Standard streams and redirection 109

14.1 Execution flow of a robust script 131

30.1 Backup tool workflow 250

34.1 Structure of the menu-driven toolkit 271

1 Foundations

Chapters 1 to 4

What CMD and Batch are, how to set up, the essential commands, and your first script

Part 1 gives you the vocabulary and the working environment. You will learn what `cmd.exe` is and how it differs from other shells, set up an editor and a safe practice area, learn the commands you will use daily, and write and run your first batch file.

Chapters in this Part

- Chapter 1: What CMD and Batch Scripting Are
- Chapter 2: Setting Up Your Environment
- Chapter 3: Essential CMD Commands
- Chapter 4: Your First Batch Script
- Part 1 Quiz

Chapter 1: What CMD and Batch Scripting Are

LEARNING OBJECTIVES

- Distinguish the console host, the shell and a program
- Explain what a batch file is and how it relates to the Command Prompt
- Describe the history of the Windows command interpreter
- Compare CMD with PowerShell and with Unix shells
- Describe, at a simplified level, how `cmd.exe` processes a line

1.1 Three layers: console host, shell, program

People say "command prompt" to mean three different things. Separating them removes much confusion later.

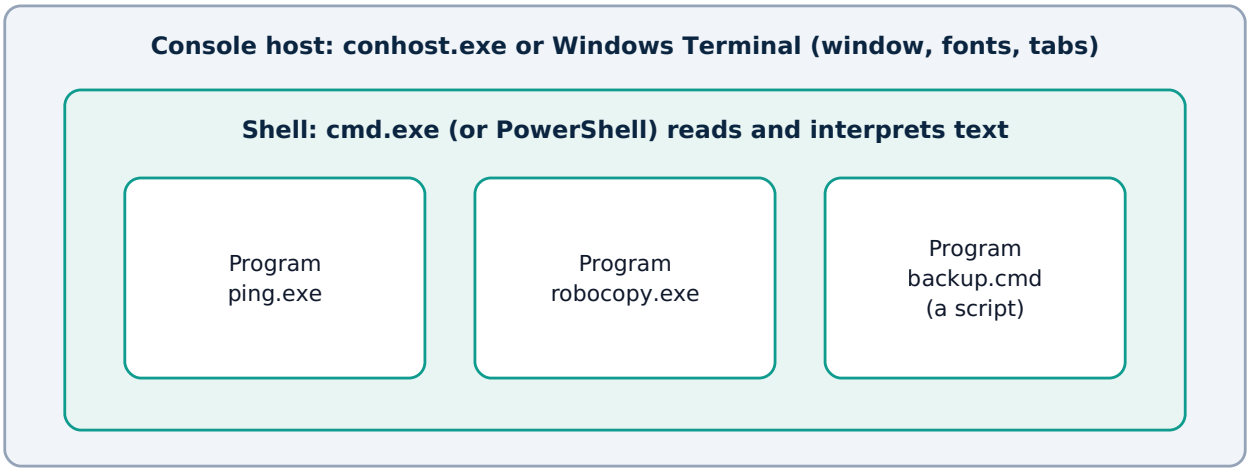


Figure 1.1. The console host draws the window, the shell interprets text, programs do the work.

- The **console host** is the window and text engine. On Windows 10 it is usually `conhost.exe`; on Windows 11 the default host can be **Windows Terminal**, which can host any shell in tabs.
- The **shell** is the program that reads what you type and decides what to run. The classic Windows shell is `cmd.exe`, the **Command Processor**. PowerShell (`powershell.exe`, `pwsh.exe`) is another shell that can run in the same window.
- A **program** is anything the shell launches: `ping.exe`, `robocopy.exe`, or another script.

A **batch file** is a plain text file containing commands that `cmd.exe` reads and executes line by line. Writing a batch file is nothing more than saving the lines you would otherwise type, plus control-flow features (variables, conditions, loops, subroutines) that `cmd.exe` provides.

> **Windows Terminal:** Windows Terminal is the default terminal application on current Windows 11 releases and is available for Windows 10 from the Microsoft Store. Choosing Windows Terminal changes the window, fonts and tabs. It does **not** change the behavior of `cmd.exe` or of your batch files.

1.2 A short history

Period	Development
1981	MS-DOS 1.0 ships with `COMMAND.COM`, a small interpreter. Batch files (`.BAT`) exist from the start and offer `ECHO`, `PAUSE`, `REM`, `GOTO`, `IF` and `FOR`.
1993	Windows NT 3.1 introduces `cmd.exe`, a 32-bit interpreter that is not a DOS program. Extensions such as `set /a`, `for /f`, delayed expansion and `call :label` appear over the following years.
1995 to 2000s	Windows 95, 98 and Me keep `COMMAND.COM`. Windows NT, 2000 and XP use `cmd.exe`. The `.CMD` extension becomes common.
2006	PowerShell (originally "Monad") is released as a new object-based shell.
2015 onward	Windows 10 and later add ANSI escape sequence support to the console, and `curl.exe` and `tar.exe` become part of the system.
2020 onward	Windows Terminal is released as an open-source application.

The key consequence of this history is **backward compatibility**. A batch file written for Windows NT often still runs today. The language was extended, never redesigned, which explains the unusual syntax: new features had to fit into grammar that already existed.

1.3 What Batch is good at, and what it is not

Batch excels at gluing together command-line programs: copying files, calling tools in sequence, checking exit codes, and running on any Windows machine with nothing installed. It is weak at structured data, text manipulation, arithmetic beyond 32-bit integers, and anything needing libraries.

> **Best Practice:** Choose Batch when the job is "run these programs in this order, with simple decisions". Choose PowerShell when you need objects, JSON, web requests, or complex data. Hybrid scripts (Chapter 19) let each language do what it does best.

1.4 CMD compared with PowerShell and Unix shells

Aspect	CMD (Batch)	PowerShell	Bash (Linux/ macOS)
Data model	Plain text only	.NET objects	Plain text
Script extension	`.bat`, `.cmd`	`.ps1`	`.sh` (by convention)
Variable syntax	`%name%` and `!name!`	`$name`	`$name`
Command names	Short, legacy (`dir`, `copy`)	Verb-Noun (`Get-ChildItem`)	Short (`ls`, `cp`)
Functions	Emulated with `call :label`	Real functions	Real functions
Arrays and dictionaries	Emulated with naming conventions	Built in	Built in
Path separator	Backslash (`\`)	Backslash or slash	Slash (`/`)
Case sensitivity	Commands and file names are not case-sensitive	Mostly not	Case-sensitive
Preinstalled	Yes	Windows PowerShell 5.1 is	Yes
Escape character	Caret (`^`)	Backtick (`` ` ``)	Backslash (`\`)

Two differences matter most for how you think about code.

First, text only. When you run `dir` in CMD, you receive characters, not file objects. To extract the size of a file you must parse text, or use a feature such as `%~z1` (Chapter 6). PowerShell would return an object with a `Length` property.

Second, the parse model. Bash and PowerShell parse a command and then run it. `cmd.exe` expands variables *before* it understands the structure of the line. This one fact explains delayed expansion (Chapter 7), the behavior of variables inside parentheses, and most of the "strange" bugs people meet.

> **Mental model:** A batch file is not compiled and not parsed as a whole. `cmd.exe` reads it one line (or one parenthesized block) at a time, expands percent variables, and then executes. Variables are replaced by their text **before** the command runs.

1.5 How cmd.exe processes a line (simplified)

The complete parsing rules are intricate. The following simplified model is accurate enough for everything in this book, and each phase is explained in the chapter where it matters.

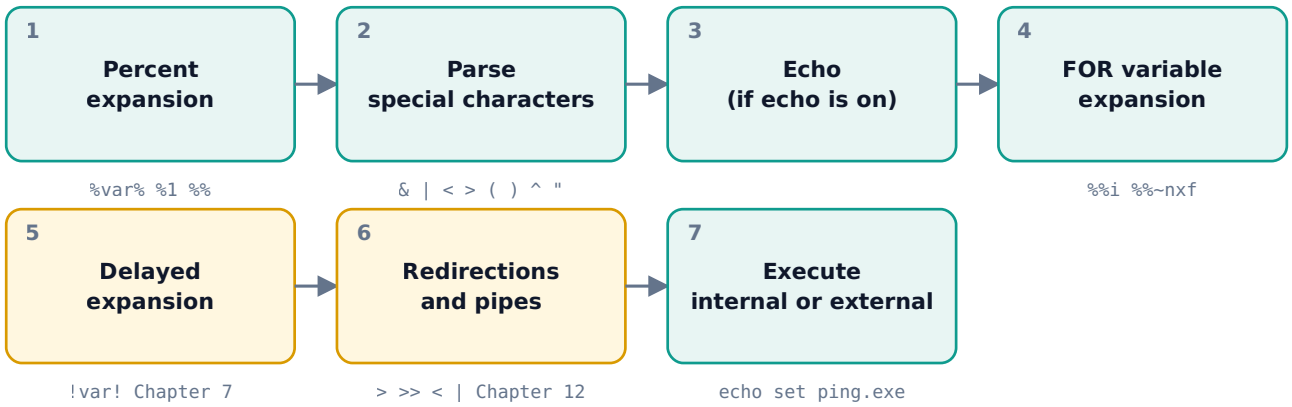


Figure 1.2. Simplified phases through which a batch line passes. Phases marked with a chapter number are covered there.

1. **Percent expansion.** In a batch file, `%1` to `%9`, `%*`, `%name%` and `%%` are replaced by their text. A `%%` becomes a single `%`.
2. **Parsing.** The line is split at special characters: `&`, `&&`, `||`, `|`, `<`, `>` and parentheses, taking quotes and the caret escape into account.
3. **Echo.** If echo is on, the line is displayed.
4. **FOR variable expansion.** Loop variables such as `%%i` are replaced inside `for` bodies.
5. **Delayed expansion.** If enabled, `!name!` is expanded now, just before execution.
6. **Redirection and pipes** are set up.
7. **Execution.** The command is run. It is either an **internal command** built into `cmd.exe` (such as `echo`, `set`, `if`, `cd`) or an **external program** found through the `PATH` variable.

A first demonstration

Percent expansion happens before execution. The following listing shows the consequence. Do not worry about the unfamiliar commands; observe the output.

demo_expansion.bat

```
@echo off
set "x=1"
rem The whole parenthesized block is read first.
rem %x% is replaced by 1 while the block is parsed, before set runs.
(
    set "x=2"
    echo x is %x%
)
echo x is now %x%
```

OUTPUT

```
x is 1
x is now 2
```

Inside the block, `%x%` was replaced with `1` when the block was parsed, before `set "x=2"` executed. After the block, a new line is parsed and `%x%` is `2`. Chapter 7 shows how to obtain the value `2` inside the block.

1.6 Internal versus external commands

Internal commands live inside `cmd.exe` itself and have no separate file. External commands are programs in files. You can tell them apart with `where`:

Command Prompt

```
C:\> where ping
C:\> where dir
```

OUTPUT

```
C:\Windows\System32\PING.EXE
INFO: Could not find files for the given pattern(s).
```

`ping` is an external program. `dir` is internal, so `where` finds no file. Knowing the difference matters because internal commands are faster (no process is started) and their help is built into the shell, while external commands may be missing on a given system or replaced by earlier entries on `PATH`.

Summary

- The console host draws the window, the shell (`cmd.exe`) interprets commands, programs do work.
- A batch file is a text file of commands read line by line by `cmd.exe`.
- CMD descends from MS-DOS and has preserved backward compatibility, at the cost of unusual syntax.
- Batch handles text only; PowerShell handles objects.
- Variables are expanded **before** a command runs. This explains the most surprising behaviors you will meet.
- Internal commands are built into `cmd.exe`; external commands are programs found via `PATH`.

Common mistakes

Common Mistake: Believing Batch and PowerShell are interchangeable. Pasting PowerShell syntax (`$x = 5`) into a batch file fails. They are separate languages with separate interpreters.

Common Mistake: Assuming a variable changes inside a parenthesized block immediately. The block is expanded as a whole first (Section 1.5). Chapter 7 explains the cure.

> **Common Mistake: Confusing the window with the shell.** Changing the terminal application or font does not change language behavior.

Exercises

1. **[Easy]** Name the three layers described in Section 1.1 and give one example of each.
2. **[Easy]** What is the file extension most associated with Windows batch scripts? Name both common extensions.
3. **[Easy]** Is `copy` an internal or external command? How can you check?
4. **[Medium]** List three differences between CMD and PowerShell from Section 1.4 and explain which one matters most for how data is handled.
5. **[Medium]** In the listing `demo_expansion.bat`, why does the first `echo` print `1` rather than `2`?
6. **[Medium]** Explain what "backward compatibility" means for batch files and why it makes the syntax unusual.
7. **[Hard]** The command `echo %date%` is typed at the prompt and `echo %%date%%` is placed in a batch file. Which prints the current date, and what does the other print? Explain using the percent-expansion phase.
8. **[Hard]** Draw, in words, the sequence of phases a line such as `echo Hello & echo World` goes through before output appears.

Chapter 2: Setting Up Your Environment

LEARNING OBJECTIVES

- Open the Command Prompt in the ways you will use most, including with administrator rights
- Choose and configure a text editor suitable for batch files
- Explain the difference between `.bat` and `.cmd`
- Run a script from Explorer and from the prompt, and keep the window open to read output
- Understand ANSI, OEM and UTF-8 encodings, and use `chcp` correctly

2.1 Opening the Command Prompt

Method	How	Notes
Start menu	Press the Windows key, type `cmd`, press Enter	Opens in your user profile folder
Run dialog	Windows key + R, type `cmd`, press Enter	Fast; same result
Administrator	Search `cmd`, then choose **Run as administrator**	Required for system changes
From Explorer	Click the address bar, type `cmd`, press Enter	Opens **in the current folder**
Windows Terminal	Open it and choose the Command Prompt profile	Tabs, themes, copy and paste

The last row of Explorer opening is the most useful for scripting: navigate to the folder holding your script, and open a prompt right there.

Reading the prompt

```
Command Prompt

C:\Users\Alex>
```

The text before `>` is the **current directory**. Commands you type are relative to it. The prompt can be customized with the `prompt` command (`prompt $P$G` is the default). The prompt text has no effect on scripts.

> **Administrator windows:** A window opened with **Run as administrator** starts in `C:\Windows\System32`, not your profile. Many scripts break because they assume the starting folder. Chapter 6 shows `%~dp0`, which locates the script's own folder reliably.

2.2 Choosing an editor

A batch file is plain text. Any plain-text editor works. Do **not** use a word processor such as Word, which stores formatting and typographic quotes that `cmd.exe` cannot parse.

Editor	Strengths	Settings to check
Notepad	Always present; simple	Save As: encoding ANSI or UTF-8; file type All Files
Notepad++	Syntax highlighting, line endings, encoding menu	Edit > EOL Conversion > Windows (CR LF)
Visual Studio Code	Extensions, integrated terminal, search across files	Set `files.eol` to `\r\n`, language mode Batch, and check the status-bar encoding

Line endings

Windows text files end lines with two characters: carriage return and line feed (**CRLF**). Unix editors may save only line feed (**LF**). A batch file with LF-only endings can behave unpredictably: `goto` and `call` to labels may fail, and multi-line blocks can misparse. Always save batch files with **CRLF** line endings.

> **Common Mistake: Saving a script as `myscript.bat.txt`.** Notepad appends `.txt` when the file type is "Text Documents". Choose **All Files** in the Save As dialog and type the full name in quotes: `"myscript.bat"`.

Show file extensions

In File Explorer open **View > Show > File name extensions**. Without this you cannot tell `backup.bat` from `backup.bat.txt`, and the most common beginner problem becomes invisible.

2.3 `.bat` versus `.cmd`

Both extensions are run by `cmd.exe`. The difference is small but real:

- In a `.bat` file, a few commands (`append`, `dpath`, `ftype`, `set`, `path`, `assoc`, `prompt`) set `ERRORLEVEL` **only when they fail**. They leave it unchanged on success.
- In a `.cmd` file those commands set `ERRORLEVEL` on success too (to 0), so it always reflects the most recent command.

Chapter 9 explains `ERRORLEVEL`. For now, the practical advice:

> **Best Practice:** Use `.cmd` for new scripts, because its `ERRORLEVEL` behavior is more predictable. Use `.bat` when an existing tool or convention requires it. This book uses `.bat` for most listings, because it is the form readers recognize, and notes where the difference could matter.

The extensions that `cmd.exe` will run without you typing them are listed in the `PATHEXT` variable:

Command Prompt

```
C:\> echo %PATHEXT%
```

OUTPUT

```
.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC
```

(Your list may be longer. It may also include `.PY` or `.CPL`.) Typing `backup` runs the first match found in the order of this list. If both `backup.exe` and `backup.bat` exist in a folder, `backup.exe` wins.

2.4 Running a script

From the prompt

Navigate to the folder, then type the name:

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> hello.bat
C:\Users\Alex\BatchBook\scripts> hello
C:\Users\Alex\BatchBook\scripts> "C:\Users\Alex\BatchBook\scripts\hello.bat"
```

All three work. Typing only the name works because the current directory is searched first for the command (and the extension comes from `PATHEXT`).

By double-clicking

Explorer opens a new window, runs the script, and **closes the window when the script ends**. If the script prints something, you will see a flash. Add `pause` at the end (Chapter 4) or run the script from an open prompt.

With `cmd /c` and `cmd /k`

Command Prompt

```
C:\> cmd /c "C:\Users\Alex\BatchBook\scripts\hello.bat"
C:\> cmd /k "C:\Users\Alex\BatchBook\scripts\hello.bat"
```

`/c` runs the command and then terminates the new shell. `/k` runs it and keeps the new shell open. `cmd /k` is a good way to test a script while keeping the window open to look at variables afterward.

From another script: `call`

When one batch file runs another, you must use `call`, or control never returns to the first file:

run_two.bat

```
@echo off
echo Step one
call "%~dp0hello.bat"
echo Step two
```

Without `call`, the second script would run, and the first would stop at that point and never print "Step two". Chapter 11 explains `call` in depth.

2.5 First debugging aids

Three techniques solve most early problems.

1. **Remove** `@echo off` (or temporarily write `echo on`) to see every command as it runs.
2. **Add** `pause` at the end to keep the window open.
3. **Run from an open prompt** so error messages stay visible.

broken.bat

```
@echo off
echo Starting
type no_such_file.txt
echo Finished
pause
```

OUTPUT

```
Starting
The system cannot find the file specified.
Finished
Press any key to continue . . .
```

Notice that `type` on a missing file does not stop the script. A command failing does **not** abort a batch file. You must check for failure yourself (Chapter 14). To see exactly which lines run, delete the first line (`@echo off`) and run the script again: `cmd.exe` then prints the prompt and the text of every command before executing it. Chapter 4 explains how to control this.

2.6 Character encoding

A computer stores characters as numbers. An **encoding** is the table that maps numbers to characters. Batch files and the console use two different tables by default, which causes the classic "garbled accents" problem.

Name	Typical use	Notes
ANSI (Windows code page, for example 1252)	Default encoding of Notepad files in many regions	Used by `cmd.exe` to read the file
OEM (console code page, for example 437 or 850)	Default for characters the console **displays**	Show with `chcp`
UTF-8 (code page 65001)	Universal	Works well when file and console agree

The `chcp` command

Command Prompt

```
C:\> chcp
```

OUTPUT

```
Active code page: 437
```

`chcp 65001` switches the console to UTF-8. After that, a script saved as **UTF-8 without BOM** can display accented letters correctly if the console font contains them.

Command Prompt

```
C:\> chcp 65001
```

OUTPUT

```
Active code page: 65001
```

Practical rules

Best Practice: Keep scripts pure ASCII. Use only English letters, digits and ordinary punctuation in code, comments and messages. A script that contains no non-ASCII character cannot suffer encoding problems at all. This book follows that rule.

If you must use non-ASCII text:

1. Save the file as **UTF-8 without BOM**. A byte order mark (BOM) at the start of a file can cause the first command to fail with a strange error.
2. Put `chcp 65001 >nul` near the top, after `@echo off`.
3. Use a console font that contains the required characters.
4. Test on Windows 10 and on Windows 11.

Windows 10 versus Windows 11: Newer builds include an option "Beta: Use Unicode UTF-8 for worldwide language support" in the region settings, which changes the system ANSI code page to 65001. It affects many programs and is **not** required for this book. Behavior with UTF-8 batch files has improved across Windows 10 builds, so test your target version rather than assume.

What to remember about `chcp`

- It changes the code page for the **current console window only**.
- It affects how bytes are interpreted when read or displayed, not how the file is stored.
- Some old programs misbehave under 65001.

Summary

- Open the prompt from the Start menu, the Run dialog, Explorer's address bar, or Windows Terminal. Use the administrator option only when you need it.
- Use a plain-text editor, save with CRLF endings and the correct extension, and show file extensions in Explorer.
- `.bat` and `.cmd` differ in `ERRORLEVEL` behavior for a few commands; prefer `.cmd` for new work.
- Run scripts by name, by full quoted path, with `cmd /c` or `cmd /k`, and use `call` from other scripts.
- A failing command does not stop a script. Debug by showing commands, adding `pause`, and running from an open window.
- Keep code ASCII. If you need UTF-8, save without a BOM and set `chcp 65001`.

Common mistakes

Common Mistake: Saving as `.txt`. The script is silently named `name.bat.txt` and does nothing. Choose **All Files** when saving and show extensions.

Common Mistake: Running from a double-click and losing the output. The window closes. Add `pause` or run from an open prompt.

Common Mistake: LF-only line endings. Labels may not be found. Use CRLF.

Common Mistake: Using a word processor. It inserts curly quotes and hidden formatting.

Common Mistake: Assuming the working directory is the script's folder. Administrator windows and shortcuts start elsewhere. Use `%~dp0` (Chapter 6).

Exercises

1. **[Easy]** Open a Command Prompt using three different methods from Section 2.1.
2. **[Easy]** Open Explorer in your practice folder and use the address-bar method to start a prompt there. What does the prompt show?
3. **[Easy]** Turn on file name extensions in Explorer. Why is this important for batch work?
4. **[Medium]** Create `hello.bat` in the `scripts` folder containing `@echo off` and `echo Hello`. Run it three ways: by name, by full path, and with `cmd /k`.
5. **[Medium]** State two differences between `.bat` and `.cmd` that matter in practice.

6. **[Medium]** Run `chcp` and write down the code page. Why might the number differ from a colleague's?
7. **[Hard]** Explain why a script saved as UTF-8 **with** BOM might show an error on its first line, and how you would save it correctly in your editor.
8. **[Hard]** `run_two.bat` calls `hello.bat` without the `call` keyword. Predict the output, then explain what `call` changes.

Chapter 3: Essential CMD Commands

LEARNING OBJECTIVES

- Navigate the file system and list its contents precisely
- Create, copy, move, rename and delete files and folders safely
- Choose between `copy`, `xcopy` and `robocopy`
- Display text files and folder structures, and change file attributes
- Locate programs with `where`, and read command help created by `/?`

All commands in this chapter are typed at the **interactive prompt** unless stated otherwise. They are also the commands you will place in batch files in later chapters. Work in your practice folder:

Command Prompt

```
C:\> cd /d "%USERPROFILE%\BatchBook\work"
```

3.1 How to read command help

Every command supports `/?`. `help dir` prints the same text as `dir /?`, and `help` alone lists the internal commands.

Command Prompt

```
C:\> copy /?
```

OUTPUT

```
Copies one or more files to another location.

COPY [/D] [/V] [/N] [/Y | /-Y] [/Z] [/L] [/A | /B ] source [/A | /B]
     [+ source [/A | /B] [+ ...]] [destination [/A | /B]]
...
```

(The output continues with a description of each parameter.) The notation is consistent:

Notation	Meaning
`[ ... ]`	Optional part
`A | B`	Choose one of the alternatives
`/X`	A **switch**: an option, almost always introduced by a slash
`source`, `destination`	Placeholders that you replace with real values
`...`	The previous item may be repeated

Switches are usually not case-sensitive (`/s` equals `/S`), and most commands let you combine them in any order.

> **Best Practice:** When you meet an unfamiliar switch, read `/?` before experimenting, and test destructive commands in the practice folder on dummy files first.

3.2 Navigating: `cd` , `pushd` , `popd`

`cd` (or `chdir`) changes the current directory. The current directory is where relative paths start.

Command Prompt

```
C:\Users\Alex> cd BatchBook\work
C:\Users\Alex\BatchBook\work> cd ..
C:\Users\Alex\BatchBook> cd \
C:\> cd /d D:\Data
D:\Data>
```

Form	Meaning
`cd folder`	Go into `folder` (relative)
`cd ..`	Go to the parent folder
`cd \`	Go to the root of the current drive
`cd /d D:\Data`	Change drive **and** folder at once
`cd` **(no argument)**	Print the current directory
`D:`	Switch to drive D (its own remembered folder)

> **Common Mistake: Using `cd D:\Data` to change drives.** Without `/d` , `cd` changes only the remembered folder on D: and leaves you on the current drive. Use `cd /d` .

`pushd path` changes directory and **remembers** the old one on a stack; `popd` returns to it. This is the clean way for a script to visit a folder and come back. `pushd` also works with network (UNC) paths such as `\\server\share` by temporarily mapping a drive letter, because `cmd.exe` cannot use a UNC path as its current directory.

Command Prompt

```
C:\Users\Alex> pushd "%USERPROFILE%\BatchBook\work"
C:\Users\Alex\BatchBook\work> popd
C:\Users\Alex>
```

Spaces in paths require quotes: `cd "C:\Program Files"` . Always quote paths in scripts.

3.3 Listing: `dir`

Command Prompt

```
C:\Users\Alex\BatchBook> dir
```

OUTPUT

```
 Volume in drive C is Windows
 Volume Serial Number is 1A2B-3C4D

 Directory of C:\Users\Alex\BatchBook

10/23/2026  09:14 AM    <DIR>          .
10/23/2026  09:14 AM    <DIR>          ..
10/23/2026  09:14 AM    <DIR>          logs
10/23/2026  09:14 AM    <DIR>          scripts
10/23/2026  09:14 AM    <DIR>          work
               0 File(s)              0 bytes
               5 Dir(s)  210,456,371,200 bytes free
```

`dir` is the most heavily used command. These switches cover almost every need:

Switch	Effect
`/b`	**Bare** list: names only, one per line. Ideal for scripts.
`/s`	Include subfolders (recursive)
`/a`	Show attributes: `/a:d` directories, `/a:-d` files only, `/a:h` hidden, `/a:r` read-only
`/o`	Sort order: `/o:n` name, `/o:s` size, `/o:d` date, `/o:-s` largest first
`/w`	Wide, multi-column
`/q`	Show the owner
`/t:w`	Use the last-**w**rite time (default); `/t:c` creation, `/t:a` last access
`/p`	Pause after each screen

Wildcards select by name. `*` matches any number of characters, `?` matches one:

Command Prompt

```
C:\> dir /b *.txt
C:\> dir /b /s /a:-d *.log
C:\> dir /b /o:-s *.*
```

> **Wildcard note:** Because of 8.3 short-name compatibility, `dir *.htm` may also list `.html` files. The behavior is a historical quirk of how `cmd.exe` matches extensions of three characters. Test wildcard patterns that rely on a three-letter extension before you trust them.

3.4 Creating and removing folders: `md`, `rd`

`md` (`mkdir`) creates folders. It creates **all missing parents** at once:

Command Prompt

```
C:\Users\Alex\BatchBook\work> md projects\alpha\docs
C:\Users\Alex\BatchBook\work> dir /b /s /a:d projects
```

OUTPUT

```
C:\Users\Alex\BatchBook\work\projects\alpha
C:\Users\Alex\BatchBook\work\projects\alpha\docs
```

`rd` (`rmdir`) removes folders. Without switches it removes **only empty** folders, which is a useful safety feature. With `/s` it removes the folder and everything inside, and with `/q` it does so without asking.

> **Warning:** `rd /s /q` deletes a whole tree **permanently**, with no Recycle Bin and no confirmation. Never run it with a variable that might be empty or wrong. A safe habit is to run `dir /s` on the same path first, and to keep tests inside the practice folder.

Command Prompt

```
C:\Users\Alex\BatchBook\work> rd projects\alpha\docs
C:\Users\Alex\BatchBook\work> rd /s projects
```

The second command asks `projects, Are you sure (Y/N)?` before deleting, because `/q` was not used.

3.5 Copying: `copy`, `xcopy`, `robocopy`

Three commands copy files. They exist because each generation fixed the limits of the previous one.

Command	Use it for	Limits
`copy`	Single files, simple wildcard copies, joining files	No folder trees, no restart, no logging
`xcopy`	Folder trees with a few options	Considered legacy; no multi-threading, weak error codes
`robocopy`	Everything serious: mirroring, retries, logs, large trees	Exit codes are a bit mask (see below)

copy

Command Prompt

```
C:\Users\Alex\BatchBook\work> echo first > a.txt
C:\Users\Alex\BatchBook\work> copy a.txt b.txt
C:\Users\Alex\BatchBook\work> copy a.txt b.txt /y
```

OUTPUT

```
        1 file(s) copied.
        1 file(s) copied.
```

The first `copy` creates `b.txt`. For the second copy, `/y` suppresses the overwrite confirmation. Without `/y`, an existing destination causes a prompt `Overwrite b.txt? (Yes/No/All):`. Inside scripts, the prompt would block the run, so scripts that overwrite must say so explicitly with `/y`.

xcopy

Command Prompt

```
C:\Users\Alex\BatchBook\work> xcopy projects projects_backup /s /e /i /y
```

Switch	Effect
`/s`	Copy subfolders (not empty ones)
`/e`	Copy subfolders, **including empty ones**
`/i`	Assume the destination is a folder if there are several sources
`/y`	Do not prompt before overwriting
`/d:mm-dd-yyyy`	Copy only files changed on or after a date; `/d` alone copies newer files only
`/h`	Include hidden and system files
`/c`	Continue after errors
`/q`	Quiet

robocopy

Robust File Copy is the preferred tool for folder copies. Its syntax is `robocopy source destination [files] [options]`, with **folders** (not files) as the first two arguments.

Command Prompt

```
C:\Users\Alex\BatchBook\work> robocopy projects projects_copy /e /r:2 /w:1 /np
```

OUTPUT

```
------------------------------------------------------------------------------
   ROBOCOPY     ::     Robust File Copy for Windows
------------------------------------------------------------------------------
```

```
  Started : Friday, October 23, 2026 9:20:15 AM
   Source : C:\Users\Alex\BatchBook\work\projects\
     Dest : C:\Users\Alex\BatchBook\work\projects_copy\
...
```

(The summary table that follows lists Dirs, Files and Bytes as Total, Copied, Skipped, Mismatch, FAILED and Extras.)

Option	Meaning
`/e`	Copy subfolders including empty ones
`/mir`	**Mirror**: make the destination identical, **deleting extra files there**
`/purge`	Delete destination files that no longer exist in the source
`/r:n`	Retry `n` times on failure (default is a million, which looks like a hang)
`/w:n`	Wait `n` seconds between retries
`/l`	**List only**: show what would happen without copying anything
`/log:file`	Write the report to a file; `/tee` also shows it on screen
`/np`	No per-file progress percentage (cleaner logs)
`/xd dir`	Exclude directories; `/xf file` excludes files

Warning: `/mir` and `/purge` **delete files in the destination**. Swapping source and destination, or mistyping a path, can erase data you wanted. Always rehearse with `/l` first.

Best Practice: Before any real `robocopy` run, run the same command with `/l` . Read the list. Then remove `/l` .

`robocopy` exit codes are not like other commands

`robocopy` returns a **bit mask**. Values **0 to 7** indicate success of some kind; **8 or higher** indicates at least one failure.

Value	Meaning
0	Nothing copied; source and destination already match
1	One or more files were copied
2	Extra files or folders found in the destination
4	Mismatches found
8	Some files or folders could not be copied (failures)
16	Fatal error: nothing was copied

Values combine by addition. An exit code of 3 means "copied some files (1) and found extras (2)". The correct test in a script is `if %errorlevel% geq 8` for failure. Chapter 14 shows this in a script.

3.6 Moving and renaming: `move`, `ren`

Command Prompt

```
C:\Users\Alex\BatchBook\work> move b.txt projects\alpha\
C:\Users\Alex\BatchBook\work> ren a.txt first.txt
```

OUTPUT

```
        1 file(s) moved.
```

- `move` relocates files or folders, and can also rename a folder (`move oldname newname`).
- `ren` (`rename`) renames within the same folder only. The new name must **not** include a path. Wildcards work: `ren *.txt *.bak` changes every extension.

Common Mistake: Giving `ren` a destination path. `ren a.txt C:\x\b.txt` fails with "The syntax of the command is incorrect." Use `move` to change location.

3.7 Deleting files: `del`

Command Prompt

```
C:\Users\Alex\BatchBook\work> echo temp > scratch.tmp
C:\Users\Alex\BatchBook\work> del /p scratch.tmp
```

OUTPUT

```
C:\Users\Alex\BatchBook\work\scratch.tmp, Delete (Y/N)?
```

Switch	Effect
`/p`	**Prompt** for each file
`/q`	Quiet: no confirmation, even for `*`
`/f`	Force deletion of read-only files
`/s`	Also delete matching files in all subfolders
`/a:attr`	Select by attribute

`del` deletes **files**, never folders, and bypasses the Recycle Bin. Used on a folder name it deletes the files inside that folder but not the folder.

> **Warning:** `del /s /q *.*` run in the wrong folder is a classic disaster. The safe workflow: list first with `dir /b /s pattern`, review, then delete with `/p` or in a practice folder.

3.8 Looking at content: `type`, `more`, `cls`, `tree`

`type` writes a text file to the screen. `more` pages it, and it can also page the output of another command:

Command Prompt

```
C:\Users\Alex\BatchBook\work> type first.txt
C:\Users\Alex\BatchBook\work> type biglog.txt | more
C:\Users\Alex\BatchBook\work> more < biglog.txt
```

OUTPUT

```
first
```

In `more`, press **Space** for the next page, **Enter** for the next line and **Q** to quit. The `|` (pipe) and `<` (redirection) symbols are explained fully in Chapter 12. `cls` clears the screen.

`tree` draws a folder structure:

Command Prompt

```
C:\Users\Alex\BatchBook\work> tree projects /f /a
```

OUTPUT

```
Folder PATH listing for volume Windows
Volume serial number is 1A2B-3C4D
C:\USERS\ALEX\BATCHBOOK\WORK\PROJECTS
\---alpha
        b.txt
```

`/f` includes files; `/a` uses plain ASCII lines rather than box-drawing characters, which avoids encoding problems when you redirect the output to a file.

3.9 Attributes: `attrib`

Files carry attribute flags: **R** read-only, **A** archive (changed since last backup), **S** system, **H** hidden.

Command Prompt

```
C:\Users\Alex\BatchBook\work> attrib first.txt
C:\Users\Alex\BatchBook\work> attrib +r +h first.txt
C:\Users\Alex\BatchBook\work> attrib first.txt
C:\Users\Alex\BatchBook\work> attrib -r -h first.txt
```

```
OUTPUT
A           C:\Users\Alex\BatchBook\work\first.txt
A  H  R     C:\Users\Alex\BatchBook\work\first.txt
```

`+` sets and `-` clears a flag. `/s` applies to subfolders, and `/d` includes folders in the operation. A read-only file cannot be deleted by plain `del`; you must add `/f`.

3.10 Finding programs: `where`

`where` searches the folders in `PATH` (and optionally others) for files:

Command Prompt

```
C:\> where notepad
C:\> where /r "%USERPROFILE%\BatchBook" *.txt
C:\> where /q robocopy && echo robocopy is available
```

```
OUTPUT
C:\Windows\System32\notepad.exe
C:\Windows\notepad.exe
C:\Users\Alex\BatchBook\work\first.txt
robocopy is available
```

`where /q` prints nothing and only sets the exit code: **0** found, **1** not found, **2** error. Scripts use it to confirm that a required tool exists before using it (Chapter 14).

3.11 A quick tour of other everyday commands

Command	Purpose	Example
`echo`	Print text (Chapter 4)	`echo Hello`
`date /t`, `time /t`	Show the current date or time without asking to change it	`date /t`
`ver`	Windows version string	`ver`
`hostname`	Computer name	`hostname`
`whoami`	Current user	`whoami`
`find`, `findstr`	Search inside text (Chapter 13)	`findstr /i error log.txt`
`sort`	Sort lines (Chapter 13)	`sort names.txt`
`mklink`	Create symbolic links (needs privileges)	`mklink /d link target`

Summary

- Use `/?` or `help` to read syntax. Brackets mean optional; the vertical bar means choice.

- `cd /d` changes drive and folder; `pushd` and `popd` visit and return, and handle UNC paths.
- `dir /b` is the script-friendly listing; `/s`, `/a` and `/o` filter and sort.
- `md` creates parents; `rd` without switches removes only empty folders.
- Choose `robocopy` for serious copying; use `/l` to rehearse and remember that exit codes 0 to 7 are success.
- `del` removes files only, permanently. `rd /s /q` and `/mir` are dangerous and deserve rehearsal.
- `where` finds programs and, with `/q`, tests for their existence.

Common mistakes

Common Mistake: Forgetting quotes around paths with spaces. `cd C:\Program Files` fails. Write `cd "C:\Program Files"`.

Common Mistake: Treating `robocopy` exit code 1 or 3 as failure. Only 8 and above signal failure.

Common Mistake: Relying on `copy` to copy folder trees. `copy` handles files. Use `robocopy` or `xcopy /e`.

Common Mistake: Using `del` on a folder and expecting the folder to vanish. It deletes the files only; use `rd`.

Exercises

1. **[Easy]** Display the current directory, then change to your practice folder and list it in bare format.
2. **[Easy]** Create `work\notes\2026\drafts` with a single command.
3. **[Easy]** Use `help` to find what the `/o:-s` switch of `dir` does, then list `C:\Windows` files from largest to smallest.
4. **[Medium]** Create three text files with `echo text > name.txt`. Rename them all to `.bak` with one command.
5. **[Medium]** Copy a folder tree with `robocopy`, first as a rehearsal (`/l`) and then for real. Describe how the two outputs differ.
6. **[Medium]** Set a file hidden and read-only. Show it with `dir /a:h`. Then try `del` on it, observe the message, and delete it properly.
7. **[Hard]** `robocopy` returns 3 after a run. Decode the value and say whether a script should treat it as success.

8. **[Hard]** Write a command sequence that visits `work\notes`, lists its contents, and returns to the starting folder without using the original path again.

Chapter 4: Your First Batch Script

LEARNING OBJECTIVES

- Create, save and run a batch file
- Control what is displayed with `@echo off`, `echo` and its variants
- Write comments with `rem` and `::`, and know when each is safe
- Use `pause`, `title`, `cls` and `exit` appropriately
- Organize a script with a consistent header and structure

4.1 The smallest useful script

Open your editor and create `hello.bat` in `%USERPROFILE%\BatchBook\scripts`:

hello.bat

```
@echo off
echo Hello, world!
pause
```

Run it from a prompt opened in that folder:

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> hello.bat
```

OUTPUT

```
Hello, world!
Press any key to continue . . .
```

Each line is a command. `cmd.exe` runs them from top to bottom.

4.2 `@echo off`: controlling command echo

By default, `cmd.exe` **echoes** (prints) each command before running it. Compare the same script without the first line:

noecho_off.bat

```
echo Hello, world!
```

OUTPUT

```
C:\Users\Alex\BatchBook\scripts>echo Hello, world!
Hello, world!
```

The echoed command line (with the prompt) is noise. `echo off` turns echoing off for the following lines. The `@` prefix suppresses the echo of that single line, which is why the very first line of a script is `@echo off`: the `@` hides the `echo off` command itself.

Statement	Effect
`echo off`	Stop echoing following commands (the line itself is still echoed)
`@echo off`	Same, and the line itself is not echoed
`echo on`	Resume echoing (useful for debugging)
`echo` **(no text)**	Print the current state: `ECHO is on.` or `ECHO is off.`
`@command`	Run one command without echoing it

> **Best Practice:** Start every script with `@echo off`. Temporarily remove it, or use `echo on` before a suspect block, to debug.

4.3 `echo` in detail

`echo text` prints the text. It prints everything after `echo` up to the end of the line, including leading spaces beyond the first.

echo_basics.bat

```
@echo off
echo Plain text
echo     Indented text
echo.
echo Line after a blank line
echo "Quotes are printed too"
echo Tab    separated
```

OUTPUT

```
Plain text
    Indented text

Line after a blank line
"Quotes are printed too"
Tab     separated
```

Printing an empty line

`echo.` prints a blank line, but it fails if a file named `echo` exists in the current folder. More robust forms are `echo:` and `echo(`:

blank_lines.bat

```
@echo off
echo First
echo:
echo Second
```

```
echo(
echo Third
```

OUTPUT

```
First

Second

Third
```

Special characters

Several characters have a meaning to the parser and need an escape. The caret `^` escapes the next character:

echo_special.bat

```
@echo off
echo Tom ^& Jerry
echo 5 ^> 3
echo Use ^| to pipe
echo Caret is ^^
echo 100%% done
```

OUTPUT

```
Tom & Jerry
5 > 3
Use | to pipe
Caret is ^
100% done
```

In a batch file a literal percent sign is written `%%`. Chapter 12 contains the complete escaping rules, and Appendix B tabulates them.

4.4 Comments: `rem` and `::`

A **comment** is text for human readers that the interpreter ignores.

comments.bat

```
@echo off
rem This is a classic comment
:: This is a label-style comment
echo Visible output
rem echo This line is disabled and never runs
```

OUTPUT

```
Visible output
```

Form	Notes
`rem text`	The official comment. Safe everywhere.
`:: text`	A **label** that is never jumped to. Slightly faster and visually lighter, but **unsafe inside parenthesized blocks** such as `if` and `for` bodies.
`rem.` **and** `rem/` **forms**	Avoid; they behave unpredictably with some characters

Common Mistake: Using `::` comments inside `( ... )` blocks. A label inside a block can end the block early or raise "The system cannot find the drive specified" or a syntax error. Use `rem` inside any block.

Best Practice: Use `rem` everywhere for comments, or `::` only at the top level of a script outside any block. Be consistent. This book uses `rem` throughout.

Rem lines are still partly parsed. In particular a `rem` line that ends with a caret or that contains `%` expansions is processed; avoid special characters in comments.

4.5 `pause`, `cls`, `title`

Command	What it does
`pause`	Prints `Press any key to continue . . .` and waits for a key
`cls`	Clears the console
`title text`	Sets the window title
`timeout /t 5`	Waits 5 seconds (any key skips it); `/nobreak` ignores keys

title_demo.bat

```
@echo off
title My First Batch Tool
cls
echo Starting the tool...
timeout /t 2 /nobreak >nul
echo Done.
pause >nul
```

OUTPUT

```
Starting the tool...
Done.
```

`>nul` discards a command's own message. Here it hides the "Waiting for 2 seconds" countdown from `timeout` and the "Press any key" text from `pause`. Redirection is explained in Chapter 12.

> **Common Mistake: Putting `pause` at the end of a script meant to be run by another program or from the Task Scheduler.** The script waits forever for a key that nobody will press. Reserve `pause` for scripts run by hand.

4.6 Ending a script: `exit` and `exit /b`

A script ends when it runs out of lines. To end earlier use:

Form	Effect
`exit /b`	End the **script** (or the current subroutine), return to the caller. The console stays open.
`exit /b n`	The same, setting the exit code to `n`
`goto :eof`	Equivalent of `exit /b` for the current routine (Chapter 11)
`exit`	Close the **whole** `cmd.exe` **session**. Inside a script run by double-click, this closes the window.

early_exit.bat

```
@echo off
echo Checking prerequisites...
if not exist "%windir%\System32\ping.exe" (
    echo ping.exe is missing.
    exit /b 1
)
echo All prerequisites found.
exit /b 0
```

OUTPUT

```
Checking prerequisites...
All prerequisites found.
```

The **exit code** (here 0 or 1) tells the caller whether the script succeeded. By convention 0 means success and any other value means failure. You read it afterwards as `%errorlevel%` (Chapter 9).

> **Common Mistake: Using plain `exit` in a script you call from another script or from an open prompt.** It closes the entire session. Prefer `exit /b` .

4.7 Script structure

Real scripts follow a predictable skeleton. This template is used throughout the book:

template.bat

```
@echo off
setlocal EnableExtensions
rem ======================================================================
rem  Name:        template.bat
rem  Purpose:     One sentence describing what the script does
```

```
rem  Usage:       template.bat [argument]
rem  Requires:    Windows 10 or later
rem  Author:      Your name
rem  Version:     1.0
rem ================================================================

rem ---- Configuration ---------------------------------------------
set "APP_NAME=Template"

rem ---- Main ------------------------------------------------------
call :main %*
exit /b %errorlevel%

rem ---- Subroutines -----------------------------------------------
:main
echo %APP_NAME% started.
exit /b 0
```

```
OUTPUT
Template started.
```

The pattern has five parts: a header comment documenting the script; `setlocal` to keep variables from leaking into the caller (Chapter 5); configuration constants gathered at the top; a main section that calls subroutines; and subroutines placed after an `exit /b` so execution can never "fall into" them. You have not yet met `setlocal`, `call :label` or `%*`; each is explained in Chapters 5, 6 and 11. Use the template now, and understand it fully later.

4.8 A realistic first script

greet.bat

```
@echo off
setlocal
title Greeting Tool
cls

echo ========================================
echo   Welcome, %USERNAME%
echo ========================================
echo.
echo Computer : %COMPUTERNAME%
echo Folder   : %CD%
echo Date     : %DATE%
echo Time     : %TIME%
echo.
pause
endlocal
```

```
OUTPUT
========================================
  Welcome, Alex
========================================

Computer : DESKTOP-4F2K9
Folder   : C:\Users\Alex\BatchBook\scripts
Date     : Fri 10/23/2026
Time     :  9:41:07.52
```

```
Press any key to continue . . .
```

`%USERNAME%`, `%COMPUTERNAME%`, `%CD%`, `%DATE%` and `%TIME%` are **predefined variables**. Chapter 5 describes them. Note that the format of `%DATE%` and `%TIME%` depends on your regional settings, and that `%TIME%` has a leading space when the hour is a single digit (a detail that matters in Chapter 15).

Summary

- A batch file is a text file saved with `.bat` or `.cmd` and run by name.
- `@echo off` should open nearly every script; `echo on` aids debugging.
- Use `echo:` or `echo(` for blank lines, `^` to escape special characters and `%%` for a literal percent sign.
- Use `rem` for comments everywhere; `::` only outside blocks.
- `pause` is for interactive use; `exit /b` ends a script cleanly with an exit code, while `exit` closes the session.
- Use a consistent skeleton: header, `setlocal`, configuration, main, subroutines.

Common mistakes

Common Mistake: Naming a script after a command, such as `ping.bat`, `copy.bat` or `test.bat`. Calling `ping` inside such a script can start the script itself, creating an endless loop. Choose distinctive names.

Common Mistake: Forgetting `@echo off`. The output is cluttered by echoed commands.

Common Mistake: Special characters unescaped. `echo Tom & Jerry` runs `echo Tom` and then tries to run a command named `Jerry`.

Common Mistake: Leaving `pause` in unattended scripts.

Exercises

1. **[Easy]** Write `hello.bat` that prints "Hello, world!" and run it from an open prompt.
2. **[Easy]** What is the difference between `echo off` and `@echo off`?
3. **[Easy]** Write a script that prints three lines separated by blank lines, using `echo:`.
4. **[Medium]** Write a script that prints `Profit & Loss: 100% (50 > 25)` exactly, using escapes.
5. **[Medium]** Explain why `rem` is preferred over `::` inside an `if` block.

6. **[Medium]** Write a script that sets the title to "Info", clears the screen, shows the computer name and the current folder, and waits for a key without printing the "Press any key" message.
7. **[Hard]** A script contains `exit` on its last line. A colleague runs it by typing its name at an open prompt. What happens, and what should it be?
8. **[Hard]** Create `ping.bat` containing `@echo off`, `ping 127.0.0.1`. Predict what happens when you run it from its own folder and explain why. Then delete it.

Part 1 Quiz

1. **Q1.** Which program is the shell: `conhost.exe`, `cmd.exe` or `ping.exe`?
2. **Q2.** True or false: a failing command inside a batch file stops the script automatically.
3. **Q3.** Which switch of `cd` changes both drive and folder?
4. **Q4.** What does `robocopy` return when it copied files and found no failures: 0, 1, or 8?
5. **Q5.** Why is `del /s /q *.*` dangerous?
6. **Q6.** Which line ending should batch files use: LF or CRLF?
7. **Q7.** Which is preferred inside a parenthesized block: `rem` or `::`?
8. **Q8.** What does `exit /b 2` do?
9. **Q9.** Name the three layers: console host, shell, _____.
10. **Q10.** What does `where /q tool` return if `tool` is not found?

2

Core Language

Chapters 5 to 12

Variables, parameters, strings, arithmetic, conditions, loops, subroutines and redirection

Part 2 teaches the language itself. Every construct that a batch file can use to hold data, make decisions, repeat work, and move information in and out of commands appears here, in an order where no concept is used before it is introduced.

Chapters in this Part

- Chapter 5: Variables
- Chapter 6: Arguments and Parameters
- Chapter 7: String Handling and Delayed Expansion
- Chapter 8: Arithmetic and Logic
- Chapter 9: Conditionals
- Chapter 10: Loops
- Chapter 11: Labels, goto, call, and Subroutines
- Chapter 12: Input, Output, Redirection and Escaping
- Part 2 Quiz

Chapter 5: Variables

LEARNING OBJECTIVES

- Create, read, change and delete variables with `set`
- Read input with `set /p` and perform simple arithmetic with `set /a`
- Use environment variables and the most important predefined variables
- Control scope with `setlocal` and `endlocal`
- Avoid the classic spacing and quoting errors of `set`

5.1 Creating and reading variables

A **variable** is a named piece of text. `cmd.exe` stores every value as a string, including numbers. Create one with `set`, read it by surrounding its name with percent signs:

Command Prompt

```
C:\> set name=Alex
C:\> echo Hello, %name%
```

OUTPUT

```
Hello, Alex
```

Variable names are **not case-sensitive**: `%NAME%`, `%Name%` and `%name%` are the same variable. The value, however, keeps the case you gave it.

Always use the quoted form

The recommended way to assign is to quote the whole assignment:

set_quoted.bat

```
@echo off
set "city=New York"
set "note=Tom & Jerry"
echo City: %city%
echo Note: %note%
```

OUTPUT

```
City: New York
Note: Tom & Jerry
```

The quotes are **not** part of the value. They protect against two problems: special characters such as `&`, `|`, `<`, `>` and `^` in the value, and invisible trailing spaces. Without quotes, `set note=Tom & Jerry` would assign `Tom` and then try to run `Jerry` as a command.

> **Best Practice:** Write `set "name=value"` for every assignment in every script.

Spaces around the equals sign

set_spaces.bat

```
@echo off
set x = 5
echo [%x%]
echo [%x %]
set "y=5"
echo [%y%]
```

OUTPUT

```
[]
[ 5]
[5]
```

`set x = 5` created a variable named `x` (with a trailing space) whose value is `5` (with a leading space). `%x%` is undefined, so a batch file prints it as empty text (the interactive prompt would print the literal `%x%`). Never put spaces around `=`.

> **Common Mistake:** **Writing `set x = 5` or `set x=5 `.** A space before `=` becomes part of the name, and a space after the value becomes part of the value. A trailing space can make `if "%x%"=="5"` fail with no visible cause. The quoted form `set "x=5"` avoids both.

Displaying, listing and deleting

Command	Effect
`set`	List all variables
`set na`	List all variables whose names **start with** `na`
`set name`	Show `name=value` if it exists, otherwise an error
`set "name="`	**Delete** the variable `name`

Command Prompt

```
C:\> set user
C:\> set "name="
C:\> set name
```

OUTPUT

```
USERDOMAIN=DESKTOP-4F2K9
USERDOMAIN_ROAMINGPROFILE=DESKTOP-4F2K9
USERNAME=Alex
USERPROFILE=C:\Users\Alex
Environment variable name not defined
```

Undefined variables

When a variable does not exist, in a **batch file** `%name%` expands to nothing. At the **interactive prompt** it is left as the literal text `%name%`. This difference is a frequent cause of "it works in the prompt but not in the script" confusion. To test whether a variable exists use `if defined name` (Chapter 9).

5.2 Reading input: `set /p`

`set /p` prompts the user and stores what they type:

ask_name.bat

```
@echo off
set /p "name=Enter your name: "
echo Hello, %name%!
```

OUTPUT

```
Enter your name: Alex
Hello, Alex!
```

(The text `Alex` is typed by the user.) Important behaviors:

- If the user presses **Enter** without typing, the variable is **not changed**. If it did not exist, it remains undefined. Clear it first (`set "name="`) if you need to detect an empty answer.
- The prompt text is whatever follows the `=`. Leading spaces in the prompt are preserved only when you quote the whole assignment.
- `set /p` reads **one line**. It cannot read a password without echoing it.

ask_default.bat

```
@echo off
set "folder=C:\Temp"
set /p "folder=Folder [%folder%]: "
echo Using folder: %folder%
```

OUTPUT

```
Folder [C:\Temp]:
Using folder: C:\Temp
```

This idiom provides a **default**: pressing Enter keeps the existing value.

Reading a line from a file

Redirecting a file into `set /p` stores its **first line**:

Command Prompt

```
C:\Users\Alex\BatchBook\work> echo version 1.4 > ver.txt
C:\Users\Alex\BatchBook\work> set /p "v=" < ver.txt
C:\Users\Alex\BatchBook\work> echo [%v%]
```

OUTPUT

```
[version 1.4 ]
```

The trailing space comes from the `echo ... > file` form (the space before `>` is written to the file). This is the first of many reasons to write `echo text>file` without a space before the symbol when the exact content matters.

5.3 Arithmetic: `set /a` (preview)

`set /a` evaluates an arithmetic expression. Variable names inside the expression need **no percent signs**. Full details are in Chapter 8.

arithmetic_preview.bat

```
@echo off
set /a "a=7"
set /a "b=3"
set /a "sum=a+b"
set /a "product=a*b"
echo Sum: %sum%  Product: %product%
set /a "count=0"
set /a "count+=1"
set /a "count+=1"
echo Count: %count%
```

OUTPUT

```
Sum: 10  Product: 21
Count: 2
```

Quoting the expression (`set /a "x=a+b"`) is again recommended, because operators like `&` , `|` , `<<` , `^` and `%` are special to the parser.

5.4 Environment variables

Every process has a block of **environment variables**. When `cmd.exe` starts a program, the program receives a **copy**. Changes made by the program do not flow back; changes you make with `set` apply only to the current `cmd.exe` and the programs it starts afterward.

Making changes permanent: `setx`

`setx` writes a variable into the user or machine environment in the registry. It affects **future** processes, not the window in which you run it:

Command Prompt

```
C:\> setx PROJECT_HOME "C:\Work"
C:\> echo %PROJECT_HOME%
```

OUTPUT

```
SUCCESS: Specified value was saved.
%PROJECT_HOME%
```

The value appears only in newly opened windows. Use `setx /m` (administrator rights required) for machine-wide values.

> **Warning:** Do not use `setx` to modify `PATH` by writing `setx PATH "%PATH%;C:\Tool"`. It copies the **combined** user and machine path into the user path, duplicates entries, and the tool has historically truncated long values. Edit `PATH` through the System Properties dialog or with PowerShell, and keep a copy of the old value first.

5.5 Predefined variables

Windows defines many variables for you. Some are fixed when the session starts; some are **dynamic** and recomputed at every use (they do not appear in the plain `set` list).

Variable	Contents	Notes
`%USERNAME%`	Current user name	
`%USERPROFILE%`	Profile folder, e.g. `C:\Users\Alex`	
`%COMPUTERNAME%`	Computer name	
`%USERDOMAIN%`	Domain or computer name	
`%HOMEDRIVE%` and `%HOMEPATH%`	Drive and path of the home folder	
`%OS%`	`Windows_NT`	Always the same on current systems
`%PROCESSOR_ARCHITECTURE%`	`AMD64`, `ARM64`, `x86`	May show `x86` for a 32-bit process on a 64-bit system
`%NUMBER_OF_PROCESSORS%`	Logical processor count	
`%SystemRoot%` and `%windir%`	`C:\Windows`	
`%SystemDrive%`	`C:`	
`%ProgramFiles%`	`C:\Program Files`	
`%ProgramFiles(x86)%`	32-bit programs folder	Contains parentheses; see below

Variable	Contents	Notes
`%ProgramData%`	Shared application data	
`%APPDATA%`	Roaming application data of the user	
`%LOCALAPPDATA%`	Local application data of the user	
`%TEMP%` and `%TMP%`	Temporary folder of the user	
`%PATH%`	Folders searched for programs	
`%PATHEXT%`	Executable extensions	See Chapter 2
`%COMSPEC%`	Full path of `cmd.exe`	
`%CD%`	Current directory	Dynamic
`%DATE%` and `%TIME%`	Current date and time	Dynamic; format depends on locale
`%RANDOM%`	Random integer from 0 to 32767	Dynamic
`%ERRORLEVEL%`	Exit code of the last command	Dynamic; Chapter 9
`%CMDCMDLINE%`	Command line that started this `cmd.exe`	Dynamic
`%CMDEXTVERSION%`	Command extensions version	Dynamic

predefined.bat

```
@echo off
echo User     : %USERNAME%
echo Computer : %COMPUTERNAME%
echo Windows  : %SystemRoot%
echo Temp     : %TEMP%
echo CPUs     : %NUMBER_OF_PROCESSORS%
echo Random   : %RANDOM%
echo Random   : %RANDOM%
```

OUTPUT

```
User     : Alex
Computer : DESKTOP-4F2K9
Windows  : C:\Windows
Temp     : C:\Users\Alex\AppData\Local\Temp
CPUs     : 8
Random   : 21408
Random   : 7339
```

Common Mistake: Using `%ProgramFiles(x86)%` inside a parenthesized block. The `)` in the variable name ends the block early and causes "was unexpected at this time". Copy it to a simpler name outside any block: `set "PF86=%ProgramFiles(x86)%"` and use `%PF86%` inside.

Common Mistake: **Defining your own variable named `ERRORLEVEL`, `DATE`, `TIME`, `CD` or `RANDOM`.** A user-defined variable with the same name **overrides** the dynamic one for that session and breaks code that depends on it.

5.6 Scope: `setlocal` and `endlocal`

By default, variables a script creates live on in the session that ran it, which can corrupt later work. `setlocal` starts a **local environment**: all changes after it are discarded by `endlocal`, or automatically when the script ends.

scope_demo.bat

```
@echo off
set "color=blue"
echo Before : %color%
setlocal
set "color=red"
set "temp_value=123"
echo Inside : %color%
endlocal
echo After  : %color%
if defined temp_value (echo temp_value survived) else (echo temp_value is gone)
```

OUTPUT

```
Before : blue
Inside : red
After  : blue
temp_value is gone
```

`setlocal` also saves the current directory and the state of two settings, `EnableExtensions` and `EnableDelayedExpansion`, and `endlocal` restores them. `setlocal` calls nest; each `endlocal` closes the most recent one.

Best Practice: Put `setlocal` near the top of every script, so that nothing it sets leaks into the caller's session. An implicit `endlocal` happens when the script ends. Chapter 7 adds `EnableDelayedExpansion` and Chapter 11 shows how to **return** a value out of a local environment.

Interactive prompt versus script: `setlocal` is meaningful only in a batch file. Typed at the prompt it has no effect. Environment changes made at the prompt persist until the window closes.

5.7 Naming and organization

- Use **UPPER_CASE** for constants and configuration (`BACKUP_DIR`), **lower_case** for working variables (`count`).
- Use only letters, digits and underscores in names. Special characters make variables impossible or hard to reference.

- Avoid names that clash with predefined variables, with commands, or with `for` variable letters.
- Put all configuration in one block at the top of the script.

config_block.bat

```
@echo off
setlocal
rem ---- Configuration ----
set "SOURCE_DIR=%USERPROFILE%\Documents"
set "BACKUP_DIR=%USERPROFILE%\BatchBook\work\backup"
set "KEEP_DAYS=7"

echo Source : %SOURCE_DIR%
echo Target : %BACKUP_DIR%
echo Keep   : %KEEP_DAYS% days
endlocal
```

OUTPUT

```
Source : C:\Users\Alex\Documents
Target : C:\Users\Alex\BatchBook\work\backup
Keep   : 7 days
```

Summary

- Assign with `set "name=value"`; read with `%name%`; delete with `set "name="`.
- Names are case-insensitive; every value is a string.
- `set /p` reads a line; pressing Enter leaves the variable unchanged, which provides a default.
- `set /a` evaluates arithmetic; names inside need no percent signs.
- Changes to the environment are local to the process; `setx` persists values for future processes only.
- Many predefined variables exist; some are dynamic. Never override them.
- `setlocal` and `endlocal` give you scope; use `setlocal` in every script.

Common mistakes

Common Mistake: Spaces around `=`. Creates a variable with a trailing space in the name.

Common Mistake: Not quoting assignments that contain `&`, `|` or `^`.

Common Mistake: Reusing a stale value after `set /p`. If the user presses Enter, the old value stays; clear the variable first.

Common Mistake: Forgetting that variables leak out of scripts. Use `setlocal`.

Exercises

1. **[Easy]** Create a variable `greeting` with the value `Good morning`, and print it.
2. **[Easy]** Delete the variable you just created and show that it is gone.
3. **[Easy]** List all variables whose names begin with `PROGRAM`.
4. **[Medium]** Write a script that asks for a name and an age and prints `<name> is <age> years old`.
5. **[Medium]** Write a script that asks for a folder with a default of `C:\Temp`.
6. **[Medium]** Explain what is wrong with `set count = 0` followed by `echo %count%`.
7. **[Hard]** A script sets `x=1`, then calls `setlocal`, sets `x=2`, calls `endlocal`. What is `x` afterward? Then repeat with the `setlocal` removed.
8. **[Hard]** Write a script that prints two random numbers and explain why they differ.

Chapter 6: Arguments and Parameters

LEARNING OBJECTIVES

- Receive values from the command line with `%1` to `%9` and `%*`
- Process an arbitrary number of arguments with `shift`
- Remove quotes and extract path components with parameter modifiers
- Locate the script's own folder reliably with `%~dp0`
- Validate that required arguments were supplied

6.1 Positional parameters

When you run `copyit.bat a.txt b.txt`, the words after the script name are **arguments**. Inside the script they are available as `%1`, `%2`, and so on. `%0` is the name used to run the script itself. Arguments are separated by spaces, commas, semicolons or equals signs, unless they are quoted.

args_demo.bat

```
@echo off
echo Script    : %0
echo Argument 1: %1
echo Argument 2: %2
echo Argument 3: %3
echo All       : %*
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> args_demo.bat alpha "two words" gamma
```

OUTPUT

```
Script    : args_demo.bat
Argument 1: alpha
Argument 2: "two words"
Argument 3: gamma
All       : alpha "two words" gamma
```

Observe that `%2` **keeps its quotes**: the quotes are part of the argument as received. `%*` expands to all arguments exactly as typed. Only nine positional parameters, `%1` to `%9`, are addressable directly; `shift` (Section 6.3) reaches the rest.

Common Mistake: Using `%10`. The parser reads it as `%1` followed by the character `0`. There is no tenth parameter without `shift`.

6.2 Parameter modifiers

A modifier inserted after the percent sign transforms an argument that is a file or path. The syntax is `%~modifiers` followed by the parameter number.

Modifier	Meaning	Example for `"C:\Data\My Files\report.txt"`
`%~1`	Argument with surrounding quotes **removed**	`C:\Data\My Files\report.txt`
`%~f1`	Fully qualified path	`C:\Data\My Files\report.txt`
`%~d1`	Drive letter only	`C:`
`%~p1`	Path only (no drive, no name)	`\Data\My Files\`
`%~n1`	File name only (no extension)	`report`
`%~x1`	Extension only (with the dot)	`.txt`
`%~s1`	Short (8.3) names; combine with others	`C:\Data\MYFILE~1\report.txt`
`%~a1`	File attributes	`--a--------`
`%~t1`	Date and time of last modification	`10/23/2026 09:14 AM`
`%~z1`	Size in bytes	`2048`
`%~$PATH:1`	Search the folders in `PATH` for the argument	Full path of the first match, or empty

Modifiers combine: `%~dp1` is drive plus path, `%~nx1` is name plus extension, `%~dpnx1` is the full path.

modifiers.bat

```
@echo off
set "FILE=%~1"
echo Quoted removed : %~1
echo Full path      : %~f1
echo Drive          : %~d1
echo Path           : %~p1
echo Name           : %~n1
echo Extension      : %~x1
echo Size (bytes)   : %~z1
echo Name+extension : %~nx1
echo Drive+path     : %~dp1
```

Command Prompt

```
C:\Users\Alex\BatchBook\work> ..\scripts\modifiers.bat "first.txt"
```

OUTPUT

```
Quoted removed : first.txt
Full path      : C:\Users\Alex\BatchBook\work\first.txt
Drive          : C:
Path           : \Users\Alex\BatchBook\work\
Name           : first
```

```
Extension      : .txt
Size (bytes)   : 8
Name+extension : first.txt
Drive+path     : C:\Users\Alex\BatchBook\work\
```

The modifiers need the file or folder to exist for `%~z`, `%~t` and `%~a` to return a value; for `%~f`, `%~d`, `%~p`, `%~n`, `%~x` the path text is analysed even if the file does not exist.

> **Best Practice:** Strip quotes with `%~1` when you store an argument, then re-add quotes deliberately when you use it: `set "src=%~1"` and later `copy "%src%" "%dest%"`. This prevents the double-quote problems that appear when a quoted argument is placed inside another pair of quotes.

6.3 `shift`: more than nine arguments, and loops over arguments

`shift` discards `%1` and moves every later argument one position down: `%2` becomes `%1`, `%3` becomes `%2`, and so on. `%*` is **not** changed by `shift`.

list_args.bat

```
@echo off
:next
if "%~1"=="" goto :done
echo Argument: %~1
shift
goto :next
:done
echo Finished.
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> list_args.bat one "two three" four
```

OUTPUT

```
Argument: one
Argument: two three
Argument: four
Finished.
```

The loop tests whether the current first argument is empty; if not it processes it and shifts. `shift /2` starts shifting at the second argument and leaves `%0` and `%1` in place.

> **Common Mistake: Testing `if "%1"==""` when the argument may be quoted.** If the argument is `"abc"`, `"%1"` becomes `""abc""` and the comparison misbehaves. Use `"%~1"` for tests.

6.4 `%0` and the script's own location

`%~dp0` is the single most useful modifier in practice. It expands to the **drive and folder of the running script**, with a trailing backslash, regardless of the current directory:

where_am_i.bat

```
@echo off
echo Current folder : %CD%
echo Script folder  : %~dp0
echo Script file    : %~f0
echo Script name    : %~nx0
```

Command Prompt

```
C:\Users\Alex> BatchBook\scripts\where_am_i.bat
```

OUTPUT

```
Current folder : C:\Users\Alex
Script folder  : C:\Users\Alex\BatchBook\scripts\
Script file    : C:\Users\Alex\BatchBook\scripts\where_am_i.bat
Script name    : where_am_i.bat
```

Use `%~dp0` to refer to files that ship next to the script (configuration files, helper scripts):

use_neighbor.bat

```
@echo off
set "HERE=%~dp0"
call "%HERE%helper.bat"
if exist "%HERE%settings.ini" (echo Settings found.) else (echo No settings file.)
```

Remember the trailing backslash already included in `%~dp0`; write `%~dp0helper.bat`, not `%~dp0\helper.bat`.

> **Administrator and shortcut launches:** When a script is run as administrator, from a shortcut, or by the Task Scheduler, the current directory is often `C:\Windows\System32` or another unexpected place. `%CD%` then misleads, while `%~dp0` remains correct. If a script must work with relative paths, start with `pushd "%~dp0"` and end with `popd`.

UNC paths

If the script is launched from a network share `\\server\share\scripts\x.bat`, `cmd.exe` cannot make a UNC path its current directory and prints `CMD.EXE was started with the above path as the current directory. UNC paths are not supported.` Using `pushd "%~dp0"` maps a temporary drive letter and solves this.

6.5 The same modifiers on `for` variables

The identical modifiers apply to the variables of a `for` loop (Chapter 10). In a batch file, a loop variable `%%f` becomes `%%~nxf`:

list_names.bat

```
@echo off
for %%f in (*.txt) do echo %%~nxf is %%~zf bytes
```

OUTPUT

```
first.txt is 8 bytes
notes.txt is 120 bytes
```

6.6 Validating arguments

A well-behaved script checks its arguments, prints usage, and returns a non-zero exit code on misuse:

greet_user.bat

```
@echo off
setlocal

if "%~1"=="" goto :usage
if /i "%~1"=="/?" goto :usage
if /i "%~1"=="-h" goto :usage

set "who=%~1"
set "greeting=Hello"
if not "%~2"=="" set "greeting=%~2"

echo %greeting%, %who%!
exit /b 0

:usage
echo Usage: %~nx0 name [greeting]
echo.
echo   name      The name to greet (use quotes if it has spaces)
echo   greeting  Optional greeting word (default: Hello)
exit /b 1
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> greet_user.bat "Ada Lovelace" Welcome
C:\Users\Alex\BatchBook\scripts> greet_user.bat
```

OUTPUT

```
Welcome, Ada Lovelace!
Usage: greet_user.bat name [greeting]

  name      The name to greet (use quotes if it has spaces)
  greeting  Optional greeting word (default: Hello)
```

The script displays usage when called with no arguments, `/?` or `-h`, and each branch ends with `exit /b` carrying a code. Labels and `goto` are introduced properly in Chapter 11; their use here is simple enough to follow.

Handling switches

A common convention is to accept options such as `/v` or `/verbose` in any order. Process them in a loop with `shift`:

options.bat

```
@echo off
setlocal
set "verbose=0"
set "target="

:parse
if "%~1"=="" goto :run
if /i "%~1"=="/v" (set "verbose=1") else set "target=%~1"
shift
goto :parse

:run
if not defined target (echo Usage: %~nx0 [/v] target & exit /b 1)
if "%verbose%"=="1" echo Verbose mode on.
echo Target: %target%
exit /b 0
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> options.bat /v D:\Data
```

OUTPUT

```
Verbose mode on.
Target: D:\Data
```

Summary

- `%1` to `%9` hold arguments; `%0` is the script; `%*` is the full argument text and is not affected by `shift`.
- Quotes stay on the argument unless you remove them with `%~1`.
- Modifiers `f d p n x s a t z` extract parts of a path or file; they combine (`%~dpnx1`).
- `%~dp0` is the script's own folder with a trailing backslash; prefer it to `%CD%`.
- `shift` walks through arguments one at a time.
- Validate arguments, print usage and return a non-zero code on misuse.

Common mistakes

Common Mistake: **Comparing `"%1"` instead of `"%~1"`.** Quoted arguments produce doubled quotes.

Common Mistake: **Assuming `%CD%` is the script's folder.**

Common Mistake: **Writing `%~dp0\file`.** Produces a double backslash; harmless in most places but wrong.

Common Mistake: **Reading `%~z1` for a file that does not exist**, which returns an empty string.

Exercises

1. **[Easy]** Write a script that echoes its first argument and its second argument.
2. **[Easy]** Using `%~nx1`, print only the file name of a path argument.
3. **[Easy]** Print the full path of the script itself.
4. **[Medium]** Write a script that prints the extension and the size in bytes of the file given as its first argument.
5. **[Medium]** Write a script that prints every argument on its own line using `shift`.
6. **[Medium]** Explain why `if "%1"==""` fails when the argument is `"a b"` and how to fix it.
7. **[Hard]** Write a script `sum.bat` that adds all numeric arguments (any count) and prints the total. (Use `set /a` as previewed in Chapter 5.)
8. **[Hard]** Write a script that works from any current directory and reads `settings.ini` stored next to it, and prints an error if it is missing.

Chapter 7: String Handling and Delayed Expansion

LEARNING OBJECTIVES

- Concatenate, extract, replace and remove parts of strings
- Compute the length of a string and trim spaces
- Convert case with the replacement trick
- Explain why variables inside blocks keep stale values
- Enable and use delayed expansion correctly, including its hazards

7.1 Strings and concatenation

Every variable holds a string. To join strings, place the pieces next to each other:

concat.bat

```
@echo off
set "first=Ada"
set "last=Lovelace"
set "full=%first% %last%"
echo %full%
set "path_part=C:\Data"
set "path_part=%path_part%\Reports"
echo %path_part%
```

OUTPUT

```
Ada Lovelace
C:\Data\Reports
```

The second assignment appends to a variable by including the variable's own current value on the right side.

7.2 Substrings: `%var:~start,length%`

The form `%var:~start,length%` extracts part of a string. Positions start at **0**. A negative `start` counts from the end. A negative `length` means "stop that many characters before the end".

substring.bat

```
@echo off
set "s=Hello, World"
echo [%s:~0,5%]
echo [%s:~7%]
echo [%s:~-5%]
echo [%s:~0,-7%]
```

```
echo [%s:~-5,3%]
echo [%s:~3,4%]
```

OUTPUT

```
[Hello]
[World]
[World]
[Hello]
[Wor]
[lo, ]
```

Expression	Meaning
`%s:~N%`	From position N to the end
`%s:~N,L%`	L characters starting at position N
`%s:~-N%`	The last N characters
`%s:~0,-N%`	Everything except the last N characters
`%s:~-N,L%`	L characters starting N characters from the end

If the start is beyond the end the result is empty. If the length exceeds what remains, you get what remains.

Common Mistake: Using a variable as an offset inside percent syntax. `%s:~%n%,3%` does not work, because the parser reads `%s:~%` as one variable reference. Section 7.5 shows two correct ways.

7.3 Replacement: `%var:old=new%`

replace.bat

```
@echo off
set "s=Hello, World"
echo %s:World=Batch%
echo %s:o=0%
echo [%s:,=%]
echo [%s: =%]
echo %s:*,=%
```

OUTPUT

```
Hello, Batch
Hell0, W0rld
[Hello World]
[Hello,World]
 World
```

- `%s:old=new%` replaces **every** occurrence of `old` with `new`. The match is **not case-sensitive**.
- An empty replacement (`%s:,=%`) **deletes** the matches.

- `%s:*,=%` uses a leading wildcard: it removes everything **up to and including** the first match of `,`. This is the only wildcard supported, and only at the start of the search text.
- The search text cannot start with `=` and cannot contain `=`; the characters `%` and `!` need special care.

Realistic example: a safe file name

safe_name.bat

```
@echo off
setlocal
set "title=Report: Q1/Q2 draft?"
set "safe=%title::=-%"
set "safe=%safe:/=-%"
set "safe=%safe:?=%"
set "safe=%safe: =_%"
echo %safe%.txt
```

OUTPUT

```
Report-_Q1-Q2_draft.txt
```

Each line removes or substitutes one character that is not allowed in a Windows file name (or that is awkward in one). The first replacement, written `%title::=-%`, searches for `:` and replaces it with `-`. Chaining replacements, each reading the result of the previous one, is the standard way to sanitize a string.

> **Warning:** A search text that **begins with an asterisk** is the wildcard-prefix form. `%t:*=%` does not remove asterisks; it removes everything up to and including the first one. Exercise 7 explores this.

7.4 Why delayed expansion exists

The problem

Recall from Chapter 1 that `%var%` is replaced **when the line is parsed**, and that a parenthesized block counts as **one** line. Therefore a variable set inside a block cannot be read with `%` inside the same block:

loop_problem.bat

```
@echo off
set "total=0"
for %%n in (1 2 3) do (
    set /a "total+=%%n"
    echo Running total: %total%
)
echo Final total: %total%
```

OUTPUT

```
Running total: 0
Running total: 0
Running total: 0
Final total: 6
```

The `for` body is one parsed unit. `%total%` was replaced by `0` once, before the loop started. The additions did happen (the final total proves it), but the `echo` inside could not see them.

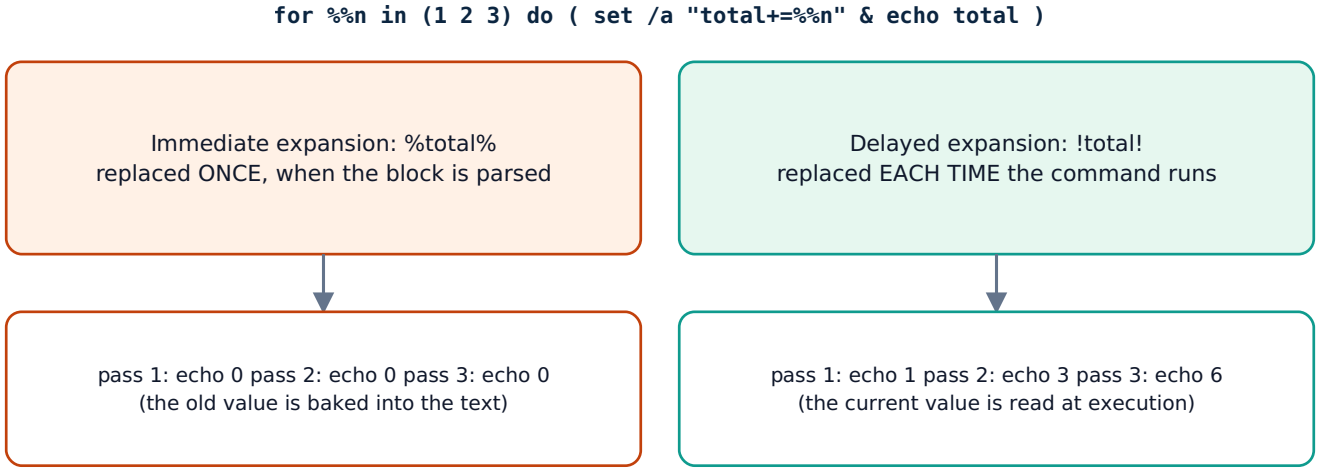


Figure 7.1. Percent expansion happens once when the block is parsed. Delayed expansion happens each time the command is executed.

The cure

Delayed expansion replaces `!var!` at **execution time**, immediately before each command runs. Enable it with `setlocal EnableDelayedExpansion` and use exclamation marks instead of percent signs where you need the current value:

loop_fixed.bat

```
@echo off
setlocal EnableDelayedExpansion
set "total=0"
for %%n in (1 2 3) do (
    set /a "total+=%%n"
    echo Running total: !total!
)
echo Final total: %total%
```

OUTPUT

```
Running total: 1
Running total: 3
Running total: 6
Final total: 6
```

Inside the block `!total!` is read freshly for each iteration. After the block both syntaxes give the same result.

> **Best Practice:** Use `%var%` where the value cannot change within the line, and `!var!` inside blocks and loops where it can. Turn delayed expansion on with `setlocal EnableDelayedExpansion` at the top of any script that uses loops with variables.

At the prompt

At the interactive prompt delayed expansion is off. Start `cmd /v:on` to enable it for a session, or use `cmd /v:on /c "..."` for one command.

Where else you need it

The same stale-value problem affects any parenthesized block: `if (...)` bodies, `for` bodies, and any group of commands joined with `&` on one line:

same_line.bat

```
@echo off
setlocal EnableDelayedExpansion
set "x=before"
set "x=after" & echo percent: %x% delayed: !x!
```

OUTPUT

```
percent: before delayed: after
```

7.5 Variables in substring and replacement syntax

To use a variable as the start or length, combine delayed expansion with percent expansion. The percent variables are expanded first (in the parse phase); the delayed substring syntax then sees plain numbers:

dynamic_substring.bat

```
@echo off
setlocal EnableDelayedExpansion
set "s=Hello, World"
set "start=7"
set "len=3"
echo [!s:~%start%,%len%!]
echo --- one character at a time ---
for /l %%i in (0,1,4) do echo !s:~%%i,1!
```

OUTPUT

```
[Wor]
--- one character at a time ---
H
e
l
l
o
```

In the loop `%%i` is replaced by the `for` machinery before delayed expansion runs, so `!s:~%%i,1!` becomes `!s:~0,1!`, `!s:~1,1!` and so on.

An alternative without delayed expansion uses `call`, which forces a second round of percent expansion:

call_expand.bat

```
@echo off
set "s=Hello, World"
set "start=7"
call echo [%%s:~%start%,3%%]
```

OUTPUT

```
[Wor]
```

This works but is **slow**, because `call` starts a new parsing pass each time. Prefer delayed expansion.

7.6 Length of a string

There is no built-in length operator. Two techniques follow.

A simple loop

strlen_simple.bat

```
@echo off
setlocal
set "text=Hello, World"
set "work=%text%"
set "len=0"
:count
if defined work (
    set "work=%work:~1%"
    set /a "len+=1"
    goto :count
)
echo Length of "%text%" is %len%.
```

OUTPUT

```
Length of "Hello, World" is 12.
```

Each pass removes the first character and counts it. The loop does one pass per character, which is acceptable for short strings.

The bisection method (fast)

For long strings, test lengths in decreasing powers of two. The method does about twelve steps regardless of length:

strlen_fast.bat

```
@echo off
setlocal EnableDelayedExpansion
call :strlen length "Hello, World"
echo Length is %length%
call :strlen length ""
echo Length is %length%
exit /b 0

:strlen  result  string
rem Returns the length of a string in the variable named by the first argument.
setlocal EnableDelayedExpansion
set "s=#%~2"
set "len=0"
for %%P in (4096 2048 1024 512 256 128 64 32 16 8 4 2 1) do (
    if not "!s:~%%P,1!"=="" (
        set /a "len+=%%P"
        set "s=!s:~%%P!"
    )
)
endlocal & set "%~1=%len%"
exit /b 0
```

OUTPUT

```
Length is 12
Length is 0
```

The routine prepends `#` so that the string is never empty (an empty variable cannot be tested with substring syntax). It then removes chunks of 4096, 2048, ... 1 characters while any remain, summing the chunk sizes. The last line uses the **return trick** explained in Chapter 11: `endlocal & set ...` ends the local scope and assigns the value to the caller's variable in a single parsed line.

> **Warning:** Strings containing `!` or `^` are damaged by delayed expansion. The routine above is suitable for ordinary text, not for arbitrary user input. Section 7.9 explains the hazard.

7.7 Trimming spaces

Leading spaces are removed by a `for /f` loop, which skips leading delimiters. Trailing spaces need a loop:

trim.bat

```
@echo off
setlocal EnableDelayedExpansion
set "text=    padded text    "
echo [%text%]
call :trim text
echo [%text%]
exit /b 0

:trim  varname
setlocal EnableDelayedExpansion
set "s=!%~1!"
rem Remove leading spaces: for /f skips leading delimiters.
for /f "tokens=* delims= " %%a in ("!s!") do set "s=%%a"
rem Remove trailing spaces one at a time.
```

```
:trim_loop
if not defined s goto :trim_done
if not "!s:~-1!"==" " goto :trim_done
set "s=!s:~0,-1!"
goto :trim_loop
:trim_done
endlocal & set "%~1=%s%"
exit /b 0
```

OUTPUT

```
[    padded text    ]
[padded text]
```

`set "s=!%~1!"` reads a variable **whose name was passed as an argument**: `%~1` expands first to `text`, leaving `!text!`, which delayed expansion then resolves.

7.8 Changing case

Replacement is case-insensitive, and the replacement text keeps the case you give. Replacing each letter with its upper-case or lower-case form therefore converts the whole string:

case_convert.bat

```
@echo off
setlocal EnableDelayedExpansion
set "s=Mixed Case Text 123"

set "upper=!s!"
for %%L in (A B C D E F G H I J K L M N O P Q R S T U V W X Y Z) do (
    set "upper=!upper:%%L=%%L!"
)

set "lower=!s!"
for %%L in (a b c d e f g h i j k l m n o p q r s t u v w x y z) do (
    set "lower=!lower:%%L=%%L!"
)

echo Upper: !upper!
echo Lower: !lower!
```

OUTPUT

```
Upper: MIXED CASE TEXT 123
Lower: mixed case text 123
```

For simply **comparing** text regardless of case, use `if /i` (Chapter 9) instead of converting.

7.9 Hazards of delayed expansion

Once delayed expansion is enabled, `!` becomes a special character everywhere in the script. Strings that contain `!` are affected both when set and when read.

bang_problem.bat

```
@echo off
setlocal EnableDelayedExpansion
set "msg=Hello, World!"
echo Message: %msg%
echo Message: !msg!
echo Literal: Hello^^!
```

OUTPUT

```
Message: Hello, World
Message: Hello, World
Literal: Hello!
```

The `!` in the **assignment** line was consumed because the line `set "msg=Hello, World!"` was itself scanned for `!var!` patterns. To handle text with `!`:

1. Disable delayed expansion around the code that handles the text (`setlocal DisableDelayedExpansion`), or
2. Escape each `!` as `^^!` when delayed expansion is on (the caret must be doubled because it is processed twice).

Best Practice: Enable delayed expansion only in the part of a script that needs it, and keep user-supplied text (file names, passwords, arguments) away from delayed expansion. File names can legally contain `!` . See Chapter 23.

Summary

- Concatenate by placing values side by side; append by including the variable itself.
- Extract with `%v:~start,length%` ; replace and delete with `%v:old=new%` ; the match ignores case; `*old` removes a prefix.
- Variables set inside a block are not visible to `%var%` in the same block; use `!var!` with `setlocal EnableDelayedExpansion` .
- To use variables as offsets, combine `!v:~%a%,%b%!` with percent expansion.
- Length requires a loop or the bisection routine; trimming needs `for /f` plus a loop; case conversion is done with case-insensitive replacement.
- Delayed expansion consumes `!` ; scope it carefully.

Common mistakes

Common Mistake: Reading a changing variable with `%` inside a loop. It shows the value from before the loop.

Common Mistake: Enabling delayed expansion globally and then processing file names. A file named `Hello!.txt` loses its `!`.

Common Mistake: Expecting replacement to be case-sensitive. `%s:a=X%` also replaces `A`.

Common Mistake: Putting spaces around `=` in a replacement. `%s:a = X%` searches for `a` (with a space).

Exercises

1. **[Easy]** Set `s=Batch Scripting` and print the first five characters.
2. **[Easy]** Print the last four characters of the same string.
3. **[Easy]** Replace `Batch` with `Windows` and print the result.
4. **[Medium]** Remove all spaces from `a b c d` using replacement.
5. **[Medium]** Write a loop that adds the numbers 1 to 5 and prints the running total each time, correctly.
6. **[Medium]** Given `set "start=2"` and `set "s=ABCDEFG"`, print three characters starting at `start` using delayed expansion.
7. **[Hard]** Explain why `set "t=a*b*c"` followed by `echo %t:*=%` prints `b*c` and not `abc`. Then write code that removes every `*` character (hint: loop with `for /l` and rebuild).
8. **[Hard]** Write a routine `:toupper` that takes a variable name and converts the variable's value to upper case, returning it to the caller.

Chapter 8: Arithmetic and Logic

LEARNING OBJECTIVES

- Evaluate integer expressions with `set /a`, using all operators and correct precedence
- Perform bitwise operations and use bit flags
- Compare values with `if` using `EQU`, `NEQ`, `LSS`, `LEQ`, `GTR` and `GEQ`
- Predict when a comparison is numeric and when it is textual
- Validate that input is numeric before computing with it

8.1 `set /a`: integer arithmetic

`set /a` evaluates an arithmetic expression and stores the result. It supports **32-bit signed integers only**: whole numbers from -2,147,483,648 to 2,147,483,647. There are no decimals.

Command Prompt

```
C:\> set /a 5+3
C:\> set /a "x=10*4"
C:\> set /a "x=x/3"
C:\> echo %x%
```

OUTPUT

```
8
40
13
13
```

At the **interactive prompt**, `set /a` prints the result. Inside a **batch file** it does not print assignments, so you display the value yourself with `echo`.

Variable names in the expression do **not** need percent signs: `set /a "total=price*qty"` reads the variables `price` and `qty` directly. A variable that is not defined counts as 0.

Operators

From highest to lowest precedence:

Group	Operators	Notes
Grouping	`( )`	Use freely to make precedence explicit
Unary	`!` `~` `-`	Logical NOT, bitwise NOT, negation
Multiplicative	`*` `/` `%`	`/` is integer division; `%` is modulo (`%%` in a batch file)
Additive	`+` `-`	

Group	Operators	Notes
Shift	`<<` `>>`	Arithmetic shift
Bitwise AND	`&`	
Bitwise XOR	`^`	
Bitwise OR	`|`	
Assignment	`=` `+=` `-=` `*=` `/=` `%=` `&=` `^=` `|=` `<<=` `>>=`	Compound assignment changes the variable
Comma	`,`	Separates several expressions

There is **no** `++` or `--` operator. Write `set /a "i+=1"`.

arithmetic.bat

```
@echo off
setlocal
set /a "a=17"
set /a "b=5"
set /a "sum=a+b"
set /a "diff=a-b"
set /a "prod=a*b"
set /a "quot=a/b"
set /a "rem=a %% b"
set /a "mix=2+3*4"
set /a "grp=(2+3)*4"
set /a "neg=-a/b"
echo Sum        : %sum%
echo Difference : %diff%
echo Product    : %prod%
echo Quotient   : %quot%
echo Remainder  : %rem%
echo 2+3*4      : %mix%
echo (2+3)*4    : %grp%
echo -17/5      : %neg%
```

OUTPUT

```
Sum        : 22
Difference : 12
Product    : 85
Quotient   : 3
Remainder  : 2
2+3*4      : 14
(2+3)*4    : 20
-17/5      : -3
```

Division **truncates toward zero**: `17/5` is 3, and `-17/5` is -3. The remainder keeps the sign of the dividend.

Common Mistake: Writing a single `%` for modulo in a batch file. `set /a "r=a % b"` is parsed as a variable reference. Always write `%%` inside batch files and a single `%` only at the prompt.

> **Common Mistake: Leaving an operator unquoted.** `set /a x=5&3` runs `set /a x=5` and then tries to run a command called `3`. Quote the expression: `set /a "x=5&3"`.

Multiple expressions

multi_expr.bat

```
@echo off
set /a "a=1, b=a+1, c=b*10"
echo a=%a% b=%b% c=%c%
set /a "count=0"
set /a "count+=5, count*=2"
echo count=%count%
```

OUTPUT

```
a=1 b=2 c=20
count=10
```

A comma-separated list is evaluated from left to right, and each part can use the results of earlier parts. (Here percent expansion shows the final values because the `echo` is a separate line.)

Number bases and the leading-zero trap

Written as	Interpreted as
`255`	Decimal 255
`0xFF`	Hexadecimal 255
`0377`	**Octal** 255

A number with a leading zero is **octal**. That makes `08` and `09` invalid:

Command Prompt

```
C:\> set /a "x=010"
C:\> set /a "x=08"
```

OUTPUT

```
8
Invalid number.  Numeric constants are either decimal (17), hexadecimal (0x11), or octal (021).
```

This matters when you take parts of the time or date. `%TIME:~3,2%` can produce `08` or `09`, which `set /a` rejects. Chapter 15 gives the standard fix of prefixing `1` and subtracting `100`.

Overflow

Results outside the 32-bit range wrap around silently:

Command Prompt

```
C:\> set /a "x=2147483647+1"
```

OUTPUT

```
-2147483648
```

For values that might exceed the range (file sizes in bytes of large disks) compute in smaller units or call PowerShell (Chapter 19).

8.2 Bitwise operations

Bitwise operators work on the binary form of numbers.

bitwise.bat

```
@echo off
set /a "andv=5 & 3"
set /a "orv=5 | 3"
set /a "xorv=5 ^ 3"
set /a "notv=~5"
set /a "shl=1 << 4"
set /a "shr=-16 >> 2"
echo 5 AND 3  : %andv%
echo 5 OR 3   : %orv%
echo 5 XOR 3  : %xorv%
echo NOT 5    : %notv%
echo 1 shl 4  : %shl%
echo -16 shr 2: %shr%
```

OUTPUT

```
5 AND 3  : 1
5 OR 3   : 7
5 XOR 3  : 6
NOT 5    : -6
1 shl 4  : 16
-16 shr 2: -4
```

For 5 (binary 101) and 3 (binary 011): AND keeps bits set in both (001, which is 1), OR keeps bits set in either (111, which is 7), XOR keeps bits that differ (110, which is 6). The shift `>>` is arithmetic: the sign bit is preserved.

Realistic example: bit flags

Flags pack several yes/no options into one number:

flags.bat

```
@echo off
setlocal
set /a "FLAG_READ=1, FLAG_WRITE=2, FLAG_EXEC=4"
set /a "perm=FLAG_READ | FLAG_WRITE"
echo Permission value: %perm%
```

```
set /a "can_write=perm & FLAG_WRITE"
if %can_write% neq 0 (echo Write is allowed) else (echo Write is denied)

set /a "can_exec=perm & FLAG_EXEC"
if %can_exec% neq 0 (echo Execute is allowed) else (echo Execute is denied)

set /a "perm|=FLAG_EXEC"
echo After adding execute: %perm%
set /a "perm&=~FLAG_WRITE"
echo After removing write: %perm%
```

OUTPUT

```
Permission value: 3
Write is allowed
Execute is denied
After adding execute: 7
After removing write: 5
```

Realistic example: decimal to binary

to_binary.bat

```
@echo off
setlocal EnableDelayedExpansion
set "n=%~1"
if "%n%"=="" set "n=42"
set "bin="
set /a "work=n"
if %work% equ 0 set "bin=0"
:loop
if %work% gtr 0 (
    set /a "bit=work & 1"
    set "bin=!bit!!bin!"
    set /a "work>>=1"
    goto :loop
)
echo %n% in binary is %bin%
```

OUTPUT

```
42 in binary is 101010
```

Each pass takes the lowest bit (`work & 1`), puts it in front of the result, and shifts the number right by one.

8.3 Practical arithmetic recipes

Percentages and fixed-point decimals

Because there are no decimals, scale first and divide last:

percent.bat

```
@echo off
setlocal
set /a "used=37, total=120"
set /a "pct=used*100/total"
```

```
echo Used: %pct%%

rem Two decimal places by working in hundredths.
set /a "hund=used*10000/total"
set /a "whole=hund/100, frac=hund%%100"
if %frac% lss 10 set "frac=0%frac%"
echo Used: %whole%.%frac% percent
```

```
OUTPUT
Used: 30%
Used: 30.83 percent
```

Multiply **before** dividing to keep precision (`used*100/total` , not `used/total*100`). Take care that the intermediate value stays below 2,147,483,647.

Random numbers

`%RANDOM%` gives 0 to 32767. To get a number in a range, take the modulo and add the offset:

dice.bat

```
@echo off
setlocal
set /a "die=%RANDOM% %% 6 + 1"
echo You rolled a %die%.
set /a "pick=%RANDOM% %% 100"
echo Random 0 to 99: %pick%
```

```
OUTPUT
You rolled a 4.
Random 0 to 99: 57
```

Modulo of a range that does not divide 32768 evenly slightly favors low values. For games this is irrelevant; for anything security-related, do not use `%RANDOM%` .

8.4 Comparing with `if`

The `if` command compares two values with these operators:

Operator	Meaning
`EQU`	Equal
`NEQ`	Not equal
`LSS`	Less than
`LEQ`	Less than or equal
`GTR`	Greater than
`GEQ`	Greater than or equal
`==`	Equal, **always as text**

(`<` and `>` cannot be used because they mean redirection.) Add `/i` to ignore case in text comparisons, and `not` to invert the result.

compare.bat

```
@echo off
set "age=25"
if %age% geq 18 (echo Adult) else (echo Minor)
if %age% equ 25 echo Age is exactly 25
if not %age% lss 10 echo Age is not below 10
```

OUTPUT

```
Adult
Age is exactly 25
Age is not below 10
```

Numeric versus text comparison

`if` compares **numerically** when **both** sides are numbers written as plain decimal, hexadecimal or octal literals. Otherwise it compares **as text**, character by character. A quoted value is always text.

numeric_vs_text.bat

```
@echo off
if 10 gtr 9 (echo 10 gtr 9 : true) else (echo 10 gtr 9 : false)
if "10" gtr "9" (echo "10" gtr "9" : true) else (echo "10" gtr "9" : false)
if 10 equ 010 (echo 10 equ 010 : true) else (echo 10 equ 010 : false)
if "10"=="010" (echo "10"=="010" : true) else (echo "10"=="010" : false)
if "abc" lss "abd" (echo abc lss abd : true) else (echo abc lss abd : false)
```

OUTPUT

```
10 gtr 9 : true
"10" gtr "9" : false
10 equ 010 : false
"10"=="010" : false
abc lss abd : true
```

- Unquoted `10 gtr 9` is a numeric comparison: true.
- `"10" gtr "9"` is text: the character `1` sorts before `9`, so `"10"` is **less** than `"9"`.
- `010` is **octal 8**, so `10 equ 010` is false as numbers.
- The `==` operator is always textual.

Common Mistake: Quoting numbers in `if` when you want numeric comparison. `if "%a%" gtr "%b%"` compares text, giving wrong answers for 10 versus 9. For numbers, leave the values unquoted, but only after confirming that both are defined and numeric.

Empty variables break unquoted comparisons

If `%a%` is empty, `if %a% gtr 5` becomes `if gtr 5` and fails with a syntax error such as "5 was unexpected at this time." Two defences:

1. Validate and default first (`if not defined a set "a=0"`).
2. Use quoted text comparison `==` when numbers are not needed.

Combining conditions

Batch has no `and` or `or`. Use these patterns:

logic.bat

```
@echo off
setlocal
set "age=30"
set "member=yes"

rem AND: nest the tests.
if %age% geq 18 if /i "%member%"=="yes" echo Adult member: access granted.

rem OR: set a flag.
set "ok=0"
if %age% lss 12 set "ok=1"
if %age% gtr 65 set "ok=1"
if %ok%==1 (echo Discount applies) else (echo No discount)

rem XOR / NOT: use "if not".
if not "%member%"=="yes" echo Not a member.
```

OUTPUT

```
Adult member: access granted.
No discount
```

8.5 Validating numeric input

Before using a user-supplied value in arithmetic or numeric `if`, confirm it contains only digits. `findstr` with a regular expression does this:

isnumber.bat

```
@echo off
setlocal EnableDelayedExpansion
set "value=%~1"
if not defined value (echo Usage: %~nx0 number & exit /b 2)

call :isnumber value
if errorlevel 1 (
    echo The value is not a whole number.
    exit /b 1
)
set /a "double=value*2"
echo Double of %value% is %double%.
exit /b 0

:isnumber  varname
rem Sets errorlevel 0 if the variable holds one or more digits, otherwise 1.
rem The text is handed over by NAME and read with delayed expansion, so special
```

```
rem characters in it are never pasted into the command line (see Chapter 23).
echo(!%~1!| findstr /r /x "[0-9][0-9]*" >nul
exit /b %errorlevel%
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> isnumber.bat 21
C:\Users\Alex\BatchBook\scripts> isnumber.bat 2x1
```

OUTPUT

```
Double of 21 is 42.
The value is not a whole number.
```

`findstr /r /x "[0-9][0-9]*"` requires the **whole** line to be digits (`/x`). The routine's exit code (`exit /b %errorlevel%`) carries the result of `findstr`. The value is passed **by name** (`value`) and read as `!%~1!` ; this keeps characters such as `&` in the text from being interpreted as commands. Chapter 23 explains why this matters. Chapter 9 explains `errorlevel` tests, and Chapter 13 explains `findstr` fully.

Warning: Validate numbers before using them in `set /a` . A user who supplies text such as `1&calc` could otherwise cause the parser to run unintended commands. This is a form of **command injection**; see Chapter 23.

Summary

- `set /a` does 32-bit integer math; division truncates; there are no decimals, and results wrap silently on overflow.
- Quote expressions; use `%%` for modulo in batch files; leading zeros mean octal.
- Bitwise operators and shifts support flags and conversions.
- `if` offers `EQU NEQ LSS LEQ GTR GEQ` ; comparisons are numeric only if both sides are unquoted numbers, else textual.
- Combine conditions by nesting (AND) or flags (OR). Validate numeric input first.

Common mistakes

Common Mistake: Octal surprises: `08` and `09` fail in `set /a` .

Common Mistake: Dividing before multiplying: `used/total*100` yields 0 for used less than total.

Common Mistake: Comparing quoted numbers. `"10"` is less than `"9"` .

> **Common Mistake:** **Ignoring overflow** when handling sizes in bytes.

Exercises

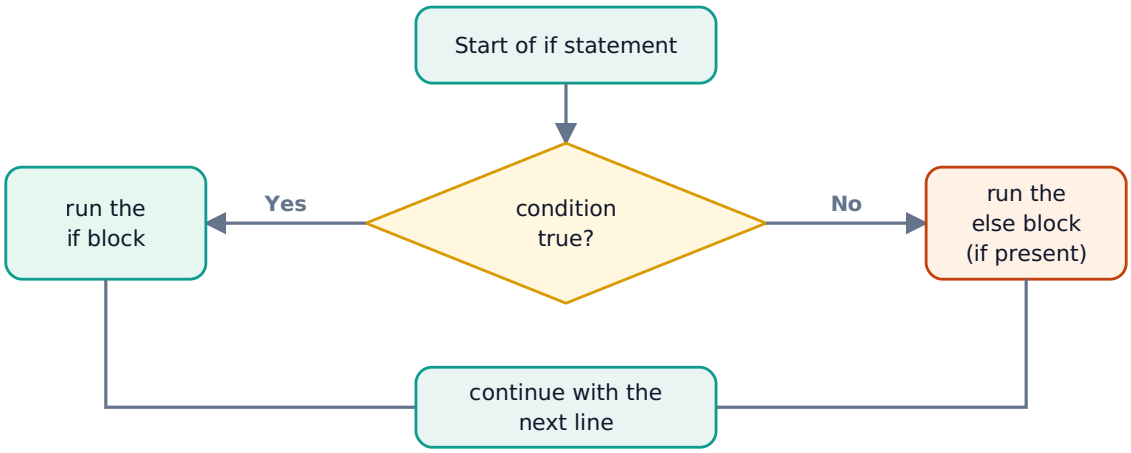
1. **[Easy]** Compute `(8+4)*3-5` using `set /a` and print the result.
2. **[Easy]** What is `set /a "x=19/4"` ? What is `19 %% 4` in a batch file?
3. **[Easy]** Write an `if` that prints "big" when a variable `n` is greater than 100.
4. **[Medium]** Write a script that reads two numbers with `set /p` and prints their sum, difference, product and integer quotient.
5. **[Medium]** Predict and then verify: `if "100" lss "20"`, `if 100 lss 20`, `if "abc" equ "ABC"`, `if /i "abc" equ "ABC"`.
6. **[Medium]** Convert 3725 seconds into hours, minutes and seconds.
7. **[Hard]** Write a function that tests whether a number is even or odd using `&` or `%%`.
8. **[Hard]** Use flags to model three options (bold=1, italic=2, underline=4). Turn on bold and underline, turn off bold, and print the value at each step.

Chapter 9: Conditionals

LEARNING OBJECTIVES

- Use `if`, `if not`, `if exist`, `if defined` and `if /i` correctly
- Understand `ERRORLEVEL`, the difference between `if errorlevel` and `%ERRORLEVEL%`
- Apply the `else` rules and avoid the syntax traps around parentheses
- Test for files, folders, empty folders and empty variables
- Write multi-way decisions with chained `if` and `goto`

9.1 The shape of `if`



The condition is evaluated once; exactly one branch runs.

Figure 9.1. An if statement evaluates a condition once and runs exactly one branch.

The general form is:

```
if [not] [/i] condition command
if [not] [/i] condition (block) else (block)
```

The command after the condition can be a single command or a parenthesized block of several commands. `not` reverses the test. All the forms of `condition` appear in the table below.

Form	True when
`if "%a%"=="%b%"`	The two strings are identical (case-sensitive unless `/i`)
`if %a% EQU %b%`	Equal; numeric when both are numbers (Chapter 8)
`if exist path`	A file or folder matching `path` exists

Form	True when
`if defined name`	The variable `name` has a value
`if errorlevel n`	The last exit code is **n or greater**
`if %errorlevel% EQU n`	The last exit code is **exactly** n
`if cmdextversion n`	Command extensions version is at least n

9.2 Comparing strings

string_if.bat

```
@echo off
setlocal
set /p "answer=Continue? (yes/no): "
if /i "%answer%"=="yes" (
    echo Continuing.
) else (
    echo Stopping.
)
```

OUTPUT

```
Continue? (yes/no): YES
Continuing.
```

`/i` makes the comparison case-insensitive, so `YES`, `Yes` and `yes` all match. The **quotes around both sides** protect against an empty answer: `if ""=="yes"` is valid, while `if ==yes` is a syntax error.

Common Mistake: Omitting quotes in a string comparison. `if %answer%==yes` fails when `answer` is empty or contains spaces. Always write `if "%answer%"=="yes"`, or better `if /i "%answer%"=="yes"`.

Characters that break comparisons even inside quotes

If the variable contains a double quote itself (`say "hi"`), the quoting is unbalanced and the comparison fails. Strip or reject quotes in user input first. Chapter 23 shows how.

9.3 Testing for files and folders: `if exist`

exist_demo.bat

```
@echo off
setlocal
set "file=%~dp0data.txt"
set "dir=%~dp0logs"

if exist "%file%" (echo File exists) else (echo File missing)
if exist "%dir%\" (echo Folder exists) else (echo Folder missing)
```

```
if not exist "%dir%\" md "%dir%"
if exist "%dir%\" echo Folder is now ready
```

```
OUTPUT
File missing
Folder missing
Folder is now ready
```

A **trailing backslash** (`"%dir%\"`) makes the test true only for **folders**; without it, `if exist` is true for a file or a folder with that name. Wildcards are allowed: `if exist "*.log"` is true if at least one such file exists in the current folder.

> **Testing for a folder:** You may see the older form `if exist "%dir%\nul"` . It still works on current Windows for local drives, but the trailing-backslash form is clearer and is recommended here. To tell a file from a folder precisely, inspect attributes with `%%~ai` in a `for` loop (Chapter 10): a `d` as the first attribute character means a directory.

Testing for an empty folder

empty_folder.bat

```
@echo off
setlocal
set "target=%~1"
if not defined target (echo Usage: %~nx0 folder & exit /b 2)
if not exist "%target%\" (echo Not a folder: %target% & exit /b 1)

dir /b "%target%" 2>nul | findstr "^" >nul
if errorlevel 1 (echo %target% is empty) else (echo %target% has content)
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> empty_folder.bat ..\logs
```

```
OUTPUT
..\logs is empty
```

`dir /b` lists names; `findstr "^"` matches every line, so it succeeds only if there was at least one line.

9.4 Testing for variables: `if defined`

`if defined name` is true when the variable has a non-empty value. It takes the variable's **name**, without percent signs:

defined_demo.bat

```
@echo off
setlocal
```

```
if defined USERNAME (echo USERNAME is set) else (echo USERNAME is missing)
if defined NOPE (echo NOPE is set) else (echo NOPE is missing)
set "config=test"
if defined config echo config holds: %config%
set "config="
if not defined config echo config was cleared
```

```
OUTPUT
USERNAME is set
NOPE is missing
config holds: test
config was cleared
```

It is the safest way to test for presence, because it never expands the value, so spaces and special characters in the value cannot break the line. A typical use is defaulting:

default_value.bat

```
@echo off
setlocal
if not defined LOG_DIR set "LOG_DIR=%~dp0logs"
echo Logging to %LOG_DIR%
```

```
OUTPUT
Logging to C:\Users\Alex\BatchBook\scripts\logs
```

9.5 ERRORLEVEL

Every program ends with an **exit code**, a number reporting success (0) or a kind of failure (non-zero). `cmd.exe` stores the exit code of the most recent command. You can read it in two ways, and they differ.

Form 1: `if errorlevel n`

True when the exit code is **n or higher**. This is a legacy construct, and its "or higher" meaning is its most important property:

errorlevel_old.bat

```
@echo off
ping -n 1 127.0.0.1 >nul
if errorlevel 1 (echo ping failed) else (echo ping succeeded)
choice /c YNC /n /m "Yes, No or Cancel? "
if errorlevel 3 echo You chose Cancel
if errorlevel 2 echo You chose No or Cancel
if errorlevel 1 echo You chose a choice
```

```
OUTPUT
ping succeeded
Yes, No or Cancel? C
You chose Cancel
```

```
You chose No or Cancel
You chose a choice
```

The `choice` example shows why order matters. When the user presses C (exit code 3), **all three** `if errorlevel` lines are true. To get one message per case test from the highest value down and jump (`goto`) or use `else` .

Form 2: `%ERRORLEVEL%`

The dynamic variable `%ERRORLEVEL%` holds the exit code and can be used with the numeric comparison operators:

errorlevel_new.bat

```
@echo off
setlocal
findstr /c:"needle" "%~dp0haystack.txt" >nul 2>&1
if %errorlevel% equ 0 (
    echo Found
) else if %errorlevel% equ 1 (
    echo Not found
) else (
    echo Error while searching: code %errorlevel%
)
```

OUTPUT

```
Error while searching: code 2
```

(`findstr` returns 0 for a match, 1 for no match and 2 for an error such as a missing file; the sample output assumes `haystack.txt` does not exist.) This form can test equality, ranges (`geq 8`) and negative values.

Common Mistake: Reading `%errorlevel%` inside a parenthesized block after a command in the same block. The percent expansion happens when the block is parsed, so it sees the **old** value. Use `if errorlevel 1` there, or enable delayed expansion and write `!errorlevel!` .

Best Practice: Use `if errorlevel 1` for the simple test "did the previous command fail" (it is safe in blocks), and `%errorlevel%` with `equ` when you need an exact code or a range outside of blocks.

Do not set ERRORLEVEL yourself

Never write `set errorlevel=0` . That creates a normal variable that **hides** the real dynamic value. To set the exit code use `exit /b n` , or the trick `cmd /c exit n` for a one-off.

Resetting the exit code

Some commands do not reset the code on success in `.bat` files (Chapter 2). To start from a known state use a command that always succeeds, such as `ver >nul` , or run a check immediately after the command of interest:

reset_level.bat

```
@echo off
ver >nul
rem errorlevel is now 0 for sure
where /q ping
if errorlevel 1 (echo ping is not available) else (echo ping is available)
```

OUTPUT

```
ping is available
```

9.6 `&&` and `||`: conditional chaining

`command1 && command2` runs `command2` only if `command1` succeeded (exit code 0). `command1 || command2` runs `command2` only if `command1` failed. They give compact checks:

chain_demo.bat

```
@echo off
where /q robocopy && echo robocopy found || echo robocopy missing
md "%~dp0tmp_demo" 2>nul && echo Created || echo Could not create
rd "%~dp0tmp_demo" 2>nul
```

OUTPUT

```
robocopy found
Created
```

The pattern `A && B || C` looks like "if A then B else C" but it is **not** equivalent: if `B` itself fails, `C` also runs. Chapter 12 covers chaining in full.

9.7 The `else` rules and parenthesis traps

`else` must be on the **same line as the closing parenthesis** of the `if` block. Placing it on its own line is a syntax error.

else_rules.bat

```
@echo off
set "n=5"

rem CORRECT: ) else ( on one line
if %n% gtr 3 (
    echo Large
) else (
    echo Small
)

rem CORRECT: all on one line
if %n% gtr 3 (echo Large) else (echo Small)

rem WRONG (commented out): else on its own line
rem if %n% gtr 3 (
rem     echo Large
```

```
rem )
rem else (
rem     echo Small
rem )
```

OUTPUT

```
Large
Large
```

Trap 1: parentheses inside blocks

A closing parenthesis inside a block ends it, unless it is quoted or escaped:

paren_trap.bat

```
@echo off
setlocal
set "dir=C:\Program Files (x86)"
if exist "%dir%" (
    echo Found "%dir%"
)
if exist "%dir%" (
    echo Found %dir%
)
```

TYPICAL OUTPUT (THE EXACT TOKEN NAMED IN THE MESSAGE VARIES)

```
Found "C:\Program Files (x86)"
was unexpected at this time.
```

The second block fails on systems where the folder exists: the unquoted `)` in `Program Files (x86)` ends the block early, leaving a stray `)` and causing "was unexpected at this time". Quote the text, or escape the parenthesis as `^)`, or use delayed expansion (`!dir!`), whose value is not parsed as syntax.

Trap 2: spaces before `else`

`)else(` and `) else (` both work, but `) else` with a different token arrangement may not. Keep the form `) else (`.

Trap 3: `else if`

else_if.bat

```
@echo off
set "n=15"
if %n% lss 10 (
    echo Small
) else if %n% lss 20 (
    echo Medium
) else (
    echo Large
)
```

OUTPUT

```
Medium
```

Chained `else if` works, and each `else` stays on the closing-parenthesis line.

Trap 4: `if` with a `goto` in a block

`goto` inside a block jumps out and cancels the rest of the block. This is fine, but be aware that for loops are cancelled too (Chapter 10).

9.8 Multi-way decisions

Chained `if` with `goto`

The classic menu dispatch:

dispatch.bat

```
@echo off
setlocal
set "cmd=%~1"
if /i "%cmd%"=="start"   goto :do_start
if /i "%cmd%"=="stop"    goto :do_stop
if /i "%cmd%"=="status"  goto :do_status
echo Unknown command: "%cmd%"
echo Usage: %~nx0 start^|stop^|status
exit /b 1

:do_start
echo Starting...
exit /b 0

:do_stop
echo Stopping...
exit /b 0

:do_status
echo Everything is fine.
exit /b 0
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> dispatch.bat STATUS
C:\Users\Alex\BatchBook\scripts> dispatch.bat fly
```

OUTPUT

```
Everything is fine.
Unknown command: "fly"
Usage: dispatch.bat start|stop|status
```

Computed `goto`

When the choice is a short key you can build the label name:

computed_goto.bat

```
@echo off
set "mode=%~1"
if not defined mode set "mode=help"
goto :mode_%mode% 2>nul || goto :mode_unknown

:mode_help
echo Help text.
exit /b 0

:mode_run
echo Running.
exit /b 0

:mode_unknown
echo Unknown mode.
exit /b 1
```

OUTPUT

```
Help text.
```

The line `goto :mode_%mode% 2>nul || goto :mode_unknown` jumps to `mode_help`, `mode_run`, and so on. If a label does not exist, `goto` fails, and the `||` branch handles the error. **Only use this when the key comes from your own code or has been validated**, because user-controlled text inside a `goto` label is an injection hazard (Chapter 23).

The `choice` command

`choice` waits for one of a set of keys and returns its position as the exit code (1 for the first):

choice_demo.bat

```
@echo off
choice /c ABQ /n /m "(A)dd, (B)rowse or (Q)uit? "
if errorlevel 3 goto :quit
if errorlevel 2 goto :browse
if errorlevel 1 goto :add

:add
echo Adding.
exit /b
:browse
echo Browsing.
exit /b
:quit
echo Goodbye.
exit /b
```

OUTPUT

```
(A)dd, (B)rowse or (Q)uit? B
Browsing.
```

Testing from the highest value down with `goto` ensures only one branch runs. Chapter 16 uses `choice` for full menus.

Summary

- Quote both sides of string comparisons; use `/i` for case-insensitive tests.
- `if exist "path"` tests files and folders; add a trailing backslash to test only for folders.
- `if defined name` tests whether a variable has a value, without expanding it.
- `if errorlevel n` means "n or greater"; `%errorlevel%` with `equ` tests exact values but is stale inside blocks.
- `else` belongs on the same line as the closing parenthesis; unquoted `)` inside a block ends it early.
- Multi-way decisions use chained `if` plus `goto`, or `choice` tested from the highest value down.

Common mistakes

Common Mistake: `if %var%==value` **without quotes.** Breaks when `var` is empty.

Common Mistake: `else` **on a separate line.**

Common Mistake: Using `%errorlevel%` **inside the same block that sets it.**

Common Mistake: Unquoted paths with parentheses inside blocks.

Common Mistake: Order of `if errorlevel` **tests ascending.** All lower tests are also true.

Exercises

1. **[Easy]** Write a script that prints "Windows" if `%OS%` equals `Windows_NT`, otherwise "Unknown".
2. **[Easy]** Check whether `C:\Windows\notepad.exe` exists and print a message either way.
3. **[Easy]** Check whether the variable `HOMEPATH` is defined.
4. **[Medium]** Ask for a password with `set /p` and compare it case-insensitively with `Secret`.
5. **[Medium]** Write a script that creates a folder `logs` only if it does not already exist.
6. **[Medium]** Run `ping -n 1 192.0.2.1` (a documentation address that should not answer) and report success or failure based on `findstr` output rather than ping's exit code. Why is the exit code unreliable?
7. **[Hard]** Write `size_class.bat` that takes a number of megabytes and prints Small (under 10), Medium (10 to 99) or Large (100 or more), rejecting non-numeric input.
8. **[Hard]** Explain why `echo %errorlevel%` inside `if exist x ( dir x & echo %errorlevel% )` shows a stale value, and rewrite it correctly.

Chapter 10: Loops

LEARNING OBJECTIVES

- Write `for` loops over lists, files, numbers, folders and directory trees
- Use every important option of `for /f` : `tokens` , `delims` , `skip` , `eol` , `usebackq`
- Parse text, files and command output
- Nest loops, and break out of or skip within loops
- Combine loops with delayed expansion safely

All loop variables in this chapter are written for **batch files**, with two percent signs (`%%i`). At the **interactive prompt** use one (`%i`).

10.1 `for` over a list

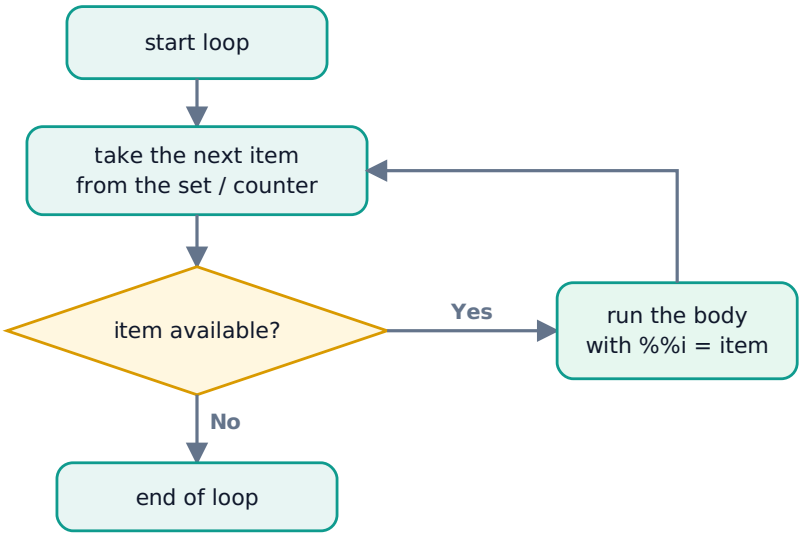


Figure 10.1. A for loop takes the next item from its set, runs the body, and repeats until the set is exhausted. The same shape applies to for /l, with a counter as the source.

The basic form is `for %%v in (set) do command` . The set is a list of items separated by spaces, commas, semicolons or tabs.

for_list.bat

```
@echo off
for %%c in (red green blue) do echo Colour: %%c
for %%n in (1,2,3) do echo Number: %%n
```

OUTPUT

```
Colour: red
Colour: green
```

```
Colour: blue
Number: 1
Number: 2
Number: 3
```

The loop variable is a **single letter or symbol** and it **is case-sensitive**: `%%a` and `%%A` are different variables. Items that look like wildcards (`*` or `?`) are expanded to matching **files**:

for_files.bat

```
@echo off
setlocal
cd /d "%~dp0"
echo hello> a.tmp
echo hello> b.tmp
for %%f in (*.tmp) do echo File: %%f  Size: %%~zf bytes
del a.tmp b.tmp
```

OUTPUT

```
File: a.tmp  Size: 7 bytes
File: b.tmp  Size: 7 bytes
```

`echo hello> a.tmp` writes `hello` plus a line break (7 bytes). The `~` modifiers of Chapter 6 (`%%~zf` , `%%~nxf` , `%%~dpf` and the others) work on loop variables.

Common Mistake: Putting a space before the redirection symbol when exact file content matters. `echo hello > a.tmp` writes `hello ` with a trailing space (8 bytes).

Quoted items and spaces

Items containing spaces must be quoted, and then `%%~f` removes the quotes:

for_quoted.bat

```
@echo off
for %%p in ("C:\Program Files" "C:\Windows") do echo Path: %%~p
```

OUTPUT

```
Path: C:\Program Files
Path: C:\Windows
```

10.2 `for /l`: counting

`for /l %%i in (start,step,end) do command` counts from `start` to `end` in increments of `step` . It is the batch equivalent of a counting loop.

for_count.bat

```
@echo off
echo Counting up:
for /l %%i in (1,1,5) do echo %%i
```

```
echo Counting down by 2:
for /l %%i in (10,-2,0) do echo %%i
```

OUTPUT

```
Counting up:
1
2
3
4
5
Counting down by 2:
10
8
6
4
2
0
```

The loop ends when the counter passes `end`. A negative step counts down. `for /l %%i in (1,1,3) do command` is also the standard way to repeat a command a fixed number of times, ignoring the variable.

Best Practice: `for /l` is much faster than building a loop from `goto` and `set /a`. Prefer it whenever you know the count in advance.

10.3 `for /d` and `for /r`: folders and trees

`for /d` loops over **directories** matching a wildcard in the current folder. `for /r` recurses into every subfolder below a starting folder and matches **files**.

for_dirs.bat

```
@echo off
setlocal
set "root=%~dp0demo_tree"
md "%root%\alpha\docs" "%root%\beta" 2>nul
echo one> "%root%\alpha\a.txt"
echo two> "%root%\alpha\docs\b.txt"
echo three> "%root%\beta\c.txt"

echo --- folders directly under demo_tree ---
for /d %%d in ("%root%\*") do echo %%~nxd

echo --- all .txt files below demo_tree ---
for /r "%root%" %%f in (*.txt) do echo %%~nxf in %%~dpf

echo --- all folders below demo_tree ---
for /d /r "%root%" %%d in (*) do echo %%d

rem Clean up the demonstration tree (inside this script folder only).
if exist "%root%\" rd /s /q "%root%"
```

OUTPUT

```
--- folders directly under demo_tree ---
alpha
beta
```

```
--- all .txt files below demo_tree ---
a.txt in C:\Users\Alex\BatchBook\scripts\demo_tree\alpha\
b.txt in C:\Users\Alex\BatchBook\scripts\demo_tree\alpha\docs\
c.txt in C:\Users\Alex\BatchBook\scripts\demo_tree\beta\
--- all folders below demo_tree ---
C:\Users\Alex\BatchBook\scripts\demo_tree\alpha
C:\Users\Alex\BatchBook\scripts\demo_tree\alpha\docs
C:\Users\Alex\BatchBook\scripts\demo_tree\beta
```

Warning: The cleanup line uses `rd /s /q` on a folder that the script itself created, whose name is built from `%~dp0` and a fixed word. Never use `rd /s /q` on a path assembled from user input or from a variable that might be empty.

10.4 `for /f`: parsing text

`for /f` reads **lines** from a file, a string or the output of a command, splits each line into **tokens**, and runs the body once per line. It is the main text-processing tool in Batch.

```
for /f ["options"] %%v in (source) do command
```

The three kinds of source are distinguished by the delimiters around them:

Source	Written as	Meaning
File(s)	`in (file.txt)` or `in ("file with spaces.txt")` with `usebackq`	Read lines from the file
String	`in ("some text")`	Parse a literal string
Command	`in ('command')`	Run `command` and parse its output

With the option `usebackq` the roles change: `"..."` can be a quoted **file name**, `'...'` is a literal **string**, and the backquote `` `...` `` runs a **command**.

Options

Option	Meaning
`delims=xyz`	Characters that separate tokens. Default: space and tab. `delims=` (nothing) means **no splitting**: the whole line is token 1.
`tokens=n`	Which tokens to take: `1`, `1,3`, `2-4`, `1,*` (`*` = the rest of the line)
`skip=n`	Skip the first n lines
`eol=c`	Lines whose first character is `c` are ignored (default `;`)
`usebackq`	Alternative quoting rules, listed above

Rules to keep in mind: blank lines are always skipped; the first token goes to the loop variable you named, and the following tokens go to the **next letters in alphabetical order** (`%%a` , `%%b` , `%%c`); consecutive delimiters count as one; an option string must be quoted.

Reading a file line by line

read_lines.bat

```
@echo off
setlocal
set "f=%~dp0sample.txt"
(
    echo First line
    echo ; this line starts with a semicolon
    echo.
    echo   Indented line
    echo Last line
) > "%f%"

echo --- 1. default options ---
for /f "usebackq" %%a in ("%f%") do echo [%%a]

echo --- 2. whole lines ---
for /f "usebackq delims=" %%a in ("%f%") do echo [%%a]

echo --- 3. whole lines, keeping lines that start with a semicolon ---
for /f "usebackq eol=~ delims=" %%a in ("%f%") do echo [%%a]
del "%f%"
```

OUTPUT

```
--- 1. default options ---
[First]
[Indented]
[Last]
--- 2. whole lines ---
[First line]
[  Indented line]
[Last line]
--- 3. whole lines, keeping lines that start with a semicolon ---
[First line]
[; this line starts with a semicolon]
[  Indented line]
[Last line]
```

Four behaviors are visible:

1. With default options only the **first word** of each line is returned (the default delimiters are space and tab), and leading spaces are skipped.
2. A line that begins with `;` is dropped, because the default `eol` character is a semicolon.
3. **Empty lines are always skipped**, in every form. (To keep them, number the lines first with `findstr /n "^"` and strip the prefix.)
4. `delims=` with nothing after it turns splitting off, so each loop receives the whole line, leading spaces included.

Option order matters: The `delims=` option consumes **everything after it** up to the closing quote, spaces included. If you also set other options, write them **before** `delims=`, and put `delims=` last: `"usebackq eol=~ delims="`. Writing `"delims= eol=~"` would make the characters space, `e`, `o`, `l`, `=` and `~` all delimiters.

Selecting tokens

tokens_demo.bat

```
@echo off
rem Parse a string with tokens separated by slashes.
for /f "tokens=1-3 delims=/" %%a in ("10/23/2026") do (
    echo Month=%%a Day=%%b Year=%%c
)
rem Take token 2 and the rest of the line.
for /f "tokens=2,* delims= " %%a in ("alpha beta gamma delta") do (
    echo Second=%%a Rest=%%b
)
```

OUTPUT

```
Month=10 Day=23 Year=2026
Second=beta Rest=gamma delta
```

Skipping a header

csv_demo.bat

```
@echo off
setlocal
set "f=%~dp0people.csv"
(
    echo name,city,age
    echo Ada,London,36
    echo Linus,Helsinki,28
) > "%f%"
for /f "usebackq skip=1 tokens=1-3 delims=," %%a in ("%f%") do (
    echo %%a lives in %%b and is %%c years old
)
del "%f%"
```

OUTPUT

```
Ada lives in London and is 36 years old
Linus lives in Helsinki and is 28 years old
```

Parsing command output

Placing a command in single quotes makes `for /f` run it and read its output. A pipe or redirection **inside** that command must be escaped with a caret (`^|`, `^>`), because the `for` line is parsed before the command runs.

ipv4.bat

```
@echo off
setlocal
```

```
for /f "tokens=2 delims=:" %%a in ('ipconfig ^| findstr /c:"IPv4 Address"') do (
    for /f "tokens=*" %%b in ("%%a") do echo IPv4 address: %%b
)
```

```
OUTPUT
IPv4 address: 192.168.1.23
```

The outer loop splits at the colon and returns the text after it, which starts with a space. The inner `for /f "tokens=*"` removes the leading spaces. The label `IPv4 Address` appears in this form in **English** Windows; on localized systems the text differs, which is a general weakness of parsing human-readable output (see Chapter 15 and Chapter 19 for locale-independent approaches).

count_lines.bat

```
@echo off
setlocal
set "f=%~dp0demo.txt"
(echo a& echo b& echo c) > "%f%"
for /f %%n in ('find /c /v "" ^< "%f%"') do echo Lines: %%n
del "%f%"
```

```
OUTPUT
Lines: 3
```

Paths with spaces and `usebackq`

When the file name has spaces, `usebackq` allows it to be quoted:

usebackq_demo.bat

```
@echo off
setlocal
set "dir=%~dp0Folder With Spaces"
md "%dir%" 2>nul
echo hello> "%dir%\note.txt"
for /f "usebackq delims=" %%a in ("%dir%\note.txt") do echo Read: %%a
rd /s /q "%dir%"
```

```
OUTPUT
Read: hello
```

10.5 Nested loops

multiplication.bat

```
@echo off
setlocal EnableDelayedExpansion
for /l %%r in (1,1,3) do (
    set "line="
    for /l %%c in (1,1,4) do (
        set /a "p=%%r*%%c"
        set "line=!line! !p!"
    )
```

```
    echo Row %%r:!line!
)
```

OUTPUT

```
Row 1: 1 2 3 4
Row 2: 2 4 6 8
Row 3: 3 6 9 12
```

Each nested loop needs its own variable letter. Delayed expansion (`!line!`, `!p!`) is required because both change inside the block (Chapter 7).

10.6 Leaving and skipping

Batch has no `break` or `continue` keyword. Two patterns replace them.

Break: `goto` a label **outside** the loop. This cancels the loop completely.

Continue: move the loop body into a subroutine; `exit /b` then ends only that iteration.

break_continue.bat

```
@echo off
for /l %%i in (1,1,10) do (
    call :handle %%i || goto :finished
)
:finished
echo Loop ended early.
exit /b 0

:handle
rem Skip item 3 (continue) and stop at item 6 (break).
if %1 equ 3 exit /b 0
if %1 equ 6 exit /b 1
echo Item %1
exit /b 0
```

OUTPUT

```
Item 1
Item 2
Item 4
Item 5
Loop ended early.
```

Common Mistake: Using `goto` inside a `for` body and expecting the loop to continue. A `goto` out of a block cancels the loop. Inside a `for /l` body, a `goto` to a label **inside** the loop body is not reliable either. Use the subroutine pattern.

10.7 Loops with delayed expansion

Whenever a variable changes inside a loop body and is read in the same body, use delayed expansion:

sum_sizes.bat

```
@echo off
setlocal EnableDelayedExpansion
cd /d "%windir%"
set /a "count=0, total_kb=0"
for %%f in (*.exe) do (
    set /a "count+=1"
    set /a "kb=%%~zf/1024"
    set /a "total_kb+=kb"
)
echo Files: %count%
echo Total: %total_kb% KB
```

OUTPUT

```
Files: 12
Total: 38472 KB
```

Warning: `if` comparisons and `set /a` use **32-bit** numbers. A file larger than 2,147,483,647 bytes (about 2 GiB) gives a wrong `%%~zf`. Dividing the size by 1024 in a **single** step overflows when the size does not fit. For large files use PowerShell (Chapter 19).

Best Practice: When you only need a loop to print or call something per item, do not enable delayed expansion at all. Enabling it exposes any file name containing `!` to corruption (Chapter 7).

Summary

- `for %%v in (list)` loops over words, or wildcard-matched files.
- `for /l %%i in (start,step,end)` counts. `for /d` loops over folders. `for /r` descends into trees.
- `for /f` parses files, strings and command output, with options `tokens`, `delims`, `skip`, `eol` and `usebackq`.
- Blank lines are skipped; default `eol` is `;`; whole-line parsing needs `"eol=~ delims="` with `delims=` last.
- Use `goto` outside a loop to break, and a subroutine with `exit /b` to continue.
- Changing variables inside a loop body requires delayed expansion.

Common mistakes

Common Mistake: One percent sign in a script. `for %i in (...)` fails in a batch file; use `%%i`.

Common Mistake: Forgetting that `for /f` skips empty lines and `;` lines.

Common Mistake: Not escaping `|` and `>` in a `for /f` command. Use `^|` and `^>`.

Common Mistake: Placing `delims=` before other options.

Common Mistake: Changing the set while looping (for example, renaming files that the wildcard will still match).

Exercises

1. **[Easy]** Print the numbers 1 to 10, one per line, using `for /l`.
2. **[Easy]** Print the name of every `.exe` file in `C:\Windows` using `for`.
3. **[Easy]** Print the numbers from 20 down to 0 in steps of 5.
4. **[Medium]** Write a script that lists the names of all folders directly under `%USERPROFILE%`.
5. **[Medium]** Read a file, whole lines only, and print each line preceded by its line number.
6. **[Medium]** Parse the string `alpha;beta;gamma` and print each part on its own line.
7. **[Hard]** Write a script that recursively counts the `.txt` files below a folder given as an argument, without using `dir`.
8. **[Hard]** Use `for /f` to extract the time zone name from the output of `tzutil /g`, and explain why this is more reliable than parsing `%TIME%`.

Chapter 11: Labels, goto, call, and Subroutines

LEARNING OBJECTIVES

- Define labels and jump with `goto`, including `goto :eof`
- Write reusable subroutines with `call :label` and `exit /b`
- Pass arguments to subroutines and return values to the caller
- Keep subroutine variables local, and return results out of `setlocal`
- Build function libraries and write a recursive subroutine

11.1 Labels and `goto`

A **label** is a line that starts with a colon followed by a name: `:start`. `goto label` transfers control to that line; execution continues from there. Label names are case-insensitive, and `cmd.exe` searches **from the top of the file**, taking the first match.

goto_basic.bat

```
@echo off
set "count=0"

:again
set /a "count+=1"
echo Pass %count%
if %count% lss 3 goto :again

echo Done after %count% passes.
```

OUTPUT

```
Pass 1
Pass 2
Pass 3
Done after 3 passes.
```

The colon before the label in the `goto` (`goto :again`) is optional but recommended: it is required for the special label `:eof`, and it makes the intent obvious.

The special label `:eof`

`goto :eof` jumps to the **end of the current batch file** (or of the current subroutine, see below) and needs no `:eof` label to be defined. It is the standard way to stop early:

goto_eof.bat

```
@echo off
if "%~1"=="" (
```

```
    echo Nothing to do.
    goto :eof
)
echo Processing %~1
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> goto_eof.bat
C:\Users\Alex\BatchBook\scripts> goto_eof.bat report.txt
```

OUTPUT

```
Nothing to do.
Processing report.txt
```

Cautions

- A script made only of `goto` jumps becomes hard to follow. Use `goto` for structure (error exits, dispatch, loops over arguments), not as your main technique.
- A label inside a parenthesized block can cause errors. Place labels at the top level.
- Jumping to a label that does not exist prints `The system cannot find the batch label specified - name` and ends the script.
- A **label line is also executed as a no-op** if control falls through it, which is why code that follows a `:main` section can accidentally run into a subroutine (Section 11.2).

11.2 Subroutines with `call :label`

`call :label arguments` runs the code at `label` as a **subroutine**: control returns to the line after the `call` when the routine ends with `exit /b` or `goto :eof`. The arguments are available inside as `%1`, `%2`, `%*`, exactly as for a script (Chapter 6).

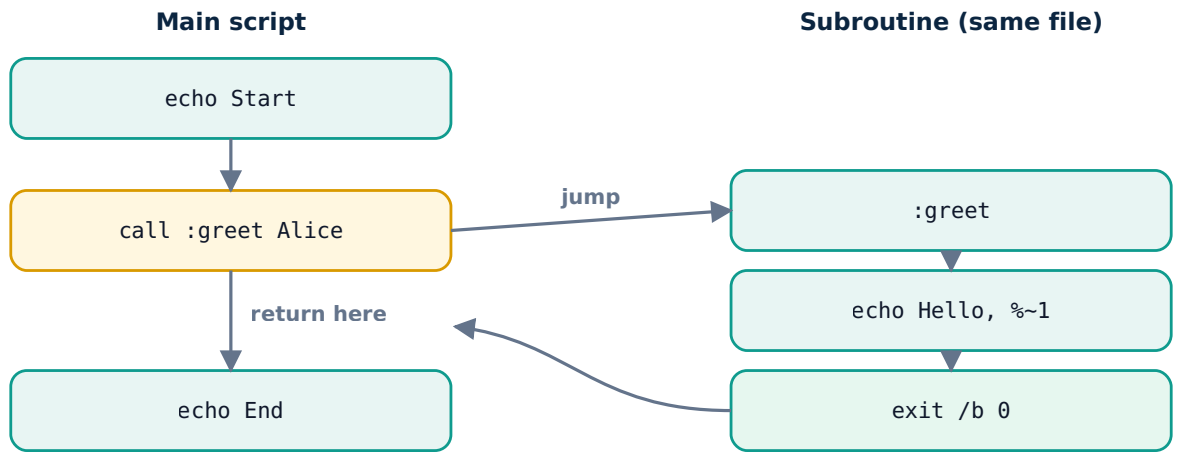


Figure 11.1. call :label transfers control to the label, exit /b returns to the next line after the call.

call_basic.bat

```
@echo off
echo Start
call :greet Alice
call :greet "Bob Smith"
echo End
exit /b 0

:greet
echo Hello, %~1!
exit /b 0
```

OUTPUT

```
Start
Hello, Alice!
Hello, Bob Smith!
End
```

Notice the `exit /b 0` just before `:greet`. **Without it, the script would run into the subroutine after printing `End` and say `Hello, !` once more. The rule: end the main code with `exit /b` or `goto :eof` before the first subroutine.**

> **Common Mistake: Falling into a subroutine.** Forgetting `exit /b` after the main section makes the first subroutine execute with empty arguments. Always put `exit /b` (or `goto :eof`) as the last line of main.

`exit /b` versus `goto :eof`

Both leave the current subroutine or script. `exit /b n` also sets the exit code `n`. `goto :eof` leaves the code unchanged. Use `exit /b` with an explicit number when the result matters.

`call` on another script

`call "other.bat" args` runs another batch file and **returns** to the caller afterward; without `call`, control never comes back (Chapter 2). A called script inherits the caller's variables, and unless it uses `setlocal`, its changes persist.

11.3 Returning values

A subroutine can return information in four ways.

1. The exit code

Good for success or failure:

return_code.bat

```
@echo off
call :is_even 10
if errorlevel 1 (echo 10 is odd) else (echo 10 is even)
call :is_even 7
if errorlevel 1 (echo 7 is odd) else (echo 7 is even)
```

```
exit /b 0

:is_even
set /a "r=%~1 %% 2"
exit /b %r%
```

```
OUTPUT
10 is even
7 is odd
```

2. A global variable

The simplest way, with a clear naming convention:

return_global.bat

```
@echo off
call :square 7
echo Square is %RESULT%
exit /b 0

:square
set /a "RESULT=%~1*%~1"
exit /b 0
```

```
OUTPUT
Square is 49
```

The subroutine and caller share a variable name by agreement. This is easy but can overwrite a caller's variable by accident.

3. An output variable named by the caller

The caller passes the **name** of the variable to receive the result. This makes the routine reusable:

return_named.bat

```
@echo off
call :add total 40 2
echo total = %total%
call :add other %total% 8
echo other = %other%
exit /b 0

:add  outvar a b
set /a "%~1=%~2+%~3"
exit /b 0
```

```
OUTPUT
total = 42
other = 50
```

`set /a "%~1=%~2+%~3"` becomes `set /a "total=40+2"` after expansion.

4. Local variables and the `endlocal` trick

A subroutine that uses its own temporary variables should protect the caller's environment with `setlocal`. But `endlocal` discards **everything**, including the result. The standard solution relies on the fact that the line is parsed **before** `endlocal` runs:

return_local.bat

```
@echo off
set "tmp=caller value"
call :compute answer 6 7
echo answer = %answer%
echo tmp    = %tmp%
exit /b 0

:compute  outvar a b
setlocal
set "tmp=scratch"
set /a "value=%~2*%~3"
endlocal & set "%~1=%value%"
exit /b 0
```

OUTPUT

```
answer = 42
tmp    = caller value
```

On the line `endlocal & set "%~1=%value%"`, `%value%` and `%~1` are replaced when the line is **parsed**, giving `endlocal & set "answer=42"`. Then `endlocal` runs, restoring the caller's environment, and the `set` stores the result in it. The caller's `tmp` survives untouched.

Returning several values

return_multi.bat

```
@echo off
call :divmod 17 5 q r
echo 17 divided by 5 = %q% remainder %r%
exit /b 0

:divmod  a b quotient_var remainder_var
setlocal
set /a "q=%~1/%~2, r=%~1%%%~2"
endlocal & set "%~3=%q%" & set "%~4=%r%"
exit /b 0
```

OUTPUT

```
17 divided by 5 = 3 remainder 2
```

Inside the routine `%%%~2` is the literal percent sign followed by the argument.

> **Warning:** The `endlocal & set` trick fails when the returned value contains `!`, `^` or unbalanced quotes and delayed expansion is on, and quotes in the value can break the `set` line. For arbitrary text values, return the value through a temporary file instead, and read it with `set /p` or `for /f`.

11.4 Conventions for good subroutines

Convention	Reason
A header comment line: `:name param1 param2`	Documents the interface at the label
Use `%~1` (quotes removed) and re-quote when using the value	Safe for paths with spaces
`setlocal` **at the start of the routine**	No variable leaks
One purpose per routine, short name with a prefix such as `:util_trim`	Avoids collisions
Document the exit code (0 = success)	Callers can test it
Put all routines after the final `exit /b` **of main**	Prevents fall-through

Best Practice: Inside a routine, `setlocal` is implicitly closed when the routine returns with `exit /b` or `goto :eof`, so you do not need `endlocal` if you return nothing.

11.5 Function libraries

Reusable routines belong in a separate file. The calling file asks the library to run one of its labels:

mathlib.bat

```
@echo off
rem Library of math helpers. Usage: call "path\mathlib.bat" :name args
rem Dispatch to the requested label.
call %*
exit /b %errorlevel%

:max  outvar a b
setlocal
set /a "m=%~2"
if %~3 gtr %m% set /a "m=%~3"
endlocal & set "%~1=%m%"
exit /b 0

:abs  outvar n
setlocal
set /a "v=%~2"
if %v% lss 0 set /a "v=-v"
endlocal & set "%~1=%v%"
exit /b 0
```

use_lib.bat

```
@echo off
call "%~dp0mathlib.bat" :max big 12 99
echo Max is %big%
call "%~dp0mathlib.bat" :abs size -250
echo Absolute value is %size%
```

```
OUTPUT
Max is 99
Absolute value is 250
```

`call %*` in the library expands to `call :max big 12 99`. Because the label is in `mathlib.bat`, it is found there. The library ends with `exit /b` so that, if no argument was given, control cannot fall into the first routine.

Common Mistake: Naming a library routine the same as a routine in the caller. Each file has its own labels, so there is no clash across files. Inside one file, duplicate labels are silently resolved to the first one.

11.6 Recursion

A subroutine can call itself. The factorial of n is n times the factorial of n-1, down to 1:

factorial.bat

```
@echo off
setlocal
call :fact result 5
echo 5! = %result%
call :fact result 10
echo 10! = %result%
exit /b 0

:fact  outvar n
setlocal
set /a "n=%~2"
if %n% leq 1 (
    endlocal & set "%~1=1"
    exit /b 0
)
set /a "m=n-1"
call :fact sub %m%
set /a "r=n*sub"
endlocal & set "%~1=%r%"
exit /b 0
```

```
OUTPUT
5! = 120
10! = 3628800
```

Each level has its own `n` thanks to `setlocal`. The base case returns 1. Beware of limits:

- `13!` exceeds the 32-bit range and wraps silently, so the result is wrong (Chapter 8).
- Every nested `call :` consumes stack space; very deep recursion (thousands of levels) can fail. Prefer loops for large depths.

11.7 A realistic example: a logging library

logger.bat

```
@echo off
setlocal
set "LOG_FILE=%~dp0demo.log"
if exist "%LOG_FILE%" del "%LOG_FILE%"

call :log INFO "Script started"
call :log WARN "Disk space is low"
call :log ERROR "Could not open the database"
call :log INFO "Script finished"

echo --- contents of the log ---
type "%LOG_FILE%"
del "%LOG_FILE%"
exit /b 0

:log  level message
rem Appends a line with date, time, level and message to LOG_FILE.
>> "%LOG_FILE%" echo %date% %time% [%~1] %~2
exit /b 0
```

OUTPUT

```
--- contents of the log ---
Fri 10/23/2026  9:50:11.04 [INFO] Script started
Fri 10/23/2026  9:50:11.05 [WARN] Disk space is low
Fri 10/23/2026  9:50:11.05 [ERROR] Could not open the database
Fri 10/23/2026  9:50:11.06 [INFO] Script finished
```

The redirection is written at the start of the line (`>> file echo ...`) so that no stray space is added to the message. Date and time formats depend on your regional settings; Chapter 15 shows locale-independent timestamps.

Summary

- A label is `:name` ; `goto :name` jumps; `goto :eof` ends the current script or routine.
- `call :name args` runs a subroutine; `exit /b n` returns with a code. Always end main with `exit /b` before the routines.
- Return values by exit code, by a global, by a named output variable, or by `endlocal & set` after `setlocal` .
- Use `setlocal` inside routines; it is closed implicitly on return.
- Libraries are scripts that begin with `call %*` . Recursion works but is limited by the stack and by 32-bit arithmetic.

Common mistakes

Common Mistake: No `exit /b` before the subroutines.

Common Mistake: Using `exit` instead of `exit /b` in a subroutine. It closes the whole session.

Common Mistake: Labels inside blocks.

Common Mistake: Losing a result to `endlocal` because the value was not passed out with the single-line trick.

Common Mistake: Forgetting `call` when running another script.

Exercises

1. **[Easy]** Write a loop with a label and `goto` that prints the numbers 1 to 5.
2. **[Easy]** Write a subroutine `:hello` that prints "Hello" and call it twice.
3. **[Easy]** What is the difference between `exit /b` and `goto :eof`?
4. **[Medium]** Write a subroutine `:cube outvar n` that returns the cube of n in a named variable.
5. **[Medium]** Write `:minmax a b minvar maxvar` returning both values.
6. **[Medium]** Why does the `endlocal & set "%~1=%value%"` line work, even though `endlocal` discards the variable `value`?
7. **[Hard]** Write a recursive routine that computes the n-th Fibonacci number (small n) and explain its performance problem.
8. **[Hard]** Create a library `strlib.bat` with a routine `:repeat outvar text count` that builds a string by repeating text, and call it from another script.

Chapter 12: Input, Output, Redirection and Escaping

LEARNING OBJECTIVES

- Name the three standard streams and redirect each of them
- Use `>`, `>>`, `<`, `2>`, `2>&1`, pipes and `nul` correctly, including the order rules
- Chain commands with `&`, `&&`, `||` and grouping
- Escape every special character with `^`, `%%`, quotes and delayed-expansion rules
- Write output without a trailing newline and continue long lines

12.1 The three standard streams

Every command-line program is connected to three **streams**, each identified by a number:

Number	Name	Default	Purpose
0	**stdin**	Keyboard	Input
1	**stdout**	Screen	Normal output
2	**stderr**	Screen	Error messages



Redirection operators reconnect a stream to a file, a device (nul) or another program.

Figure 12.1. A program reads from stdin and writes to stdout and stderr. Redirection operators reconnect these streams to files, devices or other programs.

Because stdout and stderr both appear on screen by default, they look the same, but they are separate channels. That is what makes it possible to save results in one file and errors in another.

12.2 Redirecting output

Operator	Effect
`> file`	Send stdout to `file`, **replacing** its content
`>> file`	**Append** stdout to `file`
`2> file`	Send stderr to `file`
`2>> file`	Append stderr
`2>&1`	Send stderr to wherever stdout currently goes
`1>&2`	Send stdout to wherever stderr currently goes
`> nul`	Discard stdout
`2> nul`	Discard stderr

redirect_basic.bat

```
@echo off
setlocal
cd /d "%~dp0"

echo First line> out.txt
echo Second line>> out.txt
echo Contents of out.txt:
type out.txt

echo.
echo Errors separated from results:
dir /b out.txt missing.txt > results.txt 2> errors.txt
echo --- results.txt
type results.txt
echo --- errors.txt
type errors.txt

del out.txt results.txt errors.txt
```

OUTPUT

```
Contents of out.txt:
First line
Second line

Errors separated from results:
--- results.txt
out.txt
--- errors.txt
File Not Found
```

`dir /b out.txt missing.txt` found one name and failed on the other; the successes went to `results.txt` and the error message to `errors.txt`.

Redirect both streams to one file

redirect_both.bat

```
@echo off
setlocal
cd /d "%~dp0"
echo data> out.txt
dir /b out.txt missing.txt > both.txt 2>&1
type both.txt
del both.txt out.txt
```

OUTPUT

```
out.txt
File Not Found
```

What matters is the **order**: `> both.txt 2>&1` first connects stdout to the file, then connects stderr to wherever stdout now points: the file. The reverse order `2>&1 > both.txt` connects stderr to the **screen** (where stdout points at that moment) and only then moves stdout to the file, so the error text would still appear on screen and not in the file.

Common Mistake: Writing the redirections in the wrong order. Use `> file 2>&1`, never `2>&1 > file`.

The space and digit pitfalls

redirect_pitfalls.bat

```
@echo off
setlocal
cd /d "%~dp0"

echo abc > a.txt
for %%f in (a.txt) do echo a.txt has %%~zf bytes (trailing space included)

echo abc> b.txt
for %%f in (b.txt) do echo b.txt has %%~zf bytes

rem A digit just before the operator is taken as a stream number:
echo 2> c.txt
type c.txt 2>nul
echo (c.txt holds nothing, 2 was treated as stderr)

rem Put the redirection first, or wrap the command:
> c.txt echo 2
type c.txt
del a.txt b.txt c.txt
```

OUTPUT

```
a.txt has 6 bytes (trailing space included)
b.txt has 5 bytes
ECHO is off.
```

```
(c.txt holds nothing, 2 was treated as stderr)
2
```

- `echo abc > a.txt` writes `abc` (4 characters) plus the line break (2): 6 bytes. `echo abc> b.txt` writes `abc` plus the line break: 5 bytes.
- `echo 2> c.txt` is read as `echo` with **stderr** (stream 2) redirected to `c.txt`. The digit is consumed as the stream number, so `echo` receives no text, prints `ECHO is off.` on screen, and the file stays empty. Move the redirection to the front (`> c.txt echo 2`) or write `echo 2 > c.txt`, accepting the trailing space.

Best Practice: When exact content matters, write redirection **first**: `>> "%LOG%" echo Message`. It avoids trailing spaces and digit confusion.

Grouping commands to share a redirection

group_redirect.bat

```
@echo off
setlocal
cd /d "%~dp0"
(
    echo Report
    echo ======
    echo Computer: %COMPUTERNAME%
    echo User    : %USERNAME%
) > report.txt
type report.txt
del report.txt
```

OUTPUT

```
Report
======
Computer: DESKTOP-4F2K9
User    : Alex
```

Creating empty files and clearing files

Command Prompt

```
C:\> type nul > empty.txt
C:\> copy nul empty2.txt
C:\> type nul > existing.txt
```

The last line **empties** an existing file. `nul` is a special device that reads as end-of-file and swallows anything written to it.

12.3 Redirecting input

`< file` makes the program read its stdin from a file instead of the keyboard:

redirect_input.bat

```
@echo off
setlocal
cd /d "%~dp0"
(echo pear& echo apple& echo cherry) > fruit.txt
echo --- sorted ---
sort < fruit.txt
echo --- line count ---
find /c /v "" < fruit.txt
set /p "first=" < fruit.txt
echo First line: %first%
del fruit.txt
```

OUTPUT

```
--- sorted ---
apple
cherry
pear
--- line count ---
3
First line: pear
```

12.4 Pipes

`command1 | command2` connects the **stdout** of the first command to the **stdin** of the second:

pipe_demo.bat

```
@echo off
echo --- folders in %windir% containing "sys", sorted ---
dir /b /ad "%windir%" | findstr /i "sys" | sort
```

OUTPUT

```
--- folders in C:\Windows containing "sys", sorted ---
SysWOW64
System
System32
```

Pipes of more than two commands work the same way. Three behaviors to remember:

1. **Each side of a pipe runs in its own `cmd.exe`.** Variables set on either side are lost afterward.
2. **A space before `|` is part of the data** when you pipe `echo`: `echo abc | find "abc "` matches, because the pipe receives `abc` with a trailing space. Write `echo abc| find ...`.
3. **Only stdout is piped.** To pipe errors too use `2>&1` before the pipe: `command 2>&1 | findstr x`.

pipe_variables.bat

```
@echo off
set "x=before"
echo hello | set /p "x="
echo x is still: %x%
```

OUTPUT

```
x is still: before
```

The `set /p` ran in the pipe's own `cmd.exe`, so the change disappeared with it.

12.5 Chaining commands

Form	Meaning
`a & b`	Run `a`, then `b`, always
`a && b`	Run `b` only if `a` **succeeded** (exit code 0)
`a || b`	Run `b` only if `a` **failed** (non-zero exit code)
`( a & b )`	Group: treat several commands as one

chain_forms.bat

```
@echo off
setlocal
cd /d "%~dp0"
echo one & echo two
md chain_test && echo Created chain_test
md chain_test 2>nul || echo Could not create it again
rd chain_test
```

OUTPUT

```
one
two
Created chain_test
Could not create it again
```

The `A && B || C` trap

It reads like if-then-else, but is not: if `B` fails, `C` also runs. When the middle command cannot fail (such as `echo`) the pattern is reliable; otherwise use a real `if` or a block:

chain_trap.bat

```
@echo off
rem B (a failing command) causes C to run even though A succeeded:
cmd /c exit 0 && cmd /c exit 1 || echo C ran although A succeeded
```

OUTPUT

```
C ran although A succeeded
```

12.6 Devices: `nul`, `con` and friends

Windows reserves some names for devices: `nul`, `con`, `prn`, `aux`, `com1` to `com9`, `lpt1` to `lpt9`. They are valid redirection targets, and they **cannot be used as file or folder names** (with any extension).

Device	Use
`nul`	Discard output; empty input
`con`	The console; `copy con file.txt` types into a file (press Ctrl+Z, then Enter, to finish)

The most common idioms:

- `command >nul` hides normal output.
- `command 2>nul` hides errors.
- `command >nul 2>&1` hides everything.

> **Best Practice:** Check results through the **exit code**, and silence output you do not need: `ping -n 1 host >nul 2>&1 && echo up || echo down`.

12.7 Escaping special characters

Some characters mean something to `cmd.exe`. To print or pass them literally you must **escape** them. The rules differ depending on whether you are inside quotes.

The caret

The caret `^` placed before a character removes its special meaning. It works for `& | < > ^ ( )` and also `"` when outside quotes:

To get	Write	Notes
`&`	`^&`	
`|`	`^|`	
`<` **and** `>`	`^<` and `^>`	
`^`	`^^`	
`(` **and** `)`	`^(` and `^)`	Needed for `)` inside blocks
`%` **in a batch file**	`%%`	Not a caret case
`%` **at the prompt**	`%` or `^%`	Often works unescaped
`!` **with delayed expansion**	`^^!`	Caret doubled
`"`	no reliable escape	See below

escape_demo.bat

```
@echo off
echo Fish ^& Chips
echo 5 ^> 3 and 2 ^< 4
echo a ^| b
echo caret: ^^
echo 50%% off
echo (parentheses) are fine at the top level
if 1==1 (
    echo inside a block: ^(parentheses^)
)
```

OUTPUT

```
Fish & Chips
5 > 3 and 2 < 4
a | b
caret: ^
50% off
(parentheses) are fine at the top level
inside a block: (parentheses)
```

Quotes change the rules

Inside double quotes, `& | < > ^ ( )` are **literal**, so carets are not needed (and would be printed):

quote_rules.bat

```
@echo off
set "msg=Tom & Jerry | Cat > Mouse"
echo %msg%
echo "Tom & Jerry"
echo Tom ^& Jerry
```

OUTPUT

```
Tom & Jerry | Cat > Mouse
"Tom & Jerry"
Tom & Jerry
```

The first `echo %msg%` works because the variable's content is inserted **after** parsing, as plain text. Percent signs (`%`) and, with delayed expansion, exclamation marks (`!`) remain special even inside quotes.

> **Common Mistake: Putting a caret inside quotes.** `echo "a ^& b"` prints the caret. Escape outside quotes or quote, but not both.

Double quotes themselves

`cmd.exe` has no general way to escape a double quote. Programs decide how to interpret `\"` or `""` after `cmd.exe` has passed the text. When you need a literal quote in `echo`, you can use a variable that holds it:

quote_char.bat

```
@echo off
set "Q=""
echo He said %Q%hello%Q%.
```

OUTPUT

```
He said "hello".
```

Line continuation

A caret at the **very end** of a line continues the command on the next line:

continuation.bat

```
@echo off
robocopy "%~dp0" "%~dp0copy" /l /e ^
    /r:1 /w:1 ^
    /njh /njs /np
exit /b 0
```

OUTPUT (LIST-ONLY RUN; FILE NAMES VARY)

```
           New File                 120   notes.txt
           New File                  42   todo.txt
```

There must be **no space after the caret**, and the next line must not be empty.

Printing without a newline

`set /p` can print a prompt and wait; reading from `nul` skips the wait. The result prints text with no line break:

no_newline.bat

```
@echo off
<nul set /p "=Working... "
timeout /t 1 /nobreak >nul
echo done.
```

OUTPUT

```
Working... done.
```

Escaping inside `for /f` commands

A pipe or redirection inside the quoted command of `for /f` must carry a caret (Chapter 10):

Command Prompt

```
C:\> for /f "delims=" %i in ('dir /b ^| findstr /i "txt"') do echo %i
```

12.8 Practical patterns

Logging a whole script

Run a script and capture everything it prints:

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> backup.bat > "%USERPROFILE%\BatchBook\logs\backup.log" 2>&1
```

Inside a script you can redirect an entire block (shown in `group_redirect.bat`), or send only errors to an error log with `2>> errors.log`.

Prompting with a choice

Use `choice` (Chapter 9) for single keystrokes and `set /p` for text. Always validate typed text (Chapter 8 and Chapter 23).

A quiet existence check

quiet_check.bat

```
@echo off
where /q git >nul 2>&1 && (echo git is installed) || (echo git is not installed)
```

OUTPUT

```
git is not installed
```

Summary

- Streams: 0 stdin, 1 stdout, 2 stderr. `>` replaces, `>>` appends, `<` reads, `2>&1` merges errors into output.
- Order matters: `> file 2>&1`. Put redirection first to avoid trailing spaces and digit confusion.
- Pipes link stdout to stdin; each side runs in its own `cmd.exe`.
- `&` sequences, `&&` runs on success, `||` on failure; `A && B || C` is not if-then-else.
- Escape with `^`, `%%`, quotes, and `^^!` for delayed expansion; quotes make most specials literal.
- `nul` discards output; reserved device names cannot be file names.

Common mistakes

Common Mistake: `2>&1 > file` **(wrong order).**

Common Mistake: `echo text > file` **when a trailing space would matter.**

Common Mistake: **Expecting variables set after a pipe to survive.**

Common Mistake: **Unescaped `)` in a block** or a caret inside quotes.

Common Mistake: **Space after the line-continuation caret.**

Exercises

1. **[Easy]** Write a command that saves the output of `ipconfig` to `net.txt`.
2. **[Easy]** Append the current date to `log.txt` without overwriting it.
3. **[Easy]** Hide both normal output and errors of `ping -n 1 127.0.0.1`.
4. **[Medium]** Create `results.txt` and `errors.txt` from one `dir` command that targets one real and one missing file.
5. **[Medium]** Print the text `Total: 50% (5 > 3) & more` exactly, from a batch file.
6. **[Medium]** Sort the lines of a file in reverse order using a pipe or `sort /r`.
7. **[Hard]** Explain why `echo hello | set /p "x="` does not set `x` in the calling script, and show how to capture the output of a command into a variable instead.
8. **[Hard]** Write a one-line command that prints `ok` if `findstr` finds the word `error` in `app.log` and `clean` otherwise, and explain why `&&` and `||` are safe here.

Part 2 Quiz

1. **Q1.** Which form is correct to assign a value with a space safely: `set x=a b`, `set "x=a b"`, or `set x = a b`?
2. **Q2.** What does `%~dp0` expand to?
3. **Q3.** Which of `%s:~0,3%` and `%s:~-3%` returns the last three characters?
4. **Q4.** Why does `echo %total%` inside a `for` body show an old value, and what is the fix?
5. **Q5.** What is the result of `set /a "x=17/5"` and of `if "10" lss "9"`?
6. **Q6.** In a batch file, how do you write the modulo operator in `set /a`?
7. **Q7.** `if errorlevel 2` is true for which exit codes?
8. **Q8.** Where must `else` be placed?
9. **Q9.** In a batch file, how many percent signs does a `for` variable need?
10. **Q10.** What does `for /f "tokens=1,3 delims=," %%a in (...)` assign to `%%b`?
11. **Q11.** Why must main code end with `exit /b` before the first subroutine?
12. **Q12.** Which is correct: `> log.txt 2>&1` or `2>&1 > log.txt`, to capture both streams?
13. **Q13.** How do you print a literal `&` with `echo`?
14. **Q14.** What happens to variables set on the right side of a pipe?

3 Practical Programming

Chapters 13 to 19

Files, errors, dates, menus, processes, networking and interoperability

Part 3 applies the language to real work. You will process text files, write scripts that fail safely, handle dates and times without depending on regional settings, build menus, manage processes and services, automate network diagnostics, and call PowerShell and other tools from a batch file.

Chapters in this Part

- Chapter 13: File and Folder Operations
- Chapter 14: Error Handling and Robustness
- Chapter 15: Dates, Times and Timestamps
- Chapter 16: Menus and User Interaction
- Chapter 17: Process and System Management
- Chapter 18: Networking from the Command Line
- Chapter 19: Interoperability
- Part 3 Quiz

Chapter 13: File and Folder Operations

LEARNING OBJECTIVES

- Write, append and read text files line by line, preserving blank lines
- Search text with `find` and `findstr`, including regular expressions
- Sort data and compare files
- Process multiple files and collect statistics
- Combine these tools into reusable utilities

13.1 Writing text files

You already know `>` and `>>`. For several lines, group the commands and redirect once:

write_file.bat

```
@echo off
setlocal
set "out=%~dp0settings.ini"
(
    echo [General]
    echo Name=Demo
    echo Debug=0
    echo(
    echo [Paths]
    echo Output=%~dp0output
) > "%out%"
type "%out%"
del "%out%"
```

OUTPUT

```
[General]
Name=Demo
Debug=0

[Paths]
Output=C:\Users\Alex\BatchBook\scripts\output
```

`echo(` writes an empty line safely. To write the special characters `& | < >` literally, escape them with a caret (Chapter 12). For arbitrary text that you cannot control, write the file with PowerShell instead (Chapter 19).

13.2 Reading text files

Method	Best for
`type file`	Show or pipe a whole file

Method	Best for
`more +n file`	Skip the first n lines when displaying
`set /p "x=" < file`	The first line only
`for /f`	Line-by-line processing (Chapter 10)
`findstr /n "^" file`	Number every line, **including blank ones**

Preserving blank lines

`for /f` skips empty lines (Chapter 10). To keep them, number every line with `findstr /n "^"` and strip the `number:` prefix:

keep_blanks.bat

```
@echo off
setlocal EnableDelayedExpansion
set "f=%~dp0poem.txt"
(echo Line one& echo.& echo Line three) > "%f%"
for /f "usebackq delims=" %%a in (`findstr /n "^" "%f%"`) do (
    set "line=%%a"
    set "line=!line:*:=!"
    echo [!line!]
)
del "%f%"
```

OUTPUT

```
[Line one]
[]
[Line three]
```

`findstr /n "^"` prefixes each line with its number and a colon. The replacement `!line:*:=!` deletes everything up to and including the **first** colon, which removes the prefix. Delayed expansion is used so that the contents are not parsed as syntax, but lines containing `!` will be damaged (Chapter 7).

Counting lines, words and size

file_stats.bat

```
@echo off
setlocal
set "f=%~1"
if not exist "%f%" (echo Usage: %~nx0 file & exit /b 1)
for /f %%n in ('find /c /v "" ^< "%f%"') do set "lines=%%n"
echo File  : %~nx1
echo Size  : %~z1 bytes
echo Lines : %lines%
```

OUTPUT

```
File  : notes.txt
Size  : 120 bytes
Lines : 4
```

13.3 Searching with `find`

`find` is the simple tool. It searches for a **literal** string (always in double quotes).

Switch	Meaning
`/i`	Ignore case
`/v`	Show lines that do **not** contain the string
`/c`	Print only the **count** of matching lines
`/n`	Prefix matches with line numbers

find_demo.bat

```
@echo off
setlocal
set "f=%~dp0app.log"
(
    echo 08:00 INFO  Service started
    echo 08:05 ERROR Disk full
    echo 08:06 info  Retrying
    echo 08:07 ERROR Disk full
) > "%f%"
echo --- case-insensitive "error" with numbers ---
find /i /n "error" "%f%"
echo --- count of lines containing ERROR ---
find /c "ERROR" "%f%"
echo --- lines without ERROR ---
find /v "ERROR" "%f%"
del "%f%"
```

OUTPUT

```
--- case-insensitive "error" with numbers ---

---------- C:\USERS\ALEX\BATCHBOOK\SCRIPTS\APP.LOG
[2]08:05 ERROR Disk full
[4]08:07 ERROR Disk full
--- count of lines containing ERROR ---

---------- C:\USERS\ALEX\BATCHBOOK\SCRIPTS\APP.LOG: 2
--- lines without ERROR ---

---------- C:\USERS\ALEX\BATCHBOOK\SCRIPTS\APP.LOG
08:00 INFO  Service started
08:06 info  Retrying
```

The header line with the file name appears when `find` is given a file name. When it reads from a pipe or from `<` redirection there is no header, which is why scripts prefer `find /c /v "" < file`.

13.4 Searching with `findstr`

`findstr` is the powerful tool: regular expressions, multiple search terms, recursion and more.

Switch	Meaning
`/i`	Ignore case
`/v`	Print non-matching lines
`/n`	Line numbers
`/m`	Print only the **names** of files that contain a match
`/c:"text"`	Search for the exact phrase (spaces included)
`/l`	Treat search text as literal
`/r`	Treat search text as a regular expression (the default when it contains metacharacters)
`/b`	Match at the beginning of the line
`/e`	Match at the end of the line
`/x`	The whole line must match
`/s`	Search subfolders
`/g:file`	Read search strings from a file
`/a:xx`	Colour attributes for output

Space means OR: Without `/c:`, a search string containing spaces is split into **separate words, and a line matches if it contains any of them**. `findstr "disk full" f.log` finds lines with `disk` or `full`. Use `findstr /c:"disk full" f.log` for the phrase.

Regular expressions in `findstr`

`findstr` supports a limited dialect:

Pattern	Meaning
`.`	Any single character
`*`	Zero or more of the previous character or class
`^` **and** `$`	Start and end of line
`[abc]` **and** `[^abc]`	One of those characters; none of those
`[a-z]`	A range
`\.` **and** `\\`	A literal dot, a literal backslash (any metacharacter is escaped with a backslash)
`\<` **and** `\>`	Start and end of a word

There is **no** `+`, `?`, `{n}` or `|`. Write "one or more digits" as `[0-9][0-9]*`.

findstr_demo.bat

```
@echo off
setlocal
```

```
set "f=%~dp0data.txt"
(
    echo 2026-10-23 ready
    echo order 4711 shipped
    echo host 192.168.1.20 up
    echo note: nothing here
    echo Disk Full on drive C
) > "%f%"

echo --- lines starting with a digit ---
findstr /r "^[0-9]" "%f%"
echo --- lines with a number of at least 3 digits ---
findstr /r "[0-9][0-9][0-9]" "%f%"
echo --- something that looks like an IPv4 address ---
findstr /r "[0-9][0-9]*\.[0-9][0-9]*\.[0-9][0-9]*\.[0-9][0-9]*" "%f%"
echo --- phrase, any case ---
findstr /i /c:"disk full" "%f%"
echo --- whole word "up" ---
findstr /r "\<up\>" "%f%"
del "%f%"
```

OUTPUT

```
--- lines starting with a digit ---
2026-10-23 ready
--- lines with a number of at least 3 digits ---
2026-10-23 ready
order 4711 shipped
host 192.168.1.20 up
--- something that looks like an IPv4 address ---
host 192.168.1.20 up
--- phrase, any case ---
Disk Full on drive C
--- whole word "up" ---
host 192.168.1.20 up
```

Searching many files

todo_scan.bat

```
@echo off
setlocal
cd /d "%~dp0"
echo Files that contain TODO:
findstr /s /i /m "TODO" *.bat
echo.
echo Matching lines:
findstr /s /i /n "TODO" *.bat
```

OUTPUT

```
Files that contain TODO:
tools\cleanup.bat
tools\report.bat

Matching lines:
tools\cleanup.bat:14:rem TODO add a dry-run switch
tools\report.bat:3:rem TODO read the folder from a config file
```

Findstr cautions

Common Mistake: Relying on `findstr` with Unicode files. `findstr` cannot read UTF-16 text (the default of some Windows tools). It may report no match or treat the file as binary.

Common Mistake: Forgetting that a quote or backslash in the search text needs care. Use `/c:` and `/l` for literal text containing special characters, such as `findstr /l /c:"C:\Temp" file`.

13.5 Sorting

`sort` sorts lines as **text**. Useful switches: `/r` reverse, `/+n` start comparing at column n, `/o file` write to a file.

sort_demo.bat

```
@echo off
setlocal
set "f=%~dp0nums.txt"
(echo 10& echo 9& echo 100& echo 2) > "%f%"
echo --- as text ---
sort < "%f%"
echo --- numbers padded to 4 digits sort correctly ---
(for /f "usebackq" %%n in ("%f%") do (
    set "p=000%%n"
    call echo %%p:~-4%%
)) | sort
del "%f%"
```

OUTPUT

```
--- as text ---
10
100
2
9
--- numbers padded to 4 digits sort correctly ---
0002
0009
0010
0100
```

Text order puts `10` and `100` before `2`. To sort numbers, **pad them with leading zeros** to a fixed width. Sort order for letters follows the system locale.

13.6 Comparing files

`fc` compares two files line by line; `/b` compares binary content; `/n` shows line numbers; `/c` ignores case; `/w` compresses white space. The exit code is **0 if identical** and **1 if different**.

compare_files.bat

```
@echo off
setlocal
cd /d "%~dp0"
(echo one& echo two& echo three) > a.txt
(echo one& echo 2& echo three) > b.txt
fc a.txt b.txt
if errorlevel 1 (echo Files differ) else (echo Files are identical)
fc a.txt a.txt >nul && echo A file equals itself
del a.txt b.txt
```

OUTPUT

```
Comparing files a.txt and b.txt
***** a.txt
one
two
three
***** b.txt
one
2
three
*****

Files differ
A file equals itself
```

For cryptographic comparison use `certutil -hashfile file SHA256`; it prints the hash and works on any file, which also proves integrity of copies (Chapter 23).

13.7 Working with many files

Finding files by name or date

`dir /s /b pattern` lists matches, `dir /o:d` sorts by date, and `forfiles` filters by age:

Command Prompt

```
C:\> dir /s /b "%USERPROFILE%\BatchBook\*.txt"
C:\> forfiles /p "%USERPROFILE%\BatchBook\work" /s /m *.tmp /d -7 /c "cmd /c echo @path"
```

`forfiles /d -7` selects files last modified **more than seven days ago**. `@path` and `@file` are replaced by the file's full path and name. Chapter 32 uses it in a cleanup tool.

Combining files

Command Prompt

```
C:\> copy /b part1.txt + part2.txt all.txt
C:\> type part1.txt part2.txt > all.txt
```

A realistic utility: extension report

ext_report.bat

```
@echo off
setlocal EnableDelayedExpansion
set "root=%~1"
if not defined root set "root=%CD%"
if not exist "%root%\" (echo Folder not found: %root% & exit /b 1)

for /r "%root%" %%f in (*) do (
    set "ext=%%~xf"
    if not defined ext set "ext=.none"
    set "key=cnt!ext:.=_!"
    if not defined !key! set /a "!key!=0"
    set /a "!key!+=1"
)
echo Extensions found under %root%:
for /f "tokens=1,2 delims==" %%a in ('set cnt 2^>nul') do echo   %%a = %%b
```

OUTPUT

```
Extensions found under C:\Users\Alex\BatchBook\work:
  cnt_bak = 3
  cnt_txt = 7
```

The script builds one variable per extension (`cnt_txt`, `cnt_bak`) and counts with `set /a`. The final loop lists all variables whose names start with `cnt`, which is a simple way to emulate a dictionary (Chapter 20). It is meant for ordinary extensions: an extension with unusual characters would produce an invalid variable name, and file names containing `!` would be damaged by delayed expansion (Chapter 7).

Summary

- Write files with `>`, `>>` and grouped blocks; use `echo(` for blank lines.
- Read with `type`, `set /p`, and `for /f`; use `findstr /n "^"` to keep blank lines.
- `find` is literal and simple; `findstr` supports a limited regular-expression dialect, multiple words (OR), recursion and file lists.
- `sort` is textual; pad numbers. `fc` returns 0 for identical files.
- `forfiles`, `dir /s /b` and `for /r` process many files.

Common mistakes

Common Mistake: Assuming a space in a `findstr` string means a phrase. It means OR; use `/c:`.

Common Mistake: Using `+` or `?` in `findstr` patterns. They are not supported.

Common Mistake: **Sorting numbers as text.**

Common Mistake: **Reading Unicode files with `find` / `findstr` .**

Exercises

1. **[Easy]** Write a file `names.txt` with three names, then display it.
2. **[Easy]** Count the lines in `names.txt` without `find` showing a file header.
3. **[Easy]** Search `names.txt` for a name, ignoring case.
4. **[Medium]** List all `.log` files in a folder tree that contain the word `error`, names only.
5. **[Medium]** Use `findstr` to print lines that begin with a capital letter followed by digits.
6. **[Medium]** Sort the lines of a file in reverse order and write the result to `sorted.txt`.
7. **[Hard]** Write a script that compares two files and prints "same" or "different" without printing the differences.
8. **[Hard]** Read a file preserving blank lines and print each line with its line number (without the colon format that `findstr` adds).

Chapter 14: Error Handling and Robustness

LEARNING OBJECTIVES

- Check exit codes after commands that matter, including `robocopy` 's bit-mask codes
- Validate arguments and input before acting on them
- Detect administrator rights and required tools
- Delete and overwrite data only behind safety checks
- Log to a file and exit with meaningful codes

A batch file does not stop when a command fails. Robust scripting means **deciding**, at each important step, what failure looks like and what to do about it.

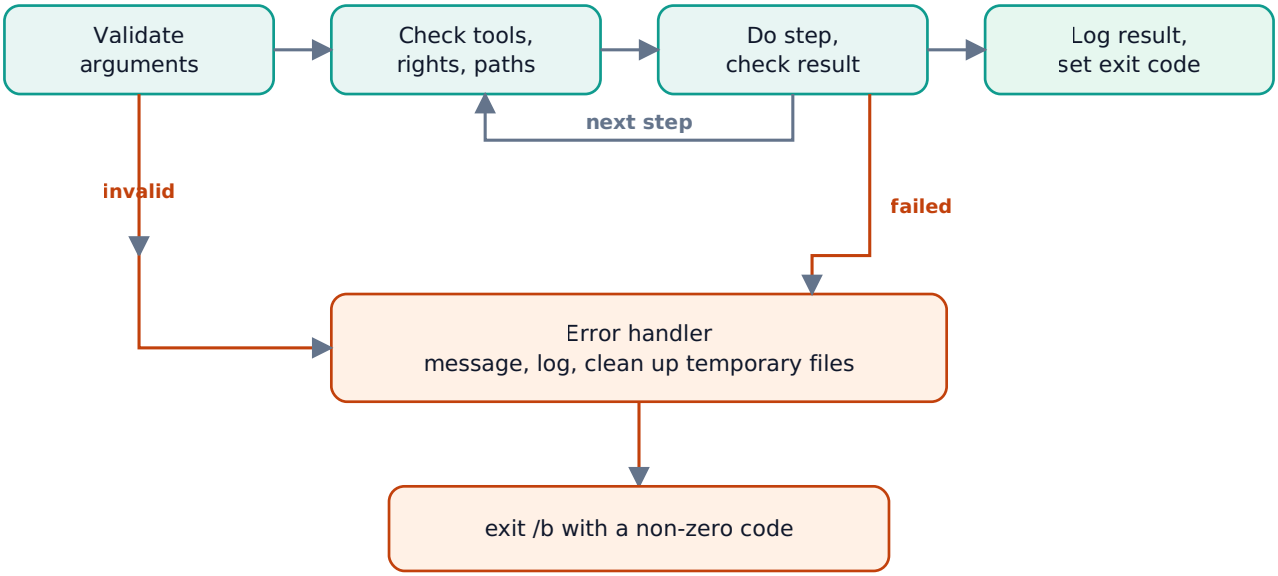


Figure 14.1. A robust script validates first, performs each step with a check, logs the result and exits with a code that tells the caller what happened.

14.1 Exit codes as a contract

By convention **0 means success** and non-zero means failure. A script should also *return* meaningful codes, so that callers (other scripts, schedulers, build systems) can react. Define them once, at the top:

exit_codes.bat

```
@echo off
setlocal
set "EXIT_OK=0"
set "EXIT_USAGE=2"
set "EXIT_MISSING=3"
set "EXIT_FAILED=4"
```

```
if "%~1"=="" (
    echo Usage: %~nx0 file
    exit /b %EXIT_USAGE%
)
if not exist "%~1" (
    echo File not found: "%~1"
    exit /b %EXIT_MISSING%
)
echo File exists: %~1
exit /b %EXIT_OK%
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> exit_codes.bat nofile.txt
C:\Users\Alex\BatchBook\scripts> echo Exit code: %errorlevel%
```

OUTPUT

```
File not found: "nofile.txt"
Exit code: 3
```

14.2 Checking commands

The pattern

check_pattern.bat

```
@echo off
setlocal
md "%~dp0work_area" 2>nul
if errorlevel 1 if not exist "%~dp0work_area\" (
    echo Could not create the work area.
    exit /b 1
)
echo Work area ready.
rd "%~dp0work_area"
```

OUTPUT

```
Work area ready.
```

`md` fails if the folder already exists, so the script tests **both** the exit code and the real condition it needs ("the folder exists"). Check the **outcome** you require, not only the exit code.

Tools with unusual exit codes

Tool	Success values	Notes
Most programs	0	Non-zero is failure
`robocopy`	0 to 7	8 or more is failure (Chapter 3)

Tool	Success values	Notes
`findstr`, `find`, `fc`	0 found/identical	1 not found/different; 2 or more is an error
`ping`	0	**Can be 0 even when the host did not answer** (for example a "Destination host unreachable" reply from a router). Search the output for `TTL=`
`choice`	The number of the key chosen	Not an error indicator

robocopy_check.bat

```
@echo off
setlocal
set "SRC=%~dp0src_demo"
set "DST=%~dp0dst_demo"
md "%SRC%" 2>nul
echo data> "%SRC%\file.txt"

robocopy "%SRC%" "%DST%" /e /r:1 /w:1 /njh /njs /np /nfl /ndl >nul
set "rc=%errorlevel%"
if %rc% geq 8 (
    echo Copy FAILED with code %rc%
) else (
    echo Copy succeeded, code %rc%
)

rd /s /q "%SRC%" "%DST%"
```

OUTPUT

```
Copy succeeded, code 1
```

Capture `%errorlevel%` **immediately** into a variable (`rc`), because the next command resets it.

14.3 Validating input

Never trust arguments, file contents or typed answers. Check **presence**, **type** and **content**:

validate_input.bat

```
@echo off
setlocal EnableDelayedExpansion
set "name=%~1"
set "count=%~2"

if not defined name (echo Usage: %~nx0 name count & exit /b 2)
if not defined count (echo Usage: %~nx0 name count & exit /b 2)

rem Allow only letters, digits, dash and underscore in the name.
echo(!name!| findstr /r /x "[A-Za-z0-9_-][A-Za-z0-9_-]*" >nul
if errorlevel 1 (echo Invalid name: only letters, digits, - and _ allowed & exit /b 2)

rem The count must be a whole number between 1 and 100.
echo(!count!| findstr /r /x "[0-9][0-9]*" >nul
if errorlevel 1 (echo Invalid count: not a number & exit /b 2)
if %count% lss 1 (echo Invalid count: too small & exit /b 2)
```

```
if %count% gtr 100 (echo Invalid count: too large & exit /b 2)

echo Valid: name=%name% count=%count%
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> validate_input.bat report_1 25
C:\Users\Alex\BatchBook\scripts> validate_input.bat "a&b" 3
```

OUTPUT

```
Valid: name=report_1 count=25
Invalid name: only letters, digits, - and _ allowed
```

The text is read with `!name!` (delayed expansion) rather than `%name%`, so that characters such as `&` inside it cannot be interpreted as commands when the check runs (Chapter 23). The approach is an **allow-list**: define exactly the characters that are acceptable and reject everything else. This is far safer than trying to remove dangerous characters (Chapter 23).

14.4 Checking the environment

Is a tool available?

check_tools.bat

```
@echo off
setlocal
set "missing=0"
for %%t in (robocopy findstr tasklist powershell) do (
    where /q %%t
    if errorlevel 1 (
        echo Required tool not found: %%t
        set "missing=1"
    )
)
if "%missing%"=="1" exit /b 3
echo All required tools are available.
```

OUTPUT

```
All required tools are available.
```

Running as administrator?

Tasks such as changing services or writing to `C:\Windows` need elevation. The command `fltmc` succeeds only for administrators and is quick:

check_admin.bat

```
@echo off
setlocal
call :is_admin
if errorlevel 1 (
    echo This script must be run as administrator.
    echo Right-click it and choose "Run as administrator".
```

```
    exit /b 5
)
echo Running with administrator rights.
exit /b 0

:is_admin
rem Returns 0 for an elevated session, 1 otherwise.
fltmc >nul 2>&1
exit /b %errorlevel%
```

```
OUTPUT
This script must be run as administrator.
Right-click it and choose "Run as administrator".
```

(The sample shows an ordinary window; an elevated window prints `Running with administrator rights.`) An alternative test is `whoami /groups | find "S-1-16-12288" >nul`, which looks for the High Mandatory Level label of an elevated token. Older examples use `net session`, which needs the Server service and is less reliable.

> **Elevation:** A user who belongs to the Administrators group still runs with a **standard** token unless the process was elevated through User Account Control. Both tests above check the elevated state, not group membership. Chapter 20 shows a script that elevates itself.

14.5 Safe deletion and overwriting

Destructive commands deserve a checklist. A safe delete routine confirms that the path **is not empty, is not a drive root, is not a system location, exists, and that the user agreed**:

safe_delete.bat

```
@echo off
setlocal EnableExtensions
set "TARGET=%~1"
set "DRYRUN=1"
if /i "%~2"=="/doit" set "DRYRUN=0"

rem 1. A target is required.
if not defined TARGET (echo Usage: %~nx0 folder [/doit] & exit /b 2)

rem 2. Resolve to a full path.
for %%f in ("%TARGET%") do set "FULL=%%~ff"

rem 3. Never allow a drive root or a system folder.
if "%FULL:~3%"=="" (echo Refusing to touch a drive root: "%FULL%" & exit /b 3)
for %%p in ("%windir%" "%ProgramFiles%" "%USERPROFILE%") do (
    if /i "%FULL%"=="%%~p" (echo Refusing protected folder: "%FULL%" & exit /b 3)
)

rem 4. It must exist and be a folder.
if not exist "%FULL%\" (echo Folder not found: "%FULL%" & exit /b 1)

rem 5. Show what would be removed.
echo Target: %FULL%
dir /s /b "%FULL%" | find /c /v ""
if "%DRYRUN%"=="1" (
    echo Dry run only. Add /doit to delete.
    exit /b 0
```

```
)

rem 6. Ask for confirmation.
choice /c YN /n /m "Delete this folder and everything in it? (Y/N) "
if errorlevel 2 (echo Cancelled. & exit /b 0)

rd /s /q "%FULL%"
if exist "%FULL%\" (echo Delete failed. & exit /b 4)
echo Deleted.
exit /b 0
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> safe_delete.bat ..\work\old_data
C:\Users\Alex\BatchBook\scripts> safe_delete.bat C:\
```

OUTPUT

```
Target: C:\Users\Alex\BatchBook\work\old_data
14
Dry run only. Add /doit to delete.
Refusing to touch a drive root: "C:\"
```

Four design decisions are worth copying: **dry run by default**; **explicit switch** to act; **protection list**; **confirmation** before the irreversible step.

> **Warning:** `rd /s /q` is permanent. The checks above reduce the risk but cannot remove it. Test with copies. Never point such a script at data you cannot restore.

14.6 Logging

A log answers "what happened?" after an unattended run. Log **start, each key action, each failure, the exit code, and the end**, each with a timestamp:

logging.bat

```
@echo off
setlocal
set "LOG=%~dp0job.log"
call :log "Job started"

robocopy "%~dp0" "%~dp0nowhere_demo" /l /njh /njs /np /nfl /ndl >nul
call :log "robocopy list-only run returned %errorlevel%"

call :log "Job finished"
type "%LOG%"
del "%LOG%"
exit /b 0

:log  message
>> "%LOG%" echo %date% %time% - %~1
exit /b 0
```

OUTPUT

```
Fri 10/23/2026  9:55:02.31 - Job started
Fri 10/23/2026  9:55:02.45 - robocopy list-only run returned 1
Fri 10/23/2026  9:55:02.46 - Job finished
```

Send errors to the same log with `2>> "%LOG%"`. Chapter 15 supplies sortable, locale-independent timestamps.

14.7 Defensive habits

Best Practice: 1. Begin with `@echo off` and `setlocal EnableExtensions`. 2. Quote every path and every `%variable%` that holds a path or text. 3. Validate arguments, then **fail early** with a usage message and a non-zero code. 4. Check each command whose failure matters; store `%errorlevel%` at once. 5. Prefer `exit /b n` over `exit`. 6. Write dry-run modes for destructive tasks. 7. Do not assume the current directory; use `%~dp0` or `pushd`. 8. Clean up temporary files in one place, including on error exits. 9. Log with timestamps; keep logs bounded (delete old ones). 10. Test failure paths deliberately, not just success.

A central failure routine

die_routine.bat

```
@echo off
setlocal
set "W=%~dp0w_demo"

md "%W%" || goto :fail
echo ok: created the work folder
echo data> "%W%\a.txt" || goto :fail
echo ok: wrote a file
copy "%W%\a.txt" "%W%\b.txt" >nul || goto :fail
echo ok: copied the file
type "%W%\missing.txt" >nul 2>&1 || goto :fail
echo ok: read the missing file
call :cleanup
exit /b 0

:fail
set "rc=%errorlevel%"
echo FAILED: a step returned a non-zero code. Aborting.
call :cleanup
exit /b 1

:cleanup
rem Cleanup lives in one place and is safe to call twice.
if exist "%W%\" rd /s /q "%W%"
exit /b 0
```

OUTPUT

```
ok: created the work folder
ok: wrote a file
ok: copied the file
FAILED: a step returned a non-zero code. Aborting.
```

Each step is followed by `|| goto :fail`, so the first failing command jumps to the failure handler. Cleanup is written once and called from both the success and the failure path. The `rd /s /q` here is aimed at a folder the script itself created from a fixed name.

Summary

- Exit codes are a contract: 0 success, non-zero failure; define named codes.
- Check outcomes, capture `%errorlevel%` at once, and remember `robocopy`, `ping` and `findstr` conventions.
- Validate with allow-lists. Test for tools with `where /q` and for elevation with `fltmc`.
- Destructive scripts: dry run by default, explicit switch, protected paths, confirmation.
- Log with timestamps and finish with `exit /b`.

Common mistakes

Common Mistake: **Reading `%errorlevel%` after another command has already replaced it.**

Common Mistake: **Treating robocopy codes 1 to 7 as errors.**

Common Mistake: **Cleaning input by deleting bad characters instead of rejecting them.**

Common Mistake: **Using `rd /s /q "%var%"` where `var` may be empty.**

Common Mistake: **Assuming a failing command stops the script.**

Exercises

1. **[Easy]** Write a script that exits with code 1 if no argument is given and prints usage.
2. **[Easy]** Print the exit code of `ping -n 1 127.0.0.1` right after running it.
3. **[Easy]** Test whether `git` is on the PATH using `where /q`.
4. **[Medium]** Validate that an argument is a whole number from 1 to 12 (a month).
5. **[Medium]** Write a script that runs `robocopy` and reports "OK", "OK with extras" (code 2 or 3 or more but below 8) or "FAILED" using the correct ranges.
6. **[Medium]** Write `:log` that appends messages to a file and call it three times.
7. **[Hard]** Extend `safe_delete.bat` so it also refuses any path shorter than 10 characters.

8. **[Hard]** Write a function `:ping_ok host` that returns 0 only if the output contains `TTL=`, and explain why checking the exit code of `ping` alone is insufficient.

Chapter 15: Working with Dates, Times and Timestamps

LEARNING OBJECTIVES

- Explain why `%DATE%` and `%TIME%` are not portable
- Create sortable, locale-independent timestamps for file names and logs
- Measure elapsed time at centisecond resolution without octal errors
- Perform date arithmetic and compute file ages with supported tools
- Choose between pure Batch and PowerShell for date work

15.1 `%DATE%` and `%TIME%`

`%DATE%` and `%TIME%` return text formatted according to the **regional settings of the user**. That makes them convenient for display and dangerous for parsing.

show_datetime.bat

```
@echo off
echo DATE = [%DATE%]
echo TIME = [%TIME%]
date /t
time /t
```

OUTPUT

```
DATE = [Fri 10/23/2026]
TIME = [ 9:55:02.31]
Fri 10/23/2026
09:55 AM
```

The sample shows a US English system. Other systems produce, for example, `23.10.2026`, `23/10/2026` or `2026-10-23`, with or without a weekday name, and the time may use a comma before the centiseconds (`9:55:02,31`). Two properties of `%TIME%` matter in every locale:

- For hours before 10 the hour is **space-padded**, not zero-padded: `9:55:02.31`.
- The format is 24-hour, with centiseconds after a decimal separator that may be a period or a comma.

Common Mistake: Cutting `%DATE%` into pieces with fixed positions. `%DATE:~-4%` for the year, `%DATE:~4,2%` for the month and similar tricks work on one locale and silently produce wrong file names on another. A backup named with the wrong month is worse than no backup.

Padding the hour

`%TIME: =0%` replaces every space with a zero and gives a safe 24-hour value:

time_pad.bat

```
@echo off
set "t=%TIME: =0%"
echo Padded time: %t%
echo Hour: %t:~0,2%  Minute: %t:~3,2%  Second: %t:~6,2%
```

OUTPUT

```
Padded time: 09:55:02.31
Hour: 09  Minute: 55  Second: 02
```

The positions of the **time** are stable across locales (hours, minutes, seconds, centiseconds), unlike the date.

15.2 Locale-independent timestamps

The robust way to obtain the date and time as fixed-format text is to ask a tool that does not depend on regional settings. PowerShell's `Get-Date -Format` is available on every supported Windows version:

timestamp.bat

```
@echo off
setlocal
call :timestamp TS
echo Timestamp: %TS%
echo Backup name: backup_%TS%.zip
exit /b 0

:timestamp  outvar
rem Returns yyyyMMdd_HHmmss, independent of regional settings.
setlocal
for /f "delims=" %%t in ('powershell -NoProfile -Command "Get-Date -Format yyyyMMdd_HHmmss"') do set "ts=%%t"
endlocal & set "%~1=%ts%"
exit /b 0
```

OUTPUT

```
Timestamp: 20261023_095502
Backup name: backup_20261023_095502.zip
```

Names in the form `yyyyMMdd_HHmmss` sort alphabetically in chronological order, which makes retention and "newest file" logic trivial.

> **Best Practice:** Use PowerShell once, at the start of a script, to compute the timestamp, and reuse the variable. Starting PowerShell costs a fraction of a second, which matters inside a loop (Chapter 21).

> **WMIC is gone from current Windows 11:** Older scripts obtain a fixed-format date with `wmic os get localdatetime` . **WMIC is deprecated.** It has been removed from supported Windows 11 releases, and Windows 10 still includes it for now. Do not use it in new scripts. The PowerShell commands in this chapter work everywhere.

Other useful formats

Need	PowerShell format string	Example result
Date only	`yyyy-MM-dd`	`2026-10-23`
Date and time for logs	`yyyy-MM-dd HH:mm:ss`	`2026-10-23 09:55:02`
Year and month	`yyyyMM`	`202610`
Day of week name	`dddd`	`Friday` (language of the system)
Day of week number	`[int](Get-Date).DayOfWeek`	`5` (Sunday is 0)

15.3 Elapsed time

To time a script, record the time before and after and subtract. Convert `%TIME%` to **centiseconds since midnight**. Beware the **octal trap** (Chapter 8): `08` and `09` are invalid numbers, so each part is prefixed with `1` and then 100 is subtracted:

timer.bat

```
@echo off
setlocal
call :to_cs start "%TIME%"

rem ---- the work being measured ----
ping -n 3 127.0.0.1 >nul
rem ---------------------------------

call :to_cs finish "%TIME%"
set /a "elapsed=finish-start"
if %elapsed% lss 0 set /a "elapsed+=8640000"

set /a "secs=elapsed/100, cs=elapsed%%100"
if %cs% lss 10 set "cs=0%cs%"
echo Elapsed: %secs%.%cs% seconds
exit /b 0

:to_cs  outvar time
setlocal
set "t=%~2"
set "t=%t: =0%"
set /a "cs=(((1%t:~0,2%-100)*60+1%t:~3,2%-100)*60+1%t:~6,2%-100)*100+1%t:~9,2%-100"
endlocal & set "%~1=%cs%"
exit /b 0
```

OUTPUT

```
Elapsed: 2.03 seconds
```

Key points: the value `8640000` is the number of centiseconds in a day and handles a run that crosses midnight; the expression `1%t:~0,2%-100` turns `09` into `109-100`, which is 9; and `%%` is the modulo operator in a batch file. This measures runs shorter than a day. `ping -n 3` takes about two seconds because it waits one second between packets.

15.4 Date arithmetic

Batch has no date type. Pure-Batch date arithmetic (leap years, month lengths) is possible but long and error-prone. For anything beyond formatting, delegate to PowerShell:

date_math.bat

```
@echo off
setlocal
call :ps_date YESTERDAY "(Get-Date).AddDays(-1).ToString('yyyy-MM-dd')"
call :ps_date IN30 "(Get-Date).AddDays(30).ToString('yyyy-MM-dd')"
call :ps_date DOW "(Get-Date).DayOfWeek"
echo Yesterday : %YESTERDAY%
echo In 30 days: %IN30%
echo Weekday   : %DOW%
exit /b 0

:ps_date  outvar expression
setlocal
for /f "usebackq delims=" %%d in (`powershell -NoProfile -Command "%~2"`) do set "v=%%d"
endlocal & set "%~1=%v%"
exit /b 0
```

OUTPUT

```
Yesterday : 2026-10-22
In 30 days: 2026-11-22
Weekday   : Friday
```

Warning: The helper uses `usebackq` so that the single quotes inside the PowerShell text do not end the command. Do not pass **untrusted text** as the expression of a helper such as `:ps_date`. The text becomes PowerShell code. In this chapter the expressions are fixed strings written by the script author.

Days between two dates

days_between.bat

```
@echo off
setlocal
set "d1=2026-01-01"
set "d2=2026-10-23"
for /f "usebackq delims=" %%n in (`powershell -NoProfile -Command "([datetime]'%d2%' - [datetime]'%d1%').Days"`)
do set "days=%%n"
echo Days from %d1% to %d2%: %days%
```

OUTPUT

```
Days from 2026-01-01 to 2026-10-23: 295
```

15.5 File times and file age

Showing a file's date

`%%~tf` (Chapter 6) returns the last-write time as text in the **locale format**. For comparing ages do not parse it; use a tool that filters by age:

Task	Command
Files older than 7 days	`forfiles /p "folder" /s /m *.log /d -7 /c "cmd /c echo @path"`
Files newer than a date	`xcopy /d:mm-dd-yyyy` (a copy filter) or `robocopy /maxage:n`
Newest file in a folder	`dir /b /o:-d` (first line)

newest_file.bat

```
@echo off
setlocal
cd /d "%~dp0"
for /f "delims=" %%f in ('dir /b /a:-d /o:-d 2^>nul') do (
    echo Newest file: %%f
    goto :done
)
echo The folder contains no files.
:done
```

OUTPUT

```
Newest file: newest_file.bat
```

`dir /o:-d` sorts newest first; the loop stops at the first entry using `goto`.

Deleting old files by age

The `forfiles` filter `/d -N` selects files last modified more than N days ago. A **dry-run first** version, only listing the candidates, is the safe way to use it:

Command Prompt

```
C:\> forfiles /p "%USERPROFILE%\BatchBook\logs" /m *.log /d -30 /c "cmd /c echo Would delete @path"
```

If the folder contains no matching file, `forfiles` reports `ERROR: No files found with the specified search criteria.` Chapter 32 turns this into a complete cleanup tool.

15.6 Waiting and scheduling

Technique	Notes
`timeout /t 10 /nobreak`	Waits; a key press ends the wait unless `/nobreak` is used. Fails when input is redirected

Technique	Notes
`ping -n 11 127.0.0.1 >nul`	Older sleep trick: about 10 seconds. Works when `timeout` does not (redirected input)
`schtasks`	Run something at a time later (Chapter 17)
`waitfor /t 10 signalname`	Waits for a named signal or times out

Common Mistake: Using `timeout` in a script whose input is redirected or run from a scheduler without a console. It stops with `ERROR: Input redirection is not supported, exiting the process immediately.` Use the `ping` trick or PowerShell `Start-Sleep` there.

Summary

- `%DATE%` and `%TIME%` depend on regional settings; never rely on positions in `%DATE%`.
- Use `%TIME: =0%` to pad the hour; positions within the time are stable.
- Obtain fixed-format timestamps with PowerShell and name files `yyyyMMdd_HHmmss` so they sort.
- Convert times with the `1xx-100` trick to avoid octal errors; add one day's centiseconds for midnight rollover.
- Delegate date arithmetic to PowerShell; use `forfiles /d` and `robocopy /maxage` for file ages.

Common mistakes

Common Mistake: Parsing `%DATE%` by position.

Common Mistake: Feeding `08` or `09` to `set /a`.

Common Mistake: Forgetting the leading space in `%TIME%` when building file names (it produces names with spaces).

Common Mistake: Passing untrusted text into PowerShell expressions.

Exercises

1. **[Easy]** Print today's date and the current time in your locale using `%DATE%` and `%TIME%`.
2. **[Easy]** Produce a zero-padded 24-hour `HHMMSS` string using only Batch.
3. **[Easy]** Create a variable `TS` containing `yyyyMMdd_HHmmss` using PowerShell.

4. **[Medium]** Write a script that creates `log_<timestamp>.txt` in the current folder.
5. **[Medium]** Measure how long `dir /s %windir% >nul` takes in seconds.
6. **[Medium]** Compute the date 14 days ago as `yyyy-MM-dd`.
7. **[Hard]** Write a function `:days_between d1 d2` that returns the number of days using PowerShell.
8. **[Hard]** Explain why `set /a "h=%TIME:~0,2%"` fails at 8 or 9 o'clock in the morning, and give two fixes.

Chapter 16: Menus and User Interaction

LEARNING OBJECTIVES

- Build menus with `choice` and with `set /p`, and validate every answer
- Structure an interactive script as a loop around a menu and a set of actions
- Control the window with `title`, `cls`, `color` and `mode`
- Use ANSI escape sequences for colour on modern terminals, and know their limits
- Show progress and status in plain text

16.1 Single-key menus with `choice`

`choice` waits for **one key** from a set you define and returns its **position** in the set as the exit code (1 for the first character). There is no Enter key and no invalid-input handling to write, because `choice` rejects other keys.

Switch	Meaning
`/c chars`	The allowed keys, for example `/c YNC`
`/n`	Hide the list of keys from the prompt
`/m "text"`	The prompt text
`/t seconds` and `/d key`	After the timeout, choose default key `key` automatically
`/cs`	Make the keys case-sensitive (default is case-insensitive)

choice_menu.bat

```
@echo off
setlocal
:menu
cls
echo ============================
echo   Main Menu
echo ============================
echo   1. Show date and time
echo   2. Show current folder
echo   3. List folder contents
echo   Q. Quit
echo ============================
choice /c 123Q /n /m "Select an option: "

rem Test from the highest value down: errorlevel means "this value or higher".
if errorlevel 4 goto :quit
if errorlevel 3 goto :list
if errorlevel 2 goto :folder
if errorlevel 1 goto :datetime
goto :menu

:datetime
echo.
echo %DATE% %TIME%
```

```
pause
goto :menu

:folder
echo.
echo %CD%
pause
goto :menu

:list
echo.
dir /b
pause
goto :menu

:quit
echo Goodbye.
```

```
OUTPUT
==============================
  Main Menu
==============================
  1. Show date and time
  2. Show current folder
  3. List folder contents
  Q. Quit
==============================
Select an option: 2

C:\Users\Alex\BatchBook\scripts
Press any key to continue . . .
```

Testing from the highest value down (`errorlevel 4` , then 3, 2, 1) is essential, because `if errorlevel n` is true for **n or higher**. The final `goto :menu` is a safety net.

A timeout default

choice_timeout.bat

```
@echo off
choice /c YN /t 5 /d N /m "Install updates now? (auto No in 5 seconds) "
if errorlevel 2 (echo Skipping.) else (echo Installing.)
```

```
OUTPUT
Install updates now? (auto No in 5 seconds) [Y,N]?N
Skipping.
```

`/t 5 /d N` presses N for the user after five seconds. This is useful in scripts that should proceed unattended if nobody answers.

16.2 Typed menus with `set /p`

Use `set /p` when the answer is more than one key: a name, a number, a path. Because the user can type anything, **validate**.

setp_menu.bat

```
@echo off
setlocal

:ask
set "pick="
echo.
echo  1) Add item
echo  2) Remove item
echo  3) Exit
set /p "pick=Choose 1-3: "

if not defined pick goto :ask
if "%pick%"=="1" goto :add
if "%pick%"=="2" goto :remove
if "%pick%"=="3" goto :eof
echo "%pick%" is not a valid choice. Try again.
goto :ask

:add
echo (adding)
goto :ask

:remove
echo (removing)
goto :ask
```

OUTPUT

```
 1) Add item
 2) Remove item
 3) Exit
Choose 1-3: 9
"9" is not a valid choice. Try again.

 1) Add item
 2) Remove item
 3) Exit
Choose 1-3: 3
```

The variable is cleared before each question (`set "pick="`) because `set /p` leaves the old value if the user only presses Enter (Chapter 5). The tests compare the **whole string**, so `13` or `1 2` or `1x` are rejected.

Asking for a number in a range

ask_number.bat

```
@echo off
setlocal
call :ask_number age "Your age (0-120)" 0 120
echo You entered %age%.
exit /b 0

:ask_number  outvar prompt min max
setlocal EnableDelayedExpansion
:retry
set "ans="
set /p "ans=%~2: "
echo(!ans!| findstr /r /x "[0-9][0-9]*" >nul
if errorlevel 1 (echo Please enter a whole number. & goto :retry)
```

```
set /a "n=%ans%" 2>nul
if %n% lss %~3 (echo Too small. & goto :retry)
if %n% gtr %~4 (echo Too large. & goto :retry)
endlocal & set "%~1=%n%"
exit /b 0
```

OUTPUT

```
Your age (0-120): abc
Please enter a whole number.
Your age (0-120): 200
Too large.
Your age (0-120): 42
You entered 42.
```

The routine can be reused for any numeric prompt. It rejects letters, symbols and out-of-range numbers before the number is used in arithmetic. (An entry with a leading zero such as `08` is not valid in `set /a`; add a zero-stripping step if you accept such input.)

> **Warning:** `set /p` echoes what the user types, so it is unsuitable for passwords. Use PowerShell's `Read-Host -AsSecureString` (Chapter 19) and never store passwords in a batch file (Chapter 23).

16.3 Structuring an interactive tool

A good interactive script has this shape:

1. Set up (title, colour, variables).
2. A **main loop** that draws the menu, reads a choice and dispatches.
3. One **subroutine per action**, so the loop stays short.
4. A single **exit** route with cleanup.

toolbox_skeleton.bat

```
@echo off
setlocal EnableExtensions
title Toolbox
set "APP_VERSION=1.0"

:main_loop
cls
call :draw_menu
choice /c 1234Q /n /m "Select: "
set "sel=%errorlevel%"
if "%sel%"=="5" goto :quit
if "%sel%"=="4" call :action_about
if "%sel%"=="3" call :action_disk
if "%sel%"=="2" call :action_ip
if "%sel%"=="1" call :action_time
echo.
pause
goto :main_loop

:quit
cls
echo Thank you for using Toolbox %APP_VERSION%.
exit /b 0
```

```
:draw_menu
echo Toolbox %APP_VERSION%
echo ------------------------
echo 1. Current time
echo 2. IP configuration
echo 3. Disk free space
echo 4. About
echo Q. Quit
echo ------------------------
exit /b 0

:action_time
echo %DATE% %TIME%
exit /b 0

:action_ip
ipconfig | findstr /c:"IPv4"
exit /b 0

:action_disk
fsutil volume diskfree %SystemDrive% 2>nul | findstr /i "avail"
exit /b 0

:action_about
echo Toolbox version %APP_VERSION%, written in Batch.
exit /b 0
```

The first step stores `%errorlevel%` in `sel` immediately, because later commands would change it. `fsutil` requires administrator rights; without them the message is hidden by `2>nul` and the action prints nothing.

16.4 Window control: `title`, `cls`, `color`, `mode`

Command	Effect
`title text`	Window title
`cls`	Clear the screen
`color BF`	Background `B`, foreground `F` (hexadecimal digits); `color` alone restores defaults
`mode con cols=100 lines=30`	Console size (works in the classic console host; Windows Terminal ignores size requests)

The colour digits:

Digit	Colour	Digit	Colour
0	Black	8	Gray
1	Blue	9	Light blue
2	Green	A	Light green
3	Aqua	B	Light aqua
4	Red	C	Light red

Digit	Colour	Digit	Colour
5	Purple	D	Light purple
6	Yellow	E	Light yellow
7	White	F	Bright white

color_demo.bat

```
@echo off
color 0A
echo Green on black
timeout /t 1 /nobreak >nul
color 4F
echo White on red
timeout /t 1 /nobreak >nul
color
echo Back to the default colours.
```

OUTPUT

```
Green on black
White on red
Back to the default colours.
```

`color` changes the **entire window**. Choose combinations with strong contrast; the same colour for foreground and background makes text invisible, and `color` then returns an error.

16.5 ANSI escape sequences

Modern consoles interpret **ANSI escape sequences**, special character sequences that start with the ESC character (code 27) and set colour or move the cursor. They allow colour for individual words, which `color` cannot do.

> **Support depends on the console:** Windows Terminal supports ANSI sequences. In the classic console host the feature (called virtual terminal processing) exists since Windows 10 version 1511 but depends on how the host is configured. If you see characters such as `[32m` printed on screen, sequences are not being interpreted in that window. Test in your target environment, and always reset attributes at the end.

You cannot type the ESC character into a batch file easily, so the script obtains it once:

ansi_demo.bat

```
@echo off
setlocal
for /f %%e in ('powershell -NoProfile -Command "[char]27"') do set "ESC=%%e"

echo %ESC%[32mOK%ESC%[0m     Disk check passed
echo %ESC%[33mWARN%ESC%[0m   Disk is 85 percent full
echo %ESC%[31mFAIL%ESC%[0m   Backup did not finish
echo %ESC%[1mBold text%ESC%[0m and normal text
```

```
OUTPUT
OK     Disk check passed
WARN   Disk is 85 percent full
FAIL   Backup did not finish
Bold text and normal text
```

(In a console that interprets the sequences, `OK` appears green, `WARN` yellow, `FAIL` red and the last line has bold text; this printed book shows plain text.)

Sequence	Effect
`ESC[0m`	Reset all attributes
`ESC[1m`	Bold or bright
`ESC[31m` ... `ESC[37m`	Foreground: red, green(32), yellow(33), blue(34), magenta(35), cyan(36), white(37)
`ESC[41m` ... `ESC[47m`	Background colours
`ESC[2J`	Clear the screen
`ESC[H`	Cursor to the top left
`ESC[?25l` and `ESC[?25h`	Hide and show the cursor

Best Practice: Provide colour as a **bonus**, never the only carrier of meaning. Print `OK`, `WARN` and `FAIL` as words, as above, so that logs and plain consoles remain readable.

16.6 Showing progress

progress.bat

```
@echo off
setlocal EnableDelayedExpansion
set "bar="
<nul set /p "=Progress: ["
for /l %%i in (1,1,20) do (
    <nul set /p "=#"
    ping -n 1 -w 100 127.0.0.1 >nul
)
echo ] done
```

```
OUTPUT
Progress: [####################] done
```

`<nul set /p "=text"` prints text with no newline (Chapter 12). The `for /l` loop adds one mark per step. For real tasks print one mark after each completed unit of work.

Summary

- Use `choice` for single-key menus and test its result from the highest value down; `/t` and `/d` supply a default after a timeout.
- Use `set /p` for typed input, clear the variable first, and validate the whole answer.
- Structure interactive scripts as a loop with one subroutine per action.
- `title`, `cls`, `color` and `mode` control the window; `color` affects the entire console.
- ANSI escape sequences colour individual words on terminals that support them; test, reset, and never rely on colour alone.

Common mistakes

Common Mistake: Testing `if errorlevel 1` before `if errorlevel 3`.

Common Mistake: Not clearing the variable before `set /p`. Pressing Enter reuses the old answer.

Common Mistake: Using `set /p` for passwords.

Common Mistake: Assuming ANSI colour works everywhere.

Common Mistake: Reading `%errorlevel%` after other commands in the menu loop.

Exercises

1. **[Easy]** Write a Yes/No question with `choice` that prints "Confirmed" or "Cancelled".
2. **[Easy]** Change the window title and colours, then restore the colours.
3. **[Easy]** Print the word `Done` in green using an ANSI sequence.
4. **[Medium]** Build a three-option `set /p` menu that rejects invalid input and repeats.
5. **[Medium]** Write a `choice` prompt that selects `N` automatically after 10 seconds.
6. **[Medium]** Write a loop that prints a 30-character progress bar.
7. **[Hard]** Write a reusable routine `:ask_yes_no outvar prompt` that returns `Y` or `N` in a variable.
8. **[Hard]** Extend `toolbox_skeleton.bat` with a fifth action that lists the five largest files in the current folder using `dir /o:-s`.

Chapter 17: Process and System Management

LEARNING OBJECTIVES

- List, start and stop processes with `tasklist`, `start` and `taskkill`
- Query and control services with `sc` and `net`
- Schedule work with `schtasks` and read the registry with `reg`
- Gather system information, and replace deprecated `wmic` calls
- Use `shutdown` safely

Warning: The commands in this chapter change the state of a running system. Practise only on a test machine or virtual machine, read each command before running it, and never copy a command that you do not understand. Administrator rights are needed where stated.

17.1 Listing processes: `tasklist`

Command Prompt

```
C:\> tasklist /fi "imagename eq explorer.exe"
```

OUTPUT

```
Image Name                     PID Session Name        Session#    Mem Usage
========================= ======== ================ =========== ============
explorer.exe                  5120 Console                    1    142,388 K
```

Option	Meaning
`/fi "filter"`	Filter: `imagename eq x.exe`, `pid eq 1234`, `status eq running`, `memusage gt 100000`
`/fo csv` or `/fo list`	Output format: table (default), CSV, list
`/nh`	No header (for parsing)
`/v`	Verbose: user name, window title, CPU time
`/svc`	Show the services hosted in each process
`/m module.dll`	Processes that loaded a module

Is a process running?

is_running.bat

```
@echo off
setlocal
set "proc=%~1"
if not defined proc (echo Usage: %~nx0 process.exe & exit /b 2)
tasklist /fi "imagename eq %proc%" /nh | find /i "%proc%" >nul
if errorlevel 1 (echo %proc% is NOT running) else (echo %proc% is running)
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> is_running.bat explorer.exe
```

OUTPUT

```
explorer.exe is running
```

`tasklist` always returns 0 and prints an information message when nothing matches, so the script searches the output with `find`.

17.2 Starting programs: `start`

`start` launches a program **without waiting** by default, in a new window if it is a console program.

Form	Effect
`start "" notepad.exe file.txt`	Start an application; the **first quoted argument is the window title**
`start "" /wait program`	Wait until it ends
`start "" /min program`	Start minimized
`start "" /b program`	Run in the **same** window, in the background
`start "" "C:\path with spaces\tool.exe"`	Path in quotes needs the empty title in front
`start "" https://example.org`	Open in the default handler (browser)

Common Mistake: Forgetting the empty title. `start "C:\Program Files\Tool\tool.exe"` treats the quoted path as the **window title** and just opens a new console window. Always write `start "" "path"`.

Running a program directly (without `start`) in a script **waits** for it, which is the usual desired behavior for tools such as `robocopy`.

17.3 Stopping processes: `taskkill`

process_demo.bat

```
@echo off
setlocal
rem Start a harmless background process and note its process ID.
for /f "usebackq" %%p in (`powershell -NoProfile -Command "(Start-Process ping -ArgumentList '-
n','120','127.0.0.1' -WindowStyle Hidden -PassThru).Id"`) do set "PID_DEMO=%%p"
echo Started demo process, PID %PID_DEMO%

tasklist /fi "pid eq %PID_DEMO%" /nh
echo Stopping it...
taskkill /pid %PID_DEMO% /f
tasklist /fi "pid eq %PID_DEMO%" /nh
```

OUTPUT

```
Started demo process, PID 12044
PING.EXE                     12044 Console                    1      4,212 K
Stopping it...
SUCCESS: The process with PID 12044 has been terminated.
INFO: No tasks are running which match the specified criteria.
```

Terminating by **PID** is precise. Terminating by **image name** (`taskkill /im notepad.exe`) ends *every* process with that name, which can destroy a user's unsaved work.

Option	Meaning
`/pid n`	Process ID
`/im name`	Image name (all matches)
`/f`	Force termination (no chance to save)
`/t`	Also end child processes
`/fi "filter"`	Filter, as for `tasklist`

Best Practice: Prefer a normal close request first (`taskkill /im app.exe` without `/f`), wait a few seconds, and force only if necessary. Never kill by image name on a shared machine without checking who owns the processes.

17.4 Services: `sc` and `net`

A **service** is a background program managed by Windows.

Task	Command
State of a service	`sc query Spooler`
Configuration	`sc qc Spooler`
Start / stop	`net start Spooler` , `net stop Spooler` (admin)

Task	Command
List running services	`net start`
List services with state	`sc query type= service state= all`

(`sc` requires a **space after the equals sign** in its own options.) In PowerShell `sc` is an alias of a different command, so always type `sc.exe` there; in `cmd.exe` plain `sc` is correct.

service_check.bat

```
@echo off
setlocal
set "SVC=%~1"
if not defined SVC set "SVC=Spooler"

sc query "%SVC%" >nul 2>&1
if errorlevel 1060 (echo Service "%SVC%" does not exist. & exit /b 1)

sc query "%SVC%" | find "RUNNING" >nul
if not errorlevel 1 (echo %SVC% is running. & exit /b 0)

echo %SVC% is not running.
fltmc >nul 2>&1 || (echo Administrator rights are required to start it. & exit /b 5)
net start "%SVC%"
```

OUTPUT

```
Spooler is running.
```

`sc` returns **1060** when the service does not exist (error code "The specified service does not exist as an installed service"). The `net start` line is reached only for an existing, stopped service, from an elevated window.

17.5 Scheduled tasks: `schtasks`

`schtasks` creates, runs, queries and deletes tasks in the Task Scheduler.

Command Prompt

```
C:\> schtasks /create /tn "BatchBook Demo" /tr "cmd /c echo hello >> %TEMP%\demo.txt" /sc once /st 23:59
C:\> schtasks /query /tn "BatchBook Demo" /fo list
C:\> schtasks /run /tn "BatchBook Demo"
C:\> schtasks /delete /tn "BatchBook Demo" /f
```

OUTPUT

```
SUCCESS: The scheduled task "BatchBook Demo" has successfully been created.

Folder: \
HostName:      DESKTOP-4F2K9
TaskName:      \BatchBook Demo
Status:        Ready
...
```

```
SUCCESS: Attempted to run the scheduled task "BatchBook Demo".
SUCCESS: The scheduled task "BatchBook Demo" was successfully deleted.
```

Option	Meaning
`/tn name`	Task name
`/tr "command"`	The program or script to run (quote the command as one string)
`/sc`	Schedule type: `once`, `daily`, `weekly`, `monthly`, `onlogon`, `onstart`
`/st HH:mm`	Start time (24-hour)
`/ru user`	Run as a given user; `/ru SYSTEM` requires administrator rights
`/f`	Force (overwrite or delete without a prompt)

Warning: A scheduled task runs without a person watching. Make sure the script never calls `pause` or waits for input, uses full paths, logs its work, and does not assume a current directory (Chapters 4, 6 and 14).

17.6 The registry: `reg`

The registry is a database of settings. `reg` reads and writes it. The main roots are `HKCU` (current user), `HKLM` (machine, changes need administrator rights).

Command Prompt

```
C:\> reg query "HKCU\Control Panel\Desktop" /v WallPaper
C:\> reg add "HKCU\Software\BatchBookDemo" /v Greeting /t REG_SZ /d "Hello" /f
C:\> reg query "HKCU\Software\BatchBookDemo"
C:\> reg export "HKCU\Software\BatchBookDemo" "%TEMP%\demo_backup.reg" /y
C:\> reg delete "HKCU\Software\BatchBookDemo" /f
```

OUTPUT

```
HKEY_CURRENT_USER\Control Panel\Desktop
    WallPaper    REG_SZ    C:\Windows\Web\Wallpaper\Windows\img0.jpg

The operation completed successfully.

HKEY_CURRENT_USER\Software\BatchBookDemo
    Greeting    REG_SZ    Hello

The operation completed successfully.
The operation completed successfully.
```

Subcommand	Use
`reg query key [/v name] [/s]`	Read a key or value; `/s` recurses
`reg add key /v name /t type /d data /f`	Create or set a value

Subcommand	Use
`reg delete key [/v name] /f`	Delete a value or a whole key
`reg export key file.reg`	Save a key to a file (back up first)
`reg import file.reg`	Restore from a file

Common types: `REG_SZ` (text), `REG_DWORD` (number), `REG_EXPAND_SZ`, `REG_MULTI_SZ`, `REG_BINARY`.

> **Warning:** Wrong registry changes can make Windows unstable. Always **export a key before changing it**, restrict work to your own test keys such as `HKCU\Software\BatchBookDemo`, and never write to `HKLM` or startup locations from a script you do not fully trust. Chapter 23 lists registry locations that malware abuses, so that you can recognize them.

17.7 System information

Command Prompt

```
C:\> systeminfo | findstr /b /c:"OS Name" /c:"OS Version" /c:"System Type" /c:"Total Physical Memory"
```

OUTPUT

```
OS Name:                   Microsoft Windows 11 Pro
OS Version:                10.0.26100 N/A Build 26100
System Type:               x64-based PC
Total Physical Memory:     16,254 MB
```

`systeminfo` is slow (several seconds) and its labels are **localized**, so a `findstr` filter that works on English Windows fails on other languages. For scripts, prefer PowerShell CIM queries that return properties by name.

Replacing `wmic`

`wmic` has been the batch way to read hardware and OS properties. It is **deprecated**, and it has been removed from supported Windows 11 releases (Chapter 15). Use `Get-CimInstance` through PowerShell:

Old command	Replacement
`wmic os get caption`	`powershell -NoProfile -Command "(Get-CimInstance Win32_OperatingSystem).Caption"`
`wmic cpu get name`	`powershell -NoProfile -Command "(Get-CimInstance Win32_Processor).Name"`
`wmic bios get serialnumber`	`powershell -NoProfile -Command "(Get-CimInstance Win32_BIOS).SerialNumber"`
`wmic logicaldisk get caption,freespace,size`	`powershell -NoProfile -Command "Get-CimInstance Win32_LogicalDisk | Select DeviceID,FreeSpace,Size"`

Old command	Replacement
`wmic os get localdatetime`	`powershell -NoProfile -Command "Get-Date -Format yyyyMMddHHmmss"`
`wmic process where name="x.exe" delete`	`taskkill /im x.exe`

sysinfo_cim.bat

```
@echo off
setlocal
call :ps OSNAME "(Get-CimInstance Win32_OperatingSystem).Caption"
call :ps CPU    "(Get-CimInstance Win32_Processor | Select-Object -First 1).Name"
call :ps RAM_GB "[math]::Round((Get-CimInstance Win32_ComputerSystem).TotalPhysicalMemory / 1GB)"
echo OS  : %OSNAME%
echo CPU : %CPU%
echo RAM : %RAM_GB% GB
exit /b 0

:ps  outvar expression
setlocal
for /f "usebackq delims=" %%a in (`powershell -NoProfile -Command "%~2"`) do set "v=%%a"
endlocal & set "%~1=%v%"
exit /b 0
```

OUTPUT

```
OS  : Microsoft Windows 11 Pro
CPU : 12th Gen Intel(R) Core(TM) i7-1260P
RAM : 16 GB
```

Because the helper receives only fixed expressions written by the script author, nothing untrusted reaches PowerShell. The pipe in the CPU line is inside double quotes, so `cmd.exe` does not treat it as its own pipe.

17.8 `shutdown`

Command	Effect
`shutdown /s /t 60`	Shut down in 60 seconds
`shutdown /r /t 60`	Restart in 60 seconds
`shutdown /a`	**Abort** a pending shutdown
`shutdown /l`	Log off the current user
`shutdown /c "text"`	Show a comment to users
`shutdown /f`	Force applications to close

> **Warning:** Never test `shutdown` without a delay and without knowing the abort command. A safe first test is: `shutdown /s /t 300 /c "Test shutdown"`, followed immediately by `shutdown /a`. `/f` can cause unsaved work to be lost.

Summary

- `tasklist` lists, filters and formats processes. It returns 0 even if nothing matches, so search the output.
- `start "" program` launches; always give the empty title when the path is quoted.
- `taskkill /pid` is precise; `/im` is broad; `/f` forces.
- `sc query` and `net start` / `net stop` manage services; `schtasks` schedules; `reg` reads and writes the registry; export before changes.
- Use CIM through PowerShell instead of `wmic`.
- Treat `shutdown`, `taskkill /f`, `reg delete` and `sc` as dangerous.

Common mistakes

> **Common Mistake:** `start "path with spaces"` with no title.

> **Common Mistake:** **Killing by image name** on a machine where another user runs the same program.

> **Common Mistake:** **Parsing localized output** such as `systeminfo` labels.

> **Common Mistake:** **Forgetting the space after `=` in `sc` options.**

Exercises

1. **[Easy]** List all processes whose image name is `svchost.exe`, without the header.
2. **[Easy]** Show whether the `Spooler` service is running.
3. **[Easy]** Start `notepad.exe` and close it again by PID.
4. **[Medium]** Write a script that reports whether a process name given as an argument is running.
5. **[Medium]** Create a test key `HKCU\Software\BatchBookDemo` with a value, export it, delete it, and re-import it.
6. **[Medium]** Create a scheduled task for 23:59 that writes a line to a file, run it manually, then delete it.

7. **[Hard]** Write a script that gets the total physical memory in GB using CIM and prints a warning if it is less than 8.
8. **[Hard]** Explain why `taskkill /im chrome.exe /f` is a poor way to close a browser and describe a more careful approach.

Chapter 18: Networking from the Command Line

LEARNING OBJECTIVES

- Inspect addresses, routes, name resolution and connections with the standard network commands
- Interpret the results of `ping`, `tracert`, `nslookup` and `netstat`
- Avoid the traps in parsing network output and in trusting exit codes
- Write a script that runs layered diagnostics and writes a report

Warning: Use these tools only on networks and hosts that you own or are authorized to examine. Scanning or probing other people's systems can violate policy or law. The scripts in this chapter diagnose **your own connection**.

18.1 A layered way to think about diagnosis

Network problems are found fastest by testing from the bottom up. Each test either rules a layer out or points to the failing one:

1. **Adapter and address:** do I have an IP address? (`ipconfig`)
2. **Local network:** can I reach the default gateway? (`ping` the gateway)
3. **Internet routing:** can I reach a public IP address? (`ping 1.1.1.1`)
4. **Name resolution:** can I turn a name into an address? (`nslookup`)
5. **Service:** can I open the port of the service? (PowerShell `Test-NetConnection`, or `curl`)

If step 3 works and step 4 fails, the problem is DNS, not the connection.

18.2 The core commands

`ipconfig`

Command	Purpose
`ipconfig`	Addresses, masks and gateways of active adapters
`ipconfig /all`	Adds MAC address, DHCP and DNS servers, lease times
`ipconfig /flushdns`	Clear the local DNS cache
`ipconfig /displaydns`	Show the cache
`ipconfig /release` and `/renew`	Drop and request a DHCP lease (briefly disconnects)

`ping`

Command Prompt

```
C:\> ping -n 2 127.0.0.1
```

OUTPUT

```
Pinging 127.0.0.1 with 32 bytes of data:
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128

Ping statistics for 127.0.0.1:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Switch	Meaning
`-n count`	Number of requests (default 4)
`-t`	Ping until stopped with Ctrl+C
`-w ms`	Timeout per reply in milliseconds
`-l size`	Payload size in bytes
`-4` / `-6`	Force IPv4 / IPv6

The exit code of ping is unreliable: `ping` can return **0** even when the target did not answer, for example when a router replies "Destination host unreachable". The reliable test is to look for `TTL=` in the output, which appears only in genuine replies:

```
ping -n 1 -w 1500 host | find "TTL=" >nul
```

`tracert` and `pathping`

`tracert -d -h 15 example.org` lists the routers on the path (`-d` skips name lookups and is much faster; `-h` limits the hops). Lines with `*` mean a router did not answer, which is often normal. `pathping` adds loss statistics per hop but takes minutes.

`nslookup`

Command Prompt

```
C:\> nslookup example.org 1.1.1.1
```

OUTPUT

```
Server:  one.one.one.one
Address:  1.1.1.1

Non-authoritative answer:
Name:    example.org
```

```
Addresses:  2606:2800:21f:cb07:6820:80da:af6b:8b2c
          93.184.216.34
```

(The addresses are sample values.) The optional second argument chooses the DNS server to ask, which separates a problem in your configured server from a problem with the name itself. `nslookup -type=MX example.org` asks for mail records.

`netstat`

Command	Meaning
`netstat -ano`	All connections and listening ports, numeric, with the owning process ID
`netstat -an | findstr LISTENING`	Only listening ports
`netstat -s`	Protocol statistics
`netstat -r`	Routing table (same as `route print`)
`netstat -b`	Show the program for each connection (administrator rights)

Command Prompt

```
C:\> netstat -ano | findstr ":443" | findstr ESTABLISHED
```

OUTPUT

```
  TCP    192.168.1.23:51512     142.250.74.46:443      ESTABLISHED     8812
```

The last column is the process ID; look it up with `tasklist /fi "pid eq 8812"` (Chapter 17) to see which program owns the connection.

`arp`, `route`, `getmac`

`arp -a` lists IP-to-MAC mappings of neighbours. `route print` shows the routing table, in which the line for destination `0.0.0.0` with mask `0.0.0.0` is the **default route**. `getmac /v` lists adapter MAC addresses.

`netsh`

`netsh` configures the network stack. Reading is safe:

Command Prompt

```
C:\> netsh interface ip show config
C:\> netsh advfirewall show allprofiles state
C:\> netsh wlan show interfaces
```

> **Warning:** `netsh` can also change addresses, firewall rules and reset the network stack (`netsh int ip reset` , `netsh winsock reset`), which needs administrator rights and usually a restart. Do not run reset commands as part of a diagnostic script.

Newer tools

`curl.exe` is included in current Windows 10 and 11. `curl -I https://example.org` shows the response headers; `curl -s -o nul -w "%%{http_code}" URL` prints only the status code (write `%` as `%%` in a batch file). PowerShell's `Test-NetConnection host -Port 443 -InformationLevel Quiet` returns `True` or `False` for a TCP connection test.

18.3 Parsing network output

Human-readable output changes between languages and Windows versions. Whenever a **PowerShell property** exists, prefer it:

network_facts.bat

```
@echo off
setlocal
call :ps GW   "(Get-NetRoute -DestinationPrefix 0.0.0.0/0 | Sort-Object RouteMetric | Select-Object -First
1).NextHop"
call :ps IP   "(Get-NetIPAddress -AddressFamily IPv4 | Where-Object { $_.PrefixOrigin -ne 'WellKnown' } | Select-
Object -First 1).IPAddress"
call :ps DNS  "(Get-DnsClientServerAddress -AddressFamily IPv4 | Where-Object { $_.ServerAddresses } | Select-
Object -First 1).ServerAddresses -join ','"
echo IP address      : %IP%
echo Default gateway : %GW%
echo DNS servers     : %DNS%
exit /b 0

:ps  outvar expression
setlocal
set "v="
for /f "usebackq delims=" %%a in (`powershell -NoProfile -Command "%~2"`) do set "v=%%a"
endlocal & set "%~1=%v%"
exit /b 0
```

OUTPUT

```
IP address      : 192.168.1.23
Default gateway : 192.168.1.1
DNS servers     : 192.168.1.1,1.1.1.1
```

The same values can be extracted from `route print` or `ipconfig` text, but only by relying on English labels and column positions.

18.4 A diagnostic script

The script below runs the layered tests of Section 18.1, prints each result with PASS or FAIL, writes a report file, and returns 0 only if all checks pass.

netcheck.bat

```
@echo off
setlocal EnableExtensions
set "REPORT=%~dp0netcheck_report.txt"
set "TARGET_IP=1.1.1.1"
set "TARGET_NAME=example.org"
set "FAILS=0"

> "%REPORT%" echo Network check %DATE% %TIME%
>> "%REPORT%" echo Computer: %COMPUTERNAME%
>> "%REPORT%" echo.

rem ---- 1. Gateway ----------------------------------------------------
set "GW="
for /f "usebackq delims=" %%a in (`powershell -NoProfile -Command "(Get-NetRoute -DestinationPrefix 0.0.0.0/0 |
Sort-Object RouteMetric | Select-Object -First 1).NextHop"`) do set "GW=%%a"
if not defined GW (
    call :result FAIL "Default gateway" "none found - no route out of this machine"
) else (
    call :result PASS "Default gateway" "%GW%"
    rem ---- 2. Gateway reachable ---------------------------------------
    call :ping_ok "%GW%"
    if errorlevel 1 (call :result FAIL "Ping gateway" "%GW% did not reply") else (call :result PASS "Ping
gateway" "%GW% replied")
)

rem ---- 3. Internet by address ----------------------------------------
call :ping_ok "%TARGET_IP%"
if errorlevel 1 (call :result FAIL "Ping %TARGET_IP%" "no reply") else (call :result PASS "Ping %TARGET_IP%" "re
ply received")

rem ---- 4. DNS --------------------------------------------------------
nslookup "%TARGET_NAME%" 2>nul | find /i "Name:" >nul
if errorlevel 1 (call :result FAIL "DNS lookup" "could not resolve %TARGET_NAME%") else (call :result PASS "DNS
lookup" "%TARGET_NAME% resolved")

rem ---- 5. HTTPS port -------------------------------------------------
set "PORTOK="
for /f "usebackq delims=" %%a in (`powershell -NoProfile -Command "Test-NetConnection %TARGET_NAME% -Port 443 -
InformationLevel Quiet -WarningAction SilentlyContinue"`) do set "PORTOK=%%a"
if /i "%PORTOK%"=="True" (call :result PASS "HTTPS port 443" "connected") else (call :result FAIL "HTTPS port
443" "could not connect")

echo.
if %FAILS% equ 0 (echo All checks passed.) else (echo %FAILS% check^(s^) failed.)
echo Report written to %REPORT%
exit /b %FAILS%

:ping_ok  host
ping -n 1 -w 1500 "%~1" | find "TTL=" >nul
exit /b %errorlevel%

:result  status name detail
set "line=%~1  %~2  -  %~3"
echo %line%
>> "%REPORT%" echo %line%
if /i "%~1"=="FAIL" set /a "FAILS+=1"
exit /b 0
```

OUTPUT

```
PASS  Default gateway  -  192.168.1.1
PASS  Ping gateway  -  192.168.1.1 replied
PASS  Ping 1.1.1.1  -  reply received
PASS  DNS lookup  -  example.org resolved
PASS  HTTPS port 443  -  connected
```

```
All checks passed.
Report written to C:\Users\Alex\BatchBook\scripts\netcheck_report.txt
```

Notes on the design:

- Each test is independent, and the script continues after failures, so the report shows **every** failing layer.
- The counter `FAILS` lives in the main scope because `:result` does not call `setlocal`; it therefore modifies the caller's variable.
- The exit code equals the number of failed checks, so a scheduler can alert on non-zero.
- `echo %FAILS% check^(s^) failed.` escapes the parentheses because the line is inside an `if` block.

18.5 Common diagnostic recipes

Question	Command
What is my public-facing IP?	`curl -s https://ifconfig.me` (an external service; check its policy)
Which program listens on port 8080?	`netstat -ano | findstr :8080`, then `tasklist /fi "pid eq <PID>"`
Is DNS caching a stale answer?	`ipconfig /flushdns`, then repeat `nslookup`
Is the problem my DNS server?	`nslookup example.org 1.1.1.1` versus `nslookup example.org`
Where does the path stop?	`tracert -d -h 20 example.org`

Summary

- Diagnose bottom-up: address, gateway, internet by IP, DNS, service port.
- `ping` exit codes can mislead; search for `TTL=`.
- `nslookup` can query a chosen DNS server; `netstat -ano` ties connections to process IDs.
- Prefer PowerShell properties to parsing localized text.
- Return a meaningful exit code and a report from automated diagnostics, and only probe networks you are authorized to examine.

Common mistakes

Common Mistake: Trusting `ping`'s exit code.

Common Mistake: Diagnosing DNS and connectivity with one test. Test by address first, then by name.

> **Common Mistake:** **Parsing English-only labels** from `ipconfig` or `route print`.

> **Common Mistake:** **Running `netsh ... reset` as a diagnostic step.**

Exercises

1. **[Easy]** Show your IPv4 address and default gateway using `ipconfig`.
2. **[Easy]** Ping `127.0.0.1` three times.
3. **[Easy]** Resolve a host name with `nslookup`, asking server `1.1.1.1`.
4. **[Medium]** List all listening TCP ports with their process IDs.
5. **[Medium]** Write `:ping_ok` and use it to report "up" or "down" for a host name given as an argument.
6. **[Medium]** Find which program owns a connection on port 443 using `netstat` and `tasklist`.
7. **[Hard]** Extend `netcheck.bat` with a check of a second DNS server (`8.8.8.8`) using `nslookup example.org 8.8.8.8`.
8. **[Hard]** Explain the difference between a ping failure to an IP address and a ping failure to a host name, and what each indicates.

Chapter 19: Interoperability

LEARNING OBJECTIVES

- Call PowerShell, VBScript and other programs from a batch file, with correct quoting
- Pass data into and out of other languages safely
- Interpret exit codes across the boundary
- Build a hybrid script that carries PowerShell inside a batch file
- Judge when to leave Batch for another tool

Batch is a glue language. Its strength is starting other programs; its weakness is everything those programs do well. This chapter shows how to hand work over and take the result back.

19.1 Calling PowerShell

`powershell.exe` is **Windows PowerShell 5.1**, installed with Windows 10 and 11. `pwsh.exe` is the separate, cross-platform PowerShell 7 and is present only if installed. Use `powershell` when you need the guaranteed tool.

Form	Use
`powershell -NoProfile -Command "expression"`	Run a short command
`powershell -NoProfile -File script.ps1 args`	Run a script file
`powershell -NoProfile -ExecutionPolicy Bypass -File script.ps1`	Run a script even if the policy blocks scripts, for this process only

`-NoProfile` skips the user's profile script, which makes startup faster and behavior predictable.

ps_basic.bat

```
@echo off
powershell -NoProfile -Command "Write-Output ('Hello from PowerShell ' + $PSVersionTable.PSVersion.Major)"
powershell -NoProfile -Command "Get-ChildItem -Path $env:windir -Filter *.exe | Select-Object -First 3 -ExpandProperty Name"
```

OUTPUT

```
Hello from PowerShell 5
bfsvc.exe
explorer.exe
HelpPane.exe
```

> **Execution policy:** The execution policy is a safety feature that stops accidental script runs. It is **not a security boundary**: `-ExecutionPolicy Bypass` is an ordinary command-line option. Use it deliberately for scripts you wrote or reviewed, and treat a script found elsewhere that uses it as something to inspect before running (Chapter 23).

Quoting between the two languages

The command passes through `cmd.exe` first, then through the PowerShell command-line parser. Three rules keep it manageable:

1. Wrap the whole `-Command` text in **double quotes**.
2. Use **single quotes** for strings *inside* PowerShell.
3. Double the percent signs for batch (`%%`) and avoid `!` when delayed expansion is on.

ps_quoting.bat

```
@echo off
powershell -NoProfile -Command "'Total: ' + (3 * 14) + ' items, ' + (100 * 0.25) + '%%'"
powershell -NoProfile -Command "Write-Output 'Single quotes inside, double outside'"
```

OUTPUT

```
Total: 42 items, 25%
Single quotes inside, double outside
```

Exit codes

`-File` returns the script's exit code. `-Command` returns **0 on success and 1 if an error occurred**, unless the command calls `exit`:

ps_exitcode.bat

```
@echo off
powershell -NoProfile -Command "exit 7"
echo Exit code: %errorlevel%
powershell -NoProfile -Command "Get-Item 'C:\no\such\file' -ErrorAction Stop" 2>nul
echo Exit code after an error: %errorlevel%
```

OUTPUT

```
Exit code: 7
Exit code after an error: 1
```

19.2 Passing data from Batch to PowerShell

Never build PowerShell code by pasting user-supplied text into the command string. If the text contains a quote or a semicolon, it changes the program (this is **injection**; see Chapter 23). The safe method is to pass the value through an **environment variable**, which PowerShell reads as data:

ps_env_data.bat

```
@echo off
setlocal
set "USER_TEXT=O'Brien; Get-Process"
powershell -NoProfile -Command "Write-Output ('Received: ' + $env:USER_TEXT)"
```

OUTPUT

```
Received: O'Brien; Get-Process
```

The apostrophe and the semicolon arrive as plain text and nothing is executed. A second safe method is to pass arguments to a script file and read them from `param(...)`:

greet.ps1

```
param([string]$Name, [int]$Times = 1)
1..$Times | ForEach-Object { "Hello, $Name" }
```

ps_file_args.bat

```
@echo off
powershell -NoProfile -ExecutionPolicy Bypass -File "%~dp0greet.ps1" -Name "Ada Lovelace" -Times 2
```

OUTPUT

```
Hello, Ada Lovelace
Hello, Ada Lovelace
```

19.3 Getting data back from PowerShell

PowerShell writes results to standard output, and `for /f` reads them (Chapter 10). Use `usebackq` when the command contains single quotes.

One value

ps_return_one.bat

```
@echo off
setlocal
for /f "usebackq delims=" %%d in (`powershell -NoProfile -Command "[Environment]::MachineName"`) do set "HOST=%%d
"
echo Machine name: %HOST%
```

OUTPUT

```
Machine name: DESKTOP-4F2K9
```

Several values

Output one `NAME=value` line per value and set them all in one loop:

ps_return_many.bat

```
@echo off
setlocal
for /f "usebackq tokens=1,* delims==" %%a in (`powershell -NoProfile -Command "$o=Get-CimInstance
Win32_OperatingSystem; 'OS_NAME='+$o.Caption; 'OS_BUILD='+$o.BuildNumber; 'OS_ARCH='+$o.OSArchitecture"`) do set
"%%a=%%b"
echo Name  : %OS_NAME%
echo Build : %OS_BUILD%
echo Arch  : %OS_ARCH%
```

OUTPUT

```
Name  : Microsoft Windows 11 Pro
Build : 26100
Arch  : 64-bit
```

The first token (before `=`) becomes the variable name; `tokens=1,*` keeps the rest of the line, which may itself contain `=`. Use this only for output produced by **your own** PowerShell text; if the names came from outside, validate them first.

Large or structured results

For tables, lists or JSON, let PowerShell write a file and read it in Batch with `for /f`, or process the data entirely in PowerShell and return only the answer. Chapter 26 (Project 2) shows a complete example: a companion PowerShell script returns `NAME=value` lines and the Batch program turns them into a report.

> **Best Practice:** Keep the boundary narrow: send a few simple values in (environment variables or parameters) and get a few simple values out (one line each). Do the complicated work on one side.

19.4 Calling VBScript

`cscript.exe` runs VBScript files in the console, `wscript.exe` in windows.

today.vbs

```
' Prints arguments and the date in a fixed format.
Dim i
For i = 0 To WScript.Arguments.Count - 1
    WScript.Echo "Argument " & i & ": " & WScript.Arguments(i)
Next
WScript.Echo "Year: " & Year(Now)
```

vbs_call.bat

```
@echo off
cscript //nologo "%~dp0today.vbs" first second
```

OUTPUT

```
Argument 0: first
Argument 1: second
Year: 2026
```

`//nologo` hides the banner of the Windows Script Host.

> **VBScript is being retired:** Microsoft has announced the **deprecation of VBScript**: on current Windows 11 releases it is an optional feature that may be disabled, and it is planned to be removed in a future release. Treat VBScript as a legacy interface for existing scripts and write new automation in PowerShell.

19.5 Other executables worth knowing

Tool	Use	Example
`curl.exe`	HTTP requests and downloads	`curl -s -o page.html https://example.org`
`tar.exe`	Create and extract archives, including `.zip`	`tar -a -c -f backup.zip folder`
`certutil -hashfile`	File hashes (integrity checks)	`certutil -hashfile file.iso SHA256`
`robocopy.exe`	Robust copying and mirroring	Chapter 3
`powershell Compress-Archive`	Create a zip with options	`Compress-Archive -Path folder -DestinationPath out.zip`
`7z.exe` (if installed)	Many archive formats	`7z a out.7z folder`

zip_folder.bat

```
@echo off
setlocal
set "SRC=%~dp0zipdemo_src"
set "ZIP=%~dp0zipdemo.zip"
md "%SRC%" 2>nul
echo hello> "%SRC%\a.txt"
tar -a -c -f "%ZIP%" -C "%SRC%" .
if errorlevel 1 (echo Archive creation failed. & exit /b 1)
for %%f in ("%ZIP%") do echo Archive size: %%~zf bytes
tar -tf "%ZIP%"
del "%ZIP%"
rd /s /q "%SRC%"
```

OUTPUT

```
Archive size: 272 bytes
./
./a.txt
```

`tar -a` chooses the format from the extension; `-C` changes into the folder first so the archive does not store the full path. Sizes vary with the Windows version.

> **Warning:** `certutil` can also download and decode files, and malware abuses that. Use it here only for hashing your own files; when you see it downloading or decoding in an unfamiliar script, that is a red flag (Chapter 23).

19.6 Hybrid scripts

A **hybrid script** is a single file that is both a batch file and the carrier of another language. The batch part starts first, calls the other interpreter on the same file, and then ends. The simplest transparent design puts the PowerShell code after a marker line that Batch never reaches because it ends first.

hybrid.bat

```
@echo off
setlocal
rem ---- Batch part: hand the rest of this file to PowerShell ----
powershell -NoProfile -Command "$lines = Get-Content -LiteralPath '%~f0'; $start = [array]::IndexOf($lines,
':PS_BEGIN'); & ([scriptblock]::Create(($lines[($start+1)..($lines.Count-1)] -join [Environment]::NewLine)))"
exit /b %errorlevel%

:PS_BEGIN
# ---- PowerShell part ----
$os = (Get-CimInstance Win32_OperatingSystem).Caption
$free = [math]::Round((Get-PSDrive -Name C).Free / 1GB, 1)
"Hybrid script running on $os"
"Free space on C: $free GB"
```

OUTPUT

```
Hybrid script running on Microsoft Windows 11 Pro
Free space on C: 182.4 GB
```

How it works:

1. `cmd.exe` executes the first lines. It never parses anything after `exit /b`, so the PowerShell text is not interpreted as batch.
2. The PowerShell command reads this very file (`%~f0`), finds the marker line, and runs everything after it as a script block.
3. PowerShell's text output reaches the console, and `exit /b %errorlevel%` passes the status on.

Limits: the file path must not contain a single quote, and the marker line must appear exactly as written. Debugging is harder than with two separate files.

> **Best Practice:** Use a hybrid only when you must ship **one file**. Otherwise keep a `.bat` and a `.ps1` side by side and call the second with `-File`; each file is easier to read, test and sign.

19.7 Choosing the right tool

Need	Prefer
Run programs in order, check exit codes	Batch
Parse JSON, XML, CSV	PowerShell
Dates, regular expressions, large numbers	PowerShell
Call Windows APIs or manage Active Directory	PowerShell
Run on any machine with no setup, very short logic	Batch
Keep a legacy tool alive	Batch or VBScript (read-only maintenance)

Summary

- Call PowerShell with `-NoProfile -Command` (double quotes outside, single inside) or `-File`.
- Exit codes cross the boundary; `-Command` returns 0 or 1 unless `exit n` is used.
- Pass untrusted values through environment variables or parameters, never by pasting text into code.
- Read results with `for /f` and `usebackq`; use `NAME=value` lines for several values.
- VBScript is deprecated; `curl`, `tar`, `certutil -hashfile` and `robocopy` are everyday helpers.
- A hybrid script carries PowerShell after a marker that Batch never reaches.

Common mistakes

Common Mistake: **Building PowerShell code from user input.** Pass data through environment variables.

Common Mistake: **Mixing quotes** so that `cmd.exe` and PowerShell disagree on where the string ends.

Common Mistake: **Forgetting** `%%` for a literal percent sign inside a batch file's PowerShell text.

Common Mistake: **Using** `-ExecutionPolicy Bypass` **on scripts you have not read.**

Exercises

1. **[Easy]** Print the PowerShell major version from a batch file.

2. **[Easy]** Run `powershell -NoProfile -Command "exit 5"` and display the resulting exit code.
3. **[Easy]** Capture the output of `[Environment]::UserName` into a batch variable.
4. **[Medium]** Pass the text `Tom & "Jerry"` to PowerShell safely and print it.
5. **[Medium]** Write a PowerShell script `add.ps1` that takes two numbers and prints their sum, and call it from Batch.
6. **[Medium]** Create a zip file of a folder with `tar` and list its contents.
7. **[Hard]** Write a hybrid script that prints the five largest files in its own folder using PowerShell.
8. **[Hard]** Explain why a `-Command` string built from a user-supplied file name is unsafe, and rewrite it with an environment variable.

Part 3 Quiz

1. **Q1.** How do you preserve blank lines when reading a file line by line?
2. **Q2.** In `findstr "disk full" f.log`, does the space mean a phrase?
3. **Q3.** What are the success values of `robocopy`?
4. **Q4.** Why is `ping`'s exit code unreliable, and what do you check instead?
5. **Q5.** Which command tests for administrator rights without needing the Server service?
6. **Q6.** Why should destructive scripts default to a dry run?
7. **Q7.** Why is `%DATE:~-4%` a poor way to get the year?
8. **Q8.** What does `%TIME: =0%` do?
9. **Q9.** Why must `if errorlevel` tests for `choice` be written from highest to lowest?
10. **Q10.** What does `start "C:\Program Files\x.exe"` actually do?
11. **Q11.** Name the replacement for `wmic os get caption`.
12. **Q12.** What is the safest way to pass user-supplied text into a PowerShell command from Batch?
13. **Q13.** What is the exit code of `powershell -Command "..."` when the command raised an error?

4 Advanced Topics

Chapters 20 to 24

Data structures, performance, debugging, security and style

Part 4 deals with the techniques that distinguish a script that merely works from one that is fast, testable, safe and maintainable: emulating data structures, reducing process starts, finding bugs systematically, recognizing unsafe patterns and keeping large scripts organized.

Chapters in this Part

- Chapter 20: Advanced Techniques
- Chapter 21: Performance and Optimization
- Chapter 22: Debugging and Testing
- Chapter 23: Security and Best Practices
- Chapter 24: Style Guide and Project Structure
- Part 4 Quiz

Chapter 20: Advanced Techniques

LEARNING OBJECTIVES

- Emulate arrays and associative arrays with indexed variable names
- Build stacks and queues with subroutines
- Pause, time and generate random numbers reliably
- Parse the output of other commands robustly
- Write a script that asks for administrator rights and relaunches itself

Batch has no arrays, dictionaries or functions in the usual sense. Experienced script writers build equivalents from what exists: **variables whose names carry an index or a key**, and **subroutines**.

20.1 Arrays

An array element is a variable named like `a[1]`, `a[2]`, ... The index is part of the name. Delayed expansion lets you compute the name at run time (`!a[%i%]!`).

array_sort.bat

```
@echo off
setlocal EnableDelayedExpansion

rem ---- Fill the array ----
set "n=0"
for %%v in (42 7 19 3 25) do (
    set /a "n+=1"
    set "a[!n!]=%%v"
)

echo Before:
for /l %%i in (1,1,%n%) do echo   a[%%i] = !a[%%i]!

rem ---- Bubble sort, ascending ----
set /a "last=n-1"
for /l %%p in (1,1,%last%) do (
    for /l %%j in (1,1,%last%) do (
        set /a "k=%%j+1"
        for %%k in (!k!) do (
            if !a[%%j]! gtr !a[%%k]! (
                set "t=!a[%%j]!"
                set "a[%%j]=!a[%%k]!"
                set "a[%%k]=!t!"
            )
        )
    )
)

echo After:
for /l %%i in (1,1,%n%) do echo   a[%%i] = !a[%%i]!
```

OUTPUT

```
Before:
  a[1] = 42
  a[2] = 7
  a[3] = 19
  a[4] = 3
  a[5] = 25
After:
  a[1] = 3
  a[2] = 7
  a[3] = 19
  a[4] = 25
  a[5] = 42
```

Three details make this work. The element names are built with `!n!` and `%%j`, so each loop pass addresses a different variable; the comparison uses unquoted values so that it is **numeric** (Chapter 8); and the helper loop `for %%k in (!k!)` turns the computed index `k` into a loop variable that can be used inside `!a[%%k]!`.

> **Best Practice:** Always keep a counter variable (`n`) with the array length. Batch cannot tell you how many elements exist; with the counter you can loop with `for /l`.

Listing all elements without a counter

Because all array elements share the prefix `a[`, the command `set a[` lists them:

Command Prompt

```
C:\> set a[
```

OUTPUT

```
a[1]=3
a[2]=7
a[3]=19
a[4]=25
a[5]=42
```

Note that this listing is **sorted as text** (`a[10]` would appear before `a[2]`).

20.2 Associative arrays (dictionaries)

Use a key instead of a number: `price[apple]`.

assoc_demo.bat

```
@echo off
setlocal EnableDelayedExpansion
set "price[apple]=3"
set "price[banana]=1"
set "price[cherry]=7"

set "key=banana"
echo Price of !key!: !price[%key%]!
```

```
set /a "price[apple]+=2"

echo --- all items ---
for /f "tokens=1,2 delims==" %%a in ('set price[') do (
    set "name=%%a"
    set "name=!name:price[=!"
    set "name=!name:]=!"
    echo !name! costs %%b
)

if defined price[mango] (echo mango is known) else (echo mango is unknown)
```

```
OUTPUT
Price of banana: 1
--- all items ---
apple costs 5
banana costs 1
cherry costs 7
mango is unknown
```

`if defined price[mango]` is a membership test. Keys must obey the naming rules of variables: **no spaces, no `=`, and avoid characters special to the parser**. Values may contain nearly anything. Listing with `set price[` gives items **sorted alphabetically by key**, a useful side effect. The counting example in Chapter 13 (`ext_report.bat`) is a dictionary of counters.

20.3 Stacks and queues

A **stack** is last-in, first-out; a **queue** is first-in, first-out. Both are an array plus index variables.

stack_queue.bat

```
@echo off
setlocal
set "sp=0"
set "qhead=1"
set "qtail=0"

echo --- stack ---
call :push one
call :push two
call :push three
call :pop item
echo popped %item%
call :pop item
echo popped %item%

echo --- queue ---
call :enqueue alpha
call :enqueue beta
call :enqueue gamma
call :dequeue item
echo dequeued %item%
call :dequeue item
echo dequeued %item%
exit /b 0

:push  value
set /a "sp+=1"
set "stack[%sp%]=%~1"
exit /b 0
```

```
:pop  outvar
if %sp% lss 1 (set "%~1=" & exit /b 1)
call set "%~1=%%stack[%sp%]%%"
set "stack[%sp%]="
set /a "sp-=1"
exit /b 0

:enqueue  value
set /a "qtail+=1"
set "queue[%qtail%]=%~1"
exit /b 0

:dequeue  outvar
if %qhead% gtr %qtail% (set "%~1=" & exit /b 1)
call set "%~1=%%queue[%qhead%]%%"
set "queue[%qhead%]="
set /a "qhead+=1"
exit /b 0
```

```
OUTPUT
--- stack ---
popped three
popped two
--- queue ---
dequeued alpha
dequeued beta
```

The line `call set "%~1=%%stack[%sp%]%%"` is a **double expansion**: the first pass turns `%sp%` into the current index and `%%` into `%`, leaving `call set "item=%stack[3]%"`, which `call` expands a second time and so reads the element.

Common Mistake: Chaining `call :pop item & echo %item%` on one line. All percent variables on a line are expanded **before any command on that line runs**, so `%item%` shows the old value. Put the `echo` on the next line, as above, or use delayed expansion (`!item!`).

20.4 Sleeping and timing

Technique	Resolution	Notes
`timeout /t N /nobreak >nul`	1 s	Not usable with redirected input
`ping -n N+1 127.0.0.1 >nul`	1 s	Works without a console; waits N seconds
`ping -n 1 -w 500 192.0.2.1 >nul`	about 0.5 s	Waits for a reply that never comes (the address is reserved for documentation); timing is approximate
`powershell -NoProfile -Command Start-Sleep -Milliseconds 250`	1 ms, plus startup cost of about 0.3 s	Precise, but slow to start

sleep_demo.bat

```
@echo off
setlocal
echo Waiting 3 seconds...
call :sleep 3
echo Done.
exit /b 0

:sleep  seconds
set /a "pings=%~1+1"
ping -n %pings% 127.0.0.1 >nul
exit /b 0
```

OUTPUT

```
Waiting 3 seconds...
Done.
```

Pinging localhost sends one echo per second, so `N+1` requests take about `N` seconds. For time measurement see Chapter 15.

20.5 Random numbers

`%RANDOM%` yields 0 to 32767. Combine two calls for a larger range, and use modulo for a range:

random_demo.bat

```
@echo off
setlocal EnableDelayedExpansion

rem A number up to about one billion from two calls.
set /a "big=%RANDOM%*32768+%RANDOM%"
echo Large random number: %big%

rem Six distinct numbers from 1 to 49.
set "count=0"
set "picked="
:draw
set /a "n=%RANDOM% %% 49 + 1"
if defined drawn[%n%] goto :draw
set "drawn[%n%]=1"
set "picked=%picked% %n%"
set /a "count+=1"
if %count% lss 6 goto :draw
echo Drawn numbers:%picked%
```

OUTPUT

```
Large random number: 482071938
Drawn numbers: 17 3 41 28 9 36
```

The `drawn[...]` variables form a set: a number is accepted only if it has not been drawn. (Output varies on every run.)

> **Warning:** `%RANDOM%` is **not** cryptographically secure. Never use it for passwords, tokens or keys. Use PowerShell with a cryptographic generator for that purpose.

20.6 Parsing command output robustly

Rules that make output parsing dependable:

1. **Ask for a machine format** when the tool has one: `/fo csv`, `/nh`, or `/fo list`.
2. **Filter first** with `find` or `findstr`, then split with `for /f`.
3. **Avoid English-only labels** where possible.
4. **Test on the oldest and newest Windows you support.**

Tool	Machine-friendly form
`tasklist`	`tasklist /fo csv /nh`
`getmac`	`getmac /fo csv /nh`
`schtasks`	`schtasks /query /fo csv /nh`
`driverquery`	`driverquery /fo csv /nh`
`netstat`	`netstat -ano` (fixed columns, numeric)

csv_parse.bat

```
@echo off
setlocal
set "shown=0"
for /f "tokens=1,2 delims=," %%a in ('tasklist /fo csv /nh') do call :show "%%~a" "%%~b" || goto :done
:done
exit /b 0

:show  name pid
set /a "shown+=1"
echo %~1  PID %~2
if %shown% geq 3 exit /b 1
exit /b 0
```

OUTPUT

```
System Idle Process  PID 0
System  PID 4
Registry  PID 100
```

Each CSV line looks like `"System","4","Services","0","144 K"`. Splitting at commas yields quoted tokens, and `%%~a` removes the quotes. Returning 1 from the subroutine after the third item triggers `|| goto :done`, which leaves the loop.

20.7 Self-elevating scripts

A script that needs administrator rights can test for them (Chapter 14) and, if absent, start a **new elevated copy of itself**. Windows will show a User Account Control prompt, which the user must accept; a script cannot elevate silently, and it should not try.

elevate.bat

```
@echo off
setlocal

rem ---- Are we already elevated? ----
fltmc >nul 2>&1 && goto :elevated

echo This script needs administrator rights. Requesting elevation...
set "SELF=%~f0"
set "SELF_ARGS=%*"
if defined SELF_ARGS (
    powershell -NoProfile -Command "Start-Process -FilePath $env:SELF -ArgumentList $env:SELF_ARGS -Verb RunAs"
) else (
    powershell -NoProfile -Command "Start-Process -FilePath $env:SELF -Verb RunAs"
)
exit /b 0

:elevated
rem An elevated window starts in System32, so go back to the script folder.
pushd "%~dp0"
echo Running with administrator rights in %CD%
echo Arguments: %*
rem ---- privileged work goes here ----
popd
echo.
pause
exit /b 0
```

OUTPUT

```
This script needs administrator rights. Requesting elevation...
```

(After the user accepts the UAC prompt, a new window opens and prints `Running with administrator rights in C:\Users\Alex\BatchBook\scripts`.) Notes:

- The path and arguments are handed to PowerShell **through environment variables**, so quotes and apostrophes in them cannot alter the command (Chapter 19).
- The original window exits immediately. The elevated copy runs in its own window and ends with `pause`, otherwise the window would close before you could read it.
- If the user declines the prompt, PowerShell reports an error and nothing is elevated.

> **Warning:** A script that elevates itself runs everything after that point with full rights. Review the privileged section carefully, and never elevate a script that builds commands from untrusted input (Chapter 23).

Summary

- Arrays are variables named `a[1]`, `a[2]`; keep a counter; use delayed expansion to address elements.
- Dictionaries use keys in the name (`price[apple]`); `set price[` lists them sorted; `if defined` tests membership.
- Stacks and queues are arrays plus index variables, managed by subroutines using `call set` double expansion.
- Sleep with `timeout` or `ping`; use `%RANDOM%` only for non-security purposes.
- Parse command output via CSV or list formats, filter first, and avoid localized labels.
- Self-elevation tests with `fltmc`, relaunches through PowerShell `-Verb RunAs`, passes data via environment variables, and restores the working folder.

Common mistakes

Common Mistake: **Forgetting delayed expansion** when addressing `a[!i!]`.

Common Mistake: **Relying on a value set earlier on the same line.**

Common Mistake: **Using `%RANDOM%` for secrets.**

Common Mistake: **Assuming the working folder after elevation.**

Exercises

1. **[Easy]** Store three names in `name[1]` to `name[3]` and print them with a loop.
2. **[Easy]** Create `color[red]=FF0000` and print the value for the key `red`.
3. **[Easy]** Write a `:sleep` routine that waits N seconds.
4. **[Medium]** Find the largest value in an array of five numbers.
5. **[Medium]** Count how many times each word occurs in `alpha beta alpha gamma beta alpha` using a dictionary.
6. **[Medium]** Implement `:push` and `:pop` and reverse a list of five words with them.
7. **[Hard]** Write a function that shuffles the elements of an array (Fisher-Yates) using `%RANDOM%`.
8. **[Hard]** Modify `elevate.bat` so that the elevated copy receives the arguments of the first call and prints them, and explain what can go wrong when arguments contain special characters.

Chapter 21: Performance and Optimization

LEARNING OBJECTIVES

- Identify the operations that dominate the running time of a batch script
- Measure code with a reusable timing routine before changing it
- Reduce process starts, repeated file opens, and `call` overhead
- Choose efficient loop and file-processing patterns
- Know when to stop optimizing, or to switch tools

Rule one: Measure first. Most scripts are fast enough, and a change made without a measurement is a guess. Optimize only code that is demonstrably slow, and keep the original readable version as a reference.

21.1 Where the time goes

Batch is interpreted line by line, but the interpreter is not the main cost. The expensive operations are:

Operation	Why it is slow	Typical cost
Starting an external program (`find`, `findstr`, `sort`, `ping`)	A new process is created	Milliseconds to tens of milliseconds each
Each side of a pipe	Each side runs in its own `cmd.exe`	Two process starts
`for /f` over a command (`in ('command')`)	The command runs in a new `cmd.exe`	One process per loop **start**
Starting PowerShell	The runtime must load	Several hundred milliseconds
`call :label`	The line is parsed again and the file position is reopened	Slower than inline code by a large factor
Redirection `>> file` on every iteration	The file is opened and closed each time	Costly in long loops
Printing to the console	The console is slow, especially with many small lines	Often the largest cost of all
Antivirus scanning	New or changed files can be inspected	Varies by system

The practical rule is to **do less work per iteration** and **start fewer processes**.

21.2 A timing harness

The routines below measure elapsed time in centiseconds, using the technique of Chapter 15. They are used by every experiment in this chapter.

bench_lib.bat

```
@echo off
rem Timing helpers. Usage: call "bench_lib.bat" :now varname
rem                        call "bench_lib.bat" :report "label" startvar
call %*
exit /b %errorlevel%

:now  outvar
setlocal
set "t=%TIME: =0%"
set /a "cs=(((1%t:~0,2%-100)*60+1%t:~3,2%-100)*60+1%t:~6,2%-100)*100+1%t:~9,2%-100"
endlocal & set "%~1=%cs%"
exit /b 0

:report  label startvar
setlocal
call :now t_end
set /a "d=t_end-%~2"
if %d% lss 0 set /a "d+=8640000"
set /a "s=d/100, c=d%%100"
if %c% lss 10 set "c=0%c%"
echo %~1: %s%.%c% s
exit /b 0
```

21.3 Experiment 1: redirect once, not on every line

bench_redirect.bat

```
@echo off
setlocal
set "LIB=%~dp0bench_lib.bat"
set "F=%~dp0bench.tmp"

call "%LIB%" :now t0
if exist "%F%" del "%F%"
for /l %%i in (1,1,2000) do >> "%F%" echo line %%i
call "%LIB%" :report "Append on every line     " t0

call "%LIB%" :now t0
(for /l %%i in (1,1,2000) do echo line %%i) > "%F%"
call "%LIB%" :report "One redirect for the loop " t0

del "%F%"
```

OUTPUT

```
Append on every line     : 0.34 s
One redirect for the loop : 0.05 s
```

The numbers are samples; the **ratio** is what matters. Opening the file once (by redirecting the whole block) is several times faster than opening it 2000 times.

21.4 Experiment 2: loop constructs

bench_loops.bat

```
@echo off
setlocal
set "LIB=%~dp0bench_lib.bat"

call "%LIB%" :now t0
set "i=0"
:again
set /a "i+=1"
if %i% lss 20000 goto :again
call "%LIB%" :report "goto loop, 20000 passes " t0

call "%LIB%" :now t0
set "j=0"
for /l %%n in (1,1,20000) do set /a "j+=1"
call "%LIB%" :report "for /l, 20000 passes    " t0

call "%LIB%" :now t0
for /l %%n in (1,1,2000) do call :nothing
call "%LIB%" :report "call :label, 2000 calls " t0
exit /b 0

:nothing
exit /b 0
```

OUTPUT

```
goto loop, 20000 passes : 0.52 s
for /l, 20000 passes    : 0.11 s
call :label, 2000 calls : 0.19 s
```

Typical findings (your times will differ, but the order does not):

- **`for /l` is much faster than a hand-built `goto` loop**, because `goto` makes `cmd.exe` search for the label and re-read the line each time.
- **`call` has a large per-call cost.** Two thousand calls cost nearly as much as twenty thousand loop passes. Use `call` for structure and reuse, not inside the innermost loop of a large job.

21.5 Experiment 3: avoid launching programs in a loop

Compare checking each file with an external program against checking all files with one:

bench_spawn.bat

```
@echo off
setlocal
set "LIB=%~dp0bench_lib.bat"
set "F=%~dp0words.tmp"
(for /l %%i in (1,1,300) do echo word%%i) > "%F%"

call "%LIB%" :now t0
for /f "usebackq delims=" %%w in ("%F%") do (echo %%w| findstr /b "word1" >nul && set /a "n+=1")
call "%LIB%" :report "findstr once per line (300 starts)" t0

set "n=0"
call "%LIB%" :now t0
```

```
for /f %%c in ('findstr /b "word1" "%F%" ^| find /c /v ""') do set "n=%%c"
call "%LIB%" :report "one findstr for the whole file     " t0
echo Matches: %n%
del "%F%"
```

```
OUTPUT
findstr once per line (300 starts): 3.10 s
one findstr for the whole file     : 0.08 s
Matches: 111
```

Starting an external program 300 times, each with a pipe (two processes), costs roughly a **hundred times more** than letting one `findstr` read the whole file. The principle holds everywhere: **give a tool the whole input once**.

21.6 Optimization techniques

Best Practice: 1. **Batch the work.** Run `findstr`, `sort`, `dir /s /b` once over all the data; avoid calling them per item. 2. **Replace external programs with built-ins.** Use `set /a` for arithmetic, `%var:~n,m%` for substrings, `if` for comparisons, instead of spawning `find` or `cmd`. 3. **Redirect once.** Wrap the loop in parentheses and redirect the whole block, or collect output in a file. 4. **Use `for /l` for counting** and avoid label-based loops for large counts. 5. **Avoid `call` in hot loops.** Use delayed expansion for double expansion where possible; inline simple routines. 6. **Reduce console output.** Print a status line every N items, or send detail to a log. 7. **Call PowerShell once.** Fetch several values in one invocation (Chapter 19) rather than starting it per value. 8. **Filter early.** Use `dir /b /a:-d *.log` instead of listing everything and testing each name. 9. **Prefer `for /f` over a file** to `type file | findstr` inside loops. 10. **Use `robocopy` or `xcopy` for bulk copies**, not a loop of `copy`.

Output is expensive: a status line every N items

Print progress at intervals instead of for every item:

progress_every.bat

```
@echo off
setlocal EnableDelayedExpansion
set "total=5000"
set /a "count=0"
for /l %%i in (1,1,%total%) do (
    set /a "count+=1"
    set /a "rem=count %% 1000"
    if !rem! equ 0 echo Processed !count! of %total%
)
```

```
OUTPUT
Processed 1000 of 5000
Processed 2000 of 5000
Processed 3000 of 5000
Processed 4000 of 5000
Processed 5000 of 5000
```

21.7 Common slowdowns checklist

Symptom	Likely cause	Fix
Script is slow only when output is on screen	Console printing	Redirect to a file or print less
Time grows with the number of files	A process started per file	Batch the work into one command
Long pause at the start of each loop pass	PowerShell or `wmic` called inside the loop	Call once before the loop
A `for /f` command loop is slow	The command runs again for every outer pass	Run it once and store the result in a file or variables
Script slows as the environment grows	Very large variables or many `setlocal` copies	Keep variables small, avoid copying large values
Redirect to the same file in a loop is slow	File reopened every time	Redirect the whole block

21.8 When to stop

If the best batch version is still too slow, or the logic is dominated by text parsing, JSON, large numbers or many files, move that part to **PowerShell** (Chapter 19) and keep Batch as the launcher. A 50-line PowerShell script that does the work in one process often replaces hundreds of lines of Batch and runs orders of magnitude faster.

Summary

- The cost of a batch script is dominated by process starts, file opens, `call` overhead and console output, not by the interpreter.
- Measure with a timing harness before and after every change.
- Redirect blocks once, prefer `for /l` over `goto`, avoid `call` in hot loops and avoid launching programs per item.
- Give tools the whole input at once; replace external commands with built-ins.
- Call PowerShell once, or switch to it when the work is data-heavy.

Common mistakes

> **Common Mistake: Optimizing without measuring.**

> **Common Mistake: Starting `findstr`, `find` or PowerShell inside a per-item loop.**

Common Mistake: **Printing every item of a large loop.**

Common Mistake: **Making code unreadable for a gain nobody needs.**

Exercises

1. **[Easy]** Use the timing harness to measure how long `ping -n 3 127.0.0.1 >nul` takes.
2. **[Easy]** Rewrite `for /l %%i in (1,1,500) do >> out.txt echo %%i` so that the file is opened once.
3. **[Easy]** Name three operations that start a new process.
4. **[Medium]** Measure the difference between `echo` to the console and `echo` redirected to `nul` for 3000 lines.
5. **[Medium]** Replace a loop that calls `findstr` once per line of a file with a single `findstr` call.
6. **[Medium]** Convert a `goto` counting loop to `for /l` and measure both.
7. **[Hard]** A script calls PowerShell four times in a loop to read four properties. Rewrite it with one call and `NAME=value` output.
8. **[Hard]** Explain why `echo %%w| findstr ...` inside a `for` loop is far slower than one `findstr` over the file, in terms of processes created.

Chapter 22: Debugging and Testing

LEARNING OBJECTIVES

- Apply a systematic method to find the cause of a failure
- Use `echo on`, tracing, `pause` and logging to observe a script
- Recognize the common error messages and their usual causes
- Write simple automated tests with assertion routines
- Add debug and dry-run switches without cluttering the code

22.1 A method

Batch has no debugger, so debugging is disciplined observation:

1. **Reproduce** the failure with the smallest input that triggers it.
2. **Observe** what the script is actually doing (not what you intended).
3. **Isolate** the failing line by cutting the script down or adding output before and after it.
4. **Form one hypothesis**, change one thing, and run again.
5. **Keep a test** that fails before the fix and passes after it.

The most common cause of a wrong result is a wrong assumption about **when a variable is expanded** (Chapter 7) or **which quote or special character the parser sees** (Chapter 12). Observation reveals both.

22.2 Seeing what runs: `echo on`

The `@echo off` at the top of a script hides commands. To see each command **after variable expansion**, turn echo back on:

trace_demo.bat

```
@echo off
set "name=World"
echo on
echo Hello, %name%
set /a "x=3*4"
echo off
echo Finished.
```

OUTPUT

```
C:\Users\Alex\BatchBook\scripts>echo Hello, World
Hello, World

C:\Users\Alex\BatchBook\scripts>set /a "x=3*4"
```

```
C:\Users\Alex\BatchBook\scripts>echo off
Finished.
```

The echoed line shows the command with `%name%` already replaced, which is exactly what `cmd.exe` is about to run. Turn echo on only around the suspect lines, or remove `@echo off` entirely while debugging.

Tracing a whole run into a file

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> cmd /c "call myscript.bat > trace.log 2>&1"
```

With `@echo off` replaced by `echo on` at the top, `trace.log` contains every command and every output line in order, ready to read at leisure.

Tracing loops and blocks

Inside `for` and `if` blocks `echo on` shows each command as it runs, which exposes stale-variable problems immediately:

trace_loop.bat

```
@echo off
setlocal EnableDelayedExpansion
set "sum=0"
echo on
for %%n in (1 2) do (
    set /a "sum+=%%n"
    echo sum is now %sum% / !sum!
)
@echo off
```

TYPICAL OUTPUT (THE LAYOUT OF ECHOED BLOCKS MAY DIFFER SLIGHTLY BETWEEN WINDOWS VERSIONS)

```
C:\Users\Alex\BatchBook\scripts>for %n in (1 2) do (
set /a "sum+=1"
 echo sum is now 0 / !sum!
)
sum is now 0 / 1

C:\Users\Alex\BatchBook\scripts>(
set /a "sum+=2"
 echo sum is now 0 / !sum!
)
sum is now 0 / 3
```

The trace makes the difference between `%sum%` and `!sum!` visible: the percent form was replaced with `0` before the loop started.

22.3 Other observation tools

Technique	How
Show a variable	`echo [%var%]` (the brackets reveal leading and trailing spaces)
Show all variables	`set` or `set prefix`
Show arguments	`echo [%1] [%2] [%~3] [%*]`
Pause at a point	`pause` or `echo Checkpoint 3 & pause`
Keep the window	Run with `cmd /k script.bat`
Check the exit code	`echo %errorlevel%` right after the command
Count what loops see	`echo iteration %%i` as the first line of a body
Check the working folder	`echo %CD%` and `echo %~dp0`

> **Best Practice:** Wrap values in brackets when you print them: `echo [%name%]` . An empty value, a trailing space and a missing variable look the same without brackets, and different with them.

22.4 A debug switch

Keep diagnostic output in the script permanently but silent, controlled by a variable:

debug_switch.bat

```
@echo off
setlocal
rem Run with:  set DEBUG=1  &  debug_switch.bat   to see the diagnostics.
call :dbg "Script started in %CD%"
set "files=0"
for %%f in (*.bat) do set /a "files+=1"
call :dbg "Counted %files% batch files"
echo Batch files here: %files%
exit /b 0

:dbg  message
if defined DEBUG echo [DEBUG] %~1
exit /b 0
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> set DEBUG=1
C:\Users\Alex\BatchBook\scripts> debug_switch.bat
```

OUTPUT

```
[DEBUG] Script started in C:\Users\Alex\BatchBook\scripts
[DEBUG] Counted 34 batch files
Batch files here: 34
```

Step mode

For long scripts a **step mode** pauses at chosen checkpoints, so you can inspect state interactively:

step_mode.bat

```
@echo off
setlocal
call :step "Reading configuration"
set "cfg=demo"
call :step "Processing"
set "result=done"
call :step "Finishing"
echo Result: %result%
exit /b 0

:step  description
if not defined STEP exit /b 0
echo.
echo [STEP] %~1
echo        cfg=[%cfg%] result=[%result%]
pause
exit /b 0
```

With `set STEP=1` defined, the script stops at each `:step`, prints the state and waits for a key; without it, `:step` costs nothing.

22.5 Logging strategies

Logs are the only debugging tool for unattended runs (scheduled tasks, remote sessions). Practices that pay off:

- **Timestamp** every line and include a **level** (INFO, WARN, ERROR) as in Chapters 11 and 14.
- Log **inputs** (arguments, configuration) at the start and **outcomes** (exit codes, counts) at the end.
- Redirect **both** streams of every external command you care about: `command >> "%LOG%" 2>&1`.
- Keep logs **bounded**: delete or rotate old logs (Chapter 32).
- Log to a path that exists and is writable under the account that runs the script (a scheduled task may run as another user).
- Never log secrets such as passwords or tokens (Chapter 23).

22.6 Common error messages

Message	Usual cause and remedy
`'name' is not recognized as an internal or external command, operable program or batch file.`	Misspelled command, program not on `PATH`, missing extension, or a variable that expanded to the wrong text. Check with `where name`
`The system cannot find the path specified.`	A folder does not exist or the path is misspelled or lacks quotes

Message	Usual cause and remedy
`The system cannot find the file specified.`	The file name is wrong, or the current folder is not what you assumed
`The system cannot find the batch label specified - name`	`goto` or `call` to a label that does not exist; a typo; or LF-only line endings
`... was unexpected at this time.`	Syntax error: unquoted `)` inside a block, an empty variable in an `if`, a missing space or bracket
`The syntax of the command is incorrect.`	Wrong arguments for the command, often an empty variable or an unquoted path with spaces
`Missing operand.`	`set /a` with an empty expression or variable
`Invalid number. Numeric constants are either decimal (17), hexadecimal (0x11), or octal (021).`	A value such as `08` or `09`, or text, in `set /a`
`Invalid number. Numbers are limited to 32-bits of precision.`	A value above 2,147,483,647
`Access is denied.`	Missing permissions, read-only file, or a folder that needs administrator rights
`The process cannot access the file because it is being used by another process.`	The file is open elsewhere; close the program or retry
`Environment variable name not defined`	`set name` on a variable that does not exist (normal; test with `if defined`)
`The input line is too long.`	A command exceeds the limit of 8191 characters
`A subdirectory or file name already exists.`	`md` of an existing folder (harmless; check with `if not exist`)
`ECHO is off.`	`echo %var%` with an empty variable printed the state instead of text; use `echo(%var%`
A `More?` prompt appears	A caret at the end of a line, an unclosed quote or an unclosed parenthesis
`CMD.EXE was started with the above path as the current directory. UNC paths are not supported.`	Script run from a network path; use `pushd "%~dp0"`

22.7 Worked example: three bugs in one script

buggy.bat

```
@echo off
set total = 0
for %%f in (*.txt) do (
    set /a total+=1
    echo Found %%f, total so far %total%
```

```
)
if %total% == 0 echo No files found.
```

TYPICAL OUTPUT (THE TOKEN NAMED IN THE FINAL MESSAGE MAY DIFFER)

```
Found a.txt, total so far
Found b.txt, total so far
== was unexpected at this time.
```

Diagnosis, one hypothesis at a time:

1. `set total = 0` creates a variable named `total` (with a trailing space). Fix: `set "total=0"` (Chapter 5).
2. `%total%` **inside the loop** shows an empty value, because the block is expanded once before it runs. Fix: enable delayed expansion and write `!total!` (Chapter 7).
3. `if %total% == 0` after the loop: `total` was never defined by the first bug, so the line becomes `if == 0 ...`, a syntax error. Fix: quote or use `equ` after initializing: `if "%total%"=="0"` (Chapter 9).

fixed.bat

```
@echo off
setlocal EnableDelayedExpansion
set "total=0"
for %%f in (*.txt) do (
    set /a "total+=1"
    echo Found %%f, total so far !total!
)
if "%total%"=="0" echo No files found.
```

OUTPUT

```
Found a.txt, total so far 1
Found b.txt, total so far 2
```

22.8 Automated tests

A script that cannot be tested is hard to change safely. The smallest useful framework is a pair of **assertion routines** and a **counter**:

test_framework.bat

```
@echo off
rem Minimal test helpers. Usage: call "test_framework.bat" :assert_equal name expected actual
rem                              call "test_framework.bat" :summary
call %*
exit /b %errorlevel%

:assert_equal  name expected actual
if "%~3"=="%~2" (
    set /a "PASSED+=1"
    echo PASS  %~1
) else (
    set /a "FAILED+=1"
    echo FAIL  %~1  expected [%~2] got [%~3]
)
exit /b 0
```

```
:summary
echo.
echo Passed: %PASSED%  Failed: %FAILED%
if %FAILED% gtr 0 exit /b 1
exit /b 0
```

test_strlen.bat

```
@echo off
setlocal EnableDelayedExpansion
set "PASSED=0"
set "FAILED=0"
set "T=%~dp0test_framework.bat"

call :strlen r "Hello"
call "%T%" :assert_equal "length of Hello" 5 "%r%"
call :strlen r ""
call "%T%" :assert_equal "length of empty string" 0 "%r%"
call :strlen r "a b c"
call "%T%" :assert_equal "length with spaces" 5 "%r%"
call :strlen r "abc"
call "%T%" :assert_equal "deliberately wrong" 4 "%r%"

call "%T%" :summary
exit /b %errorlevel%

:strlen  outvar string
setlocal EnableDelayedExpansion
set "s=#%~2"
set "len=0"
for %%P in (4096 2048 1024 512 256 128 64 32 16 8 4 2 1) do (
    if not "!s:~%%P,1!"=="" (
        set /a "len+=%%P"
        set "s=!s:~%%P!"
    )
)
endlocal & set "%~1=%len%"
exit /b 0
```

OUTPUT

```
PASS  length of Hello
PASS  length of empty string
PASS  length with spaces
FAIL  deliberately wrong  expected [4] got [3]

Passed: 3  Failed: 1
```

The framework reads and updates the variables `PASSED` and `FAILED` of the **calling** script, because it uses no `setlocal`. The final `exit /b` returns 1 when any test failed, so a build system can stop. Always include at least one **deliberately failing** test the first time you write a harness, to prove that failures are reported.

Dry-run mode

A dry-run switch prints what would happen without changing anything. It doubles as a test aid for destructive scripts:

dryrun.bat

```
@echo off
setlocal
set "DRY=1"
if /i "%~1"=="/run" set "DRY=0"

call :do copy "%~dp0a.txt" "%~dp0b.txt"
exit /b 0

:do  command args...
if "%DRY%"=="1" (echo [dry run] %*) else (%*)
exit /b 0
```

OUTPUT

```
[dry run] copy "C:\Users\Alex\BatchBook\scripts\a.txt" "C:\Users\Alex\BatchBook\scripts\b.txt"
```

Summary

- Debug by reproducing, observing, isolating, changing one thing and keeping a test.
- `echo on` shows commands after expansion; bracket values to reveal spaces; trace to a file for long runs.
- Add permanent but silent diagnostics with a `DEBUG` variable, and optional step mode.
- Log with timestamps and levels, both streams, bounded size, and no secrets.
- Learn the common messages: most come from quoting, empty variables, expansion timing, octal numbers and missing paths.
- Write assertion routines and keep regression tests; add dry-run modes to destructive scripts.

Common mistakes

Common Mistake: **Changing several things at once** while debugging.

Common Mistake: **Trusting a script's output without printing the inputs.**

Common Mistake: **Leaving `pause` in a script after debugging.**

Common Mistake: **Testing only the success path.**

Exercises

1. **[Easy]** Turn echo on for two lines of a script and read the output.
2. **[Easy]** Print a variable in brackets to reveal a trailing space you introduced on purpose.

3. **[Easy]** What does `ECHO is off.` mean when you expected text?
4. **[Medium]** Add a `DEBUG` switch to a script of your own, as in Section 22.4.
5. **[Medium]** Find and fix the bug: `set /a n=08+1`.
6. **[Medium]** A script prints `The system cannot find the batch label specified - menu`. List three possible causes.
7. **[Hard]** Write three tests for a `:max a b` routine, including a failing one, using the framework of Section 22.8.
8. **[Hard]** Write `:do` so that a dry-run prints the command and a real run executes it, and show why quotes in the arguments need care.

Chapter 23: Security and Best Practices

LEARNING OBJECTIVES

- Explain how unsanitized input changes what a batch command does, and prevent it
- Validate arguments, file names and paths with allow-lists
- Apply least privilege and protect scripts that run with elevated rights
- Handle credentials without exposing them
- Recognize risky patterns when reviewing someone else's script

Scope of this chapter: Everything here is **defensive and administrative**: how to write scripts that cannot be turned against their owner, and how to read scripts you did not write. It contains no exploit techniques.

23.1 Threat model of a batch file

Ask of every script: **who or what controls its inputs, and what can the script do with the rights it runs under?** The inputs of a script include:

Input	Who can influence it
Command-line arguments	Whoever starts the script, or a program that calls it
Typed answers (`set /p`)	The person at the keyboard
File names and file contents	Anyone who can create files in the folder being processed
Environment variables	Anything that launched the script
Output of other programs	The program, and data that reaches it (network, files)
The current folder and `PATH`	The user and the launching context
The script file itself	Anyone who can write to its folder

The danger multiplies with **privilege**. A script run as administrator, as `SYSTEM` by a scheduler, or on a server turns any flaw into a flaw with full rights.

23.2 Injection: when data becomes commands

`cmd.exe` expands variables **before** it decides where commands begin and end. A variable that contains `&`, `|`, `<`, `>`, `^` or an unbalanced quote can therefore change the structure of the line that uses it. The following demonstration uses a harmless payload that only prints a word:

injection_demo.bat

```
@echo off
setlocal EnableDelayedExpansion
rem Simulate hostile input. The payload only prints the word INJECTED.
set "name=Bob & echo INJECTED"

echo --- 1. unquoted percent expansion (unsafe) ---
echo Hello %name%

echo --- 2. delayed expansion (the value is not re-parsed) ---
echo Hello !name!

echo --- 3. value stored in quotes, shown with quotes ---
echo "Hello %name%"
```

OUTPUT

```
--- 1. unquoted percent expansion (unsafe) ---
Hello Bob
INJECTED
--- 2. delayed expansion (the value is not re-parsed) ---
Hello Bob & echo INJECTED
--- 3. value stored in quotes, shown with quotes ---
"Hello Bob & echo INJECTED"
```

In case 1, the text after `&` ran as a **second command**. With a different payload, that command could delete files or start a program. Cases 2 and 3 avoid the problem **for this payload**, but neither is a defense against every payload: a value containing a **double quote** can close the protective quotes, and delayed expansion damages `!` and `^`. The real defense is **validation**.

The dangerous places

Pattern	Why it is dangerous
`%1`, `%input%` **used unquoted in a command line**	Special characters change the command
`call %input%` **or** `%input%` **as a command**	Runs whatever the text names
`for /f ... in ('dir %arg%')`	The text is part of a command that is run
`goto :%choice%` **or** `call :%choice%`	Jumps to any label, including internal ones
`powershell -Command "... %input% ..."`	The text becomes code (Chapter 19)
`del %pattern%`, `rd /s /q %path%`	Wrong or empty text deletes the wrong things
`set /a "x=%input%"`	Text in an expression can break the line
`if "%a%"=="%b%"` **with quotes in the values**	The quoting is closed early

23.3 Validate with allow-lists

An **allow-list** states what is acceptable and rejects everything else. A **deny-list** tries to list what is bad and always misses something. Prefer allow-lists:

validators.bat

```
@echo off
setlocal EnableDelayedExpansion
call :test_name "report_2026"
call :test_name "report & echo INJECTED"
call :test_name "..\..\Windows"
call :test_number "42"
call :test_number "4 2"
exit /b 0

:test_name  text
set "v=%~1"
call :is_safe_name v
if errorlevel 1 (echo REJECTED name   "%~1") else (echo accepted name   "%~1")
exit /b 0

:test_number  text
set "v=%~1"
call :is_number v
if errorlevel 1 (echo REJECTED number "%~1") else (echo accepted number "%~1")
exit /b 0

:is_safe_name  varname
rem Letters, digits, underscore, hyphen, dot (not at the start).
echo(!%~1!| findstr /r /x "[A-Za-z0-9_-][A-Za-z0-9_.-]*" >nul
exit /b %errorlevel%

:is_number  varname
echo(!%~1!| findstr /r /x "[0-9][0-9]*" >nul
exit /b %errorlevel%
```

OUTPUT

```
accepted name   "report_2026"
REJECTED name   "report & echo INJECTED"
REJECTED name   "..\..\Windows"
accepted number "42"
REJECTED number "4 2"
```

Three design points make these validators themselves safe to run on hostile text:

1. **The text is stored in a variable inside quotes** (`set "v=%~1"`) and the check receives the variable's **name**.
2. **The check reads it with `!%~1!`.** Delayed expansion supplies the value **after** the line has been parsed, so an `&` in the text is just a character. Writing `echo(%~1| findstr ...` instead would paste the text into the command line and run anything after the `&`. This is the flaw demonstrated in Section 23.2, and it would defeat the purpose of the check.
3. `findstr /r /x` requires the **whole** text to match the pattern.

Validate and use **the same variable**. With delayed expansion on, a `!` in the text can be altered when the variable is set, so use only the value that passed the check, never the raw argument again.

Quotes inside input: A double quote inside an argument can end the protective quotes of `set "v=%~1"` early and let the rest of the text run as commands. Batch code cannot make this fully safe. When quotes are possible in the input, handle the input in a program that does not re-parse it: pass it through an environment variable or a parameter to PowerShell (Chapter 19), or take it from a file. In scripts that run with elevated rights, treat an argument that contains a quote as hostile and do not process it.

Constrain paths to a base folder

A file name that looks harmless can contain `..` and escape the folder you intended. Resolve the path first (`%%~f` normalizes `..`), then check that it starts with the allowed folder:

is_under.bat

```
@echo off
setlocal
call :check "C:\Data" "C:\Data\sub\report.txt"
call :check "C:\Data" "C:\DataBackup\x.txt"
call :check "C:\Data" "C:\Data\..\Windows\win.ini"
exit /b 0

:check  root path
call :is_under "%~1" "%~2"
if errorlevel 1 (echo OUTSIDE  %~2) else (echo inside   %~2)
exit /b 0

:is_under  root path
rem Returns 0 if the resolved path lies inside the resolved root folder.
setlocal EnableDelayedExpansion
for %%a in ("%~1") do set "r=%%~fa"
for %%b in ("%~2") do set "p=%%~fb"
if "!r:~-1!"=="\" set "r=!r:~0,-1!"
set "tmp=!r!"
set "len=0"
:measure
if defined tmp (
    set "tmp=!tmp:~1!"
    set /a "len+=1"
    goto :measure
)
set "head=!p:~0,%len%!"
set "next=!p:~%len%,1!"
if /i not "!head!"=="!r!" exit /b 1
if defined next if not "!next!"=="\" exit /b 1
exit /b 0
```

OUTPUT

```
inside   C:\Data\sub\report.txt
OUTSIDE  C:\DataBackup\x.txt
OUTSIDE  C:\Data\..\Windows\win.ini
```

Three subtleties are handled: `C:\DataBackup` starts with the text `C:\Data` but is a **different folder** (the character after the root must be a backslash); the comparison ignores case, as Windows does; and `..` is resolved before the test, so the traversal path is correctly rejected.

23.4 Safe handling of special characters

Rule	Example
Assign in quotes	`set "x=%~1"`
Quote every use of a path or text	`copy "%src%" "%dst%"`
Prefer `%~1` over `%1`	Removes surrounding quotes before you re-add your own
Do not `call` or execute data	Never write `%input%` as a command
Use delayed expansion only to pass text to a check, and avoid it when processing file names	A file named `a!b.txt` is damaged otherwise
Escape `%` as `%%` in generated batch text	
Dispatch on known keys	Use `if /i "%cmd%"=="start"`, not `goto :%cmd%`
Never build PowerShell code from input	Use environment variables or parameters

23.5 Least privilege

- **Do not run as administrator** unless the task needs it, and then only the part that does. Split the script so the privileged code is small and carefully reviewed.
- Run scheduled tasks as a **dedicated low-privilege account** where possible, and only use `SYSTEM` when required.
- Keep scripts that run with high rights in a folder that **only administrators can write to**. If an ordinary user can edit the script, they can run code with the rights of whoever starts it. Check permissions with `icacls "C:\Scripts"`.
- Write output only to folders the account owns (for example under `%ProgramData%\YourTool` or `%LOCALAPPDATA%`), not to `C:\Windows`.
- Never disable security software, firewall or logging from a utility script.

> **Command search order:** `cmd.exe` searches the **current directory first** when it looks for a program. A script that runs `ping` or `find` from a folder where someone planted `ping.bat` or `find.exe` runs the planted file (the same effect as the accidental `ping.bat` in Chapter 4). Mitigations: run from a protected folder, call important tools by full path (`%SystemRoot%\System32\find.exe`), and consider setting the environment variable `NoDefaultCurrentDirectoryInExePath=1`, which tells `cmd.exe` not to search the current directory for executables. The last measure can change the behavior of scripts that rely on running programs from the current folder, so test it.

23.6 Credentials and secrets

A password in a batch file is a password in a plain text file that backups, logs and curious users can read. Rules:

1. **Never store passwords or tokens in a script.**
2. **Do not pass secrets on the command line.** Command lines can be read by other users and tools while the program runs, and often end up in logs.
3. `set /p` **echoes** what the user types. For a hidden prompt use PowerShell (`Read-Host -AsSecureString`) and never echo the value.
4. **Do not write secrets to logs** or to the console. Redact them.
5. Use the platform's stores: the **Windows Credential Manager** (`cmdkey /list` shows stored entries), an encrypted credential file created with PowerShell `Export-Clixml` (it is protected for the same user on the same machine), a secrets manager, or integrated authentication (the task runs as an account that already has access).
6. Avoid `runas /savecred` : it saves credentials in a way that lets that user run the program without a password again.
7. Child programs **inherit environment variables**. A secret placed in a variable is visible to everything the script starts afterward. Clear it (`set "SECRET="`) as soon as possible.

> **Warning:** If a password has ever been in a script, a log, a chat or a ticket, treat it as compromised and **change it**. Removing it from the file does not remove it from backups and history.

23.7 Integrity, signing and application control

- **Batch files cannot be Authenticode-signed.** Windows has no signature format for `.bat` and `.cmd` , in contrast to PowerShell scripts and executables.
- To control which scripts run on managed machines, use **application control** (AppLocker or Windows Defender Application Control) with rules by **location** (protected folders) or by **file hash**.
- To detect tampering, record the SHA-256 hash of a trusted script and check it before running: `certutil -hashfile script.bat SHA256` prints the hash, which you compare with the recorded value.
- Keep scripts in **version control** and review changes (Chapter 24); a reviewed history is the strongest integrity control available.
- Where a signature is required, move the logic to a signed PowerShell script or a signed executable and let the batch file be a thin launcher.

23.8 Temporary files

Use the **per-user** temporary folder (`%TEMP%`), unique names, and clean up:

temp_safe.bat

```
@echo off
setlocal
set "TMPFILE=%TEMP%\mytool_%RANDOM%_%RANDOM%.tmp"
echo working data> "%TMPFILE%"
echo Using %TMPFILE%
type "%TMPFILE%"
del "%TMPFILE%"
```

OUTPUT

```
Using C:\Users\Alex\AppData\Local\Temp\mytool_18233_9071.tmp
working data
```

Predictable names in a shared writable folder (such as `C:\Temp`) let other users pre-create or replace the file. `%RANDOM%` only makes names hard to guess in casual cases; it is not a security feature (Chapter 20).

23.9 Recognizing risky patterns in scripts you did not write

When you receive a script from a colleague, a download or a ticket, **read it before you run it**: open it in a text editor, never double-click it. The following features do not prove that a script is malicious, since administrators use many of them legitimately, but each deserves a careful look and a clear explanation of why it is there.

Pattern	What to ask
Very long lines of `set` assignments and `%var:~5,1%` substring chains	**Obfuscation**: why is the author hiding what the script does?
`powershell` with `-EncodedCommand`, `-enc`, `-WindowStyle Hidden`, `-NoProfile -ExecutionPolicy Bypass` together	What is the command? Decode and read it first
`Invoke-Expression` / `iex` on downloaded text; `DownloadString`, `Invoke-WebRequest` followed by execution	Where does the code come from? Is it pinned and verified?
`certutil -urlcache`, `certutil -decode`, `bitsadmin /transfer`, `curl` to an unfamiliar address	What is being downloaded, and from whom?
`reg add ...\CurrentVersion\Run`, `schtasks /create` at logon with `SYSTEM`, `sc create`	**Persistence**: why does it need to survive reboot?
`net user ... /add`, `net localgroup administrators ... /add`	Account changes need change-control approval
`vssadmin delete shadows`, `wevtutil cl`, `bcdedit` changes, `wbadmin delete`	Destroys recovery data or logs
Commands that disable Defender, the firewall or UAC (`Set-MpPreference`, `netsh advfirewall ... off`, registry policy edits)	Why weaken protections?
`attrib +h +s`, files in odd folders, renamed system tools	Hiding things

Pattern	What to ask
`del /s /q`, `rd /s /q`, `format`, `diskpart`, `cipher /w` on computed paths	Could the path be empty or wrong?
`mshta`, `rundll32`, `regsvr32` with remote or odd arguments	Unusual launchers
Use of credentials, tokens, keys or hard-coded IP addresses	Secrets in plain text; unknown servers

A safe review routine

1. **Do not run it.** Open the file in a text editor.
2. Read it from top to bottom and write down what each section does.
3. Search for the patterns above (`findstr /i /n "powershell certutil bitsadmin reg schtasks net user vssadmin" script.bat`).
4. Decode anything encoded (Base64 text) in a safe tool and read the result as text. Do not execute it.
5. Compute the file's hash (`certutil -hashfile script.bat SHA256`) and check it against a trusted source if one exists.
6. If you must test it, use a **disposable virtual machine** without access to real data or networks.
7. If it looks malicious, **report it** to your security team and do not forward it.

23.10 A secure template

secure_template.bat

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
rem ------------------------------------------------------------------------
rem  Purpose : Copy one report file from the reports folder to the archive.
rem  Usage   : secure_template.bat report_name
rem  Rights  : Standard user. No elevation required.
rem ------------------------------------------------------------------------

set "ROOT=%~dp0reports"
set "ARCHIVE=%~dp0archive"
set "COPY_EXE=%SystemRoot%\System32\robocopy.exe"

rem 1. One argument is required. Keep it in a variable, in quotes.
if "%~1"=="" (echo Usage: %~nx0 report_name & exit /b 2)
set "NAME=%~1"

rem 2. Allow-list validation (delayed expansion hands over the text safely).
echo(!NAME!| findstr /r /x "[A-Za-z0-9_-][A-Za-z0-9_.-]*" >nul
if errorlevel 1 (echo Invalid report name. & exit /b 2)

rem 3. Build and check the path from the validated variable only.
set "SRC=%ROOT%\%NAME%.txt"
if not exist "%SRC%" (echo Report not found. & exit /b 1)

rem 4. Do the work with an explicit tool path and quoted arguments.
if not exist "%ARCHIVE%\" md "%ARCHIVE%"
"%COPY_EXE%" "%ROOT%" "%ARCHIVE%" "%NAME%.txt" /r:1 /w:1 /njh /njs /np >nul
if errorlevel 8 (echo Copy failed. & exit /b 4)
```

```
echo Archived %NAME%.
exit /b 0
```

The template validates early with an allow-list, uses the validated variable (not the raw argument) afterwards, uses a full tool path, quotes everything, never executes data, requires no elevation and returns distinct exit codes. Delayed expansion is used only to hand the unvalidated text to the check.

Summary

- Treat every input as untrusted, and remember that privilege magnifies mistakes.
- Variables expand before parsing, so special characters in data can change a command. Validate with **allow-lists**, quote consistently, never execute data, and reject input that contains quotes.
- Resolve paths and confirm they stay inside the intended folder.
- Use least privilege; protect high-rights scripts with folder permissions; mind the current-directory search order.
- Never store or pass secrets in scripts or command lines; use credential stores and hidden prompts.
- `.bat` files cannot be signed: use protected locations, hashes, application control and review.
- Review unknown scripts without running them; look for obfuscation, encoded commands, downloads, persistence, account changes, tampering and broad deletes.

Common mistakes

Common Mistake: **Using `%1` or `%input%` unquoted in commands.**

Common Mistake: **Cleaning input by removing "bad characters" instead of rejecting it.**

Common Mistake: **Placing passwords in scripts or arguments.**

Common Mistake: **Letting ordinary users write to the folder of a privileged script.**

Common Mistake: **Running an unknown script to "see what it does".**

Exercises

1. **[Easy]** Why does `echo Hello %name%` run a second command when `name` contains `&`?
2. **[Easy]** Write an allow-list test for a user name of 3 to 20 letters.
3. **[Easy]** Name two reasons not to put a password on a command line.

4. **[Medium]** Make a script accept a folder argument only if it lies below `%USERPROFILE%\BatchBook\work`, using `:is_under`.
5. **[Medium]** Find the problem: `set /p "cmd=Command: "` followed by `%cmd%`.
6. **[Medium]** List four features in a script that would make you read it more carefully before running it.
7. **[Hard]** Explain why `C:\DataBackup\x.txt` is not inside `C:\Data`, and what a naive prefix test would do.
8. **[Hard]** Describe how you would protect a scheduled script that runs as an administrator account.

Chapter 24: Style Guide and Project Structure

LEARNING OBJECTIVES

- Apply consistent naming, commenting and formatting conventions
- Document a script with a standard header
- Split a larger tool into a main script, libraries and configuration
- Load settings from a configuration file safely
- Keep scripts in version control with correct line-ending settings

Batch has no formatter and no linter. Consistency comes from a **written convention** that every script follows. The rules below are the ones used in this book's projects.

24.1 Naming

Item	Convention	Example
Script files	Lower case, words joined by underscores or hyphens, starts with a verb	`backup_documents.cmd`
Extension	`.cmd` for new work; `.bat` only when required	`.cmd`
Constants and settings	`UPPER_CASE`	`BACKUP_DIR`
Working variables	`lower_case`	`file_count`
Module-private variables	Prefix with the module name	`log_level`, `log_file`
Labels and routines	`lower_case`, verb first	`:copy_files`, `:is_admin`
Arrays	`name[index]`	`item[1]`
Loop variables	One letter, distinct per nesting level	`%%f`, `%%d`

Avoid names of existing commands for scripts (Chapter 4), avoid spaces in file and folder names that your own scripts create, and never use names that collide with predefined variables (Chapter 5).

24.2 The file header

Every script begins with the same block:

header_template.cmd

```
@echo off
setlocal EnableExtensions
rem ======================================================================
rem  Name        : backup_documents.cmd
rem  Purpose     : Copy the Documents folder to a dated backup folder.
rem  Usage       : backup_documents.cmd [destination]
rem  Parameters  : destination  Optional target folder (default: .\backups)
rem  Exit codes  : 0 success, 2 bad arguments, 3 source missing, 4 copy failed
rem  Requires    : Windows 10 or later, robocopy
rem  Rights      : Standard user
rem  Author      : Your name
rem  Version     : 1.2.0
rem  Changes     : 1.2.0  Added destination argument
rem                1.1.0  Added logging
rem                1.0.0  First version
rem ======================================================================
```

The header answers, for a reader who has never seen the script: what it does, how to call it, what it returns, what it needs and what changed. Keep the **version** and the **change list** up to date.

24.3 Comments

- Explain **why**, not what. `rem Skip the first line: it is a header` helps; `rem set the variable` does not.
- Use `rem` consistently (Chapter 4); never put `::` comments inside blocks.
- Mark each section with a divider line (`rem ---- Validate arguments ----`).
- Keep comments in plain ASCII.
- Delete dead code instead of commenting it out; version control remembers it.
- Put a short comment above every routine, stating its parameters and what it returns, as in the `:name arg1 arg2` convention of Chapter 11.

24.4 Layout and formatting

Rule	Reason
Indent with four spaces inside blocks	Makes nesting visible; tabs display differently
One command per line (avoid long `&` chains)	Readable diffs and easier debugging
Keep lines under about 100 characters; continue with `^`	Fits in editors and reviews
Open the block on the same line: `if cond (` , close with `) else (`	The only valid `else` syntax
Quote every path, and assign with `set "x=y"`	Safety (Chapters 5 and 23)
Put constants first, then main, then routines	Predictable order
End main with `exit /b` , each routine with `exit /b n`	No fall-through (Chapter 11)

Rule	Reason
Blank line between routines	Readability
Save with CRLF line endings	Required for reliable `goto` and `call` labels

layout_example.cmd

```
@echo off
setlocal EnableExtensions

rem ---- Configuration ---------------------------------------------------
set "SOURCE=%~dp0input"
set "LIMIT=100"

rem ---- Main ------------------------------------------------------------
call :count_files "%SOURCE%"
if errorlevel 1 exit /b 3

if %file_count% gtr %LIMIT% (
    echo Too many files: %file_count%
    exit /b 4
) else (
    echo File count is acceptable: %file_count%
)
exit /b 0

rem ---- Routines --------------------------------------------------------

:count_files  folder
rem Sets file_count to the number of files directly in the folder.
if not exist "%~1\" exit /b 1
set "file_count=0"
for %%f in ("%~1\*") do set /a "file_count+=1"
exit /b 0
```

24.5 Modular scripts

When a script grows past a few hundred lines, or when two scripts share code, split it:

1. **Main script** (`bin\tool.cmd`): argument handling and the overall flow.
2. **Libraries** (`lib\*.cmd`): routines used by several scripts, called with `call "%~dp0..\lib\common.cmd" :routine args` (Chapter 11).
3. **Configuration** (`config\*.ini`): values that change between machines.
4. **Tests** (`tests\*.cmd`): assertion scripts (Chapter 22).

common.cmd

```
@echo off
rem Shared routines. Usage: call "path\common.cmd" :routine args
call %*
exit /b %errorlevel%

:log  level message
rem Writes a timestamped line to %LOG_FILE% if it is defined, and always echoes it.
echo [%~1] %~2
if defined LOG_FILE >> "%LOG_FILE%" echo %date% %time% [%~1] %~2
exit /b 0
```

```
:require_tool  name
where /q "%~1"
if errorlevel 1 (
    call :log ERROR "Required tool not found: %~1"
    exit /b 1
)
exit /b 0
```

main_tool.cmd

```
@echo off
setlocal EnableExtensions
set "LIB=%~dp0common.cmd"

call "%LIB%" :log INFO "Starting"
call "%LIB%" :require_tool robocopy || exit /b 3
call "%LIB%" :require_tool nosuchtool_xyz || exit /b 3
call "%LIB%" :log INFO "Finished"
exit /b 0
```

OUTPUT

```
[INFO] Starting
[ERROR] Required tool not found: nosuchtool_xyz
```

The main script stops after the missing tool (exit code 3) and never prints `Finished`. The library has a single dispatch line, `call %*`, and its routines can be reused unchanged by every script of the project.

24.6 Configuration files

Settings that differ between machines or customers belong in a **text file**, not in code. A simple `name=value` format is easy to read with `for /f`:

settings.ini

```
# Settings for backup_documents.cmd
# Lines starting with a hash are comments.
SOURCE_DIR=C:\Users\Alex\Documents
BACKUP_DIR=D:\Backups
KEEP_DAYS=14
```

load_config.cmd

```
@echo off
setlocal EnableExtensions
call :load_config "%~dp0settings.ini" CFG_
echo Source : %CFG_SOURCE_DIR%
echo Backup : %CFG_BACKUP_DIR%
echo Keep   : %CFG_KEEP_DAYS% days
exit /b 0

:load_config  file prefix
rem Reads name=value lines, ignores lines that start with #, and sets
rem variables named <prefix><name>. Blank lines are skipped by for /f.
if not exist "%~1" (echo Config file not found: %~1 & exit /b 1)
```

```
for /f "usebackq eol=# tokens=1,* delims==" %%a in ("%~1") do set "%~2%%a=%%b"
exit /b 0
```

OUTPUT

```
Source : C:\Users\Alex\Documents
Backup : D:\Backups
Keep   : 14 days
```

Design points: a **prefix** (`CFG_`) keeps loaded settings from colliding with other variables; `eol=#` ignores comment lines; `tokens=1,*` keeps `=` characters that appear inside a value; defaults should be set **before** loading so that missing keys still have a value.

> **Warning:** A configuration file is **input**. If ordinary users can edit a file that a privileged script reads, they control that script. Protect the file with the same permissions as the script (Chapter 23), validate values before using them (for example, `KEEP_DAYS` must be digits), and never execute a value.

24.7 Version control

Keep scripts in a version-control system such as Git. Benefits: history, review, rollback, and a shared single source of truth.

Two settings are specific to batch files. First, **line endings**: a repository that converts files to LF breaks labels. Add a `.gitattributes` file that forces CRLF for batch files:

.gitattributes

```
*.bat text eol=crlf
*.cmd text eol=crlf
*.ps1 text eol=crlf
*.md  text eol=lf
```

Second, keep generated and machine-specific files out of the repository with a `.gitignore` :

.gitignore

```
logs/
*.log
*.tmp
config/*.ini
!config/*.ini.example
```

Recommended repository layout

project_layout.txt

```
mytool/
    README.md              what it is, how to run it, requirements
    CHANGELOG.md           what changed in each version
    .gitattributes         forces CRLF for batch files
    .gitignore
    bin/
```

```
    mytool.cmd          main script
lib/
    common.cmd          shared routines
config/
    mytool.ini.example  sample settings (the real .ini is not committed)
tests/
    test_common.cmd     assertion tests
logs/                   created at run time, not committed
```

Review checklist

> **Best Practice:** Before accepting a change to a batch file, a reviewer checks that: the header and version are updated; every path and variable is quoted; arguments are validated; destructive commands have dry-run and safety checks; exit codes are documented and returned; `exit /b` precedes the routines; no `pause` remains in unattended scripts; no secrets are present; tests pass; and the file uses CRLF line endings.

24.8 A style checklist for your own scripts

1. Header block with name, purpose, usage, exit codes, version.
2. `@echo off` and `setlocal` first.
3. Constants in one block at the top.
4. Arguments validated; usage printed on error.
5. Main flow short, calling routines with clear names.
6. Every routine documented, local variables protected.
7. All paths quoted; assignments quoted.
8. Errors checked, logged and reflected in the exit code.
9. No fall-through; `exit /b` after main.
10. Tested, including the failure paths; kept in version control.

Summary

- Adopt written conventions for naming, headers, comments and layout, and follow them in every script.
- Split growing tools into main script, libraries, configuration and tests.
- Read settings from a protected `name=value` file with a prefix and defaults.
- Use version control with `.gitattributes` forcing CRLF for batch files, and review changes against a checklist.

Common mistakes

Common Mistake: Inconsistent naming and no header. The next maintainer has to read all the code to learn the basics.

Common Mistake: Committing batch files with LF endings.

Common Mistake: Putting machine-specific paths and passwords in code.

Common Mistake: Copy-pasting routines between scripts instead of sharing a library.

Exercises

1. **[Easy]** Write a header block for a script `cleanup_temp.cmd` that you plan to build.
2. **[Easy]** Rename these to follow the conventions: variable `Folder Name`, script `Backup Script V2.bat`, label `:DoIt`.
3. **[Easy]** Why is `rem` preferred to `::` inside blocks?
4. **[Medium]** Write a `.gitattributes` for a repository of batch and PowerShell files.
5. **[Medium]** Create a `settings.ini` and a loader that sets default values before loading.
6. **[Medium]** Move two routines of a script of your own into `lib\common.cmd` and call them.
7. **[Hard]** Extend `load_config` so that `KEEP_DAYS` is rejected unless it is a whole number from 1 to 365.
8. **[Hard]** Write a review checklist of ten points for batch files specific to your team.

Part 4 Quiz

1. **Q1.** How do you emulate an array in Batch, and how do you read element `i` inside a loop?
2. **Q2.** Why does `call :pop item & echo %item%` show an old value?
3. **Q3.** Why is `ping -n 3 127.0.0.1` a one-line sleep of about two seconds?
4. **Q4.** Which is faster for counting to 20000: `goto` or `for /l` ?
5. **Q5.** Why is starting `findstr` once per line of a file slow?
6. **Q6.** What does `echo [%var%]` help you see?
7. **Q7.** What is the usual cause of `The system cannot find the batch label specified`?
8. **Q8.** What error does `set /a "x=08"` raise, and why?
9. **Q9.** Why is `echo(%~1| findstr ...` unsafe as a validation of hostile text, and what is the safer form?
10. **Q10.** Can `.bat` files be Authenticode-signed? What can you do instead?
11. **Q11.** Why is a folder that ordinary users can write to a bad place for a privileged script?
12. **Q12.** Name three suspicious features to look for when reviewing an unknown script.
13. **Q13.** Which line-ending style must batch files have in a Git repository, and how do you enforce it?

5 Projects

Chapters 25 to 34

Ten complete, working tools of increasing difficulty

Part 5 puts everything together. Each chapter is one project with a **goal**, **requirements**, **full source code**, a **block-by-block explanation** and **suggested improvements**. The projects use the conventions of Chapter 24 and the safety rules of Chapters 14 and 23: destructive tools default to a dry run, input is validated, and every tool returns documented exit codes.

Projects in this Part

- Chapter 25, Project 1: Number Guessing Game
- Chapter 26, Project 2: System Information Report Generator
- Chapter 27, Project 3: File Organizer by Extension
- Chapter 28, Project 4: Bulk File Renamer
- Chapter 29, Project 5: Log File Analyzer
- Chapter 30, Project 6: Automated Backup Tool with Logging
- Chapter 31, Project 7: Disk Usage Monitor
- Chapter 32, Project 8: Scheduled Cleanup Utility
- Chapter 33, Project 9: Network Diagnostics Reporter
- Chapter 34, Project 10: Menu-Driven Administration Toolkit
- Part 5 Quiz

> **Best Practice:** Type the programs yourself rather than copying them blindly, run each one first on practice data in `%USERPROFILE%\BatchBook\work`, and read the explanation after you have seen the program run. Then do the suggested improvements: they are the best exercises in the book.

Chapter 25: Project 1, Number Guessing Game

LEARNING OBJECTIVES

- Combine variables, arithmetic, loops, conditions and input validation in one program
- Use `%RANDOM%`, `set /p`, `choice` and delayed expansion together
- Structure a small interactive program with labels

Goal

A text game in which the computer picks a secret number from 1 to 100 and the player has seven attempts to find it, with hints after every guess and the option to play again.

Requirements

1. Pick a random secret number from 1 to `MAX_NUMBER`.
2. Allow at most `MAX_TRIES` guesses.
3. Validate each guess: digits only, not too long, within range. Invalid input does **not** use up an attempt.
4. After a wrong guess say whether it was too low or too high, and say when it is very close.
5. At the end show the result, offer another game, and show the score when the player quits.
6. Never fail on bad input, including leading zeros such as `08` (Chapter 8).

Source code

guess_game.cmd

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
title Number Guessing Game

rem ---- Configuration ------------------------------------------------
set "MAX_NUMBER=100"
set "MAX_TRIES=7"
set "CLOSE_RANGE=5"

rem ---- Score --------------------------------------------------------
set "games=0"
set "wins=0"

:new_game
set /a "secret=%RANDOM% %% MAX_NUMBER + 1"
set "tries=0"
cls
echo ========================================
```

```
echo   NUMBER GUESSING GAME
echo ========================================
echo I am thinking of a number from 1 to %MAX_NUMBER%.
echo You have %MAX_TRIES% attempts.
echo.

:ask
set /a "left=MAX_TRIES-tries"
set "guess="
set /p "guess=Guess (%left% left): "

rem ---- Validate the input ------------------------------------------
if not defined guess goto :ask
echo(!guess!| findstr /r /x "[0-9][0-9]*" >nul
if errorlevel 1 (
    echo Please type a whole number.
    goto :ask
)
if not "!guess:~5,1!"=="" (
    echo That number is too long.
    goto :ask
)

rem Remove leading zeros so that 08 is not read as an octal number.
set "g=!guess!"
:strip_zero
if "!g:~0,1!"=="0" if not "!g!"=="0" set "g=!g:~1!" & goto :strip_zero

if !g! lss 1 (
    echo The number must be between 1 and %MAX_NUMBER%.
    goto :ask
)
if !g! gtr %MAX_NUMBER% (
    echo The number must be between 1 and %MAX_NUMBER%.
    goto :ask
)

rem ---- Evaluate the guess ------------------------------------------
set /a "tries+=1"
if !g! equ !secret! goto :win
if !tries! geq %MAX_TRIES% goto :lose

set /a "diff=g-secret"
if !diff! lss 0 set /a "diff=-diff"
if !g! lss !secret! (echo   Too low.) else (echo   Too high.)
if !diff! leq %CLOSE_RANGE% echo   ...but you are very close!
goto :ask

:win
set /a "games+=1, wins+=1"
echo.
echo   Correct! The number was !secret!. You needed !tries! guess(es).
goto :again

:lose
set /a "games+=1"
echo.
echo   Out of attempts. The number was !secret!.
goto :again

:again
echo.
choice /c YN /n /m "Play again? (Y/N) "
if errorlevel 2 goto :finish
goto :new_game

:finish
cls
echo ========================================
```

```
echo   Final score: !wins! win(s) in !games! game(s)
echo ========================================
endlocal
exit /b 0
```

OUTPUT

```
========================================
  NUMBER GUESSING GAME
========================================
I am thinking of a number from 1 to 100.
You have 7 attempts.

Guess (7 left): 50
  Too high.
Guess (6 left): abc
Please type a whole number.
Guess (6 left): 25
  Too low.
Guess (5 left): 37
  Too high.
  ...but you are very close!
Guess (4 left): 034

  Correct! The number was 34. You needed 4 guess(es).

Play again? (Y/N) N
```

How it works

Configuration block. `MAX_NUMBER`, `MAX_TRIES` and `CLOSE_RANGE` are constants in one place (Chapter 5), so the game can be made harder by editing a single line.

Secret number. `%RANDOM% %% MAX_NUMBER + 1` yields 1 to 100. The `%%` is the modulo operator in a batch file (Chapter 8). `%RANDOM%` is expanded when the line runs, so every new game has a new secret.

The `:ask` loop. The label is the top of the guessing loop. `set "guess="` clears the variable first, because `set /p` keeps the old value if the player only presses Enter (Chapter 5); `if not defined guess goto :ask` then simply asks again.

Validation. Three checks come before the guess is used in arithmetic. `findstr /r /x` with the pattern `[0-9][0-9]*` requires digits only. The value is read with `!guess!` so that characters like `&` typed by the player are never run as commands (Chapters 14 and 23). `!guess:~5,1!` tests whether a sixth character exists, which keeps the number far below the 32-bit limit.

Leading zeros. The loop at `:strip_zero` removes zeros from the left (but leaves a single `0`), so that `034` becomes `34`. Without it, `08` or `09` would cause `Invalid number` errors in comparisons and arithmetic (Chapter 8).

Range check and attempt counting. Invalid or out-of-range input goes back to `:ask` **before** `tries` is incremented, so mistakes cost nothing.

Evaluation. An exact match jumps to `:win`. Otherwise, if the attempts are used up the game jumps to `:lose`. A wrong guess prints "Too low" or "Too high". The absolute difference `diff` triggers the "very close" hint when it is within `CLOSE_RANGE`.

Win, lose and replay. The score variables `games` and `wins` accumulate across rounds. `choice /c YN /n` returns 1 for Y and 2 for N (Chapter 16); the test `errorlevel 2` finishes the game.

> **Best Practice:** The program uses `!g!` and `!secret!` even though the code is not inside a block, where `%g%` would also work. Using one style consistently means that the code keeps working if you later move it into a loop or an `if` block.

Suggested improvements

1. **Difficulty levels:** ask for easy, normal or hard at the start and set `MAX_NUMBER` and `MAX_TRIES` accordingly.
2. **Optimal play hint:** print the minimum number of guesses a binary search needs (`log2` of the range), calculated with a loop.
3. **High-score file:** store the best number of guesses in `scores.txt` and show it at the end (Chapter 13).
4. **Hot and cold:** compare each guess with the previous one and say whether the player is getting warmer.
5. **Two-player mode:** the second player sets the secret with a hidden prompt generated by PowerShell (Chapter 19).
6. **Colour:** use ANSI colours for hints (Chapter 16), with the words kept as the real carrier of meaning.

Summary

- The project combines `set /a` , `%RANDOM%` , `set /p` , `choice` , labels and delayed expansion.
- Input is validated by an allow-list, stripped of leading zeros and range-checked before use.
- Invalid input never costs an attempt; the program cannot be crashed from the keyboard.

Common mistakes

> **Common Mistake: Not clearing the variable before `set /p` .** An empty answer reuses the previous guess.

> **Common Mistake: Using a leading-zero number in arithmetic.** `08` is invalid octal.

> **Common Mistake: Counting an invalid attempt.** Increment the counter only after validation.

Exercises

1. **[Easy]** Change the range to 1 to 20 and the attempts to 5.
2. **[Easy]** Print the secret number at the start (a debug aid) controlled by a variable `DEBUG`.
3. **[Medium]** Add a difficulty menu with `choice` that sets the range and attempts.
4. **[Medium]** Show the list of previous guesses after each attempt.
5. **[Hard]** Store the best (lowest) number of attempts across sessions in a file and display it.

Chapter 26: Project 2, System Information Report Generator

LEARNING OBJECTIVES

- Combine Batch and a companion PowerShell script through `NAME=value` lines
- Generate a formatted, timestamped report file
- Avoid `wmic` by using CIM, and keep the Batch side free of fragile text parsing
- Write report blocks safely when values contain parentheses

Goal

A tool that collects facts about the computer (operating system, hardware, disks, network address, uptime) and writes them to a dated report file, which it also displays.

Requirements

1. One PowerShell run collects every fact; Batch does no screen scraping.
2. The report goes to `reports\sysinfo_<computer>_<timestamp>.txt`, with a timestamp that does not depend on regional settings (Chapter 15).
3. Disk entries are listed for every fixed drive, however many there are.
4. The tool reports clearly if the helper script is missing or the collection fails, and returns exit codes 0, 3 or 4.
5. It must work on Windows 10 and 11 without `wmic`.

Source code

The collector prints one `NAME=value` per line (Chapter 19):

sysinfo_collect.ps1

```powershell
# Prints KEY=value lines for sysinfo.cmd. Output is plain ASCII text.
$ErrorActionPreference = 'Stop'
try {
    $now = Get-Date
    $os  = Get-CimInstance Win32_OperatingSystem
    $cs  = Get-CimInstance Win32_ComputerSystem
    $cpu = Get-CimInstance Win32_Processor | Select-Object -First 1
    $up  = $now - $os.LastBootUpTime

    'TS='          + $now.ToString('yyyyMMdd_HHmmss')
    'NOW='         + $now.ToString('yyyy-MM-dd HH:mm:ss')
    'OS_NAME='     + $os.Caption
    'OS_VERSION='  + $os.Version
    'OS_BUILD='    + $os.BuildNumber
```

```
    'OS_ARCH='     + $os.OSArchitecture
    'INSTALLED='   + $os.InstallDate.ToString('yyyy-MM-dd')
    'BOOT='        + $os.LastBootUpTime.ToString('yyyy-MM-dd HH:mm')
    'UPTIME='      + ('{0} days, {1} hours, {2} minutes' -f $up.Days, $up.Hours, $up.Minutes)
    'MAKER='       + $cs.Manufacturer
    'MODEL='       + $cs.Model
    'RAM_GB='      + [math]::Round($cs.TotalPhysicalMemory / 1GB, 1)
    'CPU='         + $cpu.Name
    'CORES='       + $cpu.NumberOfCores
    'THREADS='     + $cpu.NumberOfLogicalProcessors

    Get-CimInstance Win32_LogicalDisk -Filter 'DriveType=3' | ForEach-Object {
        $letter = $_.DeviceID.Replace(':', '')
        $total  = [math]::Round($_.Size / 1GB, 1)
        $free   = [math]::Round($_.FreeSpace / 1GB, 1)
        'DISK_' + $letter + '=' + $_.DeviceID + '  ' + $total + ' GB total, ' + $free + ' GB free'
    }

    $ip = Get-NetIPAddress -AddressFamily IPv4 |
          Where-Object { $_.IPAddress -notlike '169.254.*' -and $_.IPAddress -ne '127.0.0.1' } |
          Select-Object -First 1
    'IPV4=' + $ip.IPAddress
}
catch {
    exit 1
}
```

The Batch program loads those lines and writes the report:

sysinfo.cmd

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
rem ============================================================================
rem  sysinfo.cmd - writes a system information report
rem  Exit codes: 0 ok, 3 helper script missing, 4 collection failed
rem ============================================================================

set "HERE=%~dp0"
set "PS1=%HERE%sysinfo_collect.ps1"
set "OUT_DIR=%HERE%reports"

rem ---- Preconditions ---------------------------------------------------
if not exist "%PS1%" (
    echo Missing helper script: "%PS1%"
    exit /b 3
)
if not exist "%OUT_DIR%\" md "%OUT_DIR%"

rem ---- Collect ---------------------------------------------------------
echo Collecting information, please wait...
for /f "usebackq tokens=1,* delims==" %%a in (`powershell -NoProfile -ExecutionPolicy Bypass -File "%PS1%"`) do s
et "SI_%%a=%%b"
if not defined SI_TS (
    echo Could not collect the information.
    exit /b 4
)

rem ---- Write the report ------------------------------------------------
set "REPORT=%OUT_DIR%\sysinfo_%COMPUTERNAME%_%SI_TS%.txt"
(
    echo ================================================================
    echo  SYSTEM INFORMATION REPORT
    echo ================================================================
    echo  Generated  : !SI_NOW!
    echo  Computer   : %COMPUTERNAME%
    echo  User       : %USERNAME%
```

```
    echo(
    echo [Operating system]
    echo   Name       : !SI_OS_NAME!
    echo   Version    : !SI_OS_VERSION! ^(build !SI_OS_BUILD!^), !SI_OS_ARCH!
    echo   Installed  : !SI_INSTALLED!
    echo   Last boot  : !SI_BOOT!
    echo   Uptime     : !SI_UPTIME!
    echo(
    echo [Hardware]
    echo   Maker      : !SI_MAKER!
    echo   Model      : !SI_MODEL!
    echo   Processor  : !SI_CPU!
    echo   Cores      : !SI_CORES! cores, !SI_THREADS! threads
    echo   Memory     : !SI_RAM_GB! GB
    echo(
    echo [Disks]
    for /f "tokens=1,* delims==" %%a in ('set SI_DISK_ 2^>nul') do echo   %%b
    echo(
    echo [Network]
    echo   IPv4       : !SI_IPV4!
    echo(
    echo End of report
) > "%REPORT%"

rem ---- Show it ------------------------------------------------------
type "%REPORT%"
echo.
echo Report saved to: "%REPORT%"
exit /b 0
```

OUTPUT

```
Collecting information, please wait...
================================================================
 SYSTEM INFORMATION REPORT
================================================================
 Generated  : 2026-10-23 10:14:32
 Computer   : DESKTOP-4F2K9
 User       : Alex

[Operating system]
  Name       : Microsoft Windows 11 Pro
  Version    : 10.0.26100 (build 26100), 64-bit
  Installed  : 2025-03-02
  Last boot  : 2026-10-22 07:58
  Uptime     : 1 days, 2 hours, 16 minutes

[Hardware]
  Maker      : LENOVO
  Model      : 21CB0045US
  Processor  : 12th Gen Intel(R) Core(TM) i7-1260P
  Cores      : 12 cores, 16 threads
  Memory     : 15.7 GB

[Disks]
  C:  475.7 GB total, 210.2 GB free

[Network]
  IPv4       : 192.168.1.23

End of report
```

```
Report saved to: "C:
\Users\Alex\BatchBook\scripts\reports\sysinfo_DESKTOP-4F2K9_20261023_101432.txt"
```

How it works

The collector. `sysinfo_collect.ps1` queries the system once through CIM and prints text lines. Everything is ASCII. The timestamp line `TS` uses a fixed format, which solves the locale problem of Chapter 15. If anything fails, `exit 1` is returned and nothing partial is trusted.

Execution policy. The `-ExecutionPolicy Bypass` option on the call lets the local helper script run on systems where the default policy blocks script files. It applies only to this process (Chapter 19). The helper is a file you wrote and keep next to the batch file, not downloaded code.

Preconditions. The Batch file checks that the helper exists and creates the `reports` folder, each with its own exit code (Chapter 14).

Loading the facts. `for /f "usebackq tokens=1,* delims=="` splits each line at the **first** `=`. The first token (`OS_NAME`) becomes part of the variable name, prefixed with `SI_`, and the rest of the line is the value, even if the value contains more `=` characters. A prefix prevents collisions with existing variables (Chapter 24).

Success test. Because `SI_TS` is always printed first by a healthy run, `if not defined SI_TS` detects a failed collection.

Writing the report. The whole group in `( ... ) > "%REPORT%"` is redirected once (Chapter 21). Inside the group values are read with `!name!`, not `%name%`. This is essential: the processor name contains parentheses (`Intel(R) Core(TM)`), and an unquoted `)` inside a block would end it early (Chapter 9). Delayed expansion inserts the text **after** the block has been parsed, so the parentheses are harmless. The literal parentheses in `^(build ...^)` are escaped for the same reason.

The disk list. The disks are variables named `SI_DISK_C`, `SI_DISK_D`, and so on. `set SI_DISK_` lists all of them, and the loop prints just the value (token 2).

Showing the result. `type` displays the report and the path is printed in quotes.

Suggested improvements

1. **Add more sections:** installed memory modules, BIOS version, battery state, and the five largest folders of a given path.
2. **HTML output:** produce an HTML report with a table by writing tags with `echo`, or let PowerShell create it with `ConvertTo-Html`.
3. **Several machines:** read computer names from a text file and run the collector through PowerShell remoting, merging reports (needs administrative setup).
4. **Compare reports:** keep a baseline and use `fc` (Chapter 13) to show what changed since the last report.
5. **Retention:** delete reports older than 90 days using `forfiles` (Chapter 32).

Summary

- A companion PowerShell script returns facts as `NAME=value` lines; Batch loads them with `for /f` and a variable prefix.
- A timestamp in a fixed format makes names sortable and avoids locale problems.
- Values read inside a redirected block must use delayed expansion when they may contain parentheses.
- Precondition checks and distinct exit codes make the tool automatable.

Common mistakes

Common Mistake: Reading values with `%name%` inside the report block. A `)` in the data ends the block early.

Common Mistake: Parsing English labels of `systeminfo`. The report breaks on localized Windows. Query properties by name instead.

Common Mistake: Using `wmic`. It is removed from current Windows 11.

Exercises

1. **[Easy]** Add the BIOS version to the report (`Win32_BIOS`, property `SMBIOSBIOSVersion`).
2. **[Easy]** Change the report file name to include the date only.
3. **[Medium]** Add a `[Users]` section that lists local user names, using `Get-LocalUser` in the collector.
4. **[Medium]** Make the tool print a warning if any disk has less than 10 percent free space.
5. **[Hard]** Save a copy of the report named `latest.txt` and print the differences against the previous latest using `fc`.

Chapter 27: Project 3, File Organizer by Extension

LEARNING OBJECTIVES

- Write a tool that moves files safely, with a dry run as the default
- Use variables as a lookup table (a dictionary) to map extensions to folders
- Handle name collisions, odd file names and failures
- Apply the safety checks of Chapters 14 and 23

Goal

A utility that takes a messy folder (for example a Downloads folder) and moves its files into sub-folders by type: `Images`, `Documents`, `Audio`, `Video`, `Archives`, `Installers`, `Other` and `NoExtension`.

Requirements

1. The **default is a dry run**: show what would happen and change nothing. Moving happens only with an explicit `/run` switch.
2. Refuse drive roots and protected folders.
3. Only files directly inside the folder are handled; existing sub-folders are untouched.
4. Never overwrite: if the destination name exists, append `_1`, `_2`, and so on.
5. File names with special characters (`&`, `^`, `%`, parentheses, spaces) must not break the script or run commands.
6. Log every move and every failure; return 0 on success and 4 if any move failed.

Source code

organize_files.cmd

```
@echo off
setlocal EnableExtensions DisableDelayedExpansion
rem ================================================================
rem  organize_files.cmd - sort the files of a folder into type folders
rem  Usage     : organize_files.cmd folder [/run]
rem  Default   : dry run (shows actions, changes nothing)
rem  Exit codes: 0 ok, 2 usage, 3 refused or not found, 4 some moves failed
rem ================================================================

set "TARGET=%~1"
set "MODE=DRY"
if /i "%~2"=="/run" set "MODE=RUN"
set "LOG=%~dp0organize_log.txt"
```

```
if not defined TARGET goto :usage
if "%TARGET%"=="/?" goto :usage

rem ---- Resolve and protect the target ------------------------------
for %%f in ("%TARGET%") do set "TARGET=%%~ff"
if not exist "%TARGET%\" (
    echo Folder not found: "%TARGET%"
    exit /b 3
)
if "%TARGET:~3%"=="" (
    echo Refusing to organize a drive root.
    exit /b 3
)
for %%p in ("%windir%" "%ProgramFiles%" "%USERPROFILE%") do (
    if /i "%TARGET%"=="%%~p" (
        echo Refusing protected folder: "%TARGET%"
        exit /b 3
    )
)

rem ---- Extension to category table ---------------------------------
for %%e in (jpg jpeg png gif bmp webp tif tiff svg) do set "CAT_%%e=Images"
for %%e in (pdf doc docx xls xlsx ppt pptx txt rtf odt csv md) do set "CAT_%%e=Documents"
for %%e in (mp3 wav flac aac ogg m4a) do set "CAT_%%e=Audio"
for %%e in (mp4 mkv avi mov wmv webm) do set "CAT_%%e=Video"
for %%e in (zip rar 7z tar gz) do set "CAT_%%e=Archives"
for %%e in (exe msi) do set "CAT_%%e=Installers"

set "handled=0"
set "failed=0"

echo Mode   : %MODE%
echo Folder : "%TARGET%"
echo.

for %%f in ("%TARGET%\*") do call :process "%%~ff"

echo.
echo Files handled: %handled%   Failed: %failed%
if "%MODE%"=="DRY" echo This was a dry run. Add /run to move the files.
if %failed% gtr 0 exit /b 4
exit /b 0

:usage
echo Usage: %~nx0 folder [/run]
echo   Sorts the files directly inside "folder" into sub-folders by type.
echo   Without /run it only shows what it would do (dry run).
exit /b 2

rem ---- Routines ----------------------------------------------------

:process  fullpath
rem Decide the category, find a free name, then move (or only report).
set "ext=%~x1"
set "cat="
if not defined ext (set "cat=NoExtension") else call :categorize
if not defined cat set "cat=Other"

set "dest=%TARGET%\%cat%"
set "newname=%~nx1"
set "n=0"

:check_name
if not exist "%dest%\%newname%" goto :do_move
set /a "n+=1"
set "newname=%~n1_%n%%~x1"
goto :check_name
```

```
:do_move
if "%MODE%"=="DRY" (
    echo [dry run] "%~nx1"  to  "%cat%\%newname%"
    set /a "handled+=1"
    exit /b 0
)
if not exist "%dest%\" md "%dest%"
move "%~1" "%dest%\%newname%" >nul 2>&1
if errorlevel 1 (
    echo [FAILED]  "%~nx1"
    >> "%LOG%" echo FAILED "%~1"
    set /a "failed+=1"
) else (
    echo [moved]   "%~nx1"  to  "%cat%\%newname%"
    >> "%LOG%" echo MOVED  "%~1" to "%dest%\%newname%"
    set /a "handled+=1"
)
exit /b 0

:categorize
rem ext holds a value such as .jpg ; remove the dot, accept letters and digits only.
set "ext=%ext:~1%"
set "bad="
for /f "delims=abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789" %%z in ("%ext%") do set "bad=1"
if defined bad exit /b 0
if defined CAT_%ext% call set "cat=%%CAT_%ext%%%"
exit /b 0
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> organize_files.cmd "%USERPROFILE%\BatchBook\work\Downloads"
```

OUTPUT

```
Mode   : DRY
Folder : "C:\Users\Alex\BatchBook\work\Downloads"

[dry run] "photo.JPG"  to  "Images\photo.JPG"
[dry run] "notes.txt"  to  "Documents\notes.txt"
[dry run] "song.mp3"  to  "Audio\song.mp3"
[dry run] "setup.exe"  to  "Installers\setup.exe"
[dry run] "backup.zip"  to  "Archives\backup.zip"
[dry run] "README"  to  "NoExtension\README"
[dry run] "model.xyz"  to  "Other\model.xyz"

Files handled: 7   Failed: 0
This was a dry run. Add /run to move the files.
```

Running again with `/run` prints `[moved]` lines instead of `[dry run]` lines and creates the folders.

How it works

Delayed expansion is deliberately off. File names may contain `!`. With delayed expansion enabled such names would be damaged (Chapter 7), so the program is written without it and uses `call` and subroutines instead.

Dry run by default. `MODE` is `DRY` unless the second argument is `/run`. The decision is made once, at the top, and every action routine checks it (Chapter 22).

Protecting the target. `%%~ff` turns any argument into a full path. The tests reject a drive root (nothing after `X:\`, so `%TARGET:~3%` is empty) and three well-known protected folders, as in `safe_delete.bat` of Chapter 14.

The lookup table. Each `for` line creates variables such as `CAT_jpg=Images` (Chapter 20). Variable names are not case-sensitive, so `CAT_JPG` finds the same entry, which is why `photo.JPG` is recognized.

One call per file. The loop `for %%f in ("%TARGET%\*") do call :process "%%~ff"` examines only files (the wildcard does not match folders). Handing the full path to a routine keeps the loop body short and avoids block-expansion problems.

Categorizing safely. `:categorize` drops the dot from the extension and then checks that **only letters and digits** remain. The `for /f` trick works like this: the characters letters and digits are declared to be **delimiters**; if anything is left after removing them, the loop body runs and sets `bad`. A file called `weird.a&b` is therefore routed to `Other` and its extension is never pasted into a command. This is the allow-list principle of Chapter 23. The lookup then uses `call set "cat=%%CAT_%ext%%%"`, a double expansion (Chapter 20).

Avoiding overwrites. The label `:check_name` increments `n` and builds `name_1.ext`, `name_2.ext` until a free name is found. This loop is written with `goto` and not as a block because `n` changes and is read on every pass.

Moving and logging. `move "%~1" "%dest%\%newname%"` is quoted everywhere. `errorlevel 1` detects a failure (for example a file in use). Each success or failure is printed and appended to the log with the redirection at the start of the line (Chapter 12). In the dry run nothing is moved and nothing is logged.

Exit codes. The summary uses counters that live in the main scope, because `:process` does not call `setlocal`.

> **Warning:** The dry run checks collisions only against files that already exist in the destination. During `/run`, two source files that need the same new name are resolved one after the other, because the first move creates the file that the second check then finds, so nothing is overwritten.

Suggested improvements

1. **Undo:** write each move to a log in a format `source|destination`, and write a second script that reads the log backwards and moves the files back.
2. **Date folders:** place photos in `Images\2026\10` using the file's last-write date (obtained with PowerShell, Chapter 15).
3. **Configuration file:** load the extension table from `categories.ini` (Chapter 24) so that non-programmers can change it.
4. **Age filter:** organize only files older than N days with `forfiles /d -N`.
5. **Recursion:** optionally process sub-folders with `for /r`, skipping the category folders themselves.
6. **Report:** count files and bytes per category and print a summary table.

Summary

- A safe mover has a dry run by default, protects dangerous targets, never overwrites and logs every action.
- Variables named `CAT_<ext>` form a dictionary; case-insensitive names make lookups case-insensitive.
- Delayed expansion is avoided because file names can contain `!`; subroutines and `call set` do the work instead.
- Characters from file names are checked against an allow-list before they are used to build names.

Common mistakes

> **Common Mistake: Moving without a dry run first.** Always rehearse destructive tools.

> **Common Mistake: Enabling delayed expansion** in a tool that processes arbitrary file names.

> **Common Mistake: Overwriting by default.** `move` can silently replace files with `/y` or when it is the default in a script; check for existence first.

Exercises

1. **[Easy]** Add `heic` to the Images category and `log` to Documents.
2. **[Easy]** Print a count of files per category after the run.
3. **[Medium]** Skip files named `desktop.ini`.
4. **[Medium]** Add a `/ext:xyz` option that organizes only files with that extension.
5. **[Hard]** Write the undo script described in improvement 1.

Chapter 28: Project 4, Bulk File Renamer

LEARNING OBJECTIVES

- Parse options and positional arguments with `shift`
- Build new names with substring replacement and zero-padded counters
- Validate every user-supplied fragment with an allow-list before using it in code
- Preview changes, detect collisions and report results

Goal

A tool that renames many files at once: add a prefix or suffix, replace or remove text in names, or number files in sequence, showing a preview first.

Requirements

1. Syntax: `rename_bulk.cmd [/run] [/p:pattern] folder operation text [text2]`.
2. Operations: `prefix`, `suffix`, `replace OLD NEW`, `remove TEXT`, `number BASE`.
3. **Dry run by default**; `/run` performs the renames.
4. The extension is never changed by `prefix`, `suffix`, `replace`, `remove` or `number`.
5. Never overwrite an existing file; report collisions and skip them.
6. All text arguments and the pattern are checked against an allow-list. Names containing a caret are skipped.
7. Exit codes: 0 ok, 2 invalid usage, 3 folder not found, 4 some renames failed.

Source code

rename_bulk.cmd

```
@echo off
setlocal EnableExtensions DisableDelayedExpansion
rem ======================================================================
rem  rename_bulk.cmd - rename many files with a preview
rem  Usage : rename_bulk.cmd [/run] [/p:pattern] folder operation text [text2]
rem  Ops   : prefix TEXT | suffix TEXT | replace OLD NEW | remove TEXT | number BASE
rem  Codes : 0 ok, 2 usage, 3 folder not found, 4 some renames failed
rem ======================================================================

set "MODE=DRY"
set "PATTERN=*"

rem ---- Options come first ------------------------------------------
:parse
set "a=%~1"
if not defined a goto :usage
if /i "%a%"=="/run" (set "MODE=RUN" & shift & goto :parse)
if /i "%a:~0,3%"=="/p:" (set "PATTERN=%a:~3%" & shift & goto :parse)
```

```
rem ---- Positional arguments ------------------------------------------
set "TARGET=%~1"
set "OP=%~2"
set "ARG1=%~3"
set "ARG2=%~4"

if not defined OP goto :usage
if not defined ARG1 goto :usage

rem ---- Validate ------------------------------------------------------
set "OPOK="
for %%o in (prefix suffix replace remove number) do if /i "%OP%"=="%%o" set "OPOK=1"
if not defined OPOK (echo Unknown operation: use prefix, suffix, replace, remove or number. & exit /b 2)

call :check_text PATTERN "[A-Za-z0-9_.*?-]" "pattern" || exit /b 2
call :check_text ARG1    "[A-Za-z0-9_.-]"   "first text" || exit /b 2
if /i "%OP%"=="replace" (
    if not defined ARG2 goto :usage
    call :check_text ARG2 "[A-Za-z0-9_.-]" "second text" || exit /b 2
)

for %%f in ("%TARGET%") do set "TARGET=%%~ff"
if not exist "%TARGET%\" (
    echo Folder not found: "%TARGET%"
    exit /b 3
)

set "renamed=0"
set "skipped=0"
set "unchanged=0"
set "failed=0"
set "seq=0"

echo Mode    : %MODE%
echo Folder  : "%TARGET%"
echo Pattern : %PATTERN%
echo Action  : %OP% %ARG1% %ARG2%
echo.

rem Take a snapshot of the names first: renaming while a wildcard loop is still
rem reading the folder can make the loop meet renamed files again.
for /f "usebackq eol=? delims=" %%f in (`dir /b /a:-d "%TARGET%\%PATTERN%" 2^>nul`) do call :rename_one "%TARGET%
\%%f"

echo.
echo Renamed: %renamed%   Skipped: %skipped%   Unchanged: %unchanged%   Failed: %failed%
if "%MODE%"=="DRY" echo This was a dry run. Add /run to rename the files.
if %failed% gtr 0 exit /b 4
exit /b 0

:usage
echo Usage: %~nx0 [/run] [/p:pattern] folder operation text [text2]
echo   prefix TEXT       add TEXT before the name
echo   suffix TEXT       add TEXT after the name (before the extension)
echo   replace OLD NEW   replace OLD with NEW inside the name
echo   remove TEXT       delete TEXT from the name
echo   number BASE       rename to BASE_001, BASE_002, ... in name order
echo   Without /run only a preview is shown.
exit /b 2

rem ---- Routines ------------------------------------------------------

:check_text  varname charclass label
rem Accepts the value only if every character matches the class (allow-list).
setlocal EnableDelayedExpansion
echo(!%~1!| findstr /r /x "%~2%~2*" >nul
if errorlevel 1 (
    endlocal
```

```
    echo Invalid %~3: only the characters %~2 are allowed.
    exit /b 1
)
exit /b 0

:rename_one  fullpath
set "name=%~n1"
set "ext=%~x1"
set "newbase="

rem Names that contain a caret cannot be handled safely by call/set.
if not "%name:^=%"=="%name%" (
    echo [skipped] "%~nx1" - the name contains a caret
    set /a "skipped+=1"
    exit /b 0
)

if /i "%OP%"=="prefix"  set "newbase=%ARG1%%name%"
if /i "%OP%"=="suffix"  set "newbase=%name%%ARG1%"
if /i "%OP%"=="replace" call set "newbase=%%name:%ARG1%=%ARG2%%%"
if /i "%OP%"=="remove"  call set "newbase=%%name:%ARG1%=%%"
if /i "%OP%"=="number"  call :make_number

if not defined newbase (
    echo [skipped] "%~nx1" - the new name would be empty
    set /a "skipped+=1"
    exit /b 0
)
set "newname=%newbase%%ext%"

if "%newname%"=="%~nx1" (
    set /a "unchanged+=1"
    exit /b 0
)
if /i not "%newname%"=="%~nx1" if exist "%~dp1%newname%" (
    echo [skipped] "%~nx1" - "%newname%" already exists
    set /a "skipped+=1"
    exit /b 0
)

if "%MODE%"=="DRY" (
    echo [dry run] "%~nx1"  to  "%newname%"
    set /a "renamed+=1"
    exit /b 0
)
ren "%~1" "%newname%" >nul 2>&1
if errorlevel 1 (
    echo [FAILED]  "%~nx1"
    set /a "failed+=1"
) else (
    echo [renamed] "%~nx1"  to  "%newname%"
    set /a "renamed+=1"
)
exit /b 0

:make_number
set /a "seq+=1"
set "num=000%seq%"
set "newbase=%ARG1%_%num:~-3%"
exit /b 0
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> rename_bulk.cmd /p:*.jpg "%USERPROFILE%\BatchBook\work\photos" number holiday
```

```
OUTPUT
Mode    : DRY
Folder  : "C:\Users\Alex\BatchBook\work\photos"
Pattern : *.jpg
Action  : number holiday

[dry run] "IMG_4021.jpg"  to  "holiday_001.jpg"
[dry run] "IMG_4022.jpg"  to  "holiday_002.jpg"
[dry run] "IMG_4023.jpg"  to  "holiday_003.jpg"

Renamed: 3   Skipped: 0   Unchanged: 0   Failed: 0
This was a dry run. Add /run to rename the files.
```

Other examples: `rename_bulk.cmd /run "C:\work\docs" prefix 2026_` adds `2026_` to every file; `rename_bulk.cmd /p:*.txt folder replace draft final` changes `report_draft.txt` into `report_final.txt`.

How it works

Option parsing. The loop at `:parse` reads `%~1` into a variable and tests it. `/run` switches the mode, a value starting with `/p:` sets the pattern (`%a:~3%` cuts off the first three characters), and anything else ends the loop. Each recognized option is consumed with `shift` (Chapter 6). The assignments are quoted, so special characters in the options cannot break the line.

Positional arguments. After the options, `%~1` to `%~4` are the folder, the operation and up to two texts.

Operation check. The `for` loop compares the operation with the allowed words and sets a flag. Nothing else is accepted.

Allow-list validation. `:check_text` receives the **name** of a variable and a character class. It builds the pattern `[class][class]*` and tests the text with `findstr /r /x`, which requires the whole text to match. The text is read with `!%~1!` after `setlocal EnableDelayedExpansion` so that characters such as `&` are never executed (Chapters 14 and 23). The pattern may additionally contain `*` and `?`; the other texts may not, because they are inserted into a replacement expression. Because these checks run before any use of the texts, the later `call set` substitutions cannot be abused.

Resolving the folder. `%%~ff` converts the folder argument into a full path, and the existence test uses the trailing-backslash form.

A snapshot of the names. The loop reads the output of `dir /b /a:-d` (files only, sorted by name). Looping directly over a wildcard while renaming can make the loop meet files it has already renamed, because the folder changes under it. The `eol=?` option is used because `?` cannot occur in a file name, so no name is mistaken for a comment (the default end-of-line character is `;`). **One routine per file.** The loop calls `:rename_one` with the full path. The routine takes the name without extension (`%~n1`) and the extension (`%~x1`) and builds a new base name according to the operation:

- `prefix` and `suffix` concatenate the pieces.

- `replace` and `remove` use a **double expansion** (`call set "newbase=%%name:%ARG1%=%ARG2%%%"`) because the search and replacement texts are variables (Chapters 7 and 20). Substitution is case-insensitive.
- `number` calls `:make_number`, which pads the counter with zeros by prefixing `000` and keeping the last three digits.

Safety checks. Before renaming, the routine skips an empty result, skips unchanged names, and skips names whose target already exists (the test is case-insensitive, except that a name that only changes in letter case is allowed through). A caret in the name is rejected because `call` treats carets specially.

Renaming. `ren "%~1" "%newname%"` takes the **full path** as source and a **bare name** as target (Chapter 3). The result is tested with `errorlevel` and counted.

> **Warning:** `ren` renames immediately and offers no undo. Always read the preview. The next section suggests an undo log as an improvement.

Suggested improvements

1. **Undo log:** append `new|old` lines to a log and add an `undo` operation that reverses a run.
2. **Case operations:** add `lower` and `upper` using the replacement trick of Chapter 7.
3. **Date prefix:** prefix each file with its last-modified date in `yyyy-MM-dd` format (obtained through PowerShell, Chapter 15).
4. **Regex replace:** delegate complex renaming to PowerShell's `-replace` operator, passing patterns through environment variables (Chapter 19).
5. **Recursion:** add a `/s` switch that renames in sub-folders with `for /r`.
6. **Counter width and start:** make the width and starting number options.

Summary

- Options are parsed in a `shift` loop; positional arguments follow.
- Every user-supplied fragment is checked against an allow-list before it reaches a command or substitution.
- `call set` provides double expansion for variable search and replacement texts; `ren` needs a bare target name.
- A preview, collision checks and counters make a destructive tool predictable.

Common mistakes

> **Common Mistake: Giving `ren` a path as the new name.** It fails with a syntax error.

Common Mistake: **Renaming without checking whether the target exists.**

Common Mistake: **Putting user text into `call set` without validation.**

Common Mistake: **Forgetting that substitution ignores case**, so `replace A b` also changes `a`.

Exercises

1. **[Easy]** Run the tool in dry-run mode on the practice folder with `suffix _old`.
2. **[Easy]** Change the counter width from three to four digits.
3. **[Medium]** Add a `lower` operation that converts names to lower case.
4. **[Medium]** Make `number` start from a number given in a fifth argument.
5. **[Hard]** Write an undo log and the matching `undo` operation.

Chapter 29: Project 5, Log File Analyzer

LEARNING OBJECTIVES

- Summarize a text log: totals, levels, time range, hourly activity and the most frequent errors
- Combine `findstr`, `find`, `sort` and `for /f` into a small analysis pipeline
- Count occurrences of repeated lines using sort-and-tally
- Write a formatted report with a text histogram

Goal

Read an application log whose lines look like

`2026-10-23 08:05:12 ERROR [db] Connection timeout after 30s`

and produce a report: number of lines, time range, count per level, activity per hour as a text bar chart, and the most frequent error messages.

Requirements

1. Usage: `analyze_log.cmd logfile [topN]` with `topN` a digit from 1 to 9 (default 5).
2. Work with the log format above: date, time, level, `[module]`, message.
3. The report is printed and also saved to `analysis_report.txt`.
4. Handle empty results (for example no errors) without failing.
5. Use temporary files in `%TEMP%` with unique names and remove them at the end.
6. Exit codes: 0 ok, 2 invalid argument, 3 log file not found.

Test data

This helper creates a small sample log so that you can run the analyzer and compare with the output below:

make_sample_log.cmd

```
@echo off
setlocal
set "OUT=%~dp0sample_app.log"
(
    echo 2026-10-23 08:00:01 INFO [app] Service started
    echo 2026-10-23 08:00:05 INFO [db] Connected to database
    echo 2026-10-23 08:05:12 ERROR [db] Connection timeout after 30s
    echo 2026-10-23 08:05:40 WARN [cache] Cache miss ratio high
    echo 2026-10-23 08:07:03 ERROR [db] Connection timeout after 30s
    echo 2026-10-23 08:15:22 INFO [web] Request served in 120 ms
    echo 2026-10-23 08:30:45 ERROR [web] Upstream returned 502
    echo 2026-10-23 09:01:10 INFO [web] Request served in 95 ms
    echo 2026-10-23 09:02:33 ERROR [db] Connection timeout after 30s
```

```
    echo 2026-10-23 09:10:00 WARN [disk] Free space below 15 percent
    echo 2026-10-23 09:15:15 ERROR [web] Upstream returned 502
    echo 2026-10-23 09:45:50 INFO [app] Nightly job finished
    echo 2026-10-23 10:00:01 DEBUG [app] Heartbeat
    echo 2026-10-23 10:00:31 DEBUG [app] Heartbeat
    echo 2026-10-23 10:01:02 ERROR [db] Connection timeout after 30s
    echo 2026-10-23 10:20:20 WARN [cache] Cache miss ratio high
    echo 2026-10-23 10:21:21 ERROR [auth] Invalid token for user 42
    echo 2026-10-23 10:30:00 INFO [app] Service stopping
) > "%OUT%"
echo Created "%OUT%"
```

OUTPUT

```
Created "C:\Users\Alex\BatchBook\scripts\sample_app.log"
```

Source code

analyze_log.cmd

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
rem ============================================================================
rem  analyze_log.cmd - summary report for a log file
rem  Usage : analyze_log.cmd logfile [topN]
rem  Codes : 0 ok, 2 invalid argument, 3 log file not found
rem ============================================================================

set "LOG=%~1"
set "TOPN=%~2"
if not defined LOG goto :usage
if not defined TOPN set "TOPN=5"

echo(!TOPN!| findstr /r /x "[1-9]" >nul
if errorlevel 1 (
    echo topN must be one digit from 1 to 9.
    exit /b 2
)
if not exist "%LOG%" (
    echo Log file not found: "%LOG%"
    exit /b 3
)

set "REPORT=%~dp0analysis_report.txt"
set "MSGS=%TEMP%\alog_%RANDOM%_msgs.tmp"
set "SORTED=%TEMP%\alog_%RANDOM%_sorted.tmp"
set "COUNTS=%TEMP%\alog_%RANDOM%_counts.tmp"
for %%f in ("%LOG%") do set "LOGNAME=%%~nxf"

rem ---- Basic facts -----------------------------------------------------
set "TOTAL=0"
for /f %%n in ('find /c /v "" ^< "%LOG%"') do set "TOTAL=%%n"
set "FIRST="
set "LAST="
for /f "usebackq tokens=1,2" %%a in ("%LOG%") do (
    if not defined FIRST set "FIRST=%%a %%b"
    set "LAST=%%a %%b"
)

rem ---- Tally the error messages ----------------------------------------
if exist "%MSGS%" del "%MSGS%"
if exist "%COUNTS%" del "%COUNTS%"
for /f "usebackq tokens=4* delims= " %%a in (`findstr /c:" ERROR " "%LOG%"`) do >> "%MSGS%" echo %%a %%b
```

```
set "HAVE_ERRORS="
if exist "%MSGS%" (
    set "HAVE_ERRORS=1"
    sort "%MSGS%" /o "%SORTED%"
    set "prev="
    set "cnt=0"
    for /f "usebackq eol=~ delims=" %%l in ("%SORTED%") do (
        set "line=%%l"
        if "!line!"=="!prev!" (
            set /a "cnt+=1"
        ) else (
            if defined prev call :flush
            set "prev=!line!"
            set "cnt=1"
        )
    )
    if defined prev call :flush
)

rem ---- Write the report ------------------------------------------------
(
    echo ================================================================
    echo  LOG ANALYSIS: !LOGNAME!
    echo ================================================================
    echo  Lines      : !TOTAL!
    echo  First entry: !FIRST!
    echo  Last entry : !LAST!
    echo(
    echo [Entries per level]
    for %%L in (ERROR WARN INFO DEBUG) do (
        set "c=0"
        for /f %%c in ('findstr /c:" %%L " "%LOG%" ^| find /c /v ""') do set "c=%%c"
        echo   %%L  !c!
    )
    echo(
    echo [Entries per hour]
    for %%h in (00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23) do (
        set "c=0"
        for /f %%c in ('findstr /r /c:"^[0-9][0-9][0-9][0-9]-[0-9][0-9]-[0-9][0-9] %%h:" "%LOG%" ^| find /c /v
""') do set "c=%%c"
        if !c! gtr 0 (
            set "bar="
            set /a "w=c"
            if !w! gtr 40 set "w=40"
            for /l %%i in (1,1,!w!) do set "bar=!bar!#"
            echo   %%h:00  !bar! !c!
        )
    )
    echo(
    echo [Most frequent errors, top !TOPN!]
    if not defined HAVE_ERRORS (
        echo   No errors found.
    ) else (
        set "shown=0"
        for /f "usebackq tokens=1,* delims= " %%a in (`sort /r "%COUNTS%"`) do (
            if !shown! lss !TOPN! (
                set /a "shown+=1"
                set /a "k=1%%a-100000"
                echo   !k! x  %%b
            )
        )
    )
) > "%REPORT%"

type "%REPORT%"
echo.
echo Report saved to: "%REPORT%"

rem ---- Clean up --------------------------------------------------------
```

```
for %%t in ("%MSGS%" "%SORTED%" "%COUNTS%") do if exist %%t del %%t
exit /b 0

:usage
echo Usage: %~nx0 logfile [topN]
exit /b 2

:flush
rem Appends "count message" with the count padded to five digits for sorting.
set "pad=00000!cnt!"
>> "%COUNTS%" echo !pad:~-5! !prev!
exit /b 0
```

OUTPUT

```
================================================================
 LOG ANALYSIS: sample_app.log
================================================================
 Lines      : 18
 First entry: 2026-10-23 08:00:01
 Last entry : 2026-10-23 10:30:00

[Entries per level]
  ERROR  7
  WARN  3
  INFO  6
  DEBUG  2

[Entries per hour]
  08:00  ####### 7
  09:00  ##### 5
  10:00  ###### 6

[Most frequent errors, top 5]
  4 x  [db] Connection timeout after 30s
  2 x  [web] Upstream returned 502
  1 x  [auth] Invalid token for user 42

Report saved to: "C:\Users\Alex\BatchBook\scripts\analysis_report.txt"
```

How it works

Arguments and checks. `topN` is validated as a single digit with `findstr /r /x "[1-9]"`, reading the value with `!TOPN!` (Chapter 23). The log file's existence is tested before any work.

Temporary files. `%TEMP%` plus `%RANDOM%` gives names that do not collide (Chapter 23). The final loop deletes whichever of them exist.

Total lines. `find /c /v "" < file` prints the number of lines without a file-name header (Chapter 13).

First and last timestamps. The `for /f` loop reads the date and time (tokens 1 and 2) of every line. The first value is kept when `FIRST` is still empty; `LAST` is overwritten on every pass and therefore holds the final line's stamp.

Sort-and-tally for repeated messages. There is no built-in way to count identical lines, so the program sorts and counts runs:

1. `findstr /c:" ERROR "` selects error lines. In each, tokens 4 and following hold the module and the message; they are written to a temporary file. The redirection is written first (`>> "%MSGS%" echo ...`) so a trailing digit in a message cannot be mistaken for a stream number (Chapter 12).
2. `sort` puts identical lines next to each other.
3. The loop compares each line with the previous one. If equal it increases `cnt`; otherwise it calls `:flush` to store the finished group and starts a new group.
4. `:flush` stores `00004 [db] Connection timeout...` with the count **padded to five digits**, so that a plain text sort orders by number (Chapter 13).

All comparisons use `"!line!"=="!prev!"` with delayed expansion, so text containing quotes or `&` cannot break the `if`.

The report block. Everything between `(` and `) > "%REPORT%"` is redirected once (Chapter 21), and every value is read with `!name!` because log text may contain parentheses (Chapter 9).

Counts per level. For each level a `findstr` selects the lines and `find /c /v ""` counts them. If no line matches, `find` prints `0`, so no special case is needed.

Hourly histogram. A regular expression with the exact date layout and the hour (`^dddd-dd-dd HH:`) counts lines per hour. Only hours with entries are printed. The bar is built by appending `#` characters with `for /l`, limited to 40 so that long logs do not overflow the line.

Top errors. `sort /r` lists the tally file in descending order. Because counts are zero-padded, the biggest come first. The loop prints the first `TOPN` entries; `1%%a-100000` turns `00004` into `4` without the octal problem (Chapter 8).

> **Warning:** The program enables delayed expansion because it must compare and store arbitrary log text. A log line containing an exclamation mark may therefore be shortened in the tally. The effect is limited to that line's grouping. For logs that contain many `!` characters, perform the grouping in PowerShell (`Group-Object`) instead.

Suggested improvements

1. **Level and module filters:** accept `/level:ERROR` and `/module:db` and restrict the analysis with `findstr`.
2. **Time window:** analyze only entries between two timestamps.
3. **Other formats:** make the field positions configurable to support logs with different layouts.
4. **CSV output:** write counts as CSV so that a spreadsheet can chart them.
5. **Rate alarm:** exit with code 1 if the number of errors per hour exceeds a limit, so a scheduler can alert.
6. **Large logs:** move the grouping step to PowerShell with `Group-Object` for speed (Chapter 21).

Summary

- The analyzer combines `find /c`, `findstr`, `sort` and `for /f` into a small pipeline without a single external script.
- Counting repeated lines is done by sort-and-tally with zero-padded counts.
- A text histogram is built from repeated characters, capped to a maximum width.
- The report is generated in one redirected block and then shown with `type`.

Common mistakes

Common Mistake: Counting with `find /c` and a pattern that appears inside other words. Searching for `ERROR` also counts a message that contains the word. Search for the level with surrounding spaces, as done here.

Common Mistake: Sorting counts as text without padding. `9` would appear after `10`.

Common Mistake: Leaving temporary files behind.

Exercises

1. **[Easy]** Run the analyzer on the sample log with `topN` equal to 2.
2. **[Easy]** Add a line to the report that shows the file size in bytes.
3. **[Medium]** Add a `[Entries per module]` section for the modules `app`, `db`, `web`, `cache`, `disk` and `auth`.
4. **[Medium]** Print the percentage of lines that are errors, with one decimal place.
5. **[Hard]** Add a `/level:XYZ` option that restricts all counts to one level.

Chapter 30: Project 6, Automated Backup Tool with Logging

LEARNING OBJECTIVES

- Build a backup tool around `robocopy` with correct exit-code handling
- Create dated backup folders with locale-independent timestamps
- Enforce a retention policy that deletes only what the tool itself created
- Log every run, and protect against dangerous source and destination combinations

Goal

A tool that copies a source folder into a new dated folder `backup_yyyyMMdd_HHmmss` inside a backup root, keeps the newest N backups, removes older ones, and records each run in a history log and a per-run `robocopy` log.

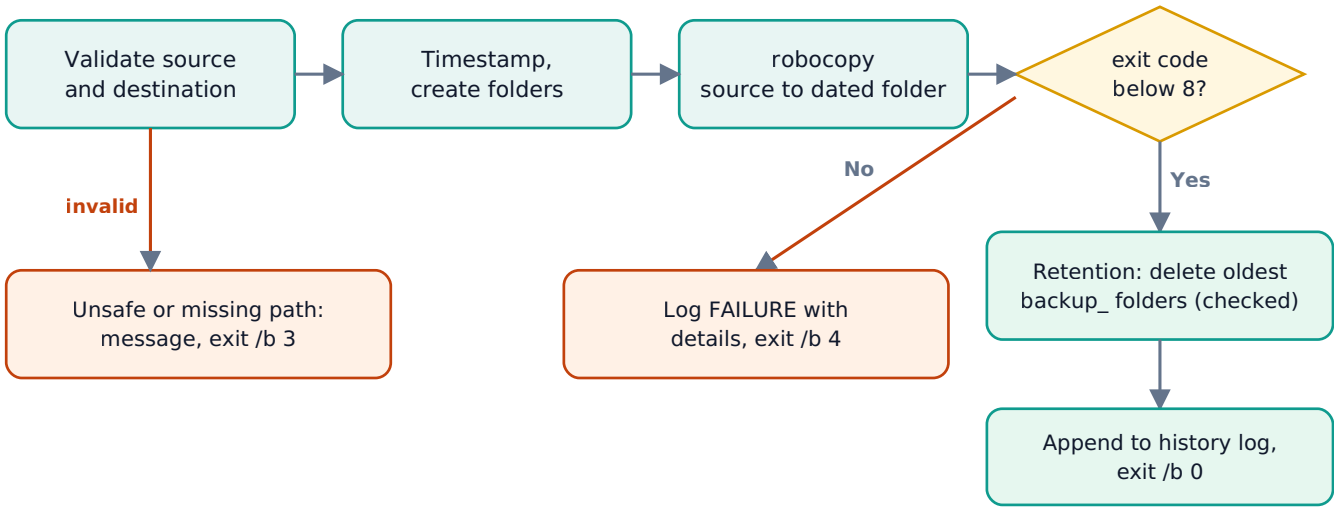


Figure 30.1. Workflow of the backup tool: validate, copy, evaluate the exit code, apply retention, log.

Requirements

1. Usage: `backup.cmd source destination [/keep:N] [/dry]`.
2. Refuse to run if the source is missing, or if the destination lies **inside** the source (it would copy itself).
3. Each run creates `destination\backup_<timestamp>`; the timestamp is independent of regional settings (Chapter 15).

4. `robocopy` exit codes of 8 or more mean failure; 0 to 7 mean success (Chapter 3).
5. **Retention:** keep the newest `N` backup folders (default 7). Only folders whose names match `backup_dddddddd_dddddd` are ever deleted.
6. `/dry` performs a list-only copy and deletes nothing.
7. Write `destination\logs\backup_<timestamp>.log` and append a line to `destination\backup_history.log`.
8. Exit codes: 0 ok, 2 usage, 3 unsafe or missing, 4 copy failed.

Source code

backup.cmd

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
rem ======================================================================
rem  backup.cmd - dated backups with retention and logging
rem  Usage : backup.cmd source destination [/keep:N] [/dry]
rem  Codes : 0 ok, 2 usage, 3 unsafe or missing, 4 copy failed
rem  Note  : folder names containing an exclamation mark are not supported.
rem ======================================================================

set "SRC="
set "DEST="
set "KEEP=7"
set "DRY=0"

rem ---- Parse arguments -------------------------------------------------
:parse
set "a=%~1"
if not defined a goto :parsed
if /i "!a!"=="/dry" (set "DRY=1" & shift & goto :parse)
if /i "!a:~0,6!"=="/keep:" (set "KEEP=!a:~6!" & shift & goto :parse)
if not defined SRC (set "SRC=!a!" & shift & goto :parse)
if not defined DEST (set "DEST=!a!" & shift & goto :parse)
echo Too many arguments.
exit /b 2
:parsed
if not defined SRC goto :usage
if not defined DEST goto :usage

rem ---- Validate --------------------------------------------------------
echo(!KEEP!| findstr /r /x "[1-9][0-9]*" >nul
if errorlevel 1 (echo /keep: needs a whole number from 1 to 99. & exit /b 2)
if not "!KEEP:~2,1!"=="" (echo /keep: needs a whole number from 1 to 99. & exit /b 2)

for %%f in ("!SRC!") do set "SRC=%%~ff"
for %%f in ("!DEST!") do set "DEST=%%~ff"

if not exist "!SRC!\" (echo Source folder not found: "!SRC!" & exit /b 3)
if "!SRC:~3!"=="" (echo Refusing to back up a whole drive root. & exit /b 3)

call :is_under "!SRC!" "!DEST!"
if not errorlevel 1 (
    echo The destination is inside the source folder. Choose another destination.
    exit /b 3
)

rem ---- Timestamp, folders ----------------------------------------------
set "TS="
for /f "usebackq delims=" %%t in (`powershell -NoProfile -Command "Get-Date -Format yyyyMMdd_HHmmss"`) do set "TS
=%%t"
```

```
if not defined TS (echo Could not read the date and time. & exit /b 3)

set "RUN_DIR=!DEST!\backup_!TS!"
set "LOG_DIR=!DEST!\logs"
set "RUN_LOG=!LOG_DIR!\backup_!TS!.log"
set "HISTORY=!DEST!\backup_history.log"

if not exist "!LOG_DIR!\" md "!LOG_DIR!"
if not exist "!LOG_DIR!\" (echo Cannot create the destination folders. & exit /b 3)

echo Source      : "!SRC!"
echo Destination : "!RUN_DIR!"
echo Keep        : !KEEP! backups
if "!DRY!"=="1" (echo Mode        : DRY RUN, nothing is copied or deleted) else (echo Mode        : real backup)
echo.

rem ---- Copy -------------------------------------------------------------
set "FLAGS=/e /copy:DAT /dcopy:T /r:2 /w:2 /np /ndl /njh /xj"
if "!DRY!"=="1" set "FLAGS=!FLAGS! /l"

robocopy "!SRC!" "!RUN_DIR!" !FLAGS! /log:"!RUN_LOG!" >nul
set "rc=!errorlevel!"

if !rc! geq 8 (
    echo Backup FAILED. robocopy exit code: !rc!
    echo Details: "!RUN_LOG!"
    >> "!HISTORY!" echo !TS! FAILED rc=!rc! source="!SRC!"
    exit /b 4
)
echo Copy finished. robocopy exit code: !rc! ^(0 to 7 means success^)
>> "!HISTORY!" echo !TS! OK rc=!rc! dry=!DRY! source="!SRC!"

rem ---- Retention --------------------------------------------------------
set "count=0"
for /f "usebackq delims=" %%d in (`dir /b /ad /o:n "!DEST!\backup_*" 2^>nul`) do set /a "count+=1"
set /a "excess=count-KEEP"
echo Backups present: !count!
if !excess! gtr 0 (
    set "i=0"
    for /f "usebackq delims=" %%d in (`dir /b /ad /o:n "!DEST!\backup_*" 2^>nul`) do (
        set /a "i+=1"
        if !i! leq !excess! call :remove_old "%%d"
    )
)
echo Done.
exit /b 0

:usage
echo Usage: %~nx0 source destination [/keep:N] [/dry]
exit /b 2

rem ---- Routines ---------------------------------------------------------

:remove_old  foldername
rem Deletes one old backup folder, but only if the name matches the pattern
rem that this tool creates. Anything else is left alone.
setlocal EnableDelayedExpansion
set "dn=%~1"
echo(!dn!| findstr /r /x "backup_[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]_[0-9][0-9][0-9][0-9][0-9][0-9]" >nul
if errorlevel 1 (
    echo   Skipped "!dn!" - the name does not match the backup pattern.
    exit /b 0
)
if "!DRY!"=="1" (
    echo   [dry run] would delete "!dn!"
    exit /b 0
)
echo   Deleting old backup "!dn!"
rd /s /q "!DEST!\!dn!"
```

```
    >> "!HISTORY!" echo !dn! DELETED by retention
    exit /b 0

    :is_under  root path
    rem Returns 0 if path lies inside root (see Chapter 23).
    setlocal EnableDelayedExpansion
    for %%a in ("%~1") do set "r=%%~fa"
    for %%b in ("%~2") do set "p=%%~fb"
    if "!r:~-1!"=="\" set "r=!r:~0,-1!"
    set "tmp=!r!"
    set "len=0"
    :measure
    if defined tmp (
        set "tmp=!tmp:~1!"
        set /a "len+=1"
        goto :measure
    )
    set "head=!p:~0,%len%!"
    set "next=!p:~%len%,1!"
    if /i not "!head!"=="!r!" exit /b 1
    if defined next if not "!next!"=="\" exit /b 1
    exit /b 0
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> backup.cmd "%USERPROFILE%\Documents" D:\Backups /keep:3
```

OUTPUT

```
Source      : "C:\Users\Alex\Documents"
Destination : "D:\Backups\backup_20261023_101500"
Keep        : 3 backups
Mode        : real backup

Copy finished. robocopy exit code: 1 (0 to 7 means success)
Backups present: 4
  Deleting old backup "backup_20261020_090000"
Done.
```

How it works

Argument parsing. The `:parse` loop reads one argument at a time. `/dry` and `/keep:N` set switches; the first two other values become `SRC` and `DEST`; a third value is an error. Delayed expansion is enabled for the whole script, so values are read as `!a!` and special characters are inert (Chapter 23). Because delayed expansion reprocesses `!`, folder names with an exclamation mark are not supported, and the header says so.

Validation. `KEEP` must be one or two digits and not start with zero. Both folders are resolved to full paths with `%%~ff`. The source must exist and must not be a drive root. `:is_under` (the routine of Chapter 23) detects a destination inside the source, which would make `robocopy` copy its own output endlessly. A destination that merely **starts with the same text** (such as `C:\DataBackup` for the source `C:\Data`) is correctly accepted.

Timestamp and paths. A single PowerShell call gives `yyyyMMdd_HHmmss`. The run folder name, log folder, log file and history file derive from it. The logs live in `destination\logs`, **outside** the run folder, so a dry run (which creates no run folder) can still write its log.

Copying. `robocopy` options: `/e` copies sub-folders including empty ones; `/copy:DAT` copies data, attributes and time stamps; `/dcopy:T` keeps folder time stamps; `/r:2 /w:2` limits retries (the default is a million); `/np /ndl /njh` keep the log compact; `/xj` skips junction points, which can create endless loops. `/l` makes it a **list-only** rehearsal. `/log:` sends all report text to a file, so the console stays quiet.

Exit code. The code is saved immediately in `rc`. Failure is `geq 8`; the success message reminds the reader that 0 to 7 are not errors. Both outcomes are appended to the history file.

Retention. Folders named `backup_*` are listed oldest first (`dir /b /ad /o:n` ; the fixed-width timestamp makes name order equal time order) and counted. If there are more than `KEEP`, the first `excess` entries are the oldest. Each is passed to `:remove_old`.

The guard in `:remove_old`. Before deleting anything, the routine checks that the name matches exactly `backup_` followed by eight digits, an underscore and six digits. A folder that merely begins with `backup_` (for example `backup_important_docs`) fails the test and is **skipped**. The delete is `rd /s /q` on a path made of the destination and a validated name: the one case in which `rd /s /q` is acceptable (Chapter 3), because every part of the path has been verified. In dry-run mode it only reports.

> **Warning:** Retention deletes data by design. Test with `/dry` first, use a destination that holds only backups of this tool, and keep at least one independent copy elsewhere. A backup on the same physical disk does not protect against disk failure.

Suggested improvements

1. **Verification:** after the copy, compare the file counts (`dir /s /b ^| find /c /v ""`) of source and backup, and warn if they differ.
2. **Compression:** zip each finished run with `tar -a -c -f` (Chapter 19) and delete the uncompressed folder.
3. **Exclusions:** accept `/xd` and `/xf` lists from a configuration file (Chapter 24).
4. **Lock file:** prevent two runs at once by creating a lock file and removing it at exit.
5. **Log retention:** delete per-run logs older than 90 days with `forfiles` (Chapter 32).
6. **Scheduling:** run daily with `schtasks` (Chapter 17), writing output to a log, and alert on non-zero codes.
7. **Incremental runs:** use `/mir` against a single folder (with great care, and only after a dry run).

Summary

- `robocopy` does the copying; the script supplies safety checks, timestamps, logging and retention.
- Exit codes below 8 are success; the code is stored immediately.
- Retention deletes only folders that match a strict name pattern and honors `/dry`.
- A destination inside the source is detected and refused.

Common mistakes

Common Mistake: Treating `robocopy` code 1 as an error.

Common Mistake: Using `/mir` for dated backups. It would delete files in the destination to match the source.

Common Mistake: Deleting with a pattern like `backup_*` without validating each name.

Common Mistake: Putting the backup on the same disk as the data and calling it protection.

Exercises

1. **[Easy]** Run the tool with `/dry` on a practice folder and read the log.
2. **[Easy]** Change the default retention to 14.
3. **[Medium]** Add an `/xd:name` option that excludes one folder from the backup.
4. **[Medium]** Show how many backups remain after retention.
5. **[Hard]** Add the verification step of improvement 1 and make the tool return code 4 if the counts differ.

Chapter 31: Project 7, Disk Usage Monitor

LEARNING OBJECTIVES

- Read disk capacity and free space without `wmic`, as integer values that Batch can compare
- Apply warning and critical thresholds and compute an overall status
- Return monitoring-style exit codes (0, 1, 2) and keep a CSV history
- Run unattended under a scheduler

Goal

A monitor that checks every fixed disk, prints a status table, appends the readings to a CSV history file, and exits with a code that a scheduler or monitoring system can act on: 0 for OK, 1 for warning, 2 for critical.

Requirements

1. Usage: `diskmon.cmd [warnPercent [critPercent]]`. Defaults: warning at 20 percent free, critical at 10 percent free.
2. Both thresholds are whole numbers from 1 to 99, and the critical value must be smaller than the warning value.
3. One PowerShell call returns a timestamp and all fixed disks as integers; Batch parses `|`-separated lines.
4. Append every reading to `disk_history.csv` (create it with a header line) and a summary line to `diskmon.log`.
5. Exit codes: 0 all disks ok, 1 at least one warning, 2 at least one critical, 3 monitor error.

Source code

diskmon.cmd

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
rem ============================================================================
rem  diskmon.cmd - free space monitor for fixed disks
rem  Usage : diskmon.cmd [warnPercent [critPercent]]
rem  Codes : 0 ok, 1 warning, 2 critical, 3 error
rem ============================================================================

set "WARN=%~1"
set "CRIT=%~2"
if not defined WARN set "WARN=20"
if not defined CRIT set "CRIT=10"
set "HIST=%~dp0disk_history.csv"
set "LOG=%~dp0diskmon.log"
```

```
rem ---- Validate thresholds ------------------------------------------
for %%v in (WARN CRIT) do (
    echo(!%%v!| findstr /r /x "[1-9][0-9]*" >nul
    if errorlevel 1 (echo %%v must be a whole number from 1 to 99. & exit /b 3)
    if not "!%%v:~2,1!"=="" (echo %%v must be a whole number from 1 to 99. & exit /b 3)
)
if %CRIT% geq %WARN% (
    echo The critical threshold must be lower than the warning threshold.
    exit /b 3
)

rem ---- One PowerShell call returns everything -----------------------
set "PSCMD=$ts=Get-Date -Format 'yyyy-MM-dd HH:mm:ss'; 'TS|'+$ts; Get-CimInstance Win32_LogicalDisk -Filter
'DriveType=3' | Where-Object { $_.Size } | ForEach-Object { 'D|{0}|{1}|{2}|{3}' -f $_.DeviceID, [int]
[math]::Round($_.Size/1GB), [int][math]::Round($_.FreeSpace/1GB), [int][math]::Floor(100*$_.FreeSpace/$_.Size) }"

set "TS="
set "worst=0"
set "disks=0"

if not exist "%HIST%" > "%HIST%" echo timestamp,drive,total_gb,free_gb,percent_free,status

for /f "usebackq tokens=1-5 delims=|" %%a in (`powershell -NoProfile -Command "%PSCMD%"`) do (
    if "%%a"=="TS" (
        set "TS=%%b"
        echo Disk usage check %%b  ^(warning at %WARN% percent free, critical at %CRIT% percent free^)
        echo.
    )
    if "%%a"=="D" call :evaluate "%%b" %%c %%d %%e
)

if not defined TS (
    echo The disk information could not be read.
    exit /b 3
)
if %disks% equ 0 (
    echo No fixed disks were found.
    exit /b 3
)

set "overall=OK"
if %worst% equ 1 set "overall=WARNING"
if %worst% equ 2 set "overall=CRITICAL"
echo.
echo Overall status: %overall%
>> "%LOG%" echo !TS! overall=%overall% warn=%WARN% crit=%CRIT% disks=%disks%
exit /b %worst%

rem ---- Routines -----------------------------------------------------

:evaluate  drive total_gb free_gb percent_free
set /a "disks+=1"
set "state=0"
set "label=OK"
if %4 leq %CRIT% (
    set "state=2"
    set "label=CRITICAL"
) else if %4 leq %WARN% (
    set "state=1"
    set "label=WARNING"
)
if !state! gtr !worst! set "worst=!state!"
set "lab=!label!         "
echo   %~1  !lab:~0,9!  free %3 GB of %2 GB, %4 percent free
>> "%HIST%" echo !TS!,%~1,%2,%3,%4,!label!
exit /b 0
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> diskmon.cmd 20 10
C:\Users\Alex\BatchBook\scripts> echo Exit code: %errorlevel%
```

OUTPUT

```
Disk usage check 2026-10-23 10:14:32  (warning at 20 percent free, critical at 10 percent free)

  C:  OK         free 210 GB of 476 GB, 44 percent free
  D:  WARNING    free 95 GB of 466 GB, 20 percent free
  E:  CRITICAL   free 8 GB of 119 GB, 6 percent free

Overall status: CRITICAL
Exit code: 2
```

The history file now contains lines such as:

disk_history.csv

```
timestamp,drive,total_gb,free_gb,percent_free,status
2026-10-23 10:14:32,C:,476,210,44,OK
2026-10-23 10:14:32,D:,466,95,20,WARNING
2026-10-23 10:14:32,E:,119,8,6,CRITICAL
```

How it works

Thresholds. Defaults are set when an argument is missing. The loop `for %%v in (WARN CRIT)` validates **both** variables with the same code: `!%%v!` reads the variable whose name is in the loop variable (delayed expansion after the loop variable has been replaced). A value must be digits, start with a non-zero digit, and have at most two digits. The critical threshold must be lower than the warning threshold.

One PowerShell call. The command text is stored in a variable and passed as a quoted string. It prints a first line `TS|<time>` and then one line `D|drive|total|free|percent` per fixed disk. Gigabytes are rounded to integers and the percentage is floored, so all values fit in Batch's 32-bit arithmetic and no decimal handling is needed. `Where-Object { $_.Size }` skips drives that report no size. Every line is plain ASCII.

Parsing. `for /f "tokens=1-5 delims=|"` splits the lines. The first token says what kind of line it is: `TS` stores the time stamp and prints the header; `D` calls `:evaluate` with the drive and the three numbers. The drive is quoted, the numbers are not (they are digits produced by PowerShell, not user input).

Evaluating a disk. The percentage is `%4`. At or below the critical threshold the state is 2, at or below the warning threshold it is 1, otherwise 0. The overall status is the **worst** state seen: `worst` only ever increases. The label is padded to nine characters with a substring (`!lab:~0,9!`) so that columns line up (Chapter 7).

Counters and variables. `:evaluate` does not call `setlocal`, so it can update the variables `disks` and `worst` of the main program.

History and log. The CSV header is written only when the file does not exist, and each reading is appended with the redirection written first (Chapter 12). The log gets one summary line per run.

Exit code. `exit /b %worst%` delivers 0, 1 or 2, the same convention used by many monitoring systems. A failure of the monitor itself returns 3 so that it is not confused with a disk problem.

Running it on a schedule

```
Command Prompt
C:\> schtasks /create /tn "BatchBook Disk Monitor" /tr "\"%USERPROFILE%\BatchBook\scripts\diskmon.cmd\" 20 10" /
sc hourly /f
```

The task runs the script every hour while you are logged on. The history file collects readings that you can chart in a spreadsheet to see trends. Check the exit code in Task Scheduler's history, or have another script read the last line of `diskmon.log`.

> **Best Practice:** An unattended monitor should never wait for input. This script does not call `pause` or `choice`, uses full paths based on `%~dp0`, and writes its results to files (Chapter 17).

Suggested improvements

1. **Top folders:** when a disk is critical, list the five largest folders of its root with a PowerShell call (Chapter 19).
2. **Trend:** read the previous reading of the same drive from the CSV and report whether free space is falling.
3. **Per-drive thresholds:** read thresholds from a configuration file (Chapter 24).
4. **Alerts:** write to the Windows event log with `eventcreate` (needs administrator rights) or send mail with PowerShell.
5. **Ignore removable drives:** the filter `DriveType=3` already excludes them; extend it to network drives (`DriveType=4`) with a flag.
6. **Absolute limit:** also warn if free space is below a fixed number of gigabytes, which matters on very large disks where 10 percent is still a lot.

Summary

- A single PowerShell call returns integer values in `|`-separated lines; Batch only compares and prints.
- Two thresholds yield three states; the overall state is the maximum.
- Monitoring exit codes (0, 1, 2) and a distinct error code (3) make the script schedulable.
- A CSV history turns each run into a data point.

Common mistakes

Common Mistake: Comparing percentages as decimals. Batch has no decimals; round or floor in PowerShell first.

Common Mistake: Mixing up "percent used" and "percent free". Thresholds here are percent **free**.

Common Mistake: Allowing a scheduled script to wait for input.

Exercises

1. **[Easy]** Run the monitor with thresholds 90 and 80 and observe the result.
2. **[Easy]** Add the computer name to every CSV line.
3. **[Medium]** Make the monitor ignore a drive letter given with `/skip:D`.
4. **[Medium]** Add an absolute rule: status CRITICAL if free space is below 5 GB.
5. **[Hard]** Print the change in free space since the previous run for each drive, using the CSV history.

Chapter 32: Project 8, Scheduled Cleanup Utility

LEARNING OBJECTIVES

- Select files by age with `forfiles` and count candidates before deleting
- Implement a policy file with per-folder retention
- Protect against dangerous folders with several independent checks
- Register and remove a scheduled task from the script itself

Warning: This project **deletes files permanently**. It is dry-run by default, refuses dangerous folders, and logs every action. Test it only on a practice folder such as `%USERPROFILE%\BatchBook\work\scratch`, with copies of files you can lose.

Goal

A utility that deletes files older than a given number of days from folders listed in a policy file. It can run by hand with a preview or, once you trust it, daily as a scheduled task.

Requirements

1. Usage: `cleanup_old.cmd policyfile [/run]`, `cleanup_old.cmd /install policyfile [HH:mm]`, `cleanup_old.cmd /uninstall`.
2. The policy file has lines `days|folder`; lines that start with `#` are comments.
3. **Dry run by default**: report how many files would be deleted. Only `/run` deletes.
4. Reject: invalid day counts, folders that do not exist, drive roots, protected system folders, and paths shorter than ten characters.
5. Log each folder processed to `cleanup.log` with a date and the numbers.
6. Installing the task asks for confirmation and warns that it will delete unattended.
7. Exit codes: 0 ok, 1 some policy lines skipped, 2 usage, 3 policy file missing.

Source code

cleanup_policy.txt

```
# days|folder
# Delete files older than the given number of days from each folder.
7|C:\Users\Alex\BatchBook\work\scratch
30|C:\Users\Alex\BatchBook\logs
```

cleanup_old.cmd

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
rem ============================================================================
rem  cleanup_old.cmd - delete old files according to a policy file
rem  Usage : cleanup_old.cmd policyfile [/run]
rem          cleanup_old.cmd /install policyfile [HH:mm]
rem          cleanup_old.cmd /uninstall
rem  Codes : 0 ok, 1 some lines skipped, 2 usage, 3 policy file missing
rem  Note  : folder names containing an exclamation mark are not supported.
rem ============================================================================

set "TASK_NAME=BatchBook Cleanup"
set "LOG=%~dp0cleanup.log"

if /i "%~1"=="/uninstall" goto :uninstall
if /i "%~1"=="/install" goto :install
if "%~1"=="" goto :usage

set "POLICY=%~1"
set "MODE=DRY"
if /i "%~2"=="/run" set "MODE=RUN"
if not exist "%POLICY%" (echo Policy file not found: "%POLICY%" & exit /b 3)

set "skipped=0"
set "folders=0"
echo Mode   : %MODE%
echo Policy : "%POLICY%"
echo.

for /f "usebackq eol=# tokens=1,* delims=|" %%a in ("%POLICY%") do call :clean_folder "%%a" "%%b"

echo.
echo Folders processed: %folders%   Lines skipped: %skipped%
if "%MODE%"=="DRY" echo This was a dry run. Add /run to delete the files.
if %skipped% gtr 0 exit /b 1
exit /b 0

rem ---- Commands that manage the scheduled task -----------------------

:install
if "%~2"=="" goto :usage
for %%f in ("%~2") do set "POLICY=%%~ff"
if not exist "!POLICY!" (echo Policy file not found: "!POLICY!" & exit /b 3)
set "AT=%~3"
if not defined AT set "AT=03:00"
echo(!AT!| findstr /r /x "[0-2][0-9]:[0-5][0-9]" >nul
if errorlevel 1 (echo The time must be written HH:mm, for example 03:00. & exit /b 2)
echo.
echo This will create a daily task that DELETES files older than the limits in
echo "!POLICY!" without asking. Run a dry run first and read its output.
choice /c YN /n /m "Create the task? (Y/N) "
if errorlevel 2 (echo Cancelled. & exit /b 0)
schtasks /create /tn "%TASK_NAME%" /tr "\"%~f0\" \"!POLICY!\" /run" /sc daily /st !AT! /f
exit /b %errorlevel%

:uninstall
schtasks /delete /tn "%TASK_NAME%" /f
exit /b %errorlevel%

:usage
echo Usage: %~nx0 policyfile [/run]
echo        %~nx0 /install policyfile [HH:mm]
echo        %~nx0 /uninstall
exit /b 2

rem ---- Routines ------------------------------------------------------
```

```
:clean_folder  days folder
set "days=%~1"
set "folder=%~2"

rem 1. The day count must be 1 to 999.
echo(!days!| findstr /r /x "[1-9][0-9]*" >nul
if errorlevel 1 (echo [skipped] invalid day count "%~1" & set /a "skipped+=1" & exit /b 0)
if not "!days:~3,1!"=="" (echo [skipped] day count too large "%~1" & set /a "skipped+=1" & exit /b 0)

rem 2. Resolve the folder and apply the safety checks.
if not defined folder (echo [skipped] no folder given & set /a "skipped+=1" & exit /b 0)
for %%f in ("!folder!") do set "folder=%%~ff"
if not exist "!folder!\" (echo [skipped] folder not found: "!folder!" & set /a "skipped+=1" & exit /b 0)
if "!folder:~3!"=="" (echo [skipped] drive root refused: "!folder!" & set /a "skipped+=1" & exit /b 0)
if "!folder:~9,1!"=="" (echo [skipped] path too short: "!folder!" & set /a "skipped+=1" & exit /b 0)
for %%p in ("%windir%" "%ProgramFiles%" "%ProgramData%" "%USERPROFILE%" "%SystemDrive%\Users") do (
    if /i "!folder!"=="%%~p" (echo [skipped] protected folder: "!folder!" & set /a "skipped+=1" & exit /b 0)
)

rem 3. Count the candidates.
set "cand=0"
for /f %%n in
('forfiles /p "!folder!" /s /d -!days! /c "cmd /c if @isdir==FALSE echo @path" 2^>nul ^| find /c /v ""') do set "
cand=%%n"
set /a "folders+=1"

if "%MODE%"=="DRY" (
    echo [dry run] "!folder!": !cand! file^(s^) older than !days! days
    exit /b 0
)

rem 4. Delete, then count what is left to report the real result.
if !cand! gtr 0 forfiles /p "!folder!" /s /d -!days! /c "cmd /c if @isdir==FALSE del /f /q @path" >nul 2>&1
set "left=0"
for /f %%n in
('forfiles /p "!folder!" /s /d -!days! /c "cmd /c if @isdir==FALSE echo @path" 2^>nul ^| find /c /v ""') do set "
left=%%n"
set /a "removed=cand-left"
echo [cleaned]  "!folder!": removed !removed! of !cand! file^(s^) older than !days! days
>> "%LOG%" echo %date% %time% folder="!folder!" days=!days! removed=!removed! left=!left!
exit /b 0
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> cleanup_old.cmd cleanup_policy.txt
```

OUTPUT

```
Mode   : DRY
Policy : "cleanup_policy.txt"

[dry run] "C:\Users\Alex\BatchBook\work\scratch": 12 file(s) older than 7 days
[dry run] "C:\Users\Alex\BatchBook\logs": 0 file(s) older than 30 days

Folders processed: 2   Lines skipped: 0
This was a dry run. Add /run to delete the files.
```

How it works

Three entry points. The first argument selects `/install`, `/uninstall` or a policy file. Everything else is a normal run. The `:install` and `:uninstall` labels sit **after** the main code's `exit /b`, so they can never be run by falling through.

Reading the policy. `for /f "usebackq eol=# tokens=1,* delims=|"` ignores comment lines and splits each line at the first `|` into the day count and the folder (everything after the bar). The loop only calls `:clean_folder`.

Layered validation. For each line the routine checks, in order: the day count (digits, no leading zero, at most three digits); that a folder is present; that the folder exists; that it is not a drive root (nothing after `X:\`); that the path has at least ten characters (which excludes very short, high-level folders); and that it is not one of the protected locations. Every failure prints a reason, counts the line as skipped, and moves on, so one bad policy line cannot cause damage or stop the others.

Selecting by age. `forfiles /p folder /s /d -N` selects files last modified **N or more days ago**; `/s` recurses. The command part `cmd /c if @isdir==FALSE ...` runs once per match; `@isdir` is `TRUE` for folders, so folders are left alone, and `@path` is the quoted full path. When nothing matches, `forfiles` prints an error to the error stream, which `2^>nul` hides, and the count is 0.

Count first, then act. The routine counts candidates with an `echo` command, then in `/run` mode runs the **same** selection with `del /f /q @path`. After deleting, it counts again; `removed = candidates - left` is the **measured** result, not a guess. Files that could not be deleted (in use, no permission) show up in `left`.

Dry run. In the default mode the routine stops after the count and prints what it **would** do.

Logging. Each cleaning is appended to `cleanup.log` with the date, folder, day limit and numbers.

Installing the task. `/install` resolves the policy path, validates the time as `HH:mm`, shows a warning and requires confirmation with `choice`. The task command is built as `"script" "policy" /run` with the inner quotes escaped as `\"` for `schtasks /tr`. The task runs daily at the chosen time as the current user (and only while that user is logged on). `/uninstall` deletes it.

> **Best Practice:** Keep the policy file and the script in a folder that only you (or only administrators) can modify. Whoever can edit the policy decides which folders are emptied (Chapter 23).

Suggested improvements

1. **Remove empty folders:** after deleting, remove empty sub-folders with `for /f "delims=" %%d in ('dir /ad /b /s "folder" ^| sort /r') do rd "%%d" 2^>nul`; `rd` without `/s` only removes empty folders.
2. **Patterns:** let a policy line contain a file mask (`7|C:\path|*.tmp`) and pass it to `forfiles /m`.
3. **Recycle Bin:** move files to a holding folder first and delete them only after another 7 days.
4. **Minimum free space:** run the cleanup only if a disk is below a threshold (combine with Chapter 31).

5. **Size limits:** also report the number of bytes freed, measured with PowerShell.
6. **Notification:** write a warning to the event log or send an e-mail when the number removed is unexpectedly large.

Summary

- `forfiles /d -N` selects files by age; counting before and after gives a measured result.
- A deleting tool needs several independent checks, a dry-run default, a log and a confirmation before it is automated.
- Scheduling is done with `schtasks`, with nested quotes escaped as `\"`.

Common mistakes

Common Mistake: Scheduling a deleting script before reading its dry-run output.

Common Mistake: Trusting a policy file that others can edit.

Common Mistake: Reporting what the script intended instead of what it achieved. Count again afterwards.

Common Mistake: Forgetting that `forfiles` returns an error when nothing matches.

Exercises

1. **[Easy]** Run a dry run on the practice folder and note the number of candidates.
2. **[Easy]** Add a third policy line for a folder of your choice and see the dry-run result.
3. **[Medium]** Make the minimum path length a configurable constant.
4. **[Medium]** Add a `/verbose` option that lists each candidate file in a dry run.
5. **[Hard]** Implement improvement 1 (removing empty sub-folders) safely.

Chapter 33: Project 9, Network Diagnostics Reporter

LEARNING OBJECTIVES

- Check a list of targets for name resolution, reachability and an open port
- Judge reachability by counting genuine replies, independent of regional settings
- Pass hostnames to PowerShell safely through environment variables
- Produce a console table, a text report and a CSV file, with an exit code for automation

Chapter 18 built a one-machine, layered check. This project checks **many targets** from a list and produces a report that can be run regularly and compared over time.

Goal

Read a list of hosts (and optional TCP ports) from `targets.txt`, test each one, and report whether it resolves, how many of four echo requests were answered, and whether the port is open.

Requirements

1. Usage: `netreport.cmd [targetsfile]` (default: `targets.txt` next to the script).
2. Target file lines: `host [port]`; lines starting with `#` are comments.
3. Host names are validated by an allow-list; invalid lines are reported and skipped.
4. Reachability is judged by counting lines that contain `TTL=` in the `ping` output, which does not depend on the Windows language, instead of reading `ping`'s exit code (Chapter 18).
5. Name resolution and port tests run in **one** PowerShell call per target, and the host and port are passed through **environment variables**, never pasted into the code (Chapter 19).
6. Write the table to the screen and to `netreport.txt`, and append rows to `netreport.csv`.
7. Exit codes: 0 all targets healthy, 1 at least one problem, 3 targets file missing.

Source code

targets.txt

```
# host [port]
1.1.1.1
example.org 443
192.168.1.1
nonexistent.invalid
```

netreport.cmd

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
rem ======================================================================
rem  netreport.cmd - check several network targets and write a report
rem  Usage : netreport.cmd [targetsfile]
rem  Codes : 0 all healthy, 1 problems found, 3 targets file missing
rem ======================================================================

set "TARGETS=%~1"
if not defined TARGETS set "TARGETS=%~dp0targets.txt"
if not exist "%TARGETS%" (
    echo Targets file not found: "%TARGETS%"
    exit /b 3
)

set "REPORT=%~dp0netreport.txt"
set "CSV=%~dp0netreport.csv"
set "SENT=4"
set "targets_total=0"
set "problems=0"

rem One PowerShell call per target: name resolution and optional port test.
rem The host and port arrive in environment variables, never inside the code.
set "PSCMD=$h=$env:T_HOST; try { [void][System.Net.Dns]::GetHostAddresses($h); 'DNS=OK' } catch { 'DNS=FAIL' };
if ($env:T_PORT) { if (Test-NetConnection -ComputerName $h -Port ([int]$env:T_PORT) -InformationLevel Quiet -
WarningAction SilentlyContinue) { 'PORT=OPEN' } else { 'PORT=CLOSED' } } else { 'PORT=-' }"

set "TS="
for /f "usebackq delims=" %%t in (`powershell -NoProfile -Command "Get-Date -Format 'yyyy-MM-dd HH:mm:ss'"`) do s
et "TS=%%t"
if not defined TS (echo Could not read the clock. & exit /b 3)

if not exist "%CSV%" > "%CSV%" echo timestamp,host,port,resolves,replies,sent,ping,port_state

> "%REPORT%" echo Network report !TS! on %COMPUTERNAME%
call :emit "Network report !TS!"
call :emit ""
call :emit "Target                        Resolves  Ping   Replies  Port"
call :emit "----------------------------  --------  -----  -------  ------"

for /f "usebackq eol=# tokens=1,2" %%a in ("%TARGETS%") do call :check "%%a" "%%b"

call :emit ""
call :emit "Targets: !targets_total!   With problems: !problems!"
echo Report written to "%REPORT%"
if !problems! gtr 0 exit /b 1
exit /b 0

rem ---- Routines --------------------------------------------------------

:emit  text
echo(%~1
>> "%REPORT%" echo(%~1
exit /b 0

:valid_host  varname
rem Letters, digits, dot and hyphen only, starting with a letter or digit.
echo(!%~1!| findstr /r /x "[A-Za-z0-9][A-Za-z0-9.-]*" >nul
exit /b %errorlevel%

:check  host port
set "host=%~1"
set "port=%~2"

call :valid_host host
if errorlevel 1 (
    call :emit "[skipped] invalid target name"
```

```
    set /a "problems+=1"
    exit /b 0
)
if defined port (
    echo(!port!| findstr /r /x "[1-9][0-9]*" >nul
    if errorlevel 1 (call :emit "[skipped] invalid port for !host!" & set /a "problems+=1" & exit /b 0)
    if not "!port:~5,1!"=="" (call :emit "[skipped] invalid port for !host!" & set /a "problems+=1" & exit /b 0)
    if !port! gtr 65535 (call :emit "[skipped] invalid port for !host!" & set /a "problems+=1" & exit /b 0)
)
set /a "targets_total+=1"

rem ---- Ping: count the replies, do not trust the exit code ------------
set "rx=0"
for /f %%c in ('ping -n %SENT% -w 1000 !host! ^| find /c "TTL="') do set "rx=%%c"
set "pingstate=OK"
if !rx! equ 0 (set "pingstate=DOWN") else if !rx! lss %SENT% set "pingstate=LOSS"

rem ---- Name resolution and port, one PowerShell run -------------------
set "T_HOST=!host!"
set "T_PORT=!port!"
set "R_DNS=?"
set "R_PORT=?"
for /f "usebackq tokens=1,* delims==" %%a in (`powershell -NoProfile -Command "%PSCMD%"`) do set "R_%%a=%%b"
set "T_HOST="
set "T_PORT="

rem ---- Judge ----------------------------------------------------------
set "bad=0"
if "!pingstate!"=="DOWN" set "bad=1"
if not "!R_DNS!"=="OK" set "bad=1"
if defined port if not "!R_PORT!"=="OPEN" set "bad=1"
if !bad! equ 1 set /a "problems+=1"

rem ---- One aligned table row ------------------------------------------
set "c1=!host!                          "
set "c2=!R_DNS!        "
set "c3=!pingstate!     "
set "c4=!rx!/%SENT%       "
set "c5=!R_PORT!"
call :emit "!c1:~0,28!  !c2:~0,8!  !c3:~0,5!  !c4:~0,7!  !c5!"
>> "%CSV%" echo !TS!,!host!,!port!,!R_DNS!,!rx!,%SENT%,!pingstate!,!R_PORT!
exit /b 0
```

Command Prompt

```
C:\Users\Alex\BatchBook\scripts> netreport.cmd
C:\Users\Alex\BatchBook\scripts> echo Exit code: %errorlevel%
```

OUTPUT

```
Network report 2026-10-23 10:30:05

Target                        Resolves  Ping   Replies  Port
----------------------------  --------  -----  -------  ------
1.1.1.1                       OK        OK     4/4      -
example.org                   OK        OK     4/4      OPEN
192.168.1.1                   OK        OK     4/4      -
nonexistent.invalid           FAIL      DOWN   0/4      -

Targets: 4   With problems: 1
Report written to "C:\Users\Alex\BatchBook\scripts\netreport.txt"
Exit code: 1
```

How it works

The PowerShell command in a variable. `PSCMD` holds one line of PowerShell. It reads the target from `$env:T_HOST` and the port from `$env:T_PORT`. A host name containing a quote or a semicolon could not alter the code, because the name is **data in the environment**, not part of the command text (Chapter 19). `GetHostAddresses` resolves names (and simply returns an IP literal), and `Test-NetConnection ... -InformationLevel Quiet` returns a true or false for the TCP connection. The output is two `NAME=value` lines.

Timestamp. The report time comes from the same locale-independent method as before (Chapter 15).

Table output. `:emit` prints a line and appends it to the report. Its argument is quoted by the callers, and `echo(` is used so that an empty string prints a blank line.

Reading targets. `for /f "eol=# tokens=1,2"` skips comments and splits each line into host and optional port. Both go to `:check` as quoted arguments.

Validation. `:valid_host` takes the **name** of the variable and tests it with `findstr /r /x` using delayed expansion, as in Chapter 23. The port must be digits, at most five of them, and not above 65535. Invalid lines are shown in the report and counted as problems.

Ping by counting replies. `ping -n 4 -w 1000 host | find /c "TTL="` counts the genuine replies. `TTL=` is not translated in the usual languages, and a router's "unreachable" answer does not contain it. The count gives three states: `DOWN` (none), `LOSS` (some), `OK` (all). The host has already passed the allow-list, so it can be used unquoted in the command.

Resolution and port. The variables `T_HOST` and `T_PORT` are set only for the PowerShell call and cleared afterwards. The loop sets `R_DNS` and `R_PORT` from the returned lines (`PORT=-` means no port was requested).

Judging. A target has a problem if it did not answer any ping, did not resolve, or has a requested port that is not open. Partial loss is shown but does not count as a problem by itself, because a single lost packet is common.

Aligned columns. Each value is padded with spaces and cut to a fixed width with a substring (`!c1:~0,28!`), a technique from Chapter 7.

Exit code. The number of problems decides between 0 and 1, so a scheduler can alert on failure.

> **Warning:** Only list hosts that you own or are authorized to test. Repeated tests against third-party systems can be treated as abuse. Do not use this tool to scan address ranges.

Suggested improvements

1. **Round-trip time:** record the average time using PowerShell's `Test-Connection`, which returns numeric values independent of language.
2. **Port list per host:** accept several ports (`example.org 80,443`) and test each.

3. **Trend:** compare the new rows with the previous run in the CSV and report targets that changed state.
4. **Parallel testing:** start several `powershell` jobs at once for long target lists, or move the whole loop into one PowerShell script.
5. **Alerting:** call a script or write to the event log when `problems` is above zero.
6. **HTML report:** convert the CSV to an HTML table through PowerShell's `ConvertTo-Html`.

Summary

- Counting `TTL=` lines is a language-independent test of real replies.
- Hostnames and ports reach PowerShell through environment variables; they are also validated with allow-lists.
- Aligned tables use padded variables and substrings; a CSV file keeps history.
- An exit code summarizes the run for automation.

Common mistakes

> **Common Mistake: Using `ping`'s exit code to decide reachability.**

> **Common Mistake: Pasting host names into a PowerShell command.**

> **Common Mistake: Testing systems you are not authorized to test.**

Exercises

1. **[Easy]** Add your router and a public DNS server to `targets.txt` and run the tool.
2. **[Easy]** Change the number of echo requests to 2.
3. **[Medium]** Report `LOSS` targets as problems when more than 50 percent of the packets were lost.
4. **[Medium]** Add a `/csvonly` switch that suppresses the screen table.
5. **[Hard]** Add an `avg_ms` column using `Test-Connection` through PowerShell and the environment-variable method.

Chapter 34: Project 10, Menu-Driven Administration Toolkit

LEARNING OBJECTIVES

- Assemble earlier tools into one menu-driven program with a consistent structure
- Run other scripts as modules, check that they exist, and record their exit codes
- Validate typed input, and offer read-only service status from an allow-list
- Use optional colour that respects the `NO_COLOR` convention, and show administrator status
- Restart the toolkit with elevated rights without losing control of the process

Goal

A single entry point, `toolkit.cmd`, with a menu that starts the tools built in this book: the system report, the disk monitor, the network reporter and the file organizer (in preview mode only), plus service status, a log viewer and elevation.

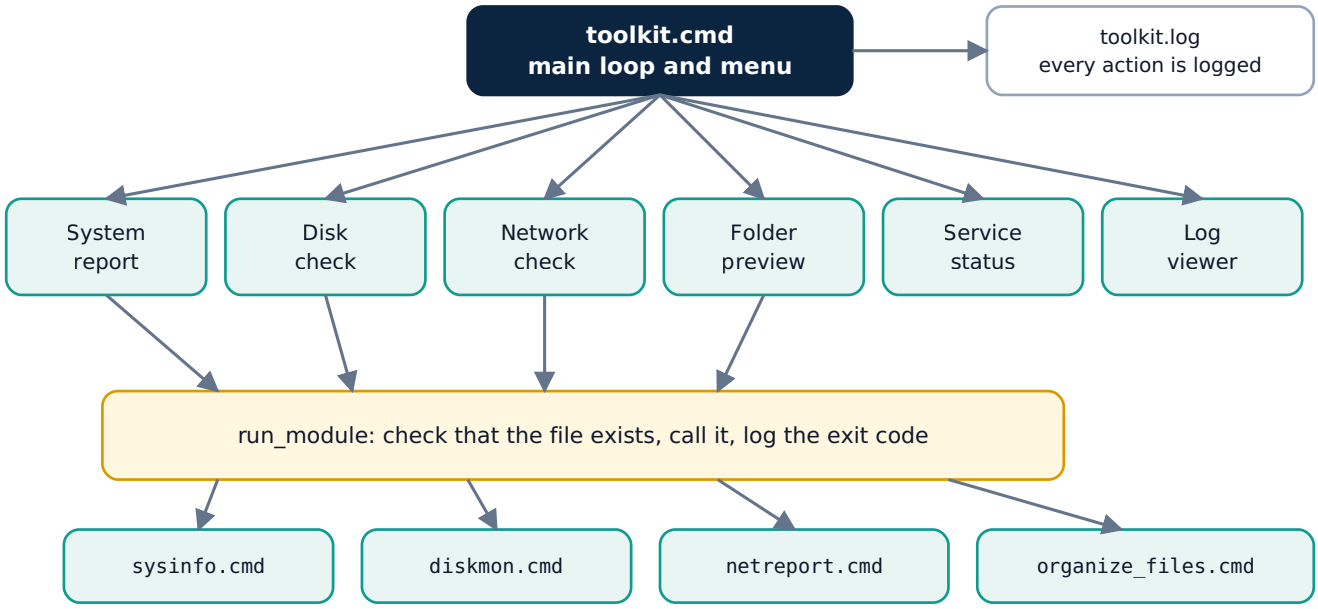


Figure 34.1. The toolkit's structure: a main loop dispatches to actions; actions call module scripts through one routine that checks, runs and logs.

Requirements

1. A menu loop with single-key choices (`choice`) and a clean exit.
2. Modules live next to the toolkit: `sysinfo.cmd`, `diskmon.cmd`, `netreport.cmd`, `organize_files.cmd`. The toolkit checks that a module exists before running it and logs its exit code.

3. The file organizer is offered **only as a preview** (its default dry-run mode). The toolkit never passes `/run`.
4. Service status uses an allow-list of four services; nothing is started or stopped.
5. Show whether the session is elevated; offer to restart the toolkit as administrator.
6. Use ANSI colour for emphasis, switched off when the environment variable `NO_COLOR` is defined. Every message is readable without colour.
7. Log menu selections and module results to `toolkit.log`.

Source code

toolkit.cmd

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
rem ======================================================================
rem  toolkit.cmd - menu-driven administration toolkit
rem  Modules (same folder): sysinfo.cmd diskmon.cmd netreport.cmd
rem                         organize_files.cmd
rem  Colour is disabled when the environment variable NO_COLOR is set.
rem ======================================================================

set "HERE=%~dp0"
set "VERSION=1.0"
set "LOGFILE=%HERE%toolkit.log"
title BatchBook Toolkit %VERSION%

rem ---- Colours (optional) ------------------------------------------------
set "C_TITLE="
set "C_OK="
set "C_BAD="
set "C_OFF="
if not defined NO_COLOR (
    for /f %%e in ('powershell -NoProfile -Command "[char]27"') do (
        set "C_TITLE=%%e[36m"
        set "C_OK=%%e[32m"
        set "C_BAD=%%e[31m"
        set "C_OFF=%%e[0m"
    )
)

call :log "Toolkit started"

:main_menu
call :admin_state
cls
echo !C_TITLE!==========================================!C_OFF!
echo !C_TITLE! BatchBook Toolkit %VERSION%   admin: !ADMIN!!C_OFF!
echo !C_TITLE!==========================================!C_OFF!
echo   1. System report
echo   2. Disk space check
echo   3. Network check
echo   4. Preview a folder tidy-up (nothing is moved)
echo   5. Service status
echo   6. Show today's toolkit log
echo   7. Restart as administrator
echo   Q. Quit
echo.
choice /c 1234567Q /n /m "Select: "
set "sel=%errorlevel%"
call :log "Menu selection !sel!"
```

```
if "!sel!"=="8" goto :quit
if "!sel!"=="7" call :act_elevate
if "!sel!"=="6" call :act_log
if "!sel!"=="5" call :act_service
if "!sel!"=="4" call :act_tidy
if "!sel!"=="3" call :run_module netreport.cmd
if "!sel!"=="2" call :run_module diskmon.cmd
if "!sel!"=="1" call :run_module sysinfo.cmd
echo.
pause
goto :main_menu

:quit
call :log "Toolkit finished"
cls
echo Goodbye.
exit /b 0

rem ---- Routines -------------------------------------------------------

:log  message
>> "%LOGFILE%" echo %date% %time% %~1
exit /b 0

:admin_state
fltmc >nul 2>&1
if errorlevel 1 (set "ADMIN=no") else (set "ADMIN=yes")
exit /b 0

:run_module  file [args]
rem Checks that the module exists, runs it, logs the exit code.
set "mod=%~1"
if not exist "%HERE%%mod%" (
    echo !C_BAD!Module not found: %mod%!C_OFF!
    call :log "Module missing: %mod%"
    exit /b 1
)
echo.
echo !C_TITLE!Running %mod% ...!C_OFF!
echo.
call "%HERE%%mod%" %2 %3
set "rc=!errorlevel!"
call :log "%mod% finished with exit code !rc!"
echo.
if "!rc!"=="0" (echo !C_OK!%mod% finished: OK!C_OFF!) else (echo !C_BAD!%mod% finished with exit code !rc!!
C_OFF!)
exit /b !rc!

:act_tidy
set "dir="
set /p "dir=Folder to preview (empty to cancel): "
if not defined dir exit /b 0
echo(!dir!| find """" >nul
if not errorlevel 1 (echo !C_BAD!Quotes are not allowed in the folder name.!C_OFF! & exit /b 1)
for %%f in ("!dir!") do set "dir=%%~ff"
if not exist "!dir!\" (echo !C_BAD!Folder not found: "!dir!"!C_OFF! & exit /b 1)
echo This is a preview only. The organizer is started WITHOUT its run switch.
call :run_module organize_files.cmd "!dir!"
exit /b 0

:act_service
echo.
echo   1. Print Spooler    2. Windows Time    3. Windows Update    4. DNS Client    B. Back
choice /c 1234B /n /m "Service: "
set "s=%errorlevel%"
if "!s!"=="5" exit /b 0
set "svc="
if "!s!"=="1" set "svc=Spooler"
if "!s!"=="2" set "svc=W32Time"
```

```
if "!s!"=="3" set "svc=wuauserv"
if "!s!"=="4" set "svc=Dnscache"
if not defined svc exit /b 0
echo.
sc query !svc! | findstr /i "SERVICE_NAME STATE"
call :log "Service status viewed: !svc!"
exit /b 0

:act_log
echo.
if not exist "%LOGFILE%" (echo No log yet. & exit /b 0)
findstr /b /l /c:"%date%" "%LOGFILE%"
if errorlevel 1 echo No entries for today.
exit /b 0

:act_elevate
if "!ADMIN!"=="yes" (echo Already running as administrator. & exit /b 0)
echo A Windows prompt will ask for permission. The toolkit restarts in a new window.
choice /c YN /n /m "Restart as administrator? (Y/N) "
if errorlevel 2 exit /b 0
set "SELF=%~f0"
call :log "Elevation requested"
powershell -NoProfile -Command "Start-Process -FilePath $env:SELF -Verb RunAs"
if errorlevel 1 (echo The request was declined or failed. & exit /b 1)
echo The elevated toolkit is starting. Closing this window.
timeout /t 2 /nobreak >nul
exit
```

OUTPUT

```
==========================================
 BatchBook Toolkit 1.0   admin: no
==========================================
  1. System report
  2. Disk space check
  3. Network check
  4. Preview a folder tidy-up (nothing is moved)
  5. Service status
  6. Show today's toolkit log
  7. Restart as administrator
  Q. Quit

Select: 5

  1. Print Spooler    2. Windows Time    3. Windows Update    4. DNS Client    B. Back
Service: 1

SERVICE_NAME: Spooler
        STATE              : 4  RUNNING
Press any key to continue . . .
```

How it works

Structure. The program follows the template of Chapter 24: configuration at the top, a **main loop** with a menu, one **routine per action**, and a handful of shared routines (`:log`, `:admin_state`, `:run_module`). The `quit` label is the single exit route.

Optional colour. Unless `NO_COLOR` is defined, one PowerShell call obtains the ESC character and four colour strings are built (Chapter 16). When `NO_COLOR` is set the strings stay empty, so every `echo` prints plain text. Colour never carries meaning alone: messages say `OK`, `Module not found` or the

exit code in words. Colour strings are expanded with delayed expansion (`!C_TITLE!`), as the ESC character cannot be typed into the file.

Administrator state. `:admin_state` uses `fltmc` (Chapter 14) each time the menu is drawn and shows `admin: yes` or `admin: no` in the header.

Menu dispatch. `choice /c 1234567Q` returns 1 to 8. The result is stored at once in `sel` (later commands would replace `%errorlevel%`, Chapter 9) and dispatched with `if` tests that call one routine each. The tests are written as independent `if` lines, and exactly one can be true, so their order does not matter. After each action `pause` lets the user read the output.

Modules. `:run_module` is the heart of the toolkit. It checks that the file exists, announces it, runs it with `call` (so control returns, Chapter 11), stores the exit code, logs it, prints an OK or failure line, and returns the code. The arguments `%2 %3` are passed unchanged, preserving their quotes, so a path with spaces works. Because it is the only place that starts modules, every module run is logged and checked in the same way.

Safe use of the organizer. `:act_tidy` reads a folder name with `set /p`, rejects an entry that contains a quote (`find """` looks for one), resolves it to a full path, checks that it exists and calls the organizer **without** `/run`. The organizer's own defaults keep it in dry-run mode (Chapter 27), so this menu item cannot move anything.

Service status. The four services come from a fixed allow-list selected by a key; **no typed text reaches a command**. The query is read-only (`sc query`), so no elevation is needed. `findstr /i "SERVICE_NAME STATE"` shows the two relevant lines.

The log viewer. `findstr /b /l /c:"%date%"` prints the lines that start with today's date text. Because both the log and the search use the same regional format, this works on any locale.

Elevation. `:act_elevate` explains the prompt, asks for confirmation, passes the path to PowerShell through an environment variable and starts an elevated copy with `-Verb RunAs` (Chapter 20). On success the **original** window closes with `exit`, the one place where a plain `exit` is appropriate: it ends the whole session on purpose, because the elevated copy has taken over. If the user declines the prompt, PowerShell returns an error and the menu continues.

> **Best Practice:** A launcher should never be the most privileged part of a system. Run the toolkit as a standard user by default, and elevate only when a chosen action needs it. This toolkit's actions are all read-only or preview-only, so elevation is rarely needed, and is offered as a convenience.

Suggested improvements

1. **Configuration file:** list the modules and their menu texts in `toolkit.ini` and build the menu from it (Chapter 24).
2. **Submenus:** group tools under System, Network and Files.
3. **Run mode for the organizer:** add a second confirmation that shows the dry-run result and requires typing the word `MOVE` before running with `/run`.
4. **Log rotation:** keep the last 30 days of entries (Chapter 32).

5. **Self-test:** a `/selftest` switch that runs each module's help or dry-run option and prints a pass/fail table.
6. **Help screen:** a menu entry that prints the contents of a `README.txt` with `more`.
7. **Localization:** move all message strings into a separate file selected by a language setting.

Summary

- A toolkit is a thin launcher: one loop, one routine per action, one routine that runs modules.
- Existence checks, logged exit codes and consistent messages make module integration reliable.
- Dangerous capabilities (the organizer's `/run`) are simply not offered; typed text never reaches a command; services come from an allow-list.
- Colour is optional and secondary; elevation is explicit and confirmed.

Common mistakes

Common Mistake: Reading `%errorlevel%` after other commands in the menu loop. Store it first.

Common Mistake: Passing typed text straight to a command. Use a key to select from a fixed list, or validate.

Common Mistake: Plain `exit` in a routine meant to return. Use `exit /b`.

Exercises

1. **[Easy]** Add a menu item that opens the practice folder in Explorer with `start "" explorer "%USERPROFILE%\BatchBook"`.
2. **[Easy]** Add a fifth service to the allow-list.
3. **[Medium]** Add a menu item that shows the last 20 lines of `toolkit.log` (use `more +N`).
4. **[Medium]** Make the toolkit refuse to start if `powershell` is not found, with a clear message and exit code.
5. **[Hard]** Build the menu text and dispatch from a `toolkit.ini` file listing `key|label|script`.

Part 5 Quiz

1. **Q1.** In the guessing game, why is the leading-zero loop needed?
2. **Q2.** Why does the report block of the system information tool read values with `!name!`?
3. **Q3.** What two properties make the file organizer safe to try?
4. **Q4.** Why does the renamer loop over `dir /b` output instead of a wildcard?
5. **Q5.** How does the log analyzer count identical messages?
6. **Q6.** Why are `robocopy` codes 1 to 7 treated as success in the backup tool?
7. **Q7.** What makes the retention step safe against deleting unrelated folders?
8. **Q8.** What exit codes does the disk monitor use, and why is 3 separate?
9. **Q9.** Why does the cleanup utility count candidates both before and after deleting?
10. **Q10.** How does the network reporter keep host names from becoming PowerShell code?
11. **Q11.** Why does the toolkit store `%errorlevel%` in `sel` immediately?
12. **Q12.** Why is plain `exit` acceptable in `:act_elevate` but not elsewhere?

A-I

Reference Material

Appendices

Command reference, escape rules, exit codes, modifiers, troubleshooting, cheat sheet, glossary, answers and resources

The appendices are for looking things up. Appendix H contains the answers to every exercise and quiz in the book.

Appendices

- A. Complete Command Reference
- B. Special Characters and Escape Rules
- C. Error Level and Exit Code Reference
- D. Parameter Modifier Reference
- E. Common Pitfalls and Troubleshooting Checklist
- F. Cheat Sheet
- G. Glossary
- H. Answers to All Exercises
- I. Recommended Next Steps and Further Resources

Appendix A: Complete Command Reference

Each table lists the command, its purpose with the essential syntax, the switches used most often and one example. Use `command /?` for the full help of your Windows version. Commands marked *internal* are part of `cmd.exe`; the others are separate programs.

A.1 Files and folders

Command	Purpose and syntax	Common switches	Example
`dir` ***internal***	List files and folders. `dir [path] [options]`	`/b` bare, `/s` recurse, `/a:d` folders, `/a:-d` files, `/o:n` `/o:s` `/o:d` sort, `/w` wide, `/q` owner	`dir /b /s *.log`
`cd`, `chdir` ***internal***	Change or show the current directory	`/d` also change the drive	`cd /d D:\Data`
`pushd`, `popd` ***internal***	Change directory and remember the previous one; return. Maps UNC paths to a drive letter	none	`pushd "%~dp0"`
`md`, `mkdir` ***internal***	Create folders, including missing parents	none	`md logs\2026`
`rd`, `rmdir` ***internal***	Remove folders (only empty ones by default)	`/s` delete the tree, `/q` no prompt	`rd "%TEMP%\demo"`
`copy` ***internal***	Copy or join files	`/y` no overwrite prompt, `/b` binary, `/v` verify, `a+b c` join	`copy /y a.txt b.txt`
`xcopy`	Copy files and trees (legacy)	`/s` `/e` trees, `/i` assume folder, `/y`, `/d` newer only, `/h` hidden, `/c` continue	`xcopy src dst /e /i /y`
`robocopy`	Robust copy and mirror	`/e`, `/mir`, `/purge`, `/l` list only, `/r:n`, `/w:n`, `/log:file`, `/np`, `/xd`, `/xf`, `/maxage:n`, `/xj`	`robocopy src dst /e /r:1 /w:1`
`move` ***internal***	Move files or folders; rename a folder	`/y` no overwrite prompt	`move a.txt sub\`
`del`, `erase` ***internal***	Delete files (never folders)	`/p` prompt, `/q` quiet, `/f` read-only too, `/s` subfolders, `/a:attr`	`del /p scratch.tmp`
`ren`, `rename` ***internal***	Rename within the same folder; target is a bare name	wildcards	`ren *.txt *.bak`

Command	Purpose and syntax	Common switches	Example
`type` ***internal***	Print a text file	pipe to `more`	`type log.txt`
`more`	Page text	`+n` skip n lines	`type big.txt | more`
`tree`	Draw a folder tree	`/f` files, `/a` ASCII lines	`tree /f /a`
`attrib`	Show or set file attributes	`+r -r +h -h +s -s +a -a` , `/s` , `/d`	`attrib +h secret.txt`
`mklink` ***internal***	Create a symbolic or hard link (privileges needed)	`/d` directory link, `/h` hard link, `/j` junction	`mklink /d link target`
`fc`	Compare two files	`/n` line numbers, `/b` binary, `/c` ignore case, `/w` white space	`fc a.txt b.txt`
`comp`	Compare file contents byte by byte	`/m` , `/a`	`comp a.bin b.bin`
`forfiles`	Run a command for files selected by name or age	`/p` path, `/s` , `/m` mask, `/d -n` older than n days, `/c "cmd"`	`forfiles /p logs /d -30 /c "cmd /c echo @path"`
`where`	Locate programs	`/q` quiet, `/r dir` recursive, `/f`	`where /q git`
`certutil -hashfile`	Compute a file hash	algorithm MD5, SHA1, SHA256	`certutil -hashfile a.iso SHA256`
`icacls`	Show or change file permissions	`/grant` , `/deny` , `/reset` (use with care)	`icacls C:\Scripts`
`clip`	Copy piped text to the clipboard	none	`dir /b | clip`
`tar`	Create and extract archives, including zip	`-a` format by extension, `-c` create, `-x` extract, `-t` list, `-f` file, `-C` change folder	`tar -a -c -f out.zip folder`

A.2 Text processing

Command	Purpose and syntax	Common switches	Example
`find`	Search for a literal string	`/i` ignore case, `/v` invert, `/c` count, `/n` numbers	`find /c /v "" < file.txt`
`findstr`	Search with regular expressions	`/i` , `/v` , `/n` , `/m` file names only, `/c:"text"` phrase, `/l` literal, `/r` regex, `/b` `/e` `/x` , `/s` subfolders, `/g:file`	`findstr /s /i /n "TODO" *.cmd`

Command	Purpose and syntax	Common switches	Example
`sort`	Sort lines as text	`/r` reverse, `/+n` column, `/o file` output	`sort /r < in.txt > out.txt`
`echo` *internal*	Print text, or show echo state	`on`, `off`, `echo.` blank line	`echo Hello`
`set` *internal*	Show, set, delete variables; `/a` arithmetic; `/p` prompt	`/a`, `/p`	`set "x=5"`

A.3 Batch language and flow

Command	Purpose and syntax	Notes	Example
`rem` *internal*	Comment	Use inside blocks	`rem note`
`goto` *internal*	Jump to a label; `goto :eof` ends the script or routine	Labels start with `:`	`goto :done`
`call` *internal*	Run a subroutine (`call :label args`) or another script	Returns afterward	`call :log "text"`
`if` *internal*	Conditions: `exist`, `defined`, `errorlevel`, comparisons	`/i`, `not`, `else` after `)`	`if exist f (echo yes)`
`for` *internal*	Loops: list, `/l` counting, `/d` folders, `/r` trees, `/f` parsing	`%%v` in scripts	`for /l %%i in (1,1,5) do echo %%i`
`shift` *internal*	Shift arguments	`/n`	`shift`
`setlocal`, `endlocal` *internal*	Local environment; `EnableDelayedExpansion`, `EnableExtensions`	Closed implicitly at end	`setlocal EnableDelayedExpansion`
`exit` *internal*	`exit /b n` ends the script with code n; plain `exit` closes the session		`exit /b 1`
`pause` *internal*	Wait for a key	Not in unattended scripts	`pause`
`choice`	One-key menu; returns the key's position	`/c`, `/n`, `/m`, `/t`, `/d`, `/cs`	`choice /c YN /m "OK? "`
`timeout`	Wait	`/t n`, `/nobreak`	`timeout /t 5 /nobreak >nul`
`title`, `color`, `cls` *internal*	Window title, colours, clear screen	`color 0A`	`title Tool`

Command	Purpose and syntax	Notes	Example
`prompt` ***internal***	Change the prompt text	`$P$G`	`prompt $P$G`
`start` ***internal***	Start a program; first quoted argument is the title	`/wait`, `/min`, `/b`	`start "" notepad`
`cmd`	Start a new interpreter	`/c` run and end, `/k` run and stay, `/v:on` delayed expansion	`cmd /c dir`
`ver`, `vol` ***internal***	Windows version; volume label		`ver`
`assoc`, `ftype` ***internal***	Show or change file associations (use with care)		`assoc .txt`

A.4 Processes, services and system

Command	Purpose and syntax	Common switches	Example
`tasklist`	List processes	`/fi`, `/fo csv`, `/nh`, `/v`, `/svc`	`tasklist /fi "imagename eq explorer.exe"`
`taskkill`	End processes	`/pid`, `/im`, `/f`, `/t`, `/fi`	`taskkill /pid 1234`
`sc`	Service control. A space follows `=` in its options	`query`, `qc`, `start`, `stop`	`sc query Spooler`
`net`	Services, users, shares, sessions	`net start`, `net stop`, `net use`, `net user`	`net start Spooler`
`schtasks`	Scheduled tasks	`/create`, `/query`, `/run`, `/delete`, `/tn`, `/tr`, `/sc`, `/st`, `/f`	`schtasks /query /tn "Task" /fo list`
`reg`	Registry	`query`, `add`, `delete`, `export`, `import`	`reg query HKCU\Software`
`systeminfo`	System summary (slow, localized)	`/fo csv`	`systeminfo`
`shutdown`	Shut down, restart, abort	`/s`, `/r`, `/t n`, `/a`, `/c`, `/f`, `/l`	`shutdown /s /t 300`
`whoami`	Current user	`/groups`, `/priv`	`whoami`
`hostname`	Computer name		`hostname`
`driverquery`	List drivers	`/fo csv`	`driverquery /fo csv`

Command	Purpose and syntax	Common switches	Example
`fltmc`	Filter manager; succeeds only when elevated		`fltmc >nul 2>&1`
`powershell`	Windows PowerShell	`-NoProfile`, `-Command`, `-File`, `-ExecutionPolicy`	`powershell -NoProfile -Command "Get-Date"`
`cscript`	Windows Script Host (console); VBScript is deprecated	`//nologo`	`cscript //nologo a.vbs`
`wmic`	**Deprecated and removed from current Windows 11.** Use `Get-CimInstance`		see Chapter 17

A.5 Networking

Command	Purpose and syntax	Common switches	Example
`ipconfig`	Address configuration	`/all`, `/flushdns`, `/displaydns`, `/release`, `/renew`	`ipconfig /all`
`ping`	Test reachability	`-n`, `-t`, `-w ms`, `-l size`, `-4`, `-6`	`ping -n 2 host`
`tracert`	Route to a host	`-d`, `-h n`, `-w ms`	`tracert -d example.org`
`pathping`	Route with loss statistics (slow)	`-n`, `-q n`	`pathping -n host`
`nslookup`	DNS queries	`nslookup name server`, `-type=MX`	`nslookup example.org 1.1.1.1`
`netstat`	Connections and listening ports	`-a`, `-n`, `-o`, `-b`, `-r`, `-s`	`netstat -ano`
`arp`	ARP cache	`-a`	`arp -a`
`route`	Routing table	`print`, `add`, `delete` (changes need admin)	`route print`
`netsh`	Network configuration (read with care, change rarely)	`interface ip show config`, `advfirewall show allprofiles`	`netsh advfirewall show allprofiles state`
`getmac`	MAC addresses	`/v`, `/fo csv`	`getmac /v`
`curl`	HTTP requests (Windows 10 1803 and later)	`-s`, `-I`, `-o`, `-w`, `-L`	`curl -I https://example.org`

Appendix B: Special Characters and Escape Rules

B.1 Characters with a meaning to cmd.exe

Character	Meaning	To use it literally
`%`	Variable and argument expansion	`%%` in a batch file
`!`	Delayed expansion (only when enabled)	`^^!` when enabled
`^`	Escape character	`^^`
`&`	Command separator	`^&`
`&&`	Run next command if the previous succeeded	`^&^&`
`||`	Run next command if the previous failed	`^|^|`
`|`	Pipe	`^|`
`<`	Redirect input	`^<`
`>`	Redirect output	`^>`
`(` and `)`	Command blocks	`^(` and `^)`
`"`	Quoting	no general escape; see Chapter 12
`=`	Delimiter in `set`, parameters	quote the text
`,` `;` and space	Argument separators	quote the text
`*` and `?`	Wildcards	none; pass inside quotes where allowed
`@`	Suppress echo of one line	not special after the first character
`:`	Label	`::` also acts as a comment outside blocks

B.2 Rules of thumb

1. **Outside quotes** use a caret before `& | < > ( ) ^`.
2. **Inside quotes** these characters are literal, but `%` still expands and `!` still expands with delayed expansion.
3. In a **batch file**, a literal percent sign is `%%`. At the prompt it is `%`.
4. With **delayed expansion**, a literal `!` is `^^!`, because the caret is processed twice.
5. In `for /f` commands in single quotes, escape `|`, `>`, `<` and `&` with a caret.

6. Passing a caret through `call` doubles it. Test any value that may contain carets.
7. Strings from outside (arguments, file names, input) must be **validated**; escaping alone is not a defense (Chapter 23).

B.3 Examples

To print	Batch line
`Tom & Jerry`	`echo Tom ^& Jerry`
`100%`	`echo 100%%`
`a > b`	`echo a ^> b`
`x ^ y`	`echo x ^^ y`
`(note)` **at top level**	`echo (note)`
`(note)` **inside a block**	`echo ^(note^)`
`Wow!` **with delayed expansion**	`echo Wow^^!`
A line with no trailing newline	`<nul set /p "=text"`

B.4 Reserved device names

`CON`, `PRN`, `AUX`, `NUL`, `COM1` to `COM9`, `LPT1` to `LPT9` cannot be used as file or folder names, with or without an extension.

Appendix C: Error Level and Exit Code Reference

C.1 Conventions

An exit code of **0** means success; any other value means some kind of failure, unless the tool documents otherwise. Read it with `%errorlevel%` (exact value, but stale inside the same block) or `if errorlevel n` (true for **n or higher**). Store it immediately: `set "rc=%errorlevel%"`.

C.2 Codes of common tools

Tool	Code	Meaning
`robocopy`	0	Nothing copied; source and destination already match
	1	Files were copied
	2	Extra files or folders in the destination
	4	Mismatched files or folders
	8	Some files or folders could not be copied
	16	Fatal error, nothing copied
		Values combine by addition. **0 to 7 are success; 8 and above are failure**
`xcopy`	0	Files copied
	1	No files found to copy
	2	Stopped by the user (Ctrl+C)
	4	Initialization error: memory, disk space or invalid syntax
	5	Disk write error
`find` , `findstr`	0	Match found
	1	No match
	2 or more	Error, such as a missing file or bad option
`fc`	0	Files identical
	1	Files differ
	greater	An error such as a missing file; see `fc /?`
`where`	0, 1, 2	Found, not found, error

Tool	Code	Meaning
`forfiles`	0	At least one file matched
	1	No file matched (an error message goes to stderr)
`choice`	1, 2, ...	Position of the pressed key in `/c`
	0	Ctrl+C was pressed
	255	Error in the command line
`ping`	0	Can be 0 even without a reply; check the output for `TTL=`
	1	Usually a failure or no reply
`tasklist`	0	Always 0, even when nothing matches; search the output
`taskkill`	0	Process ended
	128	No matching process found
`sc`	0	Success
	1060	The service does not exist
	1056	Service already running
	1062	Service not started
	5	Access denied
`net`	0	Success
	2	Common failure code; the message gives the system error number
`reg`	0, 1	Success, failure
`schtasks`	0, 1	Success, failure
`curl`	0	Success
	6	Could not resolve host
	7	Failed to connect
	22	HTTP error status (with `-f`)
	28	Operation timed out
`tar`	0, 1	Success, error
`powershell -Command`	0	Success
	1	An error occurred, unless `exit n` was called
`powershell -File`	n	The exit code of the script

C.3 Frequent Windows system error numbers

These numbers appear as exit codes of programs and in messages.

Number	Meaning
2	The system cannot find the file specified
3	The system cannot find the path specified
5	Access is denied
32	The file is being used by another process
53	The network path was not found
87	The parameter is incorrect
112	There is not enough space on the disk
183	Cannot create a file that already exists
1223	The operation was cancelled by the user (for example a declined elevation prompt)

C.4 Codes used by the projects in this book

Script	0	1	2	3	4
`organize_files.cmd`	ok		usage	refused or not found	some moves failed
`rename_bulk.cmd`	ok		usage or invalid input	folder not found	some renames failed
`backup.cmd`	ok		usage	unsafe or missing	copy failed
`diskmon.cmd`	all ok	warning	critical	monitor error	
`cleanup_old.cmd`	ok	some lines skipped	usage	policy missing	
`netreport.cmd`	all healthy	problems found		targets file missing	

Appendix D: Parameter Modifier Reference

D.1 Modifiers

Written `%~X1` for the parameters `%0` to `%9` and `%%~Xv` for `for` variables (`X` is one or more letters from the table).

Modifier	Result	Example (`C:\Data\My Files\report.txt`)
`%~1`	Argument without surrounding quotes	`C:\Data\My Files\report.txt`
`%~f1`	Fully qualified path	`C:\Data\My Files\report.txt`
`%~d1`	Drive letter and colon	`C:`
`%~p1`	Path, with a trailing backslash	`\Data\My Files\`
`%~n1`	Name without extension	`report`
`%~x1`	Extension with the dot	`.txt`
`%~s1`	Short (8.3) names	`C:\Data\MYFILE~1\report.txt`
`%~a1`	Attributes	`--a--------`
`%~t1`	Last-write date and time (locale format)	`10/23/2026 09:14 AM`
`%~z1`	Size in bytes	`2048`
`%~$PATH:1`	First match of the name in the `PATH` folders	Full path or empty

D.2 Combinations

Written	Meaning
`%~dp0`	Drive and folder of the running script, ends with `\`
`%~f0`	Full path of the running script
`%~nx0`	File name of the running script
`%~dpnx1`	Full path of argument 1 (same as `%~f1` for existing items)
`%~nx1`	Name and extension of argument 1
`%~dp1`	Drive and folder of argument 1
`%~z1`	Size of argument 1; 32-bit limit applies when used in arithmetic

D.3 Notes

1. `%~dp0` is the most useful: it locates the script's own folder regardless of the current directory.

2. Modifiers such as `%~z`, `%~t` and `%~a` need the item to exist.
3. In `for` loops the variable letter follows the modifiers: `%%~nxf`, `%%~zf`, `%%~dpf`.
4. `%*` takes **no** modifiers; `shift` does not change it.
5. `%~dp0` ends with a backslash: write `%~dp0file.txt`, not `%~dp0\file.txt`.
6. Only `%0` to `%9` exist directly; `shift` reaches further arguments.

Appendix E: Common Pitfalls and Troubleshooting Checklist

E.1 The twenty most common pitfalls

#	Pitfall	Symptom	Remedy	Chapter
1	Spaces around `=` in `set`	Variable not found; names with trailing spaces	`set "name=value"`	5
2	Unquoted paths	"The system cannot find the path specified" with spaces	Quote every path	3, 6
3	`%var%` inside a block or loop	Old value shown	`setlocal EnableDelayedExpansion` and `!var!`	7
4	Single `%` in a script's `for` loop	Syntax error	`%%i` in batch files	10
5	`else` on its own line	Syntax error	`) else (` on one line	9
6	Unquoted `)` in a path inside a block	"was unexpected at this time"	Quote, or delayed expansion	9
7	Octal numbers `08`, `09`	"Invalid number"	Strip zeros or use `1xx-100`	8, 15
8	Comparing quoted numbers	`"10"` less than `"9"`	Compare unquoted, validated numbers	8
9	Empty variable in `if`	Syntax error	Quote both sides or use `defined`	9
10	Wrong `if errorlevel` order	Several branches run	Test from highest to lowest	9
11	Reading `%errorlevel%` late	Value already replaced	Store it immediately	14
12	No `exit /b` before subroutines	Fall-through into a routine	End main with `exit /b`	11
13	`exit` instead of `exit /b`	The whole window closes	Use `exit /b`	4
14	`pause` in unattended scripts	Task never finishes	Remove `pause`	4, 17
15	`ping` exit code for reachability	False positives	Search for `TTL=`	14, 18

#	Pitfall	Symptom	Remedy	Chapter
16	`robocopy` code 1 treated as error	Needless failures	Failure is 8 or higher	3, 14
17	Redirection with a digit before `>`	Empty file, wrong output	Put redirection first	12
18	Parsing `%DATE%` by position	Wrong dates on other locales	Use PowerShell formats	15
19	User text pasted into commands	Injection	Validate with allow-lists	23
20	LF line endings	Labels not found	Save with CRLF	2, 24

E.2 Troubleshooting checklist

Work through the list in order; stop at the first problem found.

1. **Reproduce it.** Run the script from an open Command Prompt window (`cmd /k`) so error messages stay visible.
2. **Show the commands.** Remove `@echo off` or add `echo on` around the failing part.
3. **Look at the inputs.** Print arguments and variables in brackets: `echo [%1] [%var%]`.
4. **Check the folder.** Print `%CD%` and `%~dp0`. Is it where you think?
5. **Check quoting.** Are all paths quoted? Does any value contain `&` , `)` , `%` , `!` or `^` ?
6. **Check expansion timing.** Is a variable changed and read in the same block?
7. **Check numbers.** Any leading zeros, empty values, values above 2,147,483,647?
8. **Check exit codes.** Which tool returned what? Store `%errorlevel%` right after each command.
9. **Check rights.** Does the task need administrator rights? Is the file read-only or in use?
10. **Check the environment.** `where tool` , `ver` , `chcp` , `echo %PATH%` . Does it differ from the machine where the script worked?
11. **Check the file.** CRLF line endings, no BOM, correct extension, no `.txt` appended.
12. **Cut it down.** Remove code until the problem disappears; the last piece removed is the cause.
13. **Change one thing at a time**, and keep a test that fails before and passes after.

E.3 Quick diagnosis by message

Message	First thing to check
`'x' is not recognized...`	Spelling, `PATH` , extension, empty variable
`... was unexpected at this time.`	Empty variable in `if` ; unquoted `)` ; missing quote
`The syntax of the command is incorrect.`	Empty variable, unquoted path, wrong switch
`The system cannot find the batch label...`	Label spelling; LF endings; label in a block

Message	First thing to check
`Missing operand.`	Empty value in `set /a`
`Invalid number...`	Leading zero (octal), text in arithmetic, more than 32 bits
`Access is denied.`	Elevation, permissions, read-only attribute
`More?` **prompt**	Trailing caret, unclosed quote or parenthesis

Appendix F: Cheat Sheet

Basics

Task	Syntax
First lines of a script	`@echo off` and `setlocal EnableExtensions`
Comment	`rem text`
Blank line	`echo(`
Assign	`set "name=value"`
Read	`%name%`, or `!name!` with delayed expansion
Delete a variable	`set "name="`
Prompt	`set /p "name=Text: "`
Arithmetic	`set /a "x=a+b*2"`; modulo is `%%`; 32-bit integers
Script folder	`%~dp0`
Arguments	`%1` to `%9`, `%*`, `%~1` without quotes, `shift`
End a script	`exit /b n`
Labels and calls	`:label`, `goto :label`, `goto :eof`, `call :label args`
Pause, wait	`pause`, `timeout /t 5 /nobreak >nul`
Hide output	`>nul`, `2>nul`, `>nul 2>&1`

Decisions

Task	Syntax
String test	`if /i "%a%"=="text" (...) else (...)`
Number test	`if %n% geq 10 (...)` with `EQU NEQ LSS LEQ GTR GEQ`
File or folder	`if exist "path" (...)`, folder only: `if exist "path\"`
Variable defined	`if defined name (...)`
Last command failed	`if errorlevel 1 (...)` (means 1 or more)
Exact code	`if %errorlevel% equ 3 (...)`
Chain on success or failure	`a && b`, `a || b`, `a & b`

Loops

Task	Syntax
Words or files	`for %%f in (*.txt) do echo %%f`
Count	`for /l %%i in (1,1,10) do echo %%i`
Folders	`for /d %%d in (*) do echo %%d`
Tree	`for /r "folder" %%f in (*.log) do echo %%f`
File lines	`for /f "usebackq eol=~ delims=" %%a in ("file") do echo %%a`
Tokens	`for /f "tokens=1,3 delims=," %%a in (file) do ...`
Command output	`for /f %%a in ('command ^| findstr x') do ...`
Skip header	`for /f "skip=1 ..."`
Loop with changing variables	`setlocal EnableDelayedExpansion` and `!var!`

Strings

Task	Syntax
Substring	`%s:~start,length%`, `%s:~-3%`, `%s:~0,-1%`
Replace or delete	`%s:old=new%`, `%s:old=%`
Remove up to and including a text	`%s:*text=%`
Dynamic offsets	`!s:~%a%,%b%!`
Pad a number	`set "p=000%n%"` then `%p:~-3%`
Upper or lower case	replace each letter in a loop (Chapter 7)

Redirection and escapes

Task	Syntax
Overwrite, append	`> file`, `>> file`
Input	`< file`
Errors	`2> file`, `2>&1` (write `> file 2>&1`)
Pipe	`command1 | command2`
Safe write	`>> "%LOG%" echo text`
Escapes	`^&`, `^|`, `^>`, `^<`, `^^`, `%%`, `^^!`
No newline	`<nul set /p "=text"`

Useful commands

Task	Command
Robust copy	`robocopy src dst /e /r:1 /w:1` (rehearse with `/l`; 8 or more is failure)
Find text	`findstr /s /i /n "text" *.txt`
Count lines	`find /c /v "" < file`
Admin check	`fltmc >nul 2>&1`
Tool present	`where /q tool`
Timestamp	`powershell -NoProfile -Command "Get-Date -Format yyyyMMdd_HHmmss"`
Files older than N days	`forfiles /p folder /d -N /c "cmd /c echo @path"`
Reachability	`ping -n 1 host | find "TTL="`
Processes	`tasklist /fi "imagename eq x.exe"`, `taskkill /pid n`
Services	`sc query name`, `net start name`
Hash	`certutil -hashfile file SHA256`

Safety rules

1. Quote paths and assignments. 2. Validate input with allow-lists. 3. Dry run first for anything destructive. 4. Check exit codes (robocopy: below 8 is success). 5. Never put secrets in scripts. 6. CRLF endings, `exit /b` before subroutines, no `pause` in unattended scripts.

Appendix G: Glossary

Term	Meaning
Allow-list	A rule that states exactly what input is acceptable and rejects everything else.
Argument	A value passed to a script or command after its name; available as `%1`, `%2`, and so on.
ANSI escape sequence	A character sequence beginning with the ESC character that sets colours or moves the cursor on terminals that support it.
Batch file	A text file of commands run by `cmd.exe`, with the extension `.bat` or `.cmd`.
BOM (byte order mark)	Bytes at the start of a Unicode file that identify its encoding; they can break the first line of a batch file.
Caret	The character `^`, the escape character of `cmd.exe`.
CIM	Common Information Model: the management interface queried by `Get-CimInstance`, the replacement for `wmic`.
Code page	The table that maps byte values to characters; `chcp` shows or sets the console's.
Command extensions	Enhancements to `cmd.exe` (enabled by default) such as `set /a`, extended `for` and `if`.
Console host	The program that draws the console window: `conhost.exe` or Windows Terminal.
CRLF	Carriage return and line feed: the Windows line ending, required in batch files.
Delayed expansion	Replacement of `!var!` at the moment a command runs rather than when the line is parsed.
Dry run	A mode that shows what a tool would do without changing anything.
Environment variable	A named text value inherited by programs started from a process.
ERRORLEVEL	The exit code of the most recent command, read with `%errorlevel%` or `if errorlevel`.
Exit code	The number a program returns when it ends; 0 usually means success.
Injection	Changing what a command does by supplying data that the interpreter treats as syntax.
Internal command	A command built into `cmd.exe`, such as `echo`, `set` and `if`.
Label	A line beginning with a colon, the target of `goto` and `call`.
Least privilege	Running with only the rights a task needs.
Local environment	The scope created by `setlocal`, discarded by `endlocal`.
Modifier	A letter after `%~` that extracts part of a path, such as `%~dp0`.

Term	Meaning
Octal	Base-8 numbers; a leading zero makes a number octal in `set /a`.
Parameter	See *argument*.
Pipe	The operator `|` that connects the output of one command to the input of the next.
Redirection	Reconnecting a standard stream to a file, device or program.
Regular expression	A pattern language for text; `findstr` supports a limited dialect.
Shell	A program that reads and runs commands, such as `cmd.exe` or PowerShell.
Standard streams	Input (0), output (1) and error (2) of a program.
Subroutine	A labelled block of code run with `call :label` and ended with `exit /b`.
Switch	A command-line option, usually introduced by a slash.
Token	A piece of a line produced by `for /f` splitting on delimiters.
UAC	User Account Control: the prompt that grants an elevated token.
UNC path	A network path of the form `\\server\share`.
Variable expansion	Replacing a variable reference with its value.
Wildcard	The characters `*` and `?` that match parts of file names.

Appendix H: Answers to All Exercises

Answers to the exercises of every chapter and to the quiz at the end of each Part. Where an exercise has several valid solutions, one correct solution is shown. Scripts assume the practice folder of the preface and Windows 10 or 11.

Part 1: Foundations

Chapter 1

1.1 Console host (`conhost.exe` or Windows Terminal), shell (`cmd.exe` or PowerShell), program (`ping.exe` , `robocopy.exe` or another script).

1.2 `.bat` and `.cmd` .

1.3 Internal. Run `where copy` ; it reports that no files were found because `copy` is built into `cmd.exe` .

1.4 Possible answers: text versus objects, `%name%` versus `$name` , emulated versus real functions, different escape characters, different extensions. Text versus objects matters most for data handling: CMD must parse text, PowerShell accesses properties.

1.5 The whole parenthesized block is parsed in one step. `%x%` is replaced with its current value (`1`) at that moment, before `set "x=2"` runs.

1.6 A script written for an older Windows version still runs on newer ones. To keep that promise, new features had to be added within the existing grammar, producing special cases and unusual symbols.

1.7 Typed at the prompt, `echo %date%` prints the date. In a batch file, `%%date%%` becomes `%date%` during percent expansion, so the file prints the literal text `%date%` . To print the date from a batch file write `echo %date%` .

1.8 Percent expansion (nothing to expand), parsing splits the line at `&` into two commands, echo (if on) displays the line, then the first `echo` executes and prints `Hello` , then the second prints `World` .

Chapter 2

2.1 For example: Start menu search, Win+R, Explorer address bar.

2.2 The prompt shows the full path of that folder, for example `C:\Users\Alex\BatchBook>` .

2.3 Without extensions you cannot see mistakes such as `script.bat.txt` .

2.4 The three runs print `Hello` . With `cmd /k` the new window stays open afterward.

2.5 `.cmd` always updates `ERRORLEVEL` for the listed commands, `.bat` only on failure. `PATHEXT` order decides which of two same-named files runs first (`.BAT` before `.CMD`).

2.6 Typically `437` , `850` or `1252` -family values depending on region and system settings. A colleague with another system locale has another OEM code page.

2.7 The three bytes of the BOM at the start of the file can be interpreted as characters that attach to the first command, so `cmd.exe` does not recognise `@echo`. Save as "UTF-8" (not "UTF-8 with BOM") in Notepad++, VS Code or Notepad, or keep the file ASCII.

2.8 Without `call`: `Step one` then `Hello`, and the first script ends without printing `Step two`. With `call`, control returns to the caller after `hello.bat` finishes.

Chapter 3

3.1 `cd`, then `cd /d "%USERPROFILE%\BatchBook"`, then `dir /b`.

3.2 `md work\notes\2026\drafts`

3.3 `/o:-s` sorts by size, largest first. Command: `dir /o:-s /a:-d C:\Windows`.

3.4

```
echo a > one.txt & echo b > two.txt & echo c > three.txt
ren *.txt *.bak
```

3.5 With `/l` the output lists the files that would be copied and nothing appears in the destination. Without `/l` the same list is shown and the files are actually created. Command: `robocopy work\notes work\notes_copy /e /l`, then repeat without `/l`.

3.6 `attrib +h +r file.txt`, `dir /a:h`. A plain `del file.txt` reports access denied for read-only files. Use `attrib -r -h file.txt` and then `del file.txt`, or `del /f file.txt`.

3.7 3 = 1 + 2: files were copied and extra files exist in the destination. Success: values below 8 are not failures. A script tests `if %errorlevel% geq 8`.

3.8

```
pushd work\notes
dir /b
popd
```

Chapter 4

4.1

```
@echo off
echo Hello, world!
```

4.2 `echo off` turns off echoing but the line itself is still echoed once; `@echo off` also suppresses that line, because `@` hides the echo of one command.

4.3

```
@echo off
echo One
echo:
echo Two
echo:
echo Three
```

4.4

```
@echo off
echo Profit ^& Loss: 100%% (50 ^> 25)
```

4.5 A `::` comment is a label. Labels inside a parenthesized block can break the block's parsing and cause errors. `rem` is an ordinary command and is safe.

4.6

```
@echo off
title Info
cls
echo Computer: %COMPUTERNAME%
echo Folder: %CD%
pause >nul
```

4.7 `exit` terminates the whole `cmd.exe` session, so the window closes. Use `exit /b` (optionally with a code) to leave only the script.

4.8 The script calls `ping`, and because the current directory is searched first the interpreter finds `ping.bat` itself and starts it again, producing a repeating loop (each run starts a new instance). Press Ctrl+C to stop. The lesson: never name a script after an existing command.

Part 1 Quiz

Q1. `cmd.exe`. **Q2.** False. **Q3.** `/d`. **Q4.** 1. **Q5.** It deletes matching files in the current folder and all subfolders without confirmation and bypasses the Recycle Bin. **Q6.** CRLF. **Q7.** `rem`. **Q8.** Ends the script (or subroutine) and sets the exit code to 2. **Q9.** Program. **Q10.** 1.

Part 2: Core Language

Chapter 5

5.1 `set "greeting=Good morning"` then `echo %greeting%`.

5.2 `set "greeting="` then `set greeting` prints `Environment variable greeting not defined`.

5.3 `set PROGRAM`

5.4

```
@echo off
set /p "name=Name: "
set /p "age=Age: "
echo %name% is %age% years old
```

5.5

```
@echo off
set "folder=C:\Temp"
set /p "folder=Folder [%folder%]: "
echo Using %folder%
```

5.6 The variable is named `count ` (with a trailing space) and its value is `0`. `%count%` is not defined. Use `set "count=0"`.

5.7 With `setlocal` / `endlocal`, `x` is `1` again afterward. Without `setlocal`, `x` is `2`.

5.8 `echo %RANDOM% %RANDOM%` prints two numbers between 0 and 32767. `%RANDOM%` is dynamic and recomputed at each use, so the values differ.

Chapter 6

6.1

```
@echo off
echo First: %~1
echo Second: %~2
```

6.2 `echo %~nx1`

6.3 `echo %~f0`

6.4

```
@echo off
echo Extension: %~x1
echo Size: %~z1
```

6.5

```
@echo off
:loop
if "%~1"=="" goto :eof
echo %~1
shift
goto :loop
```

6.6 With a quoted argument `"a b"`, `"%1"` expands to `""a b""`, which breaks the quoting and can cause "b was unexpected" errors. Use `"%~1"`.

6.7

```
@echo off
setlocal
set /a "total=0"
:loop
if "%~1"=="" goto :done
set /a "total+=%~1"
shift
goto :loop
:done
echo Total: %total%
```

6.8

```
@echo off
set "INI=%~dp0settings.ini"
if not exist "%INI%" (
```

```
    echo Error: settings.ini not found next to the script.
    exit /b 1
)
echo Found %INI%
```

Chapter 7

7.1 `set "s=Batch Scripting"` then `echo %s:~0,5%` prints `Batch`.

7.2 `echo %s:~-4%` prints `ting`.

7.3 `echo %s:Batch=Windows%` prints `Windows Scripting`.

7.4 `set "s=a b c d"` and `echo %s: =%` prints `abcd`.

7.5

```
@echo off
setlocal EnableDelayedExpansion
set "t=0"
for /l %%i in (1,1,5) do (
    set /a "t+=%%i"
    echo Total: !t!
)
```

Output totals: 1, 3, 6, 10, 15.

7.6

```
@echo off
setlocal EnableDelayedExpansion
set "start=2"
set "s=ABCDEFG"
echo !s:~%start%,3!
```

Prints `CDE`.

7.7 The form `*=` is the wildcard-prefix replacement: it deletes everything up to and including the first `*`, leaving `b*c`. To remove every asterisk, rebuild the string character by character:

```
@echo off
setlocal EnableDelayedExpansion
set "t=a*b*c"
set "out="
for /l %%i in (0,1,20) do (
    set "ch=!t:~%%i,1!"
    if defined ch if not "!ch!"=="*" set "out=!out!!ch!"
)
echo !out!
```

Prints `abc` (the loop bound of 20 exceeds the string length; empty characters are skipped by `if defined`).

7.8

```
:toupper  varname
setlocal EnableDelayedExpansion
set "s=!%~1!"
```

```
for %%L in (A B C D E F G H I J K L M N O P Q R S T U V W X Y Z) do set "s=!s:%%L=%%L!"
endlocal & set "%~1=%s%"
exit /b 0
```

Chapter 8

8.1

```
@echo off
set /a "r=(8+4)*3-5"
echo %r%
```

Prints `31`.

8.2 `19/4` is 4 (truncated). `19 %% 4` is 3 (written `%%` in a batch file).

8.3 `if %n% gtr 100 echo big`

8.4

```
@echo off
set /p "a=First : "
set /p "b=Second: "
set /a "s=a+b, d=a-b, p=a*b"
echo Sum %s%  Diff %d%  Product %p%
if %b% neq 0 (set /a "q=a/b" & call echo Quotient %%q%%) else echo Cannot divide by zero
```

8.5 `"100" lss "20"` is true (text: `1` before `2`). `100 lss 20` is false. `"abc" equ "ABC"` is false (text, case-sensitive; `equ` on non-numbers compares text). `if /i "abc" equ "ABC"` is true.

8.6

```
@echo off
set /a "t=3725, h=t/3600, m=(t%%3600)/60, s=t%%60"
echo %h% h %m% min %s% s
```

Prints `1 h 2 min 5 s`.

8.7

```
:parity  number
set /a "bit=%~1 & 1"
if %bit%==0 (echo %~1 is even) else (echo %~1 is odd)
exit /b 0
```

8.8

```
@echo off
set /a "bold=1, italic=2, underline=4, v=0"
set /a "v|=bold" & set /a "v|=underline"
echo %v%
set /a "v&=~bold"
echo %v%
```

First value 5, then 4.

Chapter 9

9.1

```
@echo off
if "%OS%"=="Windows_NT" (echo Windows) else (echo Unknown)
```

9.2

```
@echo off
if exist "C:\Windows\notepad.exe" (echo Found) else (echo Not found)
```

9.3 `if defined HOMEPATH (echo defined) else (echo not defined)`

9.4

```
@echo off
set /p "pw=Password: "
if /i "%pw%"=="Secret" (echo Access granted) else (echo Access denied)
```

9.5

```
@echo off
if not exist "logs\" md "logs"
```

9.6 Use `ping -n 1 192.0.2.1 | find "TTL=" >nul` and test `errorlevel`. `ping` can return 0 when a router answers with "Destination host unreachable", so the exit code alone does not prove the target replied.

9.7

```
@echo off
setlocal EnableDelayedExpansion
set "mb=%~1"
echo(!mb!| findstr /r /x "[0-9][0-9]*" >nul || (echo Not a number & exit /b 1)
if %mb% lss 10 (echo Small) else if %mb% lss 100 (echo Medium) else (echo Large)
```

Note: the numeric comparison is performed only after validation. Values with leading zeros such as `08` can be misread as octal; strip leading zeros if you accept them.

9.8 The block is parsed as a whole, so `%errorlevel%` is replaced before `dir` runs. Rewrite using `if errorlevel 1 (echo failed) else (echo ok)` or enable delayed expansion and write `!errorlevel!`.

Chapter 10

10.1 `for /l %%i in (1,1,10) do echo %%i`

10.2 `for %%f in ("C:\Windows\*.exe") do echo %%~nxf`

10.3 `for /l %%i in (20,-5,0) do echo %%i`

10.4 `for /d %%d in ("%USERPROFILE%\*") do echo %%~nxd`

10.5

```
@echo off
setlocal EnableDelayedExpansion
set "n=0"
for /f "usebackq eol=~ delims=" %%a in ("%~1") do (
    set /a "n+=1"
    echo !n!: %%a
)
```

Delayed expansion is used only for the counter; avoid it when the lines may contain `!`.

10.6 `for /f "tokens=1-3 delims=;" %%a in ("alpha;beta;gamma") do (echo %%a & echo %%b & echo %%c)`

10.7

```
@echo off
setlocal EnableDelayedExpansion
set "n=0"
for /r "%~1" %%f in (*.txt) do set /a "n+=1"
echo %n% text files
```

(`!n!` is not needed because it is read after the loop.)

10.8 `for /f "delims=" %%z in ('tzutil /g') do echo %%z`. `tzutil /g` prints the time zone identifier in a fixed form (for example `W. Europe Standard Time`), independent of regional display formats, whereas `%TIME%` and `%DATE%` change with the user's locale.

Chapter 11

11.1

```
@echo off
set "i=0"
:next
set /a "i+=1"
echo %i%
if %i% lss 5 goto :next
```

11.2

```
@echo off
call :hello
call :hello
exit /b 0
:hello
echo Hello
exit /b 0
```

11.3 `exit /b n` leaves the routine and sets the exit code to n; `goto :eof` leaves it without changing the code.

11.4

```
:cube  outvar n
set /a "%~1=%~2*%~2*%~2"
exit /b 0
```

11.5

```
:minmax  a b minvar maxvar
setlocal
set /a "lo=%~1, hi=%~2"
if %~1 gtr %~2 set /a "lo=%~2, hi=%~1"
endlocal & set "%~3=%lo%" & set "%~4=%hi%"
exit /b 0
```

11.6 The whole line is parsed and its percent variables are replaced before any command on it runs. By the time `endlocal` discards `value`, the text of the `set` command already contains the number.

11.7

```
:fib  outvar n
setlocal
set /a "n=%~2"
if %n% leq 1 (endlocal & set "%~1=%n%" & exit /b 0)
set /a "a=n-1, b=n-2"
call :fib ra %a%
call :fib rb %b%
set /a "r=ra+rb"
endlocal & set "%~1=%r%"
exit /b 0
```

Performance: each call spawns two more, so the work grows exponentially; `fib 25` already takes noticeable time because every subroutine call has overhead. Use a loop instead.

11.8

```
@echo off
call %*
exit /b %errorlevel%

:repeat  outvar text count
setlocal EnableDelayedExpansion
set "out="
for /l %%i in (1,1,%~3) do set "out=!out!%~2"
endlocal & set "%~1=%out%"
exit /b 0
```

Called with `call "%~dp0strlib.bat" :repeat line "=" 20`.

Chapter 12

12.1 `ipconfig > net.txt`

12.2 `echo %date% >> log.txt` (or `>> log.txt echo %date%` to avoid the trailing space).

12.3 `ping -n 1 127.0.0.1 >nul 2>&1`

12.4 `dir /b existing.txt missing.txt > results.txt 2> errors.txt`

12.5

```
@echo off
echo Total: 50%% (5 ^> 3) ^& more
```

(The parentheses are printed correctly at the top level of a script.)

12.6 `sort /r < file.txt` or `type file.txt | sort /r`.

12.7 The right side of a pipe runs in a separate `cmd.exe`, so the variable is set in that process, which then ends. To capture output use `for /f "delims=" %%a in ('command') do set "x=%%a"`.

12.8 `findstr /c:"error" app.log >nul && echo ok || echo clean`. It is safe because `echo` cannot fail, so the `||` branch runs only when `findstr` reported no match (exit code 1).

Part 2 Quiz

Q1. `set "x=a b"`. **Q2.** The drive and folder of the running script, with a trailing backslash. **Q3.** `%s:~-3%`. **Q4.** The block is parsed once and `%total%` is replaced before the loop runs; use delayed expansion (`!total!`). **Q5.** `3`; and `if "10" lss "9"` is true (text comparison). **Q6.** `%%`. **Q7.** All codes of 2 or higher. **Q8.** On the same line as the closing parenthesis of the `if` block: `) else (`. **Q9.** Two (`%%i`). **Q10.** The third token (token 3). **Q11.** Otherwise execution continues into the subroutine. **Q12.** `> log.txt 2>&1`. **Q13.** `echo ^&` (or quote it). **Q14.** They exist only in the pipe's own `cmd.exe` and are lost.

Part 3: Practical Programming

Chapter 13

13.1 `(echo Ada& echo Linus& echo Grace) > names.txt` then `type names.txt`.

13.2 `find /c /v "" < names.txt`

13.3 `findstr /i "linus" names.txt`

13.4 `findstr /s /i /m "error" *.log`

13.5 `findstr /r "^[A-Z][0-9][0-9]*" file.txt`

13.6 `sort /r < file.txt > sorted.txt`

13.7

```
@echo off
fc "%~1" "%~2" >nul && (echo same) || (echo different)
```

13.8

```
@echo off
setlocal EnableDelayedExpansion
for /f "usebackq delims=" %%a in (`findstr /n "^" "%~1"`) do (
    set "line=%%a"
    for /f "delims=:" %%n in ("%%a") do set "num=%%n"
    set "text=!line:*:=!"
```

```
    echo !num!. !text!
)
```

Chapter 14

14.1

```
@echo off
if "%~1"=="" (echo Usage: %~nx0 argument & exit /b 1)
```

14.2 `ping -n 1 127.0.0.1 >nul` then `echo %errorlevel%`.

14.3 `where /q git && echo found || echo missing`

14.4

```
@echo off
setlocal EnableDelayedExpansion
set "m=%~1"
echo(!m!| findstr /r /x "[0-9][0-9]*" >nul || (echo Not a number & exit /b 2)
set /a "m=m+0" 2>nul
if %m% lss 1 (echo Too small & exit /b 2)
if %m% gtr 12 (echo Too large & exit /b 2)
echo Valid month %m%
```

(Values with leading zeros such as `08` are rejected by `set /a`; strip zeros first if you must accept them.)

14.5

```
robocopy "%~1" "%~2" /e >nul
set "rc=%errorlevel%"
if %rc% geq 8 (echo FAILED) else if %rc% geq 2 (echo OK with extras) else (echo OK)
```

14.6

```
@echo off
set "LOG=%~dp0x.log"
call :log "one"
call :log "two"
call :log "three"
exit /b
:log
>> "%LOG%" echo %date% %time% %~1
exit /b 0
```

14.7 After resolving `FULL`, add a length test: `if "%FULL:~9,1%"=="" (echo Path too short & exit /b 3)`. An empty tenth character means the path has fewer than 10 characters.

14.8

```
:ping_ok  host
ping -n 1 -w 1500 "%~1" | find "TTL=" >nul
exit /b %errorlevel%
```

`ping` may return 0 when a router answers with "Destination host unreachable"; only a real reply contains `TTL=`.

Chapter 15

15.1 `echo %DATE% %TIME%`

15.2

```
@echo off
set "t=%TIME: =0%"
echo %t:~0,2%%t:~3,2%%t:~6,2%
```

15.3

```
for /f "delims=" %%t in ('powershell -NoProfile -Command "Get-Date -Format yyyyMMdd_HHmmss"') do set "TS=%%t"
```

15.4

```
@echo off
for /f "delims=" %%t in ('powershell -NoProfile -Command "Get-Date -Format yyyyMMdd_HHmmss"') do set "TS=%%t"
echo Created %date% %time%> "log_%TS%.txt"
```

15.5 Use the `:to_cs` routine from `timer.bat` before and after the command, subtract, and print `elapsed/100` seconds.

15.6

```
for /f "usebackq delims=" %%d in (`powershell -NoProfile -Command "(Get-Date).AddDays(-14).ToString('yyyy-MM-dd')"`) do set "D=%%d"
echo %D%
```

15.7

```
:days_between  d1 d2
for /f "usebackq delims=" %%n in (`powershell -NoProfile -Command "([datetime]'%~2' - [datetime]'%~1').Days"`) do set "RESULT=%%n"
exit /b 0
```

The result is in `RESULT`. Dates must be in an unambiguous format such as `yyyy-MM-dd`.

15.8 At 08 or 09 the text `08` / `09` is interpreted as an invalid octal number. Fixes: prefix with `1` and subtract 100 (`set /a "h=1%TIME:~0,2%-100"`) or remove the leading zero with a substitution before use, or use `1%t:~0,2% %% 100` after padding.

Chapter 16

16.1

```
@echo off
choice /c YN /m "Proceed? "
if errorlevel 2 (echo Cancelled) else (echo Confirmed)
```

16.2 `title Demo`, `color 0A`, `color`.

16.3

```
@echo off
for /f %%e in ('powershell -NoProfile -Command "[char]27"') do set "ESC=%%e"
echo %ESC%[32mDone%ESC%[0m
```

16.4

```
@echo off
:ask
set "p="
set /p "p=Choose 1-3: "
if "%p%"=="1" echo One & goto :ask
if "%p%"=="2" echo Two & goto :ask
if "%p%"=="3" goto :eof
echo Invalid.
goto :ask
```

16.5 `choice /c YN /t 10 /d N /m "Continue? "`

16.6

```
@echo off
<nul set /p "=["
for /l %%i in (1,1,30) do <nul set /p "=#"
echo ]
```

16.7

```
:ask_yes_no  outvar prompt
choice /c YN /n /m "%~2 (Y/N) "
if errorlevel 2 (set "%~1=N") else (set "%~1=Y")
exit /b 0
```

16.8 Add the menu text `5. Largest files` and extend the dispatch (and the `choice` key list to `12345Q`, quit becoming 6). The action is:

```
:action_big
dir /a:-d /o:-s /b | findstr /n "^" | findstr /b "[1-5]:"
exit /b 0
```

This prints the first five names of the size-sorted list, numbered 1 to 5.

Chapter 17

17.1 `tasklist /fi "imagename eq svchost.exe" /nh`

17.2 `sc query Spooler | find "RUNNING"`

17.3

```
@echo off
for /f "usebackq" %%p in (`powershell -NoProfile -Command "(Start-Process notepad -PassThru).Id"`) do set "P=%%p"
```

```
timeout /t 3 /nobreak >nul
taskkill /pid %P%
```

17.4 See `is_running.bat` in the chapter.

17.5

```
reg add "HKCU\Software\BatchBookDemo" /v Greeting /t REG_SZ /d "Hello" /f
reg export "HKCU\Software\BatchBookDemo" "%TEMP%\demo.reg" /y
reg delete "HKCU\Software\BatchBookDemo" /f
reg import "%TEMP%\demo.reg"
reg delete "HKCU\Software\BatchBookDemo" /f
```

(The last line removes the test key after verifying the import.)

17.6

```
schtasks /create /tn "BatchBook Test" /tr "cmd /c echo test>>%TEMP%\sched.txt" /sc once /st 23:59
schtasks /run /tn "BatchBook Test"
schtasks /delete /tn "BatchBook Test" /f
```

17.7

```
@echo off
for /f "usebackq" %%g in (`powershell -NoProfile -Command "[math]::Round((Get-CimInstance
Win32_ComputerSystem).TotalPhysicalMemory/1GB)"`) do set "GB=%%g"
echo Memory: %GB% GB
if %GB% lss 8 echo WARNING: less than 8 GB of RAM
```

17.8 `/f` kills every browser process immediately with no chance to save tabs, sessions or downloads, and kills processes of the current user's other windows too. A careful approach: ask the user, send a normal close request (`taskkill /im chrome.exe` without `/f`), wait a few seconds, check with `tasklist`, and only then consider a forced end.

Chapter 18

18.1 `ipconfig | findstr /c:"IPv4" /c:"Default Gateway"` (English labels), or the PowerShell approach of Section 18.3.

18.2 `ping -n 3 127.0.0.1`

18.3 `nslookup example.org 1.1.1.1`

18.4 `netstat -ano | findstr LISTENING`

18.5

```
@echo off
call :ping_ok "%~1"
if errorlevel 1 (echo %~1 is down) else (echo %~1 is up)
exit /b 0
:ping_ok
ping -n 1 -w 1500 "%~1" | find "TTL=" >nul
exit /b %errorlevel%
```

18.6 `netstat -ano | findstr ":443" | findstr ESTABLISHED` shows the PID in the last column; then `tasklist /fi "pid eq <PID>"`.

18.7 Add before the final summary:

```
nslookup "%TARGET_NAME%" 8.8.8.8 2>nul | find /i "Name:" >nul
if errorlevel 1 (call :result FAIL "DNS 8.8.8.8" "no answer") else (call :result PASS "DNS 8.8.8.8" "answered")
```

18.8 A failure to an IP address means no route or no reply at the network layer (cable, gateway, firewall, remote host down). A failure only for the name, with the IP working, means name resolution (DNS) is the problem.

Chapter 19

19.1 `powershell -NoProfile -Command "$PSVersionTable.PSVersion.Major"`

19.2

```
powershell -NoProfile -Command "exit 5"
echo %errorlevel%
```

19.3 `for /f "usebackq delims=" %%u in (` powershell -NoProfile -Command "[Environment]::UserName" `) do set "WHO=%%u"`

19.4

```
@echo off
set "T=Tom & "Jerry""
powershell -NoProfile -Command "Write-Output $env:T"
```

Note: define the variable in a way that cmd accepts; the quoted form `set "T=Tom & "Jerry""` works because `set` takes everything up to the last quote. The value reaches PowerShell as plain data.

19.5 `add.ps1` : `param([double]$A,[double]$B) $A + $B` . Batch: `powershell -NoProfile -ExecutionPolicy Bypass -File "%~dp0add.ps1" -A 2 -B 3` prints `5`.

19.6

```
tar -a -c -f out.zip folder
tar -tf out.zip
```

19.7

```
@echo off
setlocal
set "HERE=%~dp0"
powershell -NoProfile -Command "$lines = Get-Content -LiteralPath '%~f0'; $start = [array]::IndexOf($lines,
':PS_BEGIN'); & ([scriptblock]::Create(($lines[($start+1)..($lines.Count-1)] -join [Environment]::NewLine)))"
exit /b %errorlevel%

:PS_BEGIN
Get-ChildItem -File -LiteralPath $env:HERE | Sort-Object Length -Descending | Select-Object -First 5 Name, Length
```

The folder is handed to PowerShell through the environment variable `HERE` (a script block has no `$PSScriptRoot`). Since `setlocal` exports its variables to child processes, `HERE` is visible to PowerShell.

19.8 The text becomes part of the code: a name containing a quote or a semicolon changes the command that runs. Safe version:

```
set "TARGET=%~1"
powershell -NoProfile -Command "Get-Item -LiteralPath $env:TARGET"
```

Part 3 Quiz

Q1. Number every line with `findstr /n "^"` and strip the prefix. **Q2.** No, it means OR: either word. **Q3.** 0 to 7 (8 or more is failure). **Q4.** It can return 0 for "Destination host unreachable" replies; search the output for `TTL=`. **Q5.** `fltmc`. **Q6.** A mistake is then visible before data is lost. **Q7.** The position depends on the regional date format. **Q8.** It pads the space before a single-digit hour with a zero. **Q9.** `if errorlevel n` is true for n or higher. **Q10.** It opens a new console window with the quoted text as its title, because the first quoted argument is the title. **Q11.** `(Get-CimInstance Win32_OperatingSystem).Caption`. **Q12.** Through an environment variable (or a script parameter). **Q13.** 1.

Part 4: Advanced Topics

Chapter 20

20.1

```
@echo off
setlocal EnableDelayedExpansion
set "name[1]=Ada" & set "name[2]=Linus" & set "name[3]=Grace"
for /l %%i in (1,1,3) do echo !name[%%i]!
```

20.2

```
@echo off
setlocal EnableDelayedExpansion
set "color[red]=FF0000"
set "k=red"
echo !color[%k%]!
```

20.3

```
:sleep  seconds
set /a "p=%~1+1"
ping -n %p% 127.0.0.1 >nul
exit /b 0
```

20.4

```
@echo off
setlocal EnableDelayedExpansion
```

```
set "n=0"
for %%v in (12 87 33 5 61) do (set /a "n+=1" & set "a[!n!]=%%v")
set "max=!a[1]!"
for /l %%i in (2,1,%n%) do if !a[%%i]! gtr !max! set "max=!a[%%i]!"
echo Largest: %max%
```

Prints `Largest: 87`.

20.5

```
@echo off
setlocal EnableDelayedExpansion
for %%w in (alpha beta alpha gamma beta alpha) do (
    if not defined cnt[%%w] set "cnt[%%w]=0"
    set /a "cnt[%%w]+=1"
)
for /f "tokens=1,2 delims==" %%a in ('set cnt[') do echo %%a = %%b
```

Prints `cnt[alpha] = 3`, `cnt[beta] = 2`, `cnt[gamma] = 1`.

20.6 Push the five words with `:push`, then call `:pop item` five times, printing `%item%` on the line after each call. The words come out in reverse order.

20.7

```
for /l %%i in (%n%,-1,2) do (
    set /a "r=!RANDOM! %% %%i + 1"
    for %%r in (!r!) do (
        set "t=!a[%%i]!"
        set "a[%%i]=!a[%%r]!"
        set "a[%%r]=!t!"
    )
)
```

Each element, from the last to the second, is swapped with a randomly chosen element at or before its position. (Inside the loop `%%i` is the loop variable and `%%r` a second loop variable that converts the computed index.)

20.8 Pass `-ArgumentList $env:SELF_ARGS` as in the chapter and print `Arguments: %*` in the `:elevated` section. Problems: arguments with embedded quotes, ampersands or carets can be re-parsed by `cmd.exe` when the elevated copy starts, so an argument may be split or altered. Validate arguments (Chapter 23) or pass data through a file or the registry instead.

Chapter 21

21.1

```
@echo off
setlocal
call "%~dp0bench_lib.bat" :now t0
ping -n 3 127.0.0.1 >nul
call "%~dp0bench_lib.bat" :report "ping" t0
```

Expect about 2 seconds.

21.2 `(for /l %%i in (1,1,500) do echo %%i) > out.txt`

21.3 Any three of: running an external program such as `find`, either side of a pipe, `for /f` over a command, starting PowerShell, `start`.

21.4 Time `(for /l %%i in (1,1,3000) do echo line %%i)` once without redirection and once with `>nul`. The second is dramatically faster because no console output is drawn.

21.5 Instead of a loop with `echo %%l| findstr pattern`, use `findstr pattern file.txt`.

21.6

```
for /l %%i in (1,1,10000) do set /a "n+=1"
```

replaces `:loop` / `set /a "i+=1"` / `if %i% lss 10000 goto :loop`. The `for /l` version is several times faster.

21.7 Call once:

```
for /f "usebackq tokens=1,* delims==" %%a in (`powershell -NoProfile -Command "$o=Get-CimInstance
Win32_OperatingSystem; 'A='+$o.Caption; 'B='+$o.BuildNumber; 'C='+$o.OSArchitecture; 'D='+$o.Version"`) do set "%
%a=%%b"
```

21.8 Each iteration creates two `cmd.exe` processes for the pipe plus a `findstr` process: three process starts per line, and their startup time dominates. A single `findstr` over the file is one process that reads all lines internally.

Chapter 22

22.1 Insert `echo on` before the lines and `echo off` after them. The output shows each command with variables expanded.

22.2 `set "x=abc "` then `echo [%x%]` prints `[abc ]`.

22.3 `echo` was run with no text, because the variable was empty (`echo %var%` became `echo`), so it reported the echo state. Use `echo(%var%`.

22.4 Add `call :dbg "message"` calls and the routine `:dbg` that tests `if defined DEBUG`.

22.5 `08` is invalid octal. Write `set /a "n=1%x%-100+1"` after prefixing, or use `set /a "n=8+1"`; in general use the prefix-with-1 trick (Chapter 15).

22.6 The label name is misspelled, or the label does not exist, or the file has LF-only line endings; it can also be a label placed inside a block, or a call to a label of another file.

22.7

```
call "%T%" :assert_equal "max 3 7" 7 "%r%"
```

after `call :max r 3 7`; similar tests for `:max 9 2` (expect 9), `:max 5 5` (expect 5) and one deliberately wrong expectation.

22.8

```
:do
if "%DRY%"=="1" (echo [dry run] %*) else (%*)
exit /b 0
```

`%*` repeats the arguments exactly as given. Quotes in them are preserved, which is needed for paths with spaces, but a `)` or `&` in a path inside the parenthesized branch could end it early. Quoted paths avoid this.

Chapter 23

23.1 Variables are expanded before the line is parsed, so the `&` in the value becomes a command separator and the text after it is executed as a separate command.

23.2

```
setlocal EnableDelayedExpansion
set "u=%~1"
echo(!u!| findstr /r /x "[A-Za-z][A-Za-z][A-Za-z][A-Za-z]*" >nul
```

This checks letters only and a minimum of three. To enforce a maximum of 20, also measure the length with a loop or the `:strlen` routine of Chapter 7 and test `leq 20`.

23.3 Command lines can be seen by other users and tools while the program runs and are often written to logs and history, so the secret is exposed to anyone who can read them.

23.4

```
call :is_under "%USERPROFILE%\BatchBook\work" "%~1"
if errorlevel 1 (echo Folder not allowed. & exit /b 3)
```

23.5 The variable `cmd` is executed as a command, so whatever the user types runs with the script's rights (including chained commands with `&`). Use a fixed menu with known keys instead.

23.6 Any four of: heavy obfuscation, encoded PowerShell, downloads followed by execution, persistence (`Run` keys, scheduled tasks), account changes, deleting shadow copies or logs, disabling security tools, hidden files, hard-coded credentials.

23.7 Different folder names share the text prefix `C:\Data`. A naive prefix test would accept `C:\DataBackup\x.txt`. The routine also checks that the character after the root is a backslash.

23.8 Store the script in a folder writable only by administrators (verify with `icacls`), record and verify its hash, run it under the least-privileged account that works, avoid secrets in the file, log its actions, and put the file under version control with review of changes.

Chapter 24

24.1 A header block with Name, Purpose, Usage, Parameters, Exit codes, Requires, Rights, Author, Version and Changes, following `header_template.cmd`.

24.2 `folder_name`, `backup_script_v2.cmd` (or `backup_script.cmd` with the version in its header), `:do_it`.

24.3 `::` is a label. A label inside a parenthesized block can break the block and cause syntax errors. `rem` is an ordinary command.

24.4

```
*.bat text eol=crlf
*.cmd text eol=crlf
*.ps1 text eol=crlf
```

24.5

```
@echo off
set "CFG_KEEP_DAYS=7"
call :load_config "%~dp0settings.ini" CFG_
echo %CFG_KEEP_DAYS%
exit /b 0
:load_config
for /f "usebackq eol=# tokens=1,* delims==" %%a in ("%~1") do set "%~2%%a=%%b"
exit /b 0
```

Defaults are set first, so a key missing from the file keeps its default.

24.6 Cut the routines into `lib\common.cmd` below the `call %*` line and call them with `call "%~dp0..\lib\common.cmd" :routine args`.

24.7 After loading, add:

```
set "ok="
for /f "delims=0123456789" %%x in ("%CFG_KEEP_DAYS%") do set "ok=bad"
if defined ok (echo KEEP_DAYS must be a whole number & exit /b 2)
if %CFG_KEEP_DAYS% lss 1 (echo KEEP_DAYS too small & exit /b 2)
if %CFG_KEEP_DAYS% gtr 365 (echo KEEP_DAYS too large & exit /b 2)
```

(The `for /f` with digits as delimiters returns a token only if some non-digit remains.) An empty value must be checked separately with `if not defined CFG_KEEP_DAYS`.

24.8 For example: header updated, paths quoted, arguments validated, dry run for destructive steps, exit codes returned, `exit /b` before routines, no `pause`, no secrets, tests pass, CRLF endings.

Part 4 Quiz

Q1. With variables named `a[1]`, `a[2]`; read element `i` with delayed expansion `!a[%%i]!` in a `for /l` loop. **Q2.** All percent variables on a line are expanded before any command on it runs; put the `echo` on the next line. **Q3.** Ping sends one request per second, and three requests take about two seconds between the first and last. **Q4.** `for /l`. **Q5.** Every start creates one or more processes whose startup time dominates the work; use one `findstr` over the whole file. **Q6.** Leading and trailing spaces and empty values. **Q7.** A misspelled or missing label, or LF-only line endings. **Q8.** `Invalid number. Numeric constants are either decimal (17), hexadecimal (0x11), or octal (021).`, because a leading zero means octal and 8 is not an octal digit. **Q9.** The text is pasted into the command line, so `&` and similar characters run other commands; store it in a quoted variable and read it with `!var!` in the check. **Q10.** No; use protected locations, hashes, application control (AppLocker or WDAC), version control and review, or move logic into a signed PowerShell script or executable. **Q11.** Anyone who can edit the script can run code with the privileged account's rights.

Q12. Obfuscation, encoded PowerShell, downloads followed by execution, persistence, account changes, deleting logs or backups, disabling security tools. **Q13.** CRLF; enforce it with `.gitattributes` (`*.bat text eol=crlf`).

Part 5: Projects

Chapter 25

25.1 Set `MAX_NUMBER=20` and `MAX_TRIES=5` in the configuration block.

25.2 After computing the secret add `if defined DEBUG echo [debug] secret=!secret!`, and start the game with `set DEBUG=1` typed beforehand.

25.3

```
choice /c 123 /n /m "Level: 1 Easy, 2 Normal, 3 Hard "
if errorlevel 3 (set "MAX_NUMBER=500" & set "MAX_TRIES=9" & goto :level_done)
if errorlevel 2 (set "MAX_NUMBER=100" & set "MAX_TRIES=7" & goto :level_done)
set "MAX_NUMBER=20" & set "MAX_TRIES=5"
:level_done
```

Place it before `:new_game` or at the start of each game.

25.4 Keep `set "history=!history! !g!"` after each valid guess, reset it with `set "history="` in `:new_game`, and print `echo Guesses so far:!history!`.

25.5 After a win, read the stored record with `set /p "best=" < best.txt` (guarding with `if exist best.txt`), compare `!tries!` to `best` with `lss`, and write the new value with `echo !tries!> best.txt` when it is lower or the file is missing.

Chapter 26

26.1 In the collector add `'BIOS=' + (Get-CimInstance Win32_BIOS).SMBIOSBIOSVersion`, and in the report add `echo BIOS : !SI_BIOS!` under Hardware.

26.2 Add to the collector `'DAY=' + $now.ToString('yyyyMMdd')` and use `sysinfo_%COMPUTERNAME%_%SI_DAY%.txt`.

26.3 In the collector: `Get-LocalUser | ForEach-Object { 'USER_' + ($_.Name -replace '[^A-Za-z0-9]','_') + '=' + $_.Name }`. In the report: `for /f "tokens=1,* delims==" %%a in ('set SI_USER_ 2^>nul') do echo %%b`.

26.4 Let the collector print a `LOW_` line when `$free / $total` is below 0.10: `if ($free / $total -lt 0.10) { 'LOW_' + $letter + '=' + $_.DeviceID + ' has less than 10 percent free' }`. In the Batch file, after loading, run `for /f "tokens=1,* delims==" %%a in ('set SI_LOW_ 2^>nul') do echo WARNING: %%b`.

26.5 After writing the report:

```
if exist "%OUT_DIR%\latest.txt" copy /y "%OUT_DIR%\latest.txt" "%OUT_DIR%\previous.txt" >nul
copy /y "%REPORT%" "%OUT_DIR%\latest.txt" >nul
if exist "%OUT_DIR%\previous.txt" fc "%OUT_DIR%\previous.txt" "%OUT_DIR%\latest.txt"
```

Differences in the `Generated` line will always appear; filter that line with `findstr /v` into temporary files first if you want a quieter comparison.

Chapter 27

27.1 Change the table lines to `for %%e in (jpg jpeg png gif bmp webp tif tiff svg heic) do ...` and `for %%e in (pdf doc docx xls xlsx ppt pptx txt rtf odt csv md log) do ...`.

27.2 In `:do_move`, after a successful move or in dry run, add `set /a "count_%cat%+=1"`. At the end:

```
for /f "tokens=1,2 delims==" %%a in ('set count_ 2^>nul') do echo   %%a = %%b
```

27.3 At the start of `:process` add `if /i "%~nx1"=="desktop.ini" exit /b 0`.

27.4 Take the third argument as the extension: `set "ONLY=%~3"` near the top (written as `xyz`, without a dot), and at the start of `:process` add `if defined ONLY if /i not ".%ONLY%"=="%~x1" exit /b 0`.

27.5 Write each successful move as `>> "%UNDO%" echo "%dest%\%newname%"^|"%~1"`; the undo script reads the lines with `for /f "usebackq tokens=1,2 delims=|" %%a in ("%UNDO%")` and runs `move %%a %%b` after checking that the original folder exists. Reverse order is only needed if names were reused.

Chapter 28

28.1 `rename_bulk.cmd "%USERPROFILE%\BatchBook\work" suffix _old` prints a preview only.

28.2 In `:make_number` use `set "num=0000%seq%"` and `%num:~-4%`.

28.3 Add `lower` to the allowed operations and in `:rename_one`:

```
if /i "%OP%"=="lower" call :make_lower
```

```
:make_lower
setlocal EnableDelayedExpansion
set "s=!name!"
for %%L in (a b c d e f g h i j k l m n o p q r s t u v w x y z) do set "s=!s:%%L=%%L!"
endlocal & set "newbase=%s%"
exit /b 0
```

(Names containing `!` would be damaged; skip those first.)

28.4 Read `set "START=%~5"` after the other arguments, validate it with `:check_text`, and in the setup set `set /a "seq=START-1"`; the routine then continues from `START`.

28.5 After each successful rename append `>> "%LOGFILE%" echo %newname%^|%~nx1`. The `undo` operation reads the log with `for /f "usebackq tokens=1,2 delims=|" %%a in ("%LOGFILE%")` and runs `ren "%TARGET%\%%a" "%%b"`, checking that `%%a` exists and `%%b` does not.

Chapter 29

29.1 `analyze_log.cmd sample_app.log 2` prints only the two most frequent errors.

29.2 Before the report block add `for %%f in ("%LOG%") do set "SIZE=%%~zf"`, and in the block `echo Size : !SIZE! bytes`.

29.3 Inside the report block:

```
echo [Entries per module]
for %%m in (app db web cache disk auth) do (
    set "c=0"
    for /f %%c in ('findstr /c:" [%%m] " "%LOG%" ^| find /c /v ""') do set "c=%%c"
    echo   %%m  !c!
)
```

29.4 Before the block compute the errors and total, then:

```
set /a "pct10=ERRORS*1000/TOTAL, whole=pct10/10, frac=pct10%%10"
```

and print `!whole!.!frac! percent`. (`ERRORS` comes from the same `findstr ... | find /c` command used in the level table.)

29.5 Parse `/level:` as in `rename_bulk.cmd` (Chapter 28), validate with the allow-list `[A-Z]`, build `set "FILTER= %LEVEL% "`, and replace `"%LOG%"` in the `findstr` commands by the filtered temporary file created with `findstr /c:"%FILTER%" "%LOG%" > "%FILTERED%"`.

Chapter 30

30.1 `backup.cmd "%USERPROFILE%\BatchBook\work" "%USERPROFILE%\BatchBook\backups" /dry`, then open the file in `backups\logs`.

30.2 Change `set "KEEP=7"` to `set "KEEP=14"`.

30.3 In `:parse` add `if /i "!a:~0,4!"=="/xd:" (set "XD=!a:~4!" & shift & goto :parse)`, validate the name with an allow-list, and after `FLAGS` is built add `if defined XD set "FLAGS=!FLAGS! /xd !XD!"`.

30.4 After the retention loop recount with the same `dir` loop and print `Backups now: !count2!`.

30.5 After a successful (non-dry) copy:

```
for /f %%n in ('dir /s /b /a:-d "!SRC!" ^| find /c /v ""') do set "n1=%%n"
for /f %%n in ('dir /s /b /a:-d "!RUN_DIR!" ^| find /c /v ""') do set "n2=%%n"
if not "!n1!"=="!n2!" (echo Verification FAILED: !n1! source files, !n2! in backup & exit /b 4)
```

(Files that were excluded or unreadable will make the counts differ; adjust for your exclusions.)

Chapter 31

31.1 `diskmon.cmd 90 80` makes almost every drive WARNING or CRITICAL, because the thresholds are percentages of free space.

31.2 Change the CSV header to `computer,timestamp,...` and each line to `>> "%HIST%" echo %COMPUTERNAME%,!TS!,%~1,...`. (Delete or move the old CSV so that header and data agree.)

31.3 Add `set "SKIP=%~3"` with a check that it matches `[A-Za-z]:`, and at the top of `:evaluate`: `if /i "%~1"=="!SKIP!" exit /b 0`.

31.4 In `:evaluate`, after the percentage rule add `if %3 lss 5 (set "state=2" & set "label=CRITICAL")` before the `worst` update.

31.5 Before appending the new line, find the previous free space of the drive:

```
set "prev="
for /f "usebackq skip=1 tokens=1-5 delims=," %%a in ("%HIST%") do if /i "%%b"=="%~1" set "prev=%%d"
```

Then `if defined prev set /a "delta=%3-prev"` and print it. This reads the file **before** the new line is written for this run.

Chapter 32

32.1 `cleanup_old.cmd cleanup_policy.txt` prints the counts and deletes nothing.

32.2 Add a line like `14|C:\Users\Alex\BatchBook\work\old` to `cleanup_policy.txt`; the dry run lists it.

32.3 Define `set "MIN_PATH=10"` near the top, compute `set /a "limit=MIN_PATH-1"` and test `if "!folder:~%limit%,1!"==""`.

32.4 Parse `/verbose`, and in the dry-run branch run `forfiles /p "!folder!" /s /d -!days! /c "cmd /c if @isdir==FALSE echo @path" 2>nul` to print each name.

32.5 After the delete step add:

```
for /f "delims=" %%d in ('dir /ad /b /s "!folder!" 2^>nul ^| sort /r') do rd "%%d" 2>nul
```

Deepest folders are processed first because of the reverse sort, and `rd` without `/s` removes a folder only if it is empty, so nothing else is touched. Run it only after the file deletion and only inside the validated folder.

Chapter 33

33.1 Append the lines `192.168.1.1` (your router) and `8.8.8.8` to `targets.txt`.

33.2 Change `set "SENT=4"` to `set "SENT=2"`.

33.3 After computing `rx`, add `set /a "lost=SENT-rx"` and `set /a "pct=lost*100/SENT"`; then `if !pct! gtr 50 set "bad=1"` in the judging section.

33.4 Parse a `/csvonly` flag into `QUIET`; change `:emit` to `if not defined QUIET echo(%~1`, while the report and CSV writes stay unconditional.

33.5 Add to `PSCMD` the segment `$r = Test-Connection -ComputerName $h -Count 2 -ErrorAction SilentlyContinue; if ($r) { 'AVG=' + [int](($r | Measure-Object -Property ResponseTime -Average).Average) } else { 'AVG=-' }`. (In Windows PowerShell 5.1 the property is `ResponseTime`; in PowerShell 7 it is `Latency`.) Read `R_AVG` like the other values and add it to the table row and CSV.

Chapter 34

34.1 Add `echo 8. Open practice folder` (and extend the `choice` keys and dispatch), with `start "" explorer "%USERPROFILE%\BatchBook"` as the action.

34.2 Add key `5` to the `choice` list (change `1234B` to `12345B` and `B` to number 6), add `if "!s!"=="5" set "svc=Themes"` and extend the menu text.

34.3

```
:act_taillog
for /f %%n in ('find /c /v "" ^< "%LOGFILE%"') do set "n=%%n"
set /a "skip=n-20"
if !skip! lss 0 set "skip=0"
more +!skip! "%LOGFILE%"
exit /b 0
```

34.4 After the colour setup add:

```
where /q powershell || (echo PowerShell was not found. & exit /b 3)
```

34.5 Read the lines with `for /f "usebackq eol=# tokens=1-3 delims=|" %%a in ("%HERE%toolkit.ini")`, build `set "ITEM_%%a=%%c"` and `set "LABEL_%%a=%%b"`, print the labels, collect the keys for `choice /c`, and dispatch with `call :run_module !ITEM_%sel%!`. The script names come from the file, so protect the file with the same permissions as the script (Chapter 23).

Part 5 Quiz

Q1. So that input such as `08` is not read as an invalid octal number. **Q2.** Values such as processor names contain parentheses; read with `%name%` inside a block, a `)` would end the block, while delayed expansion inserts the text after parsing. **Q3.** It is dry-run by default (only `/run` moves files) and it never overwrites. **Q4.** Renaming while the wildcard loop is still reading the folder can make the loop meet renamed files again; `dir /b` takes a snapshot of the names. **Q5.** It sorts the messages, compares each line with the previous one and counts runs, storing zero-padded counts that sort numerically. **Q6.** Those values are bit flags describing copies, extras or mismatches, not failures; only 8 and above mean errors. **Q7.** Only folders whose names match exactly `backup_dddddddd_dddddd` are deleted, and `/dry` only reports. **Q8.** 0 ok, 1 warning, 2 critical; 3 means the monitor itself failed, so it is not confused with a disk state. **Q9.** The second count measures the real result (files that could not be deleted remain), so the report is not just the intention. **Q10.** Host and port are passed in environment variables and read by PowerShell as data; the names are also validated by an allow-list. **Q11.** Later commands replace `%errorlevel%`, so the choice would be lost. **Q12.** There the original session is meant to end, because an elevated copy has started; elsewhere it would close the caller's session unexpectedly.

Appendix I: Recommended Next Steps and Further Resources

I.1 Where to go from here

1. **Re-read your own scripts** after a month and apply the checklist of Chapter 24.
2. **Learn PowerShell** next. Batch is the launcher; PowerShell is the engine for objects, JSON, remoting and modules. The interop techniques of Chapter 19 are the bridge.
3. **Learn Git** and keep every script under version control with `.gitattributes` set for CRLF.
4. **Automate on a schedule** with `schtasks` or the Task Scheduler, always with logging and exit codes.
5. **Practise security review**: take the checklist of Chapter 23 and read scripts from your own environment.
6. **Build something real**: pick a repetitive task in your work and automate it with the patterns of Part 5.
7. **Look at other languages for heavy work:** Python is a common choice for larger tools and cross-platform scripts.

I.2 Reference material

The addresses below were correct at the time of writing; websites change, so search for the title if a link no longer works.

Resource	What it offers
Microsoft Learn, "Windows commands" reference (`learn.microsoft.com`)	The official description of each command and switch
Microsoft Learn, PowerShell documentation	Reference for cmdlets and language
SS64 Windows CMD reference (`ss64.com/nt`)	A thorough, example-rich command index
Rob van der Woude's Scripting Pages (`robvanderwoude.com`)	Long-standing collection of batch techniques
Stack Overflow, tag `batch-file`	Questions and answers, including detailed explanations of how `cmd.exe` parses lines
Your own `command /?` output	Always authoritative for your Windows version

I.3 A last piece of advice

Read the help (`/?`) of a command before you rely on it, test destructive scripts on copies, and when a script grows beyond a few hundred lines or starts to need data structures, change tools. Batch scripting is most valuable when it stays small, clear and safe.

-

Index

Index of commands and keywords

Back matter

Index of Commands and Keywords

A

`allow-list` 134, 205, 211, 212, passim
`ANSI escape` 11, 147, 152, 154, 296
`AppLocker` 209
`assoc` 17, 282
`attrib` 30, 210, 280

B

`.bat` 4, 5, 11, 12, passim
`bisection` 65, 68
`bit mask` 26, 28
`bitwise` 70, 71, 73, 78
`BOM` 20, 21, 22, 292, 296

C

`call` 5, 11, 12, 17, passim
`call :label` 11, 12, 39, 100, 101, 189, 191, 281, 294, 297
`cd` 7, 13, 23, 24, passim
`%cd%` 39, 40, 49, 56, 58, 59, 129, 148, 187, 197 ...
`certutil` 128, 175, 176, 177, 209, 210, 211, 280, 295
`chcp` 16, 19, 20, 21, 22, 292, 296
`checklist` 135, 193, 219, 220, 278, 291, 292, 293, 323
`choice` 31, 83, 84, 87, passim
`CIM` 160, 162, 163, 228, 231, 296
`clip` 280
`cls` 30, 34, 37, 39, 147, 150, 151, 154, 223, 224 ...
`.cmd` 4, 10, 11, 12, passim
`cmd` 4, 6, 8, 9, passim
`code page` 19, 20, 22, 296
`color` 50, 147, 151, 152, 154, 188, 281
`comp` 280
`conhost` 10, 42, 296
`copy` 12, 15, 16, 23, passim
`credential` 209, 212
`CRLF` 17, 21, 42, 216, 218, 219, 292, 295, 296, 323
`cscript` 174, 283
`curl` 11, 164, 167, 169, 175, 177, 210, 283, 287

D

`date` 15, 25, 27, 31, passim
`%date%` 15, 39, 40, 49, passim
`del` 29, 30, 31, 32, passim
`delayed expansion` 8, 11, 13, 43, passim
`delims` 66, 90, 93, 94, passim
`dir` 12, 14, 23, 25, passim
`%~dp0` 16, 18, 21, 53, passim
`driverquery` 186, 282
`dry run` 135, 136, 138, 179, passim

E

`echo` 5, 11, 13, 15, passim
`else` 8, 50, 56, 58, passim
`EnableDelayedExpansion` 50, 63, 64, 66, passim
`EnableExtensions` 38, 50, 135, 137, passim
`endlocal` 39, 44, 50, 51, passim
`environment variable` 45, 172, 178, 199, 207, 208, 272, 275
`eol=` 93, 94, 95, 98, 218, 239, 241, 246, 262, 264 ...
`errorlevel` 17, 21, 50, 77, passim
`%errorlevel%` 29, 38, 39, 49, passim
`eventcreate` 259
`exit` 11, 26, 28, 29, passim
`exit /b` 38, 39, 40, 42, passim
`exit code` 29, 31, 32, 38, passim

F

`%~f0` 56, 176, 187, 262, 274, 289
`fc` 127, 128, 129, 133, 231, 232, 280, 286
`find` 14, 19, 31, 32, passim
`findstr` 31, 77, 78, 82, passim
`findstr /r` 78, 126, 133, 149, passim
`fltmc` 134, 135, 138, 158, 187, 188, 273, 275, 283, 295
`for` 6, 8, 11, 12, passim
`for /d` 92, 98, 294
`for /f` 11, 66, 68, 90, passim
`for /l` 8, 64, 69, 90, passim
`for /r` 92, 98, 129, 236, 242, 294
`forfiles` 128, 129, 144, 145, 231, 236, 254, 261, 263, 264 ...
`ftype` 17, 282

G

`Get-CimInstance` 160, 161, 174, 176, 228, 229, 257, 283
`Get-Date` 141, 142, 143, 161, 228, 251, 257, 267, 283, 295
`getmac` 166, 186, 283
`Git` 118, 138, 218, 221, 280, 323
`.gitattributes` 218, 219, 220, 323
`goto` 11, 17, 38, 43, passim
`goto :eof` 38, 100, 101, 102, 105, 107, 108, 149, 281, 294

H

`histogram` 244, 248, 249
`hostname` 31, 158, 282

I

`icacls` 208, 280
`if` 4, 8, 11, 13, passim
`if defined` 46, 50, 65, 80, passim
`if exist` 56, 80, 81, 82, passim
`if not` 38, 55, 57, 58, passim
`injection` 78, 88, 172, 204, 292, 296
`ipconfig` 96, 119, 151, 164, 167, 169, 170, 283

L

`label` 11, 12, 36, 37, passim
`least privilege` 204, 208, 212, 296
`local environment` 50, 281, 296

M

`md` 7, 26, 32, 82, passim
`mkdir` 26, 279
`mklink` 31, 280
`modifier` 54, 56, 278, 289, 290, 296
`more` 10, 17, 23, 28, passim
`move` 6, 23, 29, 43, passim

N

`net` 119, 135, 148, 155, 157, 158, 162, 210, 211, 282 ...
`netsh` 166, 167, 170, 210, 283
`netstat` 164, 166, 169, 170, 186, 283
`NO_COLOR` 271, 272, 274
`NoDefaultCurrentDirectoryInExePath` 208
`NoProfile` 210
`nslookup` 164, 165, 166, 168, 169, 170, 283
`>nul` 20, 37, 78, 82, passim
`%~nx0` 56, 57, 58, 77, passim

O

`obfuscation` 210, 212
`octal` 72, 76, 78, 140, 142, 145, 199, 202, 224, 226 ...

P

`parameter` 23, 53, 54, 207, 278, 288, 289, 290, 297
`PATH` 12, 13, 14, 17, passim
`PATHEXT` 17, 18
`pathping` 165, 283
`pause` 11, 18, 19, 21, passim
`persistence` 210, 212
`ping` 10, 13, 14, 38, passim
`pipe` 30, 36, 95, 109, passim
`popd` 24, 32, 56, 187, 279
`powershell` 10, 11, 12, 14, passim
`prompt` 5, 6, 7, 10, passim
`pushd` 24, 32, 56, 137, 187, 199, 279

Q

`queue` 183, 184

R

`%random%` 49, 75, 185, 186, 188, 210, 223, 225, 226, 245 ...
`rd` 26, 32, 85, 92, passim
`recursion` 106, 107, 124, 129, 236, 242
`redirection` 4, 8, 13, 30, passim
`reg` 155, 159, 160, 162, 210, 211, 282, 287
`regular expression` 77, 125, 248, 297
`rem` 11, 13, 34, 36, passim
`ren` 29, 240, 242, 279
`retention` 141, 231, 250, 251, 252, 253, 254, 255, 261, 277
`robocopy` 10, 23, 26, 27, passim
`route` 150, 166, 167, 168, 170, 274, 283
`RunAs` 187, 188, 209, 274, 275

S

`sc` 155, 157, 158, 162, 210, 259, 274, 275, 282, 287 ...
`schtasks` 145, 155, 158, 162, passim
`set` 9, 11, 13, 14, passim
`set /a` 11, 44, 47, 51, passim
`set /p` 44, 46, 47, 51, passim
`setlocal` 38, 39, 40, 44, passim
`setx` 47, 48, 51
`SHA256` 128, 175, 209, 211, 280, 295
`shift` 53, 55, 58, 59, 71, 73, 238, 241, 242, 251 ...
`shutdown` 155, 161, 162, 282
`skip=` 93, 95, 294
`sort` 25, 31, 32, 113, passim
`stack` 24, 106, 107, 166, 167, 183, 184, 323
`standard streams` 109, 297
`start` 11, 16, 20, 21, passim
`Start-Process` 157, 187, 274
`subroutine` 8, 38, 97, 98, passim
`switch` 23, 24, 25, 27, passim
`systeminfo` 160, 162, 232, 282

T

`tar` 11, 175, 177, 178, 234, 254, 280, 287
`Task Scheduler` 38, 56, 158, 259, 323
`taskkill` 155, 157, 161, 162, 163, 282, 287, 295
`tasklist` 134, 155, 156, 157, 162, 166, 169, 170, 186, 282 ...
`TEMP` 29, 46, 49, 52, 127, 210
`Test-NetConnection` 164, 167, 168, 267
`time` 12, 25, 31, 39, passim
`%time%` 39, 40, 49, 99, passim
`timeout` 37, 117, 144, 145, passim
`timestamp` 136, 141, 146, 198, passim
`title` 34, 37, 39, 41, passim
`token` 86, 93, 94, 95, 135, 174, 200, 231, 245, 247 ...
`tokens` 66, 90, 93, 94, passim
`tracert` 164, 165, 169, 283
`tree` 26, 30, 32, 92, 130, 279, 280, 294
`TTL=` 133, 139, 165, 168, 169, 266, 268, 269, 270, 287 ...
`type` 5, 7, 10, 16, passim
`tzutil` 99

U

`UAC` 187, 210, 297
`UNC` 24, 32, 56, 199, 279, 297
`usebackq` 90, 93, 94, 95, passim
`USERPROFILE` 45

V

`VBScript` 171, 174, 175, 177, 283
`ver` 31, 47, 84, 85, 282, 292
`version control` 209, 214, 215, 218, 219, 323

W

`where` 6, 12, 14, 17, passim
`whoami` 31, 135, 282
`wildcard` 25, 26, 62, 92, 98, 99, 236, 239, 241, 277 ...
`Windows PowerShell` 12, 171, 283
`Windows Terminal` 10, 11, 21, 151, 152, 296
`wmic` 142, 155, 160, 161, 162, 179, 193, 228, 232, 256 ...

X

`xcopy` 23, 26, 27, 32, 144, 192, 279, 286

#

`%*` 13, 39, 53, 55, passim
`%~1` 54, 55, 57, 58, passim
`2>&1` 84, 109, 110, 111, passim
`8191` 199